1. Geometrické transformace

Geometrické transformace jsou jedněmi z nejčastěji používaných operací v počítačové grafice. Patří mezi ně změna polohy objektu, změna jeho velikosti, tvaru, rotace objektu a operace vzniklé jejich skládáním.

Geometrické transformace mohou být aplikovány na **body objektu**, které tak mění svou polohu či svůj tvar ([Ježe93]). S touto operací se setkáme v počítačové třírozměrné animaci, kdy se například nějaký objekt pohybuje po své dráze, při psaní textu na křivku v DTP atp. Druhou možností je podrobit transformaci **souřadnicový systém**. To obyčejně činíme za účelem získání výhodnější reprezentace objektu pro jeho další zpracování, například pro výpočet objemu, projekci, řešení viditelnosti aj.

1.1 Transformace afinních souřadnic

Předpokládejme, že máme ve 3D dvě afinní soustavy souřadnic:

$$(P, \vec{e}_1, \vec{e}_2, \vec{e}_3) \quad (P', \vec{e}_1', \vec{e}_2', \vec{e}_3') \tag{1.1}$$

Počátek P resp. P' má souřadnice P_x, P_y, P_z resp. P_x', P_y', P_z', vektor $\vec{e}_i$ resp. $\vec{e}_i'$ má souřadnice $e_{i,x}$, $e_{i,y}$, $e_{i,z}$ resp. $e_{i,x}'$, $e_{i,y}'$, $e_{i,z}'$.

Pro bázové vektory $\vec{e}_i$ prvního a $\vec{e}_i'$, i = 1,2,3 druhého souřadnicového systému platí, že jsou **lineárně nezávislé**, tj. není možné vyjádřit kterýkoliv z vektorů jako lineární kombinaci vektorů v příslušné bázi zbývajících. V afinní soustavě souřadnic na ně není kladena žádná další podmínka stran jejich velikosti či vzájemné orientaci ([Budí83]), viz obrázek 1.1.

Mějme dále libovolný bod X (viz obrázek 1.1) o souřadnicích $X = [x,y,z] = [x',y',z']$ a hledejme vztahy pro jejich vzájemné vyjádření. Souřadnice bodu je možno zapsat jako lineární kombinaci vektorů báze vztaženou k souřadnicím počátku:

$$X = P + x\vec{e}_1 + y\vec{e}_2 + z\vec{e}_3 = P' + x\vec{e}_1' + y\vec{e}_2' + z\vec{e}_3' \tag{1.2}$$

Předpokládejme, že jsou známé rovnice přechodu od nečárkované báze k bázi druhé:

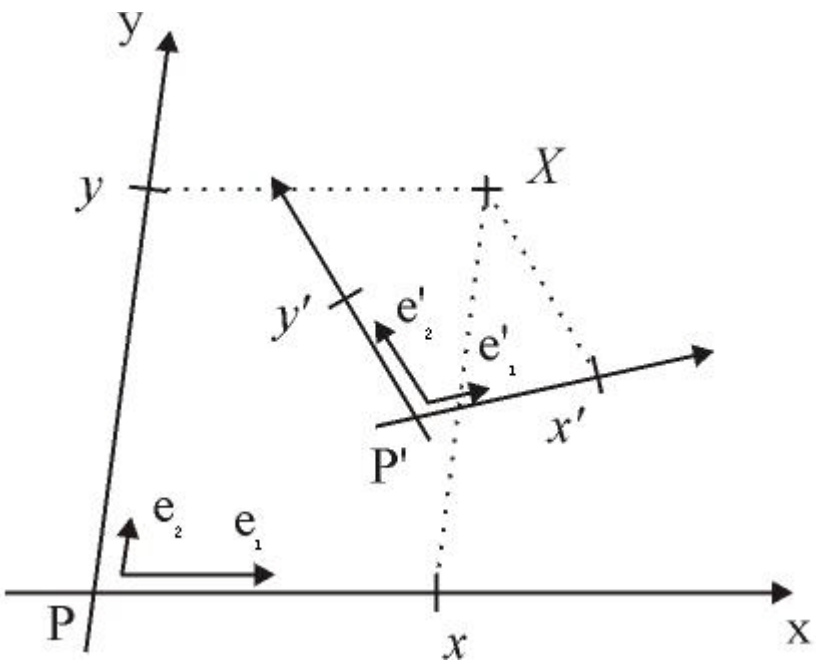
$$\begin{aligned} \vec{e}_1' &= a_{11}\vec{e}_1 + a_{12}\vec{e}_2 + a_{13}\vec{e}_3 \\ \vec{e}_2' &= a_{21}\vec{e}_1 + a_{22}\vec{e}_2 + a_{23}\vec{e}_3 \\ \vec{e}_3' &= a_{31}\vec{e}_1 + a_{32}\vec{e}_2 + a_{33}\vec{e}_3 \end{aligned} \tag{1.3}$$

tj.

$$E' = EA \tag{1.4}$$

neboli:

$$[\vec{e}_1', \vec{e}_2', \vec{e}_3'] = [\vec{e}_1, \vec{e}_2, \vec{e}_3] \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \tag{1.5}$$



Obr. 1.1: K transformaci souřadnic

Rovnice 1.3 jsou vyjádřením vektorů jedné báze pomocí vektorů báze druhé, a proto například první sloupec (souřadnice a_{11}, a_{12} a a_{13}) je souřadnicemi vektoru $\vec{e}_1'$ v bázi vektorů $\vec{e}_1$, $\vec{e}_2$, $\vec{e}_3$.

Matice A je maticí přechodu od bázových vektorů jedné soustavy souřadnic k bázovým vektorům soustavy druhé. Počátek P' má v první (nečárkované) soustavě souřadnice P'_x, P'_y, P'_z, vektor (P_x,P_y,P_z) tedy určuje posunutí čárkované soustavy souřadnic vzhledem k soustavě nečárkované.

Rovnice 1.6 je hledanou rovnicí přepočtu souřadnic bodu z čárkované soustavy souřadnic do soustavy nečárkované.

$$\begin{aligned} x &= a_{11}\,x'+a_{12}\,y'+a_{13}\,x'+P_x' \\ y &= a_{21}\,x'+a_{22}\,y'+a_{23}\,z'+P_y' \\ z &= a_{31}\,x'+a_{32}\,y'+a_{33}\,z'+P_z' \end{aligned} \tag{1.6}$$

Maticová podoba rovnice 1.6 má tvar 1.7 a 1.8.

$$X = X'A+P' \tag{1.7}$$

$$[x,y,z] = [x',y',z']\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} + \left[P_x',P_y',P_z'\right] \tag{1.8}$$

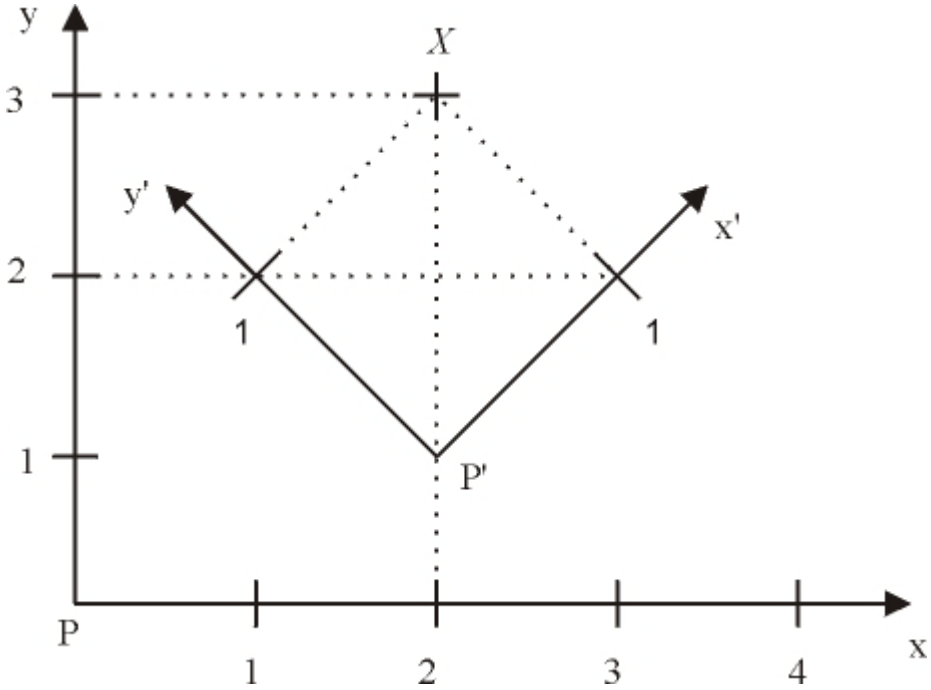
Inverzní úloha, výpočet čárkovaných souřadnic pomocí nečárkovaných, má maticovou podobu 1.9.

$$X' = XA^{-1} - P'A^{-1} \tag{1.9}$$

Matice A^{-1} musí existovat, neboť vektory $\vec{e}_1'$, $\vec{e}_2'$, $\vec{e}_3'$ jsou, jak již bylo výše uvedeno, lineárně nezávislé. Jelikož se jedná o jednoduché matice (pro operace ve 2D jsou typu 2×2, ve 3D pak 3×3), je i výpočet inverzní matice jednoduchý ([Bube94]).

Za poznámku stojí, že v případě transformace afinních souřadnic se musí při inverzní operaci vypočítat souřadnice počátku nečárkované báze vzhledem k bázi čárkované. Vztah $P'A^{-1}$ je vyjádřením souřadnic počátku P nečárkované soustavy souřadnic v soustavě čárkované.

Transformace afinních souřadnic ve 2D znamená použití vektorů báze, které mají dvě souřadnice, matice přechodu bude tedy typu 2×2, stejně jako souřadnice počátku a transformované body budou mít dvě souřadnice. Pro jejich vzájemné transformace pak platí rovnice 1.7 a 1.9.



Obr. 1.2: Příklad k převodu souřadnic

Příklad

Ve 2D jsou zadány dva souřadnicové systémy (viz obrázek 1.2). Vektory báze první soustavy mají souřadnice $\vec{e}_1$ = (1,0), $\vec{e}_2$ = (0,1) a počátek P = [0,0]. Vektory báze druhé soustavy mají v první soustavě souřadnice $\vec{e}_1'$ = (1,1), $\vec{e}_2'$ = (-1,1) a její počátek P' = [2,1]. Uvažujme dále bod X o čárkovaných souřadnicích $X = [1,1]'$. Spočtěme jeho souřadnice v nečárkované soustavě. Matici A získáme dosazením vektorů čárkované báze za její řádky:

$$A = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Dosazením do vztahu 1.3 ověříme, že tato matice opravdu transformuje vektor (1,0) na (1,1) a vektor (0,1) na (-1,1).

Po dosazení do vztahu 1.7 za souřadnice počátku spočteme, že souřadnice bodu X v nečárkované soustavě jsou [2,3], a dosazením do vztahu 1.9 se přesvědčíme o správnosti výpočtu.

1.1.1 Transformace kartézských souřadnic

Při transformaci afinních souřadnic je zásadní podmínkou kladenou na vektory báze jejich lineární nezávislost. Pro kartézský souřadnicový systém platí ([Budí83]), že bázové vektory musí být **ortonormální**, to jest

1. lineárně nezávislé,
2. po dvou na sebe kolmé a
3. každý velikosti jedna.

To ve svém důsledku způsobuje že, determinant matice přechodu k jiným kartézským souřadnicím je roven jedné nebo mínus jedné [1] a že **inverzní matice je maticí transponovanou**, neboli $A^{-1} = A^T$.

Z toho vyplývá, že inverzní operace přepočtu kartézských souřadnic prostoru spočívá jen v transposici prvků matice vyjadřující souřadnice vektorů báze a v odečtení souřadnic počátku.

1.2 Homogenní souřadnice

Pro reprezentaci transformace afinních souřadnic v počítači potřebujeme matici přechodu od jedné báze k druhé a vektor posunutí počátku. Při implementaci v nějakém vyšším programovacím jazyce to znamená použít datovou strukturu pro matici a jednorozměrný vektor. Při vlastním výpočtu se pak násobí souřadnice bodu maticí přechodu a k výsledku se přičítá vektor posunutí. Pro zjednodušení výpočtů se s výhodou používá reprezentace transformací pomocí **homogenních souřadnic** ([Polá91]).

Uspořádanou čtveřici $[X,Y,Z,w]$ nazýváme homogenními souřadnicemi bodu $[x,y,z]$ ve 3D, platí-li, že existuje bod, jehož kartézské souřadnice jsou:

$$x = \frac{X}{w},\ y = \frac{Y}{w},\ z = \frac{Z}{w} \tag{1.10}$$

Tento bod je svými homogenními souřadnicemi určen jednoznačně. Souřadnici w nazýváme *váhou* bodu. Zavedení homogenních souřadnic nám umožňuje vyjádřit pomocí jediné matice i posunutí.

Položíme-li $w = 1$, můžeme transformační vztah 1.7 zapsat v jednoduché podobě

$$X = X' A_H, \tag{1.11}$$

a podobně vztah 1.9 je možno zapsat jako

$$X' = X A_H^{-1}, \tag{1.12}$$

kde

$$A_H = \begin{bmatrix} a_{11} & a_{21} & a_{31} & 0 \\ a_{12} & a_{22} & a_{32} & 0 \\ a_{13} & a_{23} & a_{33} & 0 \\ a_{14} & a_{24} & a_{34} & 1 \end{bmatrix} \tag{1.13}$$

Prvky a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23},a_{31}, a_{32}, a_{33} matice A_H odpovídají stejným prvkům v matici A ve vztahu 1.7. Prvky a_{14}, a_{24}, a_{34} odpovídají posunutí počátku, tj. souřadnicím P_x', P_y',P_z', resp. poslední řádek je homogenními souřadnicemi počátku a první tři řádky odpovídají homogenním souřadnicím bázových vektorů.

Poznamenejme, že v případě ortonormální báze, to jest při transformaci kartézských souřadnic, již **neplatí**

$$A_H^{-1} = A_H^T. \tag{1.14}$$

Matice reprezentující transformaci ve 2D bude mít analogický tvar:

$$A_H = \begin{bmatrix} a_{11} & a_{21} & 0 \\ a_{12} & a_{22} & 0 \\ a_{13} & a_{23} & 1 \end{bmatrix} \tag{1.15}$$

a pro převod souřadnic platí vztah 1.11 a 1.12.

Příklad

Transformační matice z příkladu k obrázku 1.2 má v této podobě tvar:

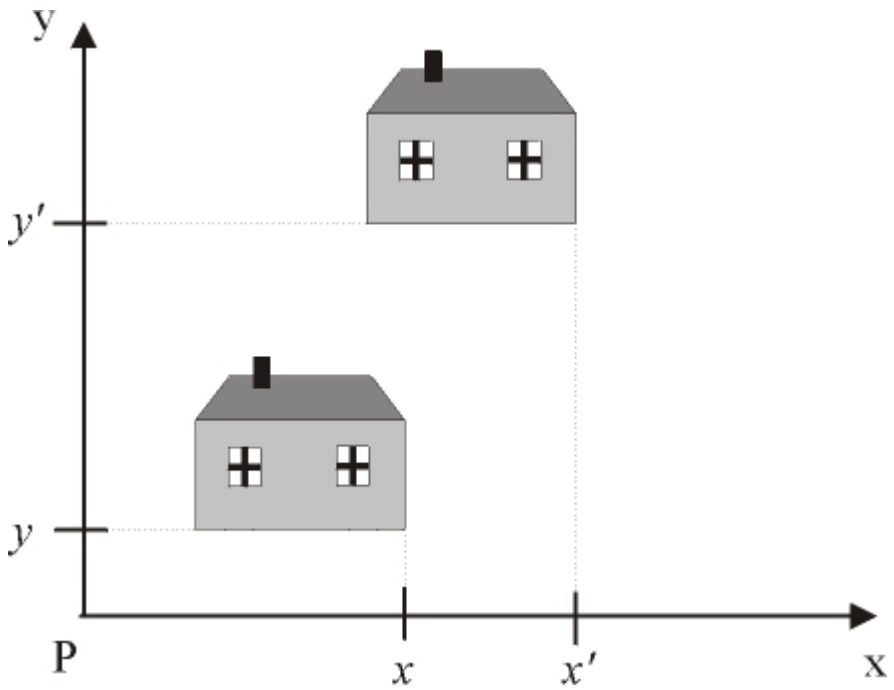
$$A = \begin{bmatrix} 1 & 1 & 0 \\ -1 & 1 & 0 \\ 2 & 1 & 1 \end{bmatrix}$$

Dosazením za bod X o čárkovaných souřadnicích $X = [1,1,1]'$ dostaneme $X = [2,3,1]$.

1.3 Transformace ve 2D

Doposud jsme se zabývali transformacemi zcela obecně, nyní uvedeme nejčastěji používané transformace v rovině a jim odpovídající transformační matice. Pro jejich reprezentaci budeme vycházet ze vztahu 1.15. V dalším textu nebudeme vždy uvádět matice pro inverzní operace, neboť jejich odvození je snadné.

Posunutí



Obr. 1.3: Posunutí

Posunutí nebo také translace (angl. *move, translate*) je transformací, při které si objekt zachovává svůj tvar (tedy i plochu a obvod) a mění pouze svou polohu. Při této operaci (viz obrázek 1.3) se v matici 1.15 uplatní pouze koeficienty a_{13} a a_{23}, které udávají změnu počátku souřadnicového systému. Matice pro vyjádření transformace transformace posunutí má tedy podobu:

$$A_M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ (X'-X) & (Y'-Y) & 1 \end{bmatrix} \tag{1.16}$$

kde (X'-X,Y'-Y) je vektor posunutí viz obrázek 1.3.

Inverzní matice má tvar:

$$A_M^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -(X'-X) & -(Y'-Y) & 1 \end{bmatrix} \tag{1.17}$$

Rotace

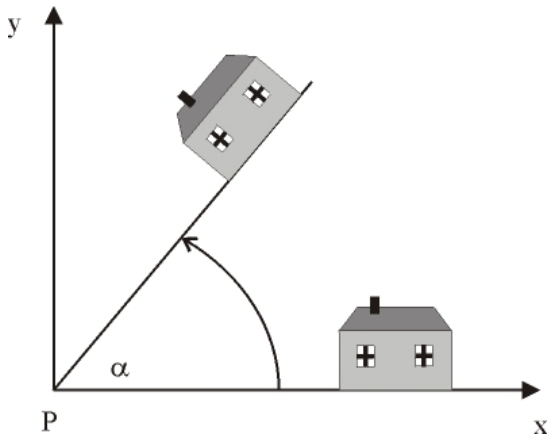
Vztah pro rotaci (angl. *rotation*) kolem počátku soustavy souřadnic o obecný úhel α (viz obrázek 1.4) lze zapsat pomocí matice

$$A_R = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) & 0 \\ -\sin(\alpha) & \cos(\alpha) & 0 \\ 1 & 1 & 1 \end{bmatrix} \tag{1.18}$$

Inverzní matice má potom tvar:

$$A_R^{-1} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 1 & 1 & 1 \end{bmatrix} \tag{1.19}$$

Vzhledem k tomu, že oba bázové vektory jsou na sebe kolmé a velikosti jedna, jedná se o transformaci kartézských souřadnic a matice inverzní je zároveň maticí transponovanou.



Obr. 1.4: Rotace

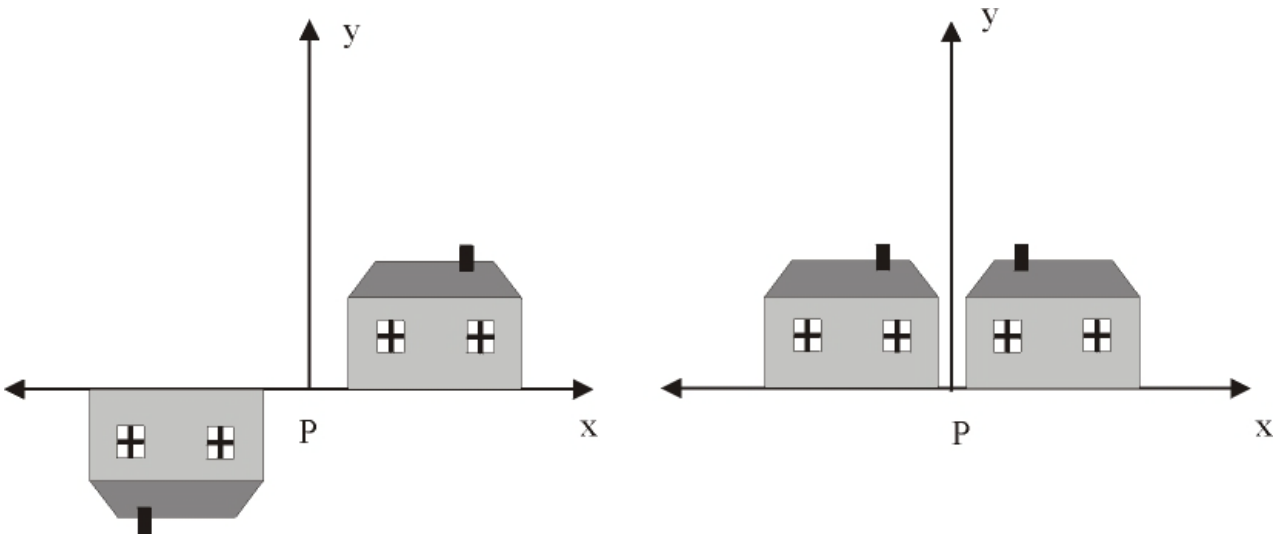
Souměrnost

Souměrnost (angl. *symetry*) je možno rozdělit na souměrnost **středovou** a **osovou** (viz obrázek 1.5). Středová souměrnost je rotací o 180 stupňů a její matici je možno odvodit z předchozího odstavce dosazením.

$$A_{Sstred} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \tag{1.20}$$

Dvě osové souměrnosti pak získáme zrcadlením podle os. Odpovídající matice mají tvar:

$$A_{Sx} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A_{Sy} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.21}$$



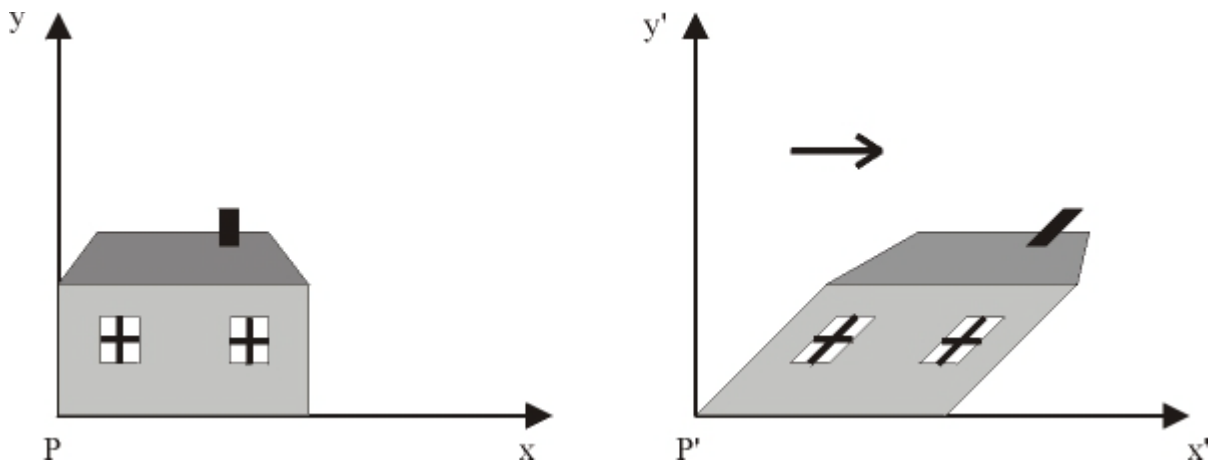
Obr. 1.5: Souměrnost podle středu a podle osy *y*

Zkosení

Matice operace zkosení (angl. *shear*) ve směru osy *x* a *y* má tvar

$$A_{Skx} = \begin{bmatrix} 1 & 0 & 0 \\ SH_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A_{Sky} = \begin{bmatrix} 1 & SH_y & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.22}$$

Obrázek 1.6 na následující stránce demonstruje použití této transformace.



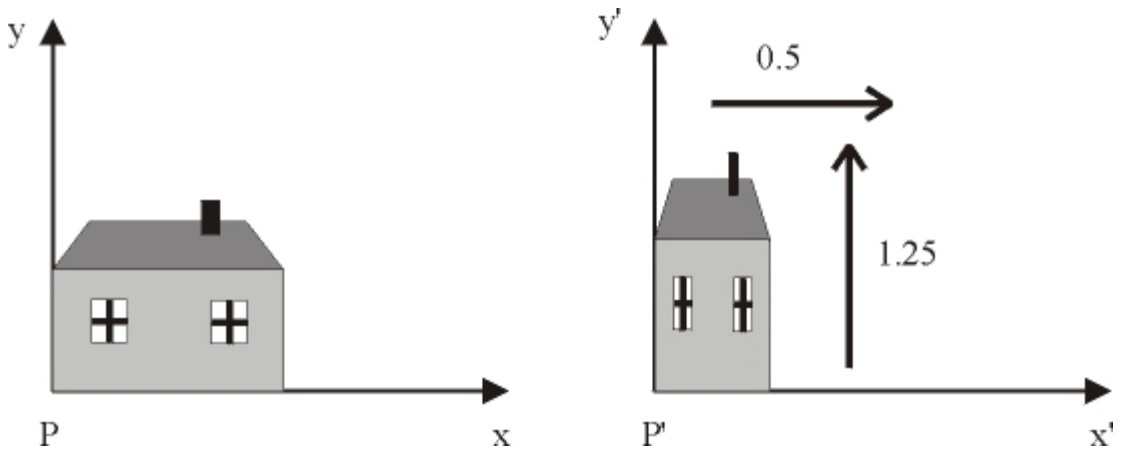
Obr. 1.6: Zkosení ve směru osy *x*

Změna měřítka

Změna měřítka (angl. *scale, zoom*) je změnou velikosti objektu ve směru měřítkování, případně jeho zvětšením či zmenšením, je-li koeficient měřítkování pro oba směry stejný. Pokud je absolutní hodnota koeficientu měřítkování v intervalu (0,1), dochází ke zmenšení v příslušném směru. Je-li absolutní hodnota koeficientu větší nežli jedna, dojde k prodloužení, je-li znaménko koeficientu záporné, dochází k prodloužení či kontrakci v opačném směru.

Obrázek 1.7 demonstruje změnu měřítka a vztah 1.23 je odpovídající transformační maticí.

$$A_S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad (1.23)$$



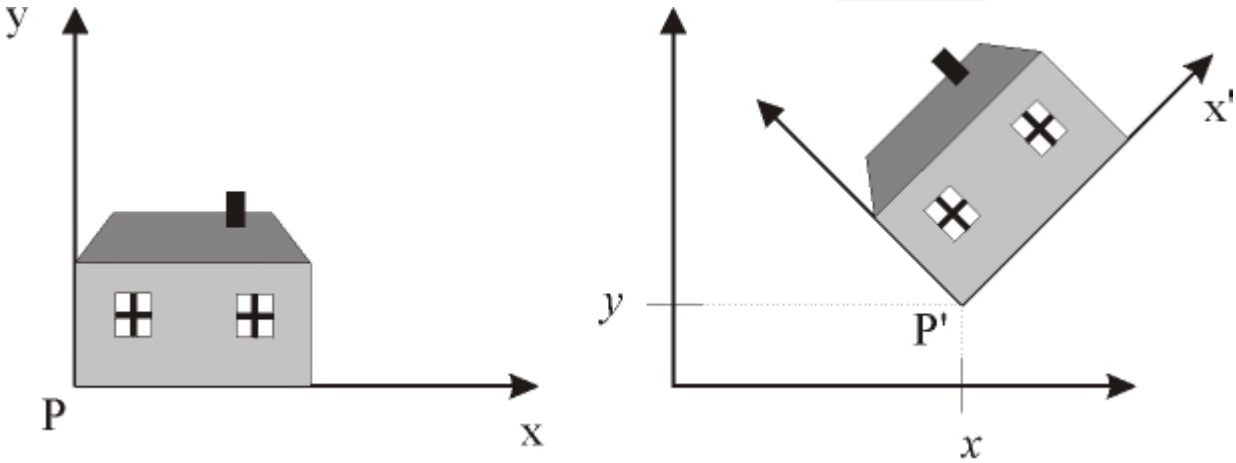
Obr. 1.7: Změna měřítka

Změna měřítka, je velmi často používanou transformací v systémech typu GKS, PHIGS ([Fole90]), ale i v technickém vybavení počítačů.

- **Uživatelský souřadnicový systém** je souřadnicový systém, ve kterém pracuje uživatel a bývá nazýván **WC** (angl. *World Coordinates*). Jeho jednotkami jsou typicky metry, palce atp. podle druhu aplikace.
- **Normalizovaný souřadnicový systém NC** (angl. *Normalized Coordinates*) je mezičlánkem mezi WC a fyzickým zařízením. Svým pojmenováním napovídá, že pracuje v intervalu hodnot < 0,1 >.
- **Souřadnicový systém zařízení DC** (angl. *Device Coordinates*) je dán zobrazovacím zařízením, tj. rozlišením monitoru, počtem bodů zobrazitelných na tiskárně atp.

Příklad

Odvoďme transformační matici pro rotaci okolo bodu [*x*,*y*] (viz obrázek 1.8).



Obr. 1.8: Rotace okolo bodu [x,y]

Operaci provedeme takto: objekt nejprve přemístíme pomocí matice A_M do počátku souřadnicového systému, poté ho otočíme pomocí matice A_R a nakonec objekt přemístíme zpět do původní polohy - inverzní matice A_M^{-1}. Vše provedeme tak, že vytvoříme matici, která vznikne složením všech tří operací. Výsledná matice je součin $A = A_M A_R A_M^{-1}$

APPLET APPLET

applet 1.1 *applet 1.2*

1.4 Transformace ve 3D

Transformace ve třech dimenzích jsou zobecněním 2D transformací. V počítačové grafice se pro ně používají matice ze vztahu 1.11, které jsou typu 4 × 4. První tři řádky jsou homogenními souřadnicemi vektorů báze a poslední z nich je homogenními souřadnicemi počátku. Opět se pro zjednodušení výpočtů pokládá $w = 1$.

Posunutí

Posunutí je snadným zjednodušením posunutí ve 2D viz obrázek 1.3.

$$A_M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ (X'-X) & (Y'-Y) & (Z'-Z) & 1 \end{bmatrix} \qquad (1.24)$$

Hodnoty (X'-X,Y'-Y,Z'-Z) jsou složkami vektoru posunutí.

Rotace

Rotace ve 3D se rozpadá na tři podpřípady rotací podle jednotlivých souřadnicových os. Matice pro rotaci kolem osy *x*,*y* a *z* mají postupně tvar:

$$A_{Rx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha) & \sin(\alpha) & 0 \\ 0 & -\sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.25}$$

$$A_{Ry} = \begin{bmatrix} \cos(\alpha) & 0 & \sin(\alpha) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\alpha) & 0 & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.26}$$

$$A_{Rz} = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) & 0 & 0 \\ -\sin(\alpha) & \cos(\alpha) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.27}$$

Změna měřítka

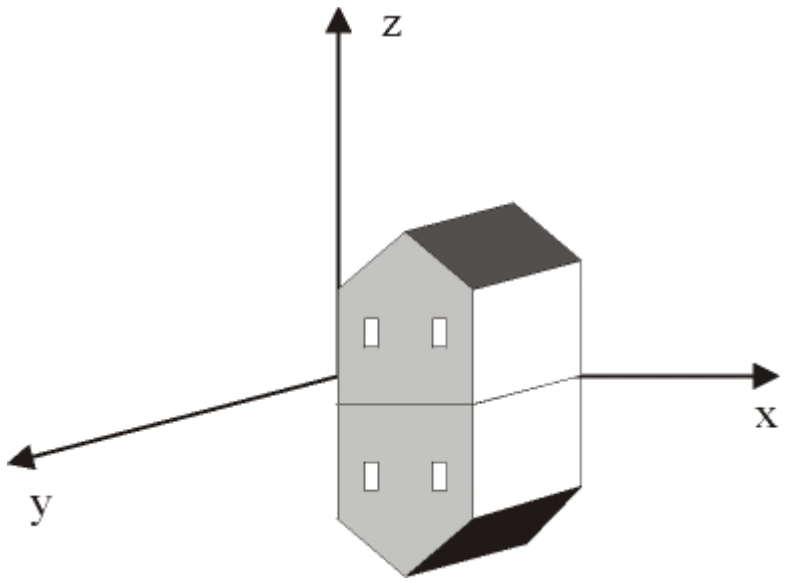
Změně měřítka (angl. *scale*) v prostoru odpovídá matice:

$$A_{Sc} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.28}$$

v níž koeficienty S_x,S_y a S_z určují změnu ve směru příslušné osy.

Souměrnost

Souměrnost (angl. *symetry*) je zvláštním případem změny měřítka. Zmenšením hodnoty koeficientu měřítka, se objekt zmenšuje a při záporném čísle se zrcadlově překlopí.



Obr. 1.9: Souměrnost podle roviny *xy*

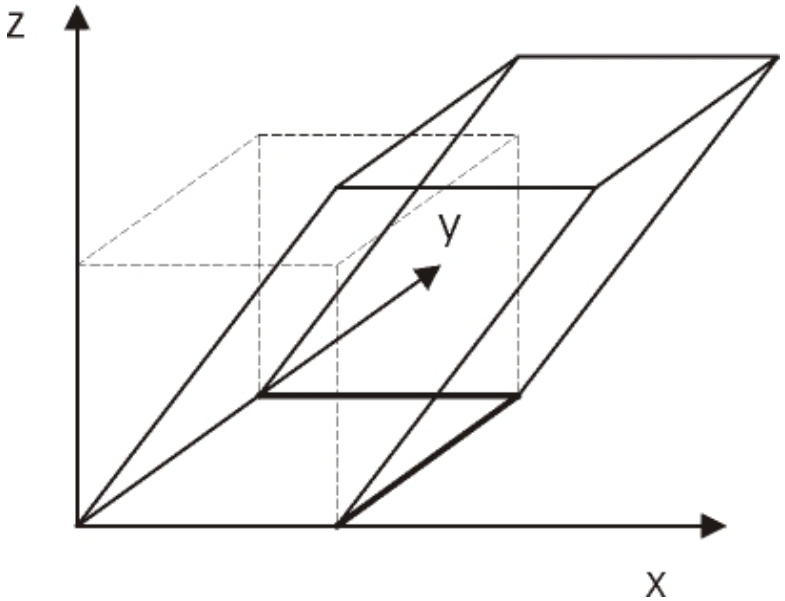
Souměrnost můžeme rozdělit do tří skupin:

1. Středová souměrnost - v matici 1.28 je definováno $S_x = S_y = S_z = -1$.
2. Souměrnost podle roviny
   a. *yz* pro $S_x = -1$,
   b. *xz* pro $S_y = -1$ a
   c. *xy* pro $S_z = -1$ (viz obrázek 1.9).
3. Souměrnost podle osy
   a. *x* pro $S_y = S_z = -1$,
   b. *y* pro $S_x = S_z = -1$ a
   c. *z* pro $S_x = S_y = -1$.

Stojí za povšimnutí, že například souměrnost podle osy *z* je možno získat složením souměrnosti podle roviny *yz* a *xz* a středovou souměrnost složením symetrií podle všech tří rovin.

Zkosení

Operaci zkosení (angl. *shear*) ve třech dimenzích (viz obrázek 1.10)



Obr. 1.10: Prostorové zkosení ve směru roviny *xy*

můžeme rozdělit na tři případy: ve směru roviny *yz*, kterou reprezentujeme maticí:

$$A_{Skyz} = \begin{bmatrix} 1 & SH_y & SH_z & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (1.29)$$

ve směru roviny *xz*

$$A_{Skxz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ SH_x & 1 & SH_z & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (1.30)$$

a ve směru roviny *xy*

$$A_{Skxy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ SH_x & SH_y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (1.31)$$

Například při zkosení ve směru roviny *xy* (viz obrázek 1.10), určuje zkosení souřadnice *z* a koeficienty SH_x a SH_y.

APPLET APPLET

applet 1.3 *applet 1.4*

- [1] Hovoříme potom o souhlasné či nesouhlasné orientaci obou souřadnicových systémů.

Literatura:

- [Bube94] Bubeník, F., Pultar, M.: *Matematické vzorce a metody. ČVUT, Praha, 1994.*
- [Budí83] Budínský, B.: *Analytická a diferenciální geometrie. SNTL, 1983.*
- [Fole90] Foley, J., van Dam, A., Feiner, S., Hughes, J.: *Computer Graphics. Principles and Practice. Second Edition. Addison-Wesley Publ. Comp., Reading Massachusets, 1990.*
- [Ježe93] Ježek, F.: *Geometrické a počítačové modelování. text přednášky, ZČU, 1993.*
- [Polá91] Poláček, J., Ježek, F., Kopincová, E.: *Počítačová grafika. ČVUT, Praha, 1991.*

2. Základní zobrazovací algoritmy

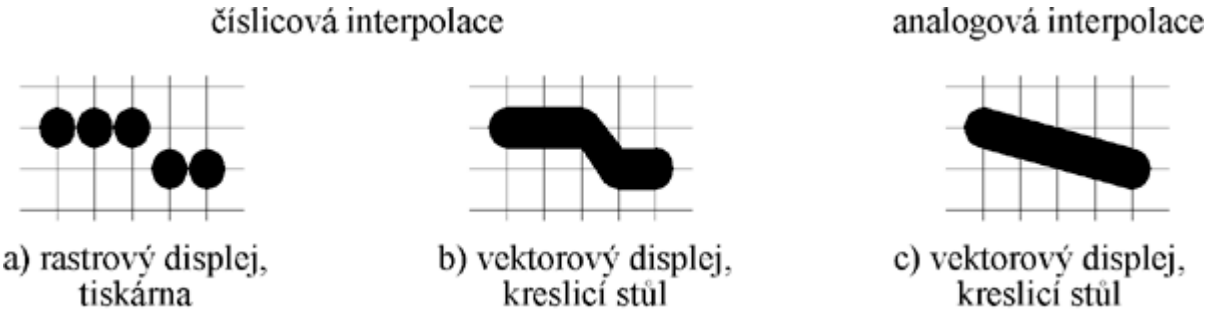
Výstupní grafická zařízení můžeme podle druhu zobrazování rozdělit na zařízení vektorového a rastrového typu. U zařízení vektorového typu jsou typickými zobrazovanými prvky úsečka, oblouk, kružnice a z nich složené vyšší útvary (např. znaky). Rastrová zařízení pracují s diskrétními body - pixely. *Pixel* (zkratka angl. *picture element*) označuje elementární obrazový prvek, nejmenší zobrazitelný bod na rastrovém zařízení (displeji, tiskárně). Pixel může mít jednak zobrazitelné atributy (barva, jas, blikání aj.), jednak atributy použité při řešení různých úloh v grafice (hloubka, pokrytí, maska aj.). Rastrové algoritmy určují vlastnosti jednotlivých obrazových bodů, jejich výstupem bývají souřadnice pixelu (poloha skupiny pixelů) a hodnoty jeho atributů, např. barvy a jasu.

Interpolátory: Dvojrozměrný obraz znázorněný na zobrazovacím zařízení je složený z bodů, úseček a částí křivek, vyplněných uzavřených oblastí a textu. Poloha a rozměry (parametry) grafických objektů jsou popsány ve světových souřadnicích, grafická zařízení však mohou zobrazovat pouze s určitou rozlišovací schopností - v určitém rastru. Při kreslení zadané hladké křivky je nutno řídit pohyb kreslicího členu nebo obarvit body na rastrové síti zobrazovací plochy tak, aby výsledné zobrazení bylo co nejvíce podobné zadané křivce. Vlastní kresbu lze realizovat dvěma různými způsoby:

Číslicová interpolace: Body ležící v daném rastru zadané křivky určíme výpočtem, křivku nahradíme (interpolujeme)schodovitou funkcí nebo množinou bodů.

Analogová interpolace: Analogový obvod, kterému jsou zadány okrajové podmínky (koncové body aj.), řídí spojitý pohyb kreslicího členu. U analogové interpolace spadají pouze krajní body do zvoleného rastru.

Příklady výsledné kresby u jednotlivých způsobů jsou uvedeny na následujícím obrázku:



obr. 2.1: Číslicová a analogová interpolace

Oba typy interpolátorů musí splňovat co nejlépe následující požadavky:

- kreslená čára musí být hladká (z hlediska vizuálního vjemu);
- kresba musí začínat a končit v zadaných (zaokrouhlených) bodech;
- intenzita (tloušťka) čáry nesmí (příliš) záviset na délce a sklonu;
- generace čáry musí být provedena co nejrychleji.

Analogové vektorové interpolátory vycházejí z integrální nebo parametrické (lineární nebo exponenciální) rovnice přímky a jsou řešeny specializovanými analogovými obvody. Používaly se u vektorových displejů a některých kreslicích stolů. Jejich řešením se zde nebudeme zabývat.

V případech ukázaných na obr.2.1.a)b) byl rozklad vektoru do rastru generován číslicovými interpolátory. I tyto interpolátory mohou být řešeny obvodově (posuvné registry a sčítačky) nebo implementovány jako programy, které jsou součástí firmware kreslicích zařízení.

Číslicové interpolátory založené na rastrových algoritmech se používají i u zařízení s čárovým výstupem (např. u kreslicích stolů). Mikroprogramová nebo obvodová řešení interpolátorů jsou základem všech moderních grafických řadičů. Technická řešení zde nebudeme uvádět.

2.1 Princip interpolace

Při interpolaci - stanovení polohy a hodnoty pixelů, které nejlépe nahradí zadanou ideální křivku - se používají dva postupy. První z nich vychází z předpokladu, že interpolovanou křivku můžeme rozložit na spojité úseky popsané jednoduchými funkčními vztahy. Nechť určitý úsek křivky začíná v bodě $[x_p, y_p]$ a končí v bodě $[x_k, y_k]$. Tento úsek pomocí soustavy rovnic $x_{n+1} = F(x_n, y_n)$, $y_{n+1} = G(x_n, y_n)$ - tj. v daném úseku se pohybujeme po křivce zvoleným směrem a z bodu $[x_n, y_n]$ - přejdeme do následujícího bodu $[x_{n+1}, y_{n+1}]$ na křivce jednoduchým výpočtem. Schéma tohoto typu interpolace je popsáno alg.2.1. Úsek začíná v bodě $[x_p, y_p]$ a končí v bodě $[x_k, y_k]$.

```
Inicializace: (xn , yn) = (xp , yp).
Opakuj
    Kresli bod (⌊xn⌋ , ⌊yn⌋).
     xn+1 = F(xn , yn)
     yn+1 = G(xn , yn)
     (xn , yn) = (xn+1 , yn+1)
    pokud  (xn , yn)  ≠ (xk , yk).
```

Algoritmus 2.1: Princip DDA interpolace

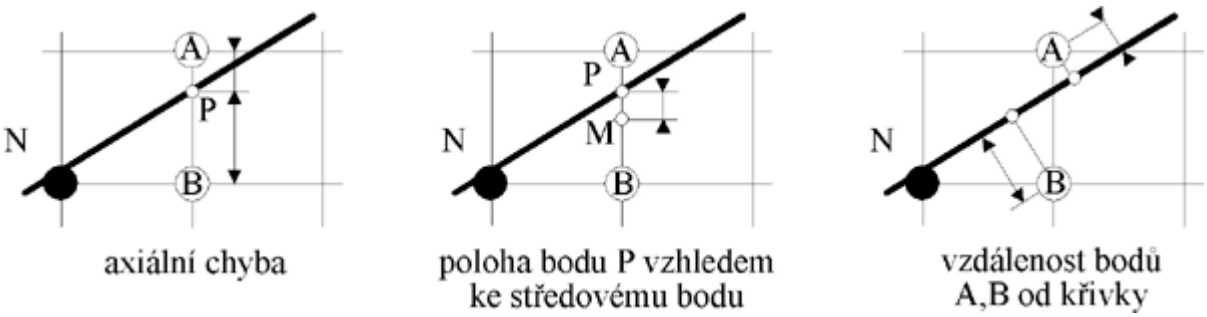
Někdy je výpočet formulován pomocí diferencí Δ_x a Δ_y (alg.2.2.).

```
Inicializace: (xn , yn) = (xp , yp).
Opakuj
    Kresli bod (⌊xn⌋ , ⌊yn⌋).
     xn+1 = xn + Δx(xn , yn)
     yn+1 = yn + Δy(xn , yn)
     (xn , yn) = (xn+1 , yn+1)
pokud  (xn , yn)  ≠ (xk , yk).
```

Algoritmus 2.2: DDA interpolace s diferencemi Δ_x a Δ_y

Tento způsob výpočtu je používán při řešení soustav diferenčních a integrálních rovnic. Zařízení zkonstruované pro uvedený účel jako blok číslicově analogového počítače bylo nazváno **d**igitální **d**iferenciální **a**nalyzátor. Uvedený historický název byl zachován při pojmenování algoritmu **DDA**.

Druhý postup opět vychází ze znalosti vlastností interpolované křivky. Při volbě jednotlivých bodů rastru se používá rozhodovací - chybový člen. Při řešení interpolace je křivka dělena na monotónní oblasti, ve kterých při daném směru interpolace je volba možných kroků omezena pouze na dvojici horizontální-diagonální krok, nebo vertikální-diagonální krok. Pro každou z těchto možností se stanoví chyba podle zvoleného chybového kritéria a volí se krok s menší chybou. Příklady používaných kritérii pro volbu kroku z bodu *N* do bodu *A* nebo *B* jsou uvedeny na obr.2.2:



obr. 2.2: Kritéria volby kroku

Interpolátor s chybovým nebo rozhodovacím členem je naznačen v alg.2.3. na následující stránce.

```
Inicializace: (xn , yn) = (xp , yp).
              D inicializuj podle druhu interpolované křivky.
Opakuj
    Kresli bod (xn , yn).
    Podle D zvol krok:
      Volba (Krok1x , Kroky):
           xn+1 = xn + Krokx;  yn+1 = yn + Kroky;
           D = D + Oprava1;
      Volba (Krokx , Kroky):
           xn+1 = xn + Krokx;  yn+1 = yn + Kroky;
           D = D + Oprava2;
    (xn , yn) = (xn+1 , yn+1)
pokud  (xn , yn)  ≠ (xk , yk).
```

Algoritmus 2.3: Interpolátor s rozhodovacím členem

V následujících odstavcích si uvedeme příklady rastrových interpolátorů, tj. odpovídajících algoritmů pro jednoduché čárové prvky.

2.2 Kresba úsečky

2.2.1 Interpolátor DDA

Jednoduchý interpolátor vychází z integrální rovnice přímky.

$$x(t) = \int k_x \, dt, \; y(t) = \int k_y \, dt \qquad (2.1)$$

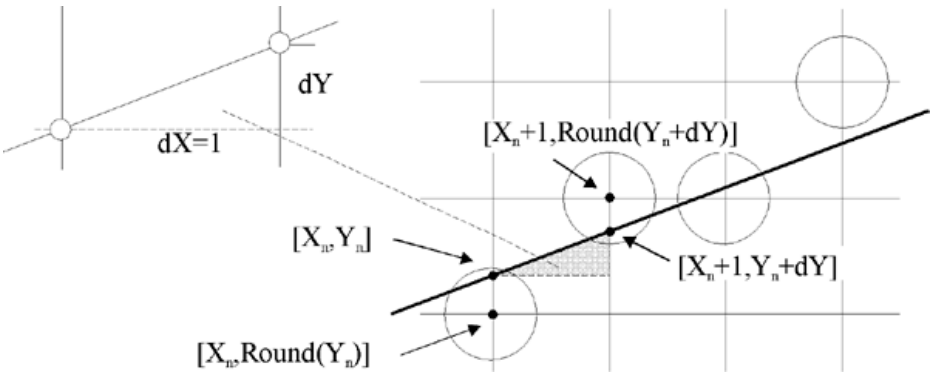
Integrace konstant k_x a k_y nahradíme postupným přičítáním konstantních přírůstků ve směrech obou os. Pro úsečku s koncovými body [$X1$, $Y1$] a [$X2$, $Y2$] můžeme použít hodnoty $k_x = X2\text{-}X1$, $k_y = Y2\text{-}Y1$. Pak z bodu [X_n, Y_n] ležícího na odpovídající přímce vypočteme další bod přímky [X_{n+1}, Y_{n+1}] pomocí rovnic

$$\begin{aligned} X_{n+1} &= X_n + k_x . \Delta t \\ Y_{n+1} &= Y_n + k_y . \Delta t; \;\; \Delta t \neq = 0 \end{aligned} \qquad (2.2)$$

Hodnotu Δt volíme tak, aby v absolutní hodnotě větší z obou přírůstků $dX = k_x.\Delta t$, $dY = k_y.\Delta t$ byl menší nebo roven vzdálenosti sousedních bodů rastrové sítě, rastrové jednotce, tj.

$$dX = \frac{X2\text{-}X1}{max(|X2\text{-}X1|,|Y2\text{-}Y1|)}, dY = \frac{Y2\text{-}Y1}{max(|X2\text{-}X1|,|Y2\text{-}Y1|)} \qquad (2.3)$$

Pro příklad uvedený na obr.2.3. zvolíme $dX = 1$, $dY = (Y2\text{-}Y1)/(X2\text{-}X1)$ a pomocí těchto přírůstků určíme následující bod [X_n+1,Y_n+dY] (obr.2.3).



obr. 2.3: DDA interpolátor - úsečka

Pokud je větší z přírůstků roven rastrové jednotce, pak ve směru odpovídající osy leží vypočtené body v rastru. Ve směru druhé osy zaokrouhlujeme zlomkovou část. Výsledkem postupného přičítání a následného zaokrouhlení jsou souřadnice jednotlivých bodů, které aproximují úsečku ve zvoleném rastru.

```
void DDA (int x1,int y1,int x2,int y2)
{ int PocetKroku,i;
  float x,y,dx,dy;
  PocetKroku=max(abs(x2-x1),abs(y2-y1));
  KresliBod((int)x1,(int)y1);           /* první bod */
  if (PocetKroku != 0)
    { dx=(x2-x1)/PocetKroku; dy=(y2-y1)/PocetKroku
      x=x1+0.5*sign(dx); y=y1+0.5*sign(dy);
      for (i=0; i<PocetKroku; i++)
        { KresliBod((int)x,(int)y);
          x+=dx; y+=dy;
        }
    }
}
```

Algoritmus 2.4: Kresba úsečky pomocí DDA

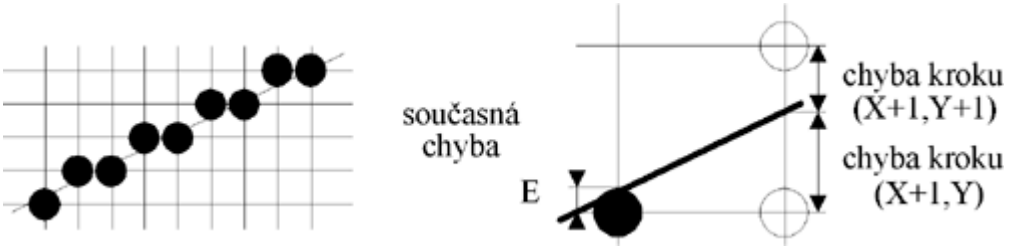
Algoritmus počítá body ležící na úsečce posunuté s ohledem na zaokrouhlování o vektor (0.5,0.5) a nastavuje pixely s nejbližší nižší celočíselnou hodnotou souřadnic. Nevýhodou algoritmu je dělení v inicializační fázi. Protože algoritmus kumuluje chyby přírůstků *dX* a *dY*, přesnost výpočtu a reprezentaci čísel musíme volit tak, aby celková chyba při kresbě nejdelší úsečky nepřesáhla 1/2 rastrové jednotky.

APPLET

applet 2.1

2.2.2 Bresenhamův algoritmus pro kresbu úsečky

J. E. Bresenham publikoval tento algoritmus v roce 1965 ([Bres65]). Byl původně určen pro řízení kreslicích stolů s číslicovými interpolátory, je však vhodný i pro jiné typy rastrových kreslicích zařízení. Algoritmus vybírá pixely ležící nejblíže skutečné úsečce na základě hodnoty axiálního chybového členu. V souřadném systému obrazové plochy vybere podle sklonu úsečky osu ve směru rychlejšího pohybu, v tomto směru se pohybuje s krokem 1 rastrové jednotky. Ve směru pomalejšího pohybu určuje axiální chyby pro alternativní kroky a podle nich volí bližší pixel k úsečce. Chybový člen zachycuje vzdálenost mezi ideální přímkou a středem uvažovaného pixelu, měřenou kolmo na směr rychlejšího pohybu (obr.2.4):



obr. 2.4: Axiální chyba kroku

Předpokládáme, že úsečka leží v 1.oktantu a rozhodujeme se mezi horizontálními a diagonálními kroky (X+1,Y) a (X+1,Y+1). Volíme pixel, který je blíže ideální přímce, tedy pixel s menší chybou kroku. Po každém kroku opravíme chybový člen následujícím způsobem:

$$E+\frac{dY}{dX}\begin{cases} \le \frac{1}{2}: & krok\ (1,0), \quad E = E+dY/dX \\ > \frac{1}{2}: & krok\ (1,1), \quad E = E+dY/dX-1 \end{cases} \tag{2.4}$$

Při vhodné transformaci chybového členu stačí testovat jen jeho znaménko a algoritmus lze upravit na tvar, který počítá pouze s celými čísly. Nerovnosti 2.4 upravíme po vynásobení 2.*dX* na tvar

$$D = 2.dX.E + 2.dY - dX \begin{cases} \le 0: & krok\ (1,0), \quad D = D+2.dY = D+D_{1,0} \\ > 0: & krok\ (1,1), \quad D = D+2.dY-2.dX = D+D_{1,1} \end{cases} \tag{2.5}$$

Takto transformovaný chybový člen se nazývá **rozhodovací člen** a označuje se písmenem *D* (angl. *decision*).V případě Bresenhamova algoritmu má počáteční hodnotu $D = 2.dY - dX$ a v každém kroku je opravován obdobně transformovanými hodnotami $D_{1,0} = 2.dY$ nebo $D_{1,1} = 2.dY - 2.dX$.

Algoritmus obsahuje v inicializační části sčítání a násobení 2, v cyklu pouze sčítání a test znaménka rozhodovacího členu. Pro svou jednoduchost je to velmi často používaný algoritmus jak v programových, tak i obvodových realizacích interpolátorů. Úplná realizace pro všechny směry obsahuje v úvodní části výběr dvojice kroků podle sklonu a orientace úsečky. Z vybrané dvojice se pak volí interpolační krok podle hodnoty chybového členu.

Porovnáme-li DDA a Bresenhamův algoritmus, vidíme, že Bresenhamův algoritmus obsahuje v cyklu test, který řídí volbu interpolačního kroku s ohledem na vzdálenost pixelu od skutečné úsečky. Tím je zaručena maximální chyba 1/2 rastrové jednotky. DDA je citlivý na přesnost a u delších úseček může dojít ke kumulaci zaokrouhlovací chyby inicializační fáze (nepřesně stanovené přírůstky *dX*,*dY*).

```
void Bresenham(int x1, int y1, int x2, int y2)

{ int dx,dy,d,d10,d11,i;
  dx=x2-x1;   dy=y2-y1;  d=2*dy-dx;  d10=2*dy; d11=2*dy-2*dx;
  for (i=0; i<=dx; i++)
    { KresliBod(x1,y1);
      if (d>0) /* volba kroku podle rozhodovacího členu */
         { x1++; y1++; d+=d11; }
      else
         { x1++; d+=d10; }
    }
}
```

Algoritmus 2.5: Bresenhamův algoritmus pro kresbu úsečky v 1. oktantu

APPLET

applet 2.2

2.3 Kresba kružnice a kruhového oblouku

Kresba kružnice pomocí úseček

S využitím transformačního vztahu pro otočení kolem zadaného středu otáčení o úhel α lze jednoduchým způsobem kreslit kružnici. Kružnici se středem v bodě [0,0] procházející bodem $[X_n, Y_n]$ popíšeme následující soustavou rovnic:

$$[\,X\,,\,Y\,] = [\,X_n\,,\,Y_n\,]\,.\begin{bmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{bmatrix}\,,\quad 0 \le \alpha < 2\pi \qquad (2.6)$$

Při aproximaci kružnice pomocí úseček vypočteme vrcholy pravidelného *n*-úhelníku, jehož vrcholy leží na obvodu kružnice. Výpočet *sin* α a *cos* α stačí provést pouze jednou a při otáčení použít pouze operace násobení a sčítání.

```
#define DvePi 6.2832
#define n 100          /* počet vrcholů N-úhelníku */

void Kruznice(float xs, float ys, float r)
/* střed [xs,ys], poloměr r */
{ float x1,y1,x2,y2,cosa,sina; int i;

  x1=r; y1=0; cosa=cos(DvePi/n); sina=sin(DvePi/n);
  for (i=1; i<=n; i++)
    { x2 = x1*cosa - y1*sina;
      y2 = x1*sina + y1*cosa;
      KresliUsecku(x1+xs,y1+ys,x2+xs,y2+ys);
      x1 = x2; y1 = y2;
    }
}
```

Algoritmus 2.6: Kresba kružnice otáčenou úsečkou v kladném směru

Pro malé úhly α je *cos*α << 1 a soustava 2.6 přejde na tvar odpovídající algoritmu DDA, tj.

$$[X_{n+1}\,,\,Y_{n+1}] = [X_n\,,\,Y_n\,]\,.\begin{bmatrix} 1 & \sin\alpha \\ -\sin\alpha & 1-\sin^2\alpha \end{bmatrix} \qquad (2.7)$$

Alg.2.7 odpovídající soustavě 2.7 zaručuje kresbu uzavřené křivky, tj. determinant matice transformace

$$\begin{vmatrix} 1 & \sin\alpha \\ -\sin\alpha & 1-\sin^2\alpha \end{vmatrix} = 1 - \sin^2\alpha + \sin^2\alpha = 1$$

Upravený algoritmus má tvar:

```
void Kruznice(float xs, float ys, float r)

{ float x1,y1,x2,y2,sa; int i;
  x1=r; y1=0; sa=sin(DvePi/n);
  for (i=1; i<=n; i++)
    { x2 = x1 - y1*sa; y2 = y1 + x2*sa;
      KresliUsecku(x1+xs,y1+ys,x2+xs,y2+ys);
      x1 = x2; y1 = y2;
    }
}
```

Algoritmus 2.7: Kresba kružnice - *cos(α)<<1*

Algoritmus patří do skupiny algoritmů DDA založených na diferenčních rovnicích křivky. Je citlivý na přesnost výpočtu a může se u něj projevit kumulovaná chyba. Přesnější algoritmus typu DDA byl publikován pod názvem HP-DDA (High precission DDA):

APPLET

applet 2.3

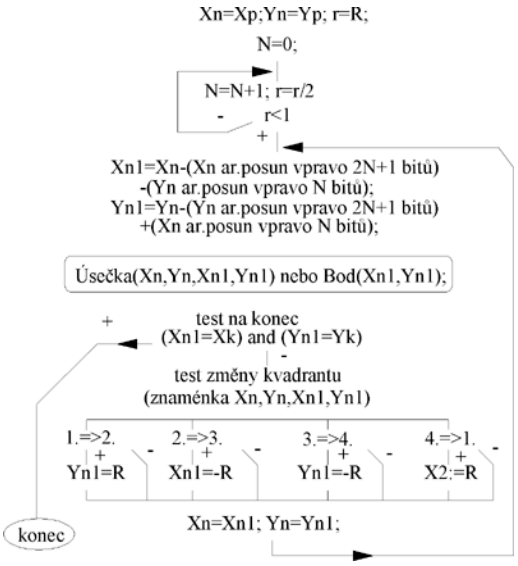
Algoritmus HP-DDA:

Základem výpočtů je opět soustava 2.6. Pro malý úhel α nahradíme $\cos\alpha << 1 - \sin^2\alpha/2$ (první 2 členy Taylorova rozvoje). Úhel α, který určuje přesnost interpolace, volíme tak, aby při daném poloměru R nepřesáhl krok 1 rastrovou jednotku. Pak můžeme odhadnout $\sin\alpha < {}^1/_R$. Nechť N je počet bitů, potřebných pro uložení hodnoty R, $2^{N-1} \le R < 2^N$. Označíme $sa = \sin\alpha$ a s ohledem na požadovanou přesnost volíme podle poloměru R hodnotu $sa = [1/(2^N)] < {}^1/_R \le [1/(2^{N-1})]$. Transformaci rotace převedeme na tvar

$$[X_{n+1}, Y_{n+1}] = [X_n, Y_n] \cdot \begin{bmatrix} 1-\frac{sa^2}{2} & sa \\ -sa & 1-\frac{sa^2}{2} \end{bmatrix} = [X_n, Y_n] \cdot \begin{bmatrix} 1-2^{-(2N+1)} & 2^{-N} \\ -2^{-N} & 1-2^{-(2N+1)} \end{bmatrix} \quad (2.8)$$

Z upravené soustavy rovnic vychází algoritmus HP-DDA, který obsahuje pouze celočíselné sčítání, odčítání a dělení 2.

Verze algoritmu uvedená na obr.2.8, kreslí kruhový oblouk (kružnici) proti směru hodinových ručiček. Při implementaci je nutno věnovat pozornost podmínce pro ukončení interpolace a řešit ji tak, aby při nepřesně zadaných souřadnicích koncového bodu nedošlo k zacyklení výpočtu. Algoritmus obsahuje test na změnu kvadrantu, který je použit pro doladění výpočtu (zmenšení vlivu kum. chyb).

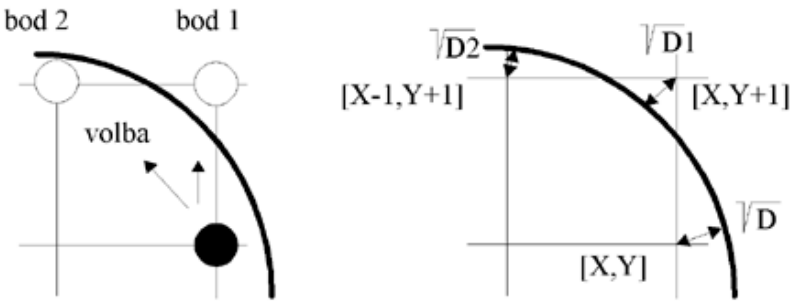


Algoritmus 2.8: Kružnice a kruhový oblouk

Bresenhamův algoritmus pro generování kružnice

Bresenhamův algoritmus pro kružnice stejně jako algoritmus pro generování úseček vyhodnocuje chybový člen a volí nejbližší bod v rastrové síti.

Při výběru bodu se používá jako rozhodovací člen čtvercová radiální chyba způsobem naznačeným na obr.2.5:



obr. 2.5: Čtvercová radiální chyba Bresenhamova algoritmu

Při rozhodování mezi body 1 a 2 stačí testovat znaménko součtu $D1+D2$. Zkrácený algoritmus (pouze 1.oktant) má tvar:

```
x=r; y=0; d=0;              /* čtvercová radiální chyba D */
while (x>=y)                /* 1. oktant */
  { KresliBod(x,y);         /* posun podle středu XS,YS */
    d1=d+2*y+1; d2=d-2*x+2*y+2;
    if ((d1+d2)>0)          /* volím bod 2 */
        { x--; y++; d=d2; }
    else                    /* volím bod 1 */
        { y++; d=d1; }
  }
```

Algoritmus 2.9: Bresenham - kresba 1. oktantu kružnice

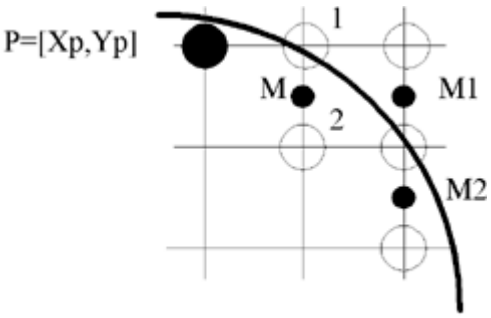
Úplný algoritmus je doplněn o test změny oktantů a nastavení volitelných kroků v závislosti na kresleném oktantu a směru kresby.

APPLET

applet 2.4

Kružnice s kritériem středového bodu

Pro volbu vhodného interpolačního kroku lze použít i jiná kritéria, než chybový člen Bresenhamova algoritmu. Příkladem je kritérium středového bodu. Na obr.2.6 je naznačena interpolace kružnice se středem v [0,0] ve 2. oktantu, v záporném směru otáčení.



obr. 2.6: Středové body M, M1, M2

Interpolátor se nachází v bodě $P = [Xp,Yp]$. Při rozhodování mezi body 1 a 2 použijeme hodnotu funkce $F(x,y) = x^2 + y^2 - R^2$ ve středovém bodě $M = [Xm,Ym]$. Zřejmě pokud je $F(Xm,Ym) > 0$, leží bod M vně kružnice a je vhodné volit bod 2, jinak volíme bod 1. Rozhodovací člen D má v bodě M hodnotu

$$D = F(\, Xp + 1\, ,\, Yp - \frac{1}{2}\,) = (Xp + 1)^2 + (Yp - \frac{1}{2})^2 - R^2 \qquad (2.9)$$

Při volbě bodu 1 (krok +1,0) má rozhodovací člen v bodě $M1$ hodnotu $D_{M1} = D + 2\,Xp + 3 = D + D_1$, při volbě bodu 2 (krok +1,-1) má v bodě $M2$ hodnotu $D_{M2} = D + 2\,Xp - 2\,Yp + 5 = D + D_2$. D_1 a D_2 představují možné přírůstky rozhodovacího členu D, jsou to diference 1.řádu. Při výpočtu nových hodnot D_1 a D_2 můžeme opět využít jednoduchých vztahů - diferencí 2.řádu . Jednoduše lze odvodit, že pokud zvolíme krok z bodu P do bodu 1, pak nové hodnoty rozhodovacího členu a diferencí jsou

$$D = D + D_1\, ,\, D_1 = D_1 + 2\, ,\, D_2 = D_2 + 2\, . \qquad (2.10)$$

Při kroku do bodu 2 jsou tyto hodnoty

$$D = D + D_2\, ,\, D_1 = D_1 + 2\, ,\, D_2 = D_2 + 4\, . \qquad (2.11)$$

S použitím těchto vztahů dostaneme velmi jednoduchý algoritmus pro kresbu kružnice. Úplné řešení, které interpoluje v záporném směru ve 2. oktantu a při kresbě ostatních bodů využívá kruhové symetrie, je obsaženo v následujícím algoritmu:

```
void Kruznice (float xs, float ys, float r)
{ int d,d1,d2,x,y;
  x=0; y=r; d=1-r;            /* počáteční hodnoty */
  d1=3; d2=-2*r+5;
  plot8(x,y);                 /* 8 bodů s využitím středové symetrie */
  while (y>x)
    { if (d<0)                /* krok x+1,y */
      { d=d+d1; d1=d1+2; d2=d2+2; x++; }
     else                     /* krok X+1, Y-1 */
      { d=d+d2; d1=d1+2; d2=d2+4; x++; y--; }
     plot8(x,y)
   }
}
```

Algoritmus 2.10: Kružnice s diferencí 2. řádu

2.4 Kresba elipsy

Algoritmy pro kresby elipsy byly odvozeny obdobným způsobem jako algoritmy pro interpolaci kružnice. Publikované algoritmy jsou založené na různých kritériích volby nejvýhodnějšího kroku, včetně dvoukrokové varianty. Jako příklad klasického řešení uvedeme algoritmus pro kresbu elipsy s kritériem středového bodu (angl. *midpoint ellipse*). Podobné algoritmy včetně odvození nalezne čtenář např. v [Earn85], [Fole90] a [WuRo89].

Elipsu určenou středem v bodě x_c,y_c , jejíž hlavní poloosy a a b jsou rovnoběžné se souřadnicovými osami, popíšeme rovnicí

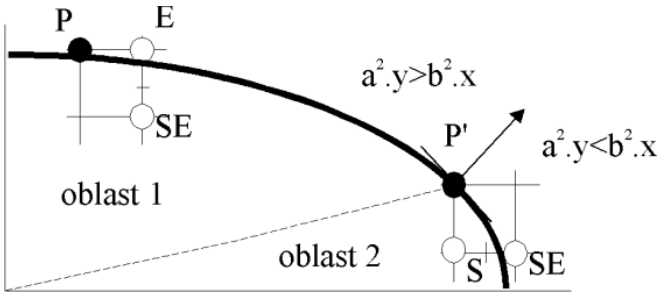
$$\frac{(x-x_c)^2}{a^2} + \frac{(y-y_c)^2}{b^2} = 1 \qquad (2.12)$$

Při výpočtu v celočíselné aritmetice je jednodušší nejprve určovat body ležící na elipse, jejíž střed je v počátku soustavy souřadnic, a takto vypočítané body posunout o $[x_c,y_c]$. Implicitní rovnice elipsy

se středem v počátku je

$$F(x,y) = b^2x^2 + a^2y^2 - a^2b^2 = 0 \qquad (2.13)$$

Výpočet pixelů ležících na elipse budeme provádět podobným způsobem jako v případě kružnice. Vyjdeme přitom z obr. 2.7:



Obr. 2.7: Interpolace elipsy v 1.kvadrantu

Vzhledem k podobnosti postupu při odvození vztahů pro predikci si všimneme jen základních rozdílů:

- pomocí symetrie lze pro jeden vypočítaný bod určit pouze 4 body ležící na elipse
- algoritmus obsahuje dvě odlišné části - výpočet části elipsy s interpolačními kroky (1,0), (1,-1) (oblast 1) a výpočet části s interpolačními kroky (0,-1), (1,-1) (oblast 2)

V prvním kvadrantu se pro zvolený směr kresby používají následující pravidla pro úpravu rozhodovacího členu (v souladu s obr. 2.7 je použito značení kroků ve směrech *E*-east, *SE*-southeast, *S*-south):

$$\begin{aligned} Oblast1:\quad D+D_E &= D+b^2(2X+3) \\ D+D_{SE} &= D+b^2(2X+3)+a^2(-2Y+2) \\ Oblast2:\quad D+D_S &= D+a^2(-2Y+3) \\ D+D_{SE} &= D+b^2(2X+2)+a^2(-2Y+3) \end{aligned}$$

Pro počáteční hodnotu predikce platí $D=b^2-ba^2+a^2/4$, ve kterém je prováděno celočíselné dělení čtyřmi. Při něm může dojít k malé a zanedbatelné chybě, pokud je *a* liché. Bod, ve kterém se mění řídicí osa, je ten, ve kterém má tečna k elipse směrnici -1. Souřadnice tohoto bodu lze odvodit pomocí derivace funkce $F(x,y)$. Obrázek ukazuje geometrický význam derivací funkce *F*. Pro derivace platí:

$$\frac{\delta F}{\delta x} = 2b^2x, \qquad \frac{\delta F}{\delta y} = 2a^2y \qquad (2.14)$$

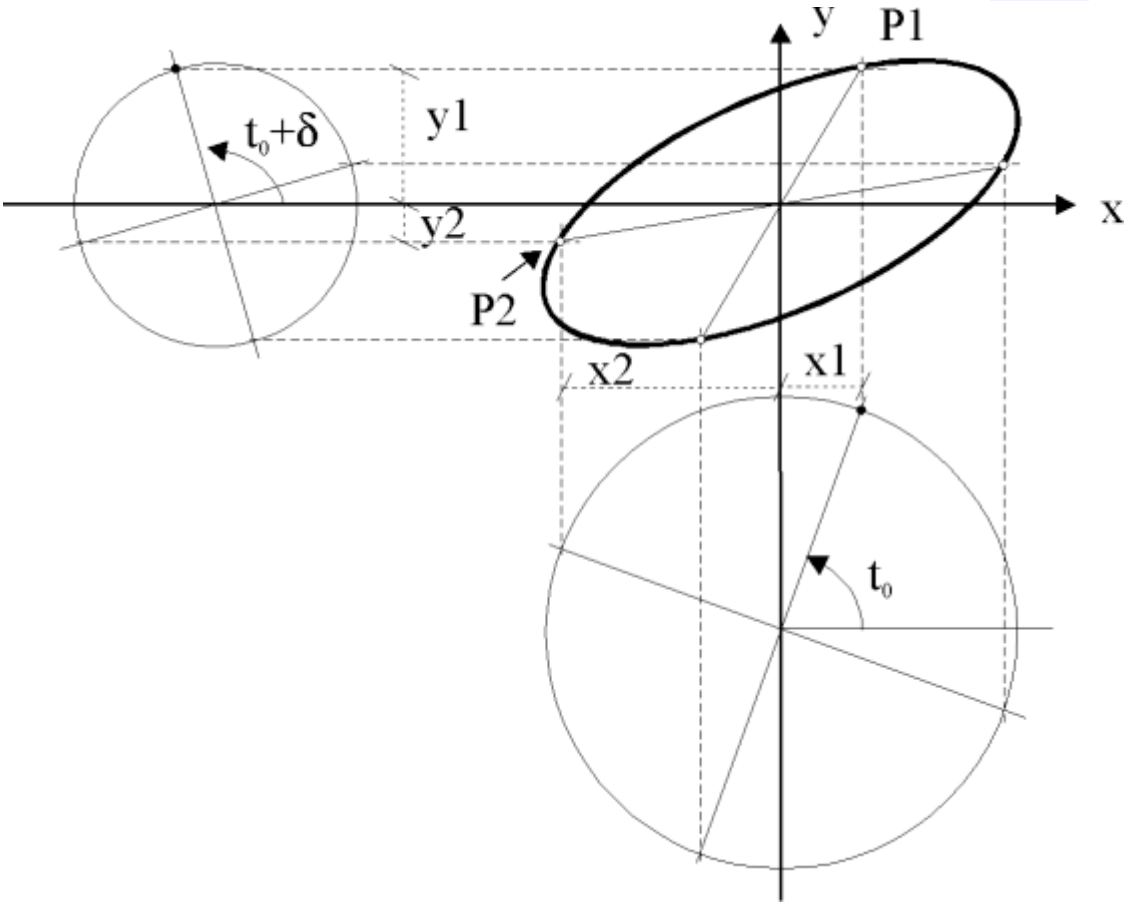
Při rovnosti derivací má tečna v daném bodě sklon 45° a algoritmus přechází do druhé části.

```
void Elipsa (int xs, int ys, int a, int b)
{ int x,y; float aa,bb,d;
  x=0; y=b; aa=a*a; bb=b*b; d= bb-aa*b+aa/4;
  Body(x,y);                          /* 4 středově symetrické body */
  while (aa*(y-0.5)>bb*(x+1))      /* oblast 1 */
   { if (d<0) { d=d+bb*(2*x+3); x++; }
       else   { d=d+bb*(2*x+3)+aa*(-2*y+2); x++; y--; }
     Body(x,y);
   }
  d=bb*(x+0.5)*(x+0.5)+aa*(y-1)*(y-1)-aa*bb;
  while (y>0)                         /* oblast 2 */
   { if (d<0) { d=d+bb*(2*x+2)+aa*(-2*y+3); x++; y--; }
       else   { d=d+aa*(-2*y+3); y--; }
     Body(x,y);
 }
}
```

Algoritmus 2.11: Kresba elipsy podle chyby středového bodu

Elipsa a eliptický oblouk v obecné poloze

Následující algoritmus umožňuje vykreslení elipsy v obecné poloze ([Fell93]). Elipsa vznikne složením dvou rotačních pohybů se stejnou úhlovou rychlostí. Souřadnice x je určena bodem rotujícím na kružnici s poloměrem A, souřadnice y obdobně bodem na kružnici s poloměrem B (obr. 2.8).



Obr. 2.8: Elipsa pomocí dvou kružnic

APPLET

applet 2.5

Elipsa se středem v počátku má parametrické vyjádření

$$f(t) = (x(t), y(t)) = (A.sin(t), B.sin(t+\delta)) \qquad t,\delta \in [0,2\pi).$$

Bod $f'(t) = (x'(t), y'(t)) = (A.cos(t), B.cos(t+\delta)) = (x(t+\pi/2), y(t+\pi/2))$ také náleží elipse. V grafických systémech a standardech (např.PHIGS, CGI) je obecná elipsa zadána pomocí dvojice koncových bodů sdružených průměrů, které spolu se středem tuto elipsu plně určují.

Nechť úsečka mezi body $P_1(t) = f(t_1)$ a $-P_1$ je libovolně zvoleným průměrem elipsy, pak *sdružený průměr* je tvořen úsečkou $P_2(t) = f'(t_1)$ a $-P_2$. A, B, které odpovídají průměrům generujících kružnic, získáme ze sdružených průměrů s koncovými body $[x_1, y_1]$, $[-x_1, -y_1]$, $[x_2, y_2]$, $[-x_2, -y_2]$ pomocí rovnic

$$\left.\begin{aligned} A.sint_0 &= x_1 \\ A.sin(t_0+\pi/2) &= x_2 \end{aligned}\right\} \Rightarrow A = \sqrt{(x_1^2+x_2^2)}$$

$$\left.\begin{aligned} B.sin(t_0+\delta) &= y_1 \\ B.sin(t_0+\delta+\pi/2) &= y_2 \end{aligned}\right\} \Rightarrow B = \sqrt{(y_1^2+y_2^2)} \qquad (2.15)$$

Sdružené průměrové páry $[x_1, y_1]$, $[-x_1, -y_1]$, $[x_2, y_2]$, $[-x_2, -y_2]$ využijeme k inicializaci algoritmu.

```
int mask,k;
void compute (int *rx, int *dx, int y);
{ *rx= *rx+(y << 1);
  *dx= *rx >> k;
  *rx= *rx & mask;
}
void ellipse(int x1, int y1, int x2, int y2);     /* průměrový pár */
{ int dx1,dx2,rx1,rx2,dy1,dy2,ry1,ry2,n;

   if ((x1*y2-y1*x2)<0) { x2=-x2; y2=-y2;}        /* jednotný směr */
   n=max(x1*x1+x2*x2,y1*y1+y2*y2);
   k=1; do {k++; n= n >> 1} while (n!=0);         /* k=domain(n) */
   k=(k>>1)+1; mask=(1<<k)-1;
   Bod(x1,y1); Bod(-x1,-y1); Bod(x2,y2); Bod(-x2,-y2);
   dx1=dx2=dy1=dy2=0;
   rx1=rx2=ry1=ry2= 1<<(k-1);
   for (n=pi*(1<<(k-1)); n>0; n--)
     { rx1=rx1-dx2;  compute(&rx1,&dx1,x2);  x1=x1+dx1; /* 1. kružnice */
       rx2=rx2+dx1;  compute(&rx2,&dx2,x1);  x2=x2-dx2;
       ry1=ry1-dy2;  compute(&ry1,&dy1,y2);  y1=y1+dy1; /* 2. kružnice */
       ry2=ry2+dy1;  compute(&ry2,&dy2,y1);  y2=y2-dy2;
       if (dx1!=0 || dy1!=0) { Bod(x1,y1); Bod(-x1,-y1); }
       if (dx2!=0 || dy2!=0) { Bod(x2,y2); Bod(-x2,-y2); }
     }
}
```

Algoritmus 2.12: Kresba elipsy v obecné poloze, střed [0,0]

Generování kružnice zvolenou úhlovou rychlostí

Při generování kružnice použijeme parametrické vyjádření

$$x(t) = r.\cos(t), \quad y(t) = r.\sin(t)$$

Odpovídající počáteční podmínky jsou

$$\dot{x} = -y, \; x(0) = r, \; \dot{y} = x, \; y(0) = 0$$

Vyjdeme z bodu $x_0 = r$ a $y_0 = 0$ a postupně řešíme rovnice pro jednotlivé kroky Δt. Po dosazení bodu $x_i = x_{i-1}+\Delta x_{i-1}$ a $y_i = y_{i-1}+\Delta y_{i-1}$ vypočteme další bod následovně:

$$x_{i+1} = x_i + \dot{x}_i\Delta t + \delta\dot{x}_{i-1}\Delta t/2 \qquad y_{i+1} = y_i + \dot{y}_i\Delta t + \delta\dot{y}_{i-1}\Delta t/2$$

Protože

$$x_i = -y_i, \qquad \Delta\dot{x}_{i-1} = \dot{x}_i - \dot{x}_{i-1} = -y_i + y_{i-1} = -\Delta y_{i-1}$$

je po úpravě

$$x_{i+1} = x_i - \underbrace{(2y_i+\Delta y_{i-1})\Delta t/2}_{\Delta x_i} \qquad y_{i+1} = y_i + \underbrace{(2x_i+\Delta x_{i-1})\Delta t/2}_{\Delta y_i} \tag{2.16}$$

$\Delta x(\Delta y)$ rozdělíme na celočíselnou část $dx(dy)$ a zbytkovou $rx(ry)$. Za předpokladu, že $\Delta t = 2^{-m}$ pro nějaké m, lze dělení nahradit aritmetickým posuvem vpravo o n bitů. Protože je ve výpočtu použit faktor $\Delta t/2$, použijeme na zlomkovou část m+1 bitů a postupujeme takto:

cosinus	*sinus*
`rx=rx+dy;`	`ry=ry-dx;`
`rx=rx+2*y;`	`ry=ry+2*x;`
`dx=rx*2^-(m+1)`	`dy=ry*2^-(m+1);`
`rx=rx and (2^(m+1)-1);`	`ry=ry and (2^(m+1)-1);`
`x=x-dx`	`y=y+dy`

s počátečními hodnotami $rx = 2^m$; $ry = 2^m$; $dx = 0$; $dy = 0$; $x = r$; $y = 0$. Pro interpolaci musíme zvolit dostatečně malý přírůstek. Protože $|x| \le r$, je $\Delta t < 1/r$ a m volíme

$$m = \lfloor \log_2 r \rfloor + 1 \tag{2.17}$$

Celkový počet iterací pro kružnici je $N = \lfloor 2^{m+1}\pi + 0.5 \rfloor$, kde $m = \lfloor log_2(r) \rfloor + 1$. Algoritmus 2.12 počítá paralelně dvě kružnice s poloměry A a B. Odpovídající přesnost (a počet iterací) je nutno zvolit podle větší z obou kružnic. S využitím vztahu 2.15 volíme počet iterací k

$$k = \lfloor log_2(\sqrt{max(x_1^2+x_2^2, y_1^2+y_2^2)} \rfloor + 2.$$

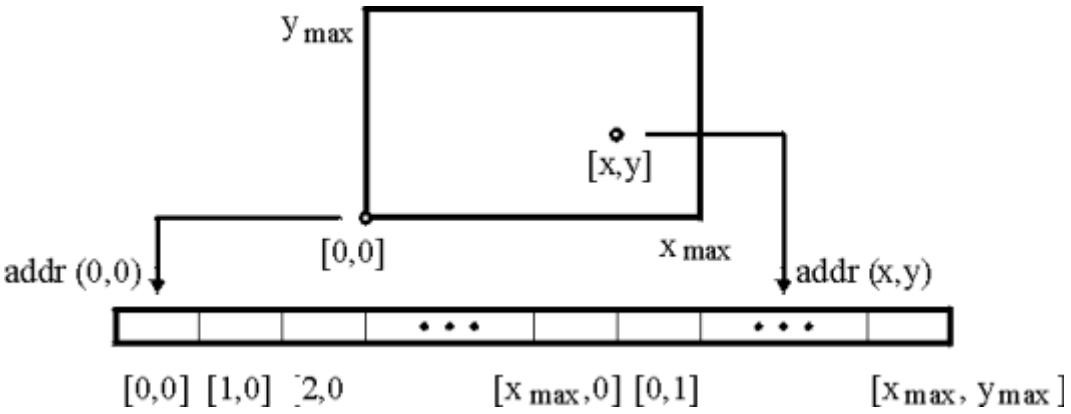
Přesný výpočet nahradíme s použitím funkce $domain(x)$, která určí počet bitů potřebných pro uložení testované hodnoty x ($2^{domain} > x \geq 2^{domain-1} \geq 1$). k získáme jako

$$k = ((\mathbf{domain}(\mathbf{max}(x_1*x_1+x_2*x_2, y_1*y_1+y_2*y_2))+1)\mathbf{div}\ 2)+1.$$

Algoritmus lze modifikovat na vykreslování eliptických oblouků a také pro techniku antialias. Intenzitu pixelů lze určit ze zbytkových hodnot rx a ry a tečného vektoru v každém bodě, které tento algoritmus poskytuje.

2.5 Zápis do obrazové paměti

Ačkoliv se zdá, že Bresenhamův algoritmus je velmi efektivní, existují další metody zrychlení kresby úsečky i dalších grafických primitiv. Kdykoliv mají být další body a úsečky zobrazeny na rastrovém displeji, musí dojít k zápisu do obrazové paměti. Zápis provádí procedura **KresliBod**, která převádí hodnoty souřadnic na adresu v obrazové paměti, do níž zapíše jas (barvu) příslušného pixelu.



Obr. 2.9: Adresace pixelů v obrazové paměti

Pro zjednodušení předpokládejme, že obrazová paměť je uspořádána jako dvourozměrné pole adresované po řádcích, které obsahují pixely obrazovky s levým dolním rohem o souřadnicích [0,0] a pravým horním rohem $[x_{max}, y_{max}]$ jako na obr. 2.9.

Adresa začátku paměti je uložena v proměnné **addr_rastr**. Pixel o souřadnicích $[x, y]$ leží v obrazové paměti na adrese

$$\text{addr}(x,y) = \text{addr_rastr} + y.(x_{max}+1) + x \qquad (2.18)$$

Tento výpočet je obsažen v proceduře **KresliBod**. V algoritmech pro kresbu úsečky je možno výpočet 2.18 zjednodušit, protože sousední body tvořící přímku jsou od sebe vzdáleny maximálně o jedničku ve směru x a y. Adresu obrazové paměti můžeme snadno určit z dříve vypočítané adresy. Například pro úsečku s kladnou směrnicí menší než 1 může být z pozice $[x, y]$ jednoho pixelu vypočítána adresa dalšího pixelu pomocí jednoho ze dvou následujících vzorců:

$$\text{addr}(x+1,y) = \text{addr}(x,y) + 1 \qquad (2.19)$$

$$\text{addr}(x+1,y+1) = \text{addr}(x,y) + x_{max} + 2 \qquad (2.20)$$

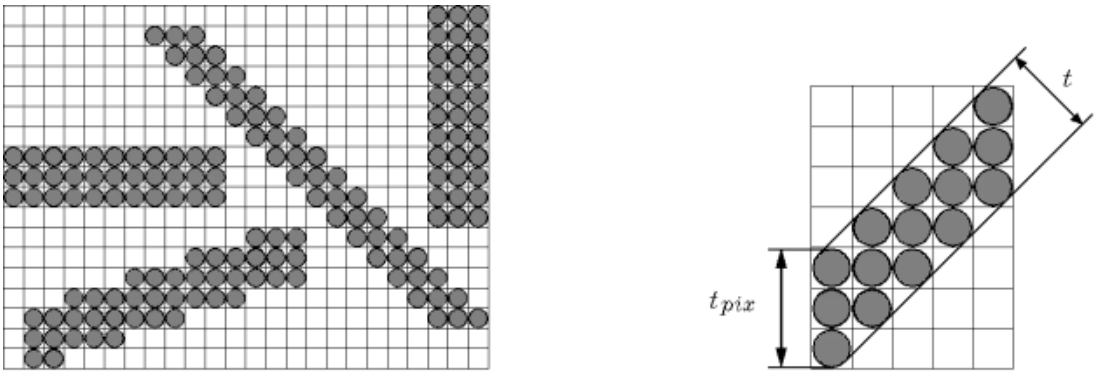
Podobně lze odvodit vzorce i pro ostatní případy změn souřadnic. Tento způsob vypočítávání adres je efektivní, neboť vyžaduje pouze celočíselné sčítání, pamatujeme-li si adresu předchozího pixelu.

Dodejme jen, že zcela nejrychleji se do obrazové paměti vykreslují **vodorovné úsečky**. Jejich pixely leží v jediném nepřerušeném bloku paměti a lze je často vykreslit jediným příkazem pro nastavení hodnot bitů v určitém úseku paměti. Tento rychlý způsob se používá v algoritmech pro vyplňování ploch, které budou uvedeny později.

2.6 Tloušťka čáry

Implementace atributu tloušťky čáry závisí na typu výstupního zařízení. Silné čáry na monitoru mohou být zobrazeny vykreslením několika sousedních rovnoběžných čar, zatímco na kreslicím zařízení lze téhož výsledku docílit výměnou pera.

Dříve uvedený Bresenhamův algoritmus (alg. 2.5) je možno modifikovat tak, aby kreslil čáru o větší než jednotkové tloušťce. Zřejmě lze upravit podprogram KresliBod tak, aby namísto jednoho bodu zapsal do obrazové paměti t bodů, je-li t tloušťka čáry v pixelech. Pokud je řídicí osa kreslené úsečky vodorovná, budou v podprogramu KresliBod kresleny pixely nad sebe, v opačném případě vedle sebe. Výsledek použití této metody je ukázán na obr. 2.10.



Obr. 2.10: Tlusté čáry Bresenhamovým algoritmem

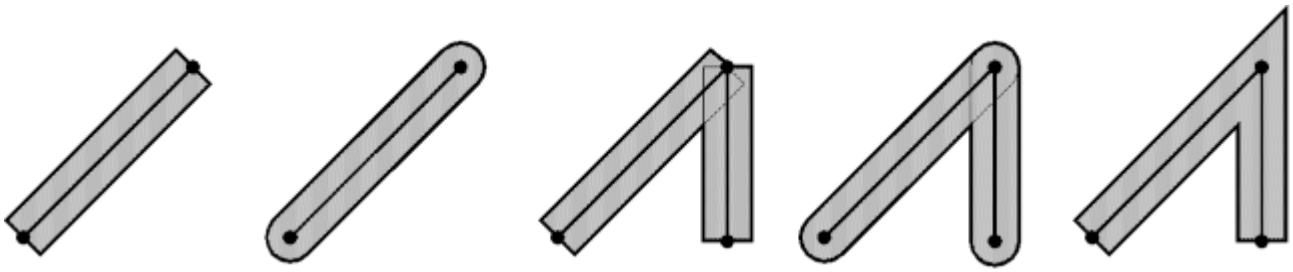
Všimneme si dvou negativních jevů:

1. *Tloušťka čáry není invariantní vzhledem ke sklonu úsečky.*
2. *Zakončení čar je nepřirozené. Je ostré a vždy rovnoběžné s některou ze souřadnicových os.*

První nedostatek zmírníme přepočítáním dané tloušťky čáry t na počet pixelů t_{pix} v závislosti na sklonu úsečky. Pro případ prvního kvadrantu a směrnice $m \leq 1$ platí:

$$t_{pix} = \frac{t.\sqrt{(dx^2+dy^2)}}{|dx|} \qquad (2.21)$$

Vodorovná úsečka bude tedy kreslena pomocí t pixelů nad sebou, úsečka s nenulovou směrnicí bude kreslena pomocí většího počtu pixelů nad sebou. Druhý nedostatek metody již nelze tak snadno napravit. U malé tloušťky čar nemusí působit rušivě, u velké je však výrazný. Proto se při větší tloušťce nahlíží na čáru jako na vyplněnou plochu a pro její kresbu se používají algoritmy pro kreslení vyplněných oblastí, které budou uvedeny dále.



a) b) c) d) e)

Obr. 2.11: Zakončení tlustých a lomených čar

Na obr. 2.11 jsou příklady možného správného zakončení čar. Nejjednodušší tvar tlusté čáry je obdélník, jeho hranice se snadno vypočítá i vyplní. Zadávané koncové body leží ve středu kratších hran obdélníku. Výpočetně náročnější, ale často vzhledově lepší, je zakončení do oblouku (obr. 2.11b). To také automaticky řeší častý problém vznikající při kresbě posloupnosti čar (obr. 2.11d). I tlusté lomené čáry

by měly navzájem dobře navazovat, čehož nelze pomocí obdélníků (obr. 2.11c) docílit. Některé systémy kreslí tlustou lomenou čáru pomocí mnohoúhelníkové hranice (obr. 2.11e).

APPLET

applet 2.6

APPLET

applet 2.7

Literatura:

- *[Bres65] Bresenham, J. E.: Algorithm for Computer Control of a Digital Plotter. IBM Systems Journal, 4 (1), 1965, pp. 25 - 30.*
- *[Earn85] Earnshaw, R.A: Fundamental Algorithms for Computer Graphics. NATO ASI series F, Springer-Verlag Berlin, 1985*
- *[Fole90] Foley, J. van Dam, A. Feiner S, Computer Graphics. Principles and Practise. Addison-Wesley, reading Messechusets, 1990*
- *[WuRo89] Wu, X.,Rokne, J.G.: Double-step Generation Of Ellipses. Computer Graphics & Application, May 1989, pp. 56-69*
- *[Fell93] Fellner, D.W.,Hellmberg, C.: Robust Rendering of General Ellipses and Elliptical Arcs. ACM Trans on Graphics, 1993, pp. 251-276*

3. Vyplňování oblastí

Naprostá většina grafických systémů obsahuje prostředky pro kresbu oblastí vyplněných barvou či vzorem. Kreslicí algoritmy zde mají za cíl nalézt všechny body ležící uvnitř dané oblasti a změnit jejich barvu. Základní informací je tedy tvar hranice omezující oblast. Hranice může být:

geometricky určena,

nejčastěji se jedná o mnohoúhelník, definovaný posloupností svých vrcholů. Tento způsob je běžný jak ve vektorové, tak v rastrové grafice. Základní metodou je řádkové vyplňování (angl. *scan conversion*), obr.3.1.a.

nakreslena v obrazové paměti,

tvar hranice může být libovolný, je třeba znát její barvu. Algoritmy jsou založeny na semínkovém vyplňování, (angl. *seed fill*), obr.3.1.b, které vychází z uživatelem vybraného vnitřního bodu oblasti.

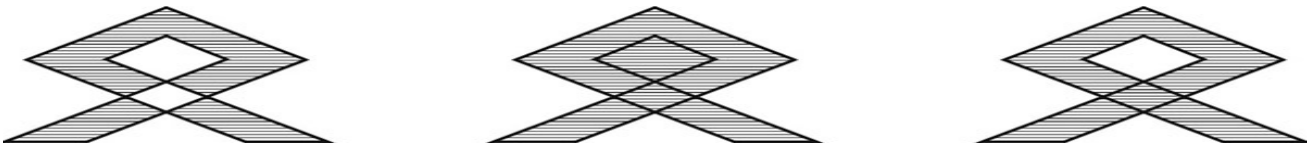
a) řádkové vyplňování b) semínkové vyplňování

obr. 3.1: Metody vyplňování podle reprezentace hranice

3.1 Řádkové vyplňování

Touto v grafice často používanou metodou je možno vyplňovat nejen mnohoúhelníky, ale i oblasti omezené obecnými (křivkovými) hranicemi. V našem výkladu ji popíšeme na příkladu oblasti omezené lomenou čarou, přičemž nebude důležité, zda zadaná oblast je konvexní či nekonvexní.

V případě, že hranice se sama protíná, je důležité rozhodnout, jak má být oblast vyplněna. Na obr.3.2. jsou tři různé možnosti. **Paritní vyplňování** je nejběžnější. Odpovídá intuitivnímu pohledu na hranici jako na entitu oddělující vyplněný a nevyplněný prostor. Ve druhém případě (obr.3.2.b) jsou vyplněny všechny body, které nejsou vně hranice. Poslední variantou je tzv. nenulové obtočení bodu hranicí, která je určena uzavřeným polygonálním tahem s možností sebeprotínání. Paritní vyplňování je přirozené a nejjednodušší, zbývající dvě varianty vyžadují složitější výpočty.



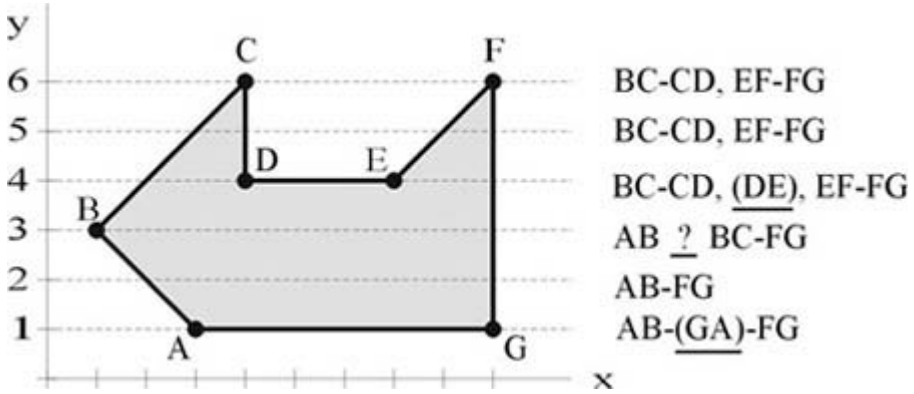
a) paritní vyplňování b) vnitřní vyplňování c) pravidlo nenulového obtočení

obr. 3.2: Varianty vyplňování téže hranice

Metodu paritního řádkového vyplňování můžeme popsat jednoduchým algoritmem alg.3.1.

```
Pro každý řádek y (v rozsahu mezních souřadnic oblasti):

   1. Vytvoř seznam průsečíků x_i řádku y se všemi hraničními úsečkami u_i

   2. Uspořádej seznam průsečíků podle souřadnic x_i

   3. Vykresli vodorovné úseky mezi lichými a sudými průsečíky v seznamu

   4. Zruš seznam průsečíků
```

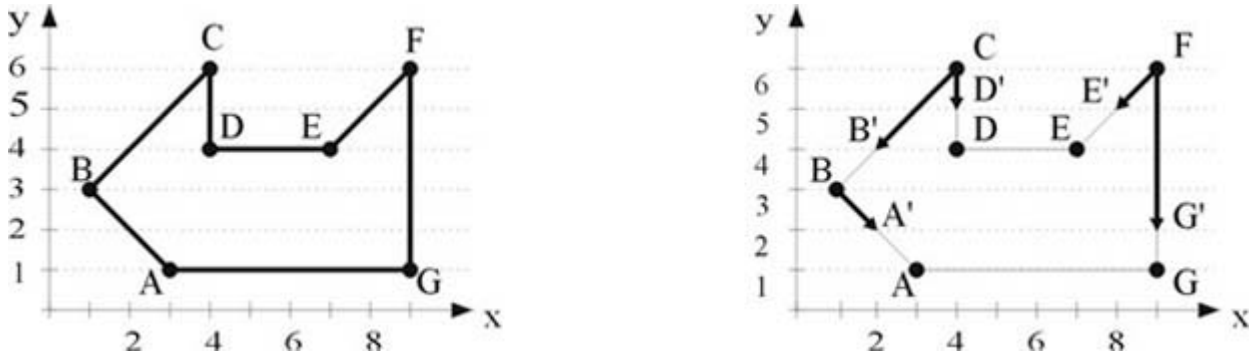
Algoritmus 3.1: Základní algoritmus řádkového vyplňování



obr. 3.3: Mnohoúhelník vyplňovaný řádkovým rozkladem

Algoritmus předpokládá, že počet průsečíků je sudý. To nemusí být splněno, pokud řádek prochází vrcholem mnohoúhelníku. Tehdy protíná obě úsečky které s vrcholem incidují a do seznamu průsečíků jsou přidány dva průsečíky namísto jednoho. Pokud tato situace nastane ve vrcholu, který je z hlediska souřadnic *y* lokálním minimem či lokálním maximem (body *C* a *F* na obr.3.3), nemá vyšší počet průsečíků vliv na funkčnost algoritmu.

Komplikace může nastat při zpracování vrcholu *B*, který není lokálním extrémem hranice. Standardním řešením takové situace je například zkrácení hrany *BC* zdola o 1 pixel ve směru *y*. Toto zdánlivé rozpojení dosud uzavřené hranice nemá vliv na činnost (diskrétního) algoritmu. Ukazuje se naopak, že systematické zkrácení všech hran vede ke snížení počtu průsečíků (na řádku 4 na obr.3.4), budou pouze dva průsečíky místo čtyř.



obr. 3.4: Mnohoúhelník a zkrácení jeho hran před vyplňováním

Při řádkovém vyplňování jsou tedy nejprve předzpracovány všechny hraniční úsečky. Ve většině případů je upravena jejich orientace (shora dolů) a jsou zkráceny u dolního vrcholu. Vodorovné úsečky mohou být z dalšího algoritmu vynechány (obr.3.4). Celkový postup je uveden v algoritmu 3.2.

```
1. Pro všechny hraniční úsečky ověř
   (a) je-li vodorovná, vynechej ji
   (b) uprav orientaci shora dolů a zkrať ji o 1 ve směru y
   (c) aktualizuj mezní souřadnice celé hranice ymax a ymin

2. Pro y od ymax do ymin s krokem -1
   (a) nalezni průsečíky hran s řádkem y
   (b) uspořádej všechny průsečíky podle souřadnice x
   (c) vykresli úseky mezi lichými a sudými průsečíky

3. Vykresli hranici oblasti (je-li třeba)
```

Algoritmus 3.2: Řádkové vyplňování s předzpracováním hran

K tomu, zda má být na závěr algoritmu znovu vykreslena hranice, se vrátíme v další části této kapitoly, kdy budeme uvažovat o tom, zda hranice je či není součástí vyplňované oblasti.

Využití koherence

Výše uvedený algoritmus řádkového vyplňování opakuje některé výpočty zbytečně a nevyužívá dříve získaných výsledků. Především kroky 2a) a 2b) lze výrazně urychlit, a to využitím různých **souvislostí** (angl.*coherence*) a vlastností zpracovávané oblasti.

Koherence (jinak též spojitost) znamená, že hodnota určité veličiny může být odvozena z hodnot již zpracovaných veličin. V našem případě využijeme dvou koherencí:

- hranová koherence umožní určit průsečík s rozkladovým řádkem ze znalosti předchozího průsečíku
- řádková koherence sníží počet třídění průsečíků

Uvažujme nejprve **hranovou koherenci**. Jakmile je jednou spočítána souřadnice x_i průsečíku rozkladové řádky s hranou, určíme snadno další průsečíky s následujícími řádkami upraveným DDA algoritmem. Je-li směrnice hrany

$$m = \frac{y_2 - y_1}{x_2 - x_1}, \qquad (3.1)$$

pak při konstantní změně $\Delta y = y_{i+1} - y_i = -1$ určíme novou souřadnici x_{i+1} průsečíku s nižším rozkladovým řádkem jako

$$x_{i+1} = x_i - \frac{1}{m}. \qquad (3.2)$$

Výraz $1/m$ je obecně neceločíselný, což však neznamená nutnost práce s reálnými proměnnými. Jmenovatel i čitatel jsou totiž celá čísla - rozdíly původních (ještě nezkrácených) souřadnic koncových bodů hrany, takže postupné přičítání $1/m$ lze realizovat pomocí vlastní aritmetiky s celočíselnými zlomky. Při vhodné inicializaci zároveň celá část složeného zlomku určuje polohu pixelu odpovídajícího průsečíku o souřadnici x, čímž odpadá zaokrouhlování.

Použití celočíselné aritmetiky je vyváženo větším počtem potřebných proměnných. Složený zlomek vyjadřující souřadnici x je reprezentován dvěma konstantami (jmenovatel a čitatel) a dvěma proměnnými - celou částí zlomku a proměnnou, která řídí inkrementaci celé části zlomku.

Uvažujme případ postupného přičítání zlomku, jehož jmenovatel je větší než čitatel. Příkladem může být posloupnost hodnot $x = 3$, $3^2/_5$, $3^4/_5$, $4^1/_5$, ..., představující řady průsečíků začínající v bodě $x = 3$. Přírůstek je tvořen zlomkem s čitatelem 2 a jmenovatelem 5. Souřadnice (resp. polohy pixelů) x potom vypočítáme podle algoritmu 3.3. Inicializace proměnné *přírůstek* určuje způsob zaokrouhlování. Je-li počáteční hodnotou 0, pak na výstupu algoritmu získáváme souřadnice x (funkce *floor*).

```
přírůstek = přírůstek + čitatel;
    if (přírůstek >  jmenovatel)
                { x = x + 1;
                  přírůstek = přírůstek - jmenovatel;
                }
```

Algoritmus 3.3: Sčítání složených zlomků

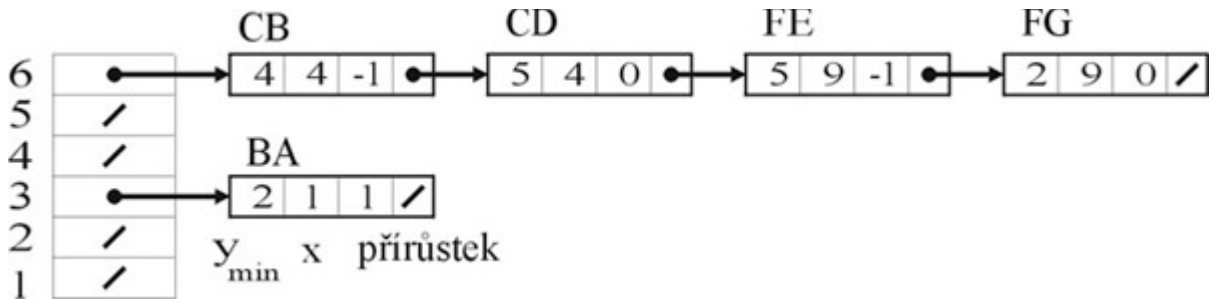
Uvedený algoritmus získávání výpočtu průsečíků na hraně je třeba upravit pro všechny čtyři kombinace znaménka a velikosti směrnice m (kladná/záporná, větší/menší než 1).

Úspora času při výpočtu průsečíků na hraně není tak výrazná, jako úspora času získaná **řádkovou koherencí**. Ta vychází z praktického předpokladu, že při přechodu na další rozkladový řádek budou protnuty především ty hrany, které byly protnuty nyní. Navíc se ve většině případů nezmění pořadí průsečíků na řádku. Z těchto úvah vychází následující velmi efektivní úprava.

3.1.1 Řádkové vyplňování se seznamem aktivních hran

Setřiďme nejprve všechny hrany podle maximální souřadnice *y*. Máme-li dostatek paměti, je možno použít, kde počet přihrádek je roven počtu potřebných rozkladových řádků. Příklad je uveden na obr.3.5., konkrétní číselné hodnoty odpovídají oblasti z obr.3.4.. Datovou strukturu, obsahující takto setříděné hrany, nazveme **tabulkou hran**.

Na obr.3.5 jsou současně uvedena i všechna data, potřebná pro popis hrany. Hrana je orientována shora dolů, hodnota souřadnice *y* jejího počátečního bodu je dána polohou v tabulce hran. V datovém záznamu hrany jsou proto pouze následující položky:

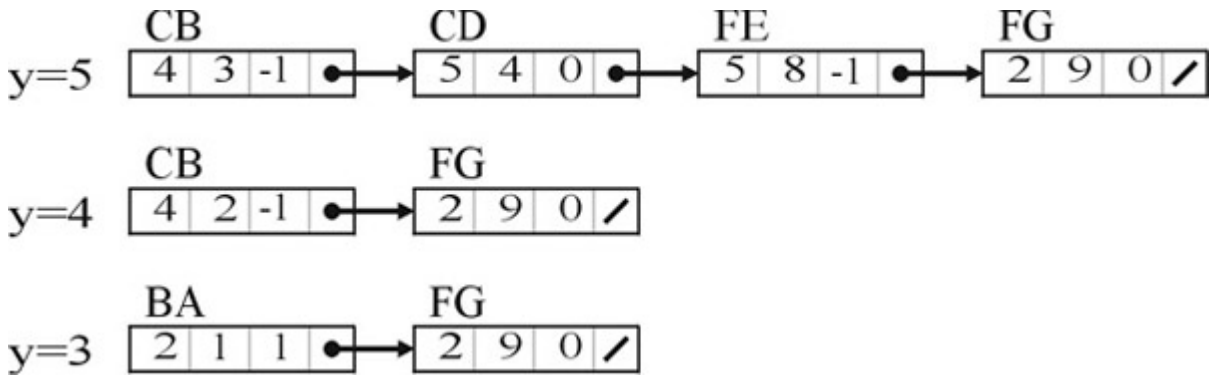


obr. 3.5:Uspořádaná tabulka hran

int	y_{min}	souřadnice *y* koncového bodu
float	*x*	aktuální souřadnice *x* průsečíku s rozkladovým řádkem
float	-1/*m*	přírůstek *x* při přechodu na další řádek

Pro zjednodušení zápisu jsme uvedli řešení s neceločíselnou aritmetikou. Odstranění neceločíselných hodnot znamená rozšířit záznam hrany o konstantu *jmenovatel* (viz alg.3.3), nutnou pro aritmetiku složených zlomků.

Další datovou strukturou bude **seznam aktivních hran**, který obsahuje pouze ty hrany, které mají průsečík s právě zpracovávaným rozkladovým řádkem. Hrany v tomto seznamu budeme udržovat setříděné podle souřadnic *x*. Především zde se projeví časová úspora metody - při přechodu na nový řádek zůstává uspořádání průsečíků ve většině případů beze změny. Uspořádání je narušeno pouze v případě protínajících se hran nebo pokud je do seznamu přidána nová hrana z tabulky hran. Tyto případy snadno řeší **bublinkové třídění** (angl.*bubble-sort*).



obr. 3.6: Seznam aktivních hran pro řádky *y* = 5, 4 a 3

Při přechodu na nový řádek tedy provedeme jednoduchou aktualizaci seznamu aktivních hran - vyloučíme nepotřebné hrany (ležící celé nad daným řádkem), určíme nové hodnoty *x* hran

v seznamu, přidáme případné nové hrany z tabulky hran a seznam uspořádáme dle *x*. Postup je zapsán v algoritmu 3.4.

```
1. Vytvoř uspořádanou tabulku hran TH

2. Vytvoř prázdný seznam aktivních hran SAH

3. Nastav y na nejvyšší souřadnici y

4. Dokud nejsou TH a SAH prázdné, opakuj

    (a) Přidej do SAH hrany z TH[y]

    (b) Uspořádej SAH (bublinkovým tříděním)

    (c) Vykresli úseky mezi lichými a sudými hranami v SAH

    (d) Zruš ze SAH hrany, jejichž y_min = y

    (e) Sniž y o 1

    (f) Aktualizuj x u všech hran v SAH
```

Algoritmus 3.4: Řádkové vyplňování se seznamem aktivních hran

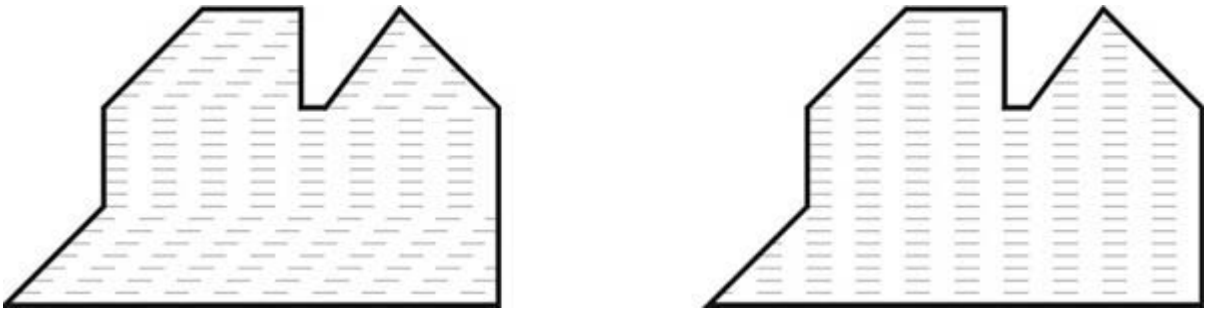
APPLET

applet 3.1

3.1.2 Vyplňování vzorem a šrafování

Přirozeným zobecněním metody řádkového vyplňování je **vyplňování vzorem** definovaným v rastru. Definice vzoru je uložena v obdélníkové matici $\mathbf{V}_{m,n}$. Hodnoty, definující cílovou barvu každého pixelu, se z matice vybírají podle souřadnic kresleného pixelu. Jeho souřadnice *modulo*[1] velikost matice **V** jsou použity jako index prvku matice. Protože se vyplňování uskutečňuje po řádcích, je možno zvýšit efektivitu tím, že do obrazové paměti budeme opakovaně kopírovat celé řádky matice **V**.

V technických aplikacích je podporováno vyplňování **šrafováním**. Jeho výhodou je možnost plynulé změny měřítka. Rozšíření řádkového vyplňování na šrafování je jednoduché. V případě vodorovných šraf stačí jen změnit krok změny souřadnice *y* z hodnoty -1 na -*m* (*m* celé a kladné) a získáme oblast vyplněnou vodorovnými šrafami s roztečí *m* pixelů.



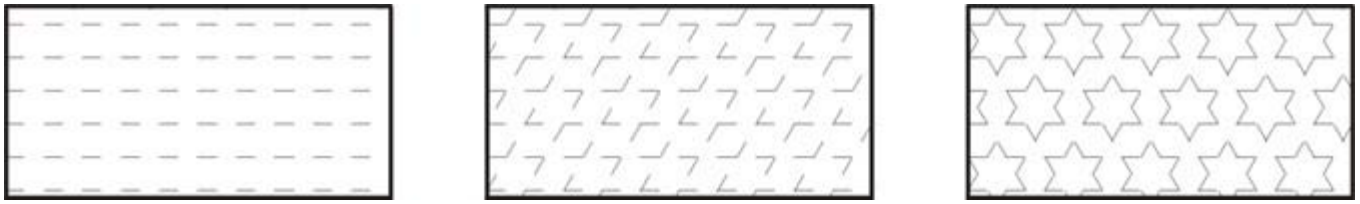
a) s pevným začátkem b) vzhledem ke vztažnému bodu

obr. 3.7: Použití přerušovaných čar při šrafování oblasti

V praxi je však častější šrafování pod obecným úhlem α. Pomocí transformace otáčení převedeme úlohu na běžné vyplňování. Nejprve otočíme všechny hraniční úsečky oblasti o úhel -α. Dále použijeme normální algoritmus řádkového vyplňování s tím, že před vykreslením vypočítaných vodorovných úseků otočíme jejich koncové body o úhel α a teprve poté je zobrazíme.

Při šrafování přerušovanými čárami dochází k optickému jevu, který je naznačen na obr.3.7. Protože je kreslení přerušovaných čar zahajováno od levého kraje vodorovného úseku na každém řádku, čárkování vytváří určitou siluetu, která je do jisté míry kopií levé hranice oblasti (obr. 3. 7.a). Tento jev je nežádoucí a projevuje se jak u oblastí se složitější hranicí, tak na místech, kde k sobě přiléhají dvě oblasti vyplňované stejným typem šrafování. Řešení spočívá v úpravě algoritmu pro kreslení přerušovaných čar tak, aby čárkování bylo nanášeno relativně k určitému vztažnému bodu, například k počátku soustavy souřadnic. Umístění jednotlivých čárek ve vyšrafované oblasti se tak stane nezávislým na tvaru a umístění oblasti (obr. 3.7.b).

Všimněme si, že v případě vyplňování rastrovým vzorem je nezávislost umístění vzoru na poloze hranice obsažena již v samotné metodě. Oblast je vlastně považována za výřez, kterým uživatel nahlíží na vzor na pozadí. V některých aplikacích, jako jsou například animace, je naopak žádoucí, aby se vzor pohyboval spolu s oblastí. Tehdy zvolíme za vztažný bod jeden z vrcholů hranice.



obr. 3.8: Vytváření vzoru opakovaným šrafováním (AutoCAD)

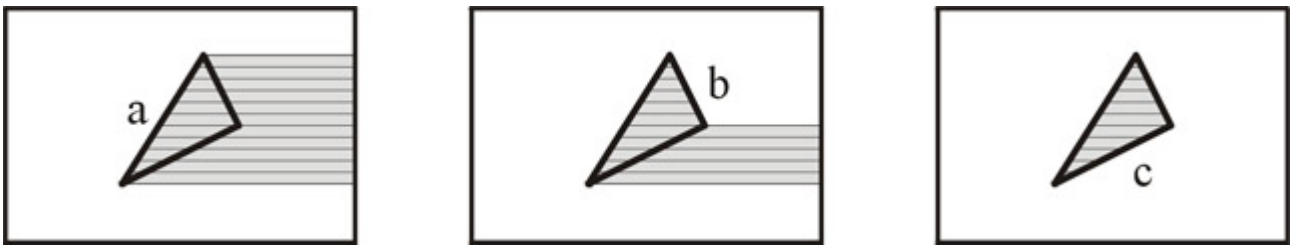
APPLET

applet 3.2

Ačkoliv se pomocí šrafování nedá docílit vyplnění libovolným barevným vzorem, je možno pomocí opakovaného šrafování téže oblasti pod různými úhly vytvořit z čárkovaných čar poměrně širokou škálu vzorů. Jeden z příkladů je na obr.3.8.

3.1.3 Inverzní vyplňování

Metoda řádkového rozkladu není jedinou vyplňovací metodou používanou v počítačové grafice. Její slabinou je nutnost uspořádávání průsečíků na každém rozkladovém řádku, které můžeme pouze různými způsoby zrychlovat, ale nikoliv odstranit. Byly proto hledány způsoby vyplňování, které by nevyžadovaly žádné třídění. Jednou z nich je inverzní vyplňování, které vykazuje lineární časovou závislost na počtu hraničních úseček.



obr. 3.9: Postup inverzního vyplňování při zpracování hranic *a, b, c*

Při metodě inverzního vyplňování je zpracovávána každá úsečka samostatně, a to tak, že pro každou souřadnici *y* hraniční úsečky je vygenerována a pomocí inverze pixelů (případně operací XOR) vykreslena vodorovná čára vedoucí od bodu na úsečce s odpovídající souřadnicí *x* k bodu s maximální souřadnicí *x* celé hranice (případně celé obrazovky). Po zpracování první úsečky je takto vyplněn lichoběžník zleva omezený touto úsečkou a zprava svislou čárou o souřadnici $x = x_{max}$. Další hraniční úsečky způsobí smazání části tohoto lichoběžníku, případně jeho doplnění o nové vyplněné oblasti. Postup je dokumentován obrázkem 3.9 a algoritmem 3.5. V tomto i v následujících algoritmech automaticky uvažujeme zkrácení hran o 1 pixel zdola.

```
1. Nalezni největší souřadnici oblasti x_max

2. Pro každou nevodorovnou hranu

   (a) nalezni řádkovým rozkladem její pixely [x,y]

   (b) invertuj pixely ve vodorovném úseku [x,y]-[x_max,y]
```

Algoritmus 3.5: Inverzní řádkového vyplňování

APPLET

applet 3.3

Časově úspornější variantou je tzv. **plotové vyplňování**. Plotem je nazývána myšlená svislá čára, která může protínat i vnitřek oblasti. V bodu 2b) algoritmu 3.5 je potom třeba určovat směr kresby podle souřadnice *x* vlevo či vpravo od plotu. Výsledkem je obecně nižší počet pixelů, které je třeba invertovat.

Inverze hodnot pixelů odstraňuje nutnost třídění průsečíků a udržování aktivního seznamu hran. Nevýhodou je, že pixely vyplněné oblasti nepřekryjí původní kresbu na obrazovce. Pokud vyplňovaná oblast nebyla zcela prázdná, zůstanou v jejím vnitřku invertované pixely z předchozích kreseb. Z praktického pohledu to znamená, že samotné inverzní vyplňování nemá velký význam, je-li prováděno přímo v obrazové paměti. Lepší je vyplnit nejprve oblast v pomocné paměti a teprve výsledek přenést do paměti obrazové. Tento postup je používán v následující metodě.

3.1.4 Vyplňování šablony

Šablonou je nazývána část operační paměti, ve které je prováděno vyplňování jedné oblasti. Jeden pixel obrazové paměti může být v šabloně reprezentován jediným bitem, i když se jedná o zpracování barevných informací. Z důvodů snazší adresace se však častěji volí reprezentace pixelu jedním bytem nebo slovem procesoru.

Vyplnění oblasti je tedy převedeno na vyplnění šablony. Údaje ze šablony, které mají charakter dvoustavové informace (vyplněno/nevyplněno) jsou po zpracování celé oblasti použity pro kresbu v obrazové paměti. Při této fázi je možno také nastavovat barvu kreslených pixelů podle barevného vzoru.

```
1. Vyčisti (vynuluj) šablonu

2. Pro každou nevodorovnou hranu
   (a) nalezni řádkovým rozkladem její pixely [x,y]
   (b) invertuj místo v šabloně na pozici [x,y]

3. Pro každý řádek šablony nalezni pozice nastavené na logické jedničky.
   Do obrazové paměti nakresli vodorovné úseky odpovídající lichým
   a sudým pozicím v šabloně.
```

Algoritmus 3.6: Vyplňování šablony

APPLET

applet 3.4

Velikost šablony musí odpovídat velikosti vyplňované oblasti, takže oproti předchozím způsobům je tato metoda vyplňování náročná na paměť. Předčí je ovšem v rychlosti, protože vyplňování šablony je převedeno na kresbu hranice. Vodorovné úseky, které mají být vyplněny, nejsou do šablony zapisovány jako posloupnost pixelů, protože významnou informaci obsahují pouze koncové body úseků. Podobně jako při inverzním vyplňování jsou zpracovávány hraniční úsečky jedna po druhé a souřadnice jejich bodů jsou zapisovány do šablony. Zápis je proveden negací odpovídajícího bitu v šabloně. Na rozdíl od inverzního vyplňování se do šablony nezapisuje žádná vodorovná čára, takže přístup do šablony je omezen na minimum (alg.3.6).

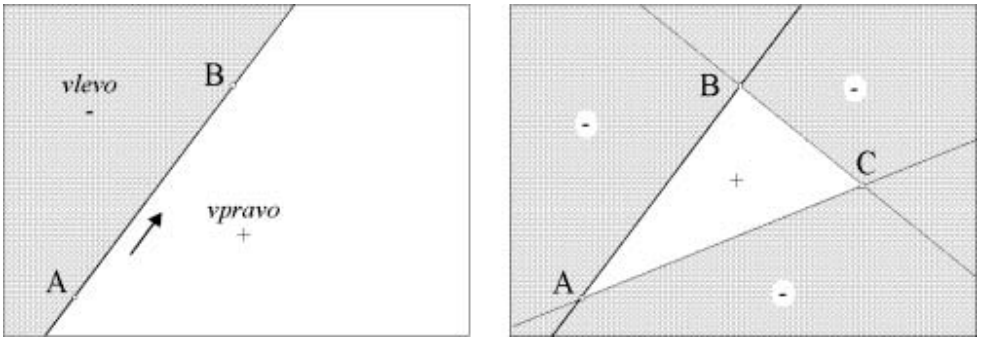
```
1. Pro každou nevodorovnou hranu

   (a) nalezni řádkovým rozkladem její pixely [x,y]

   (b) jde-li o levou hranu,
       pak šablona[y][xL] = x,
       jinak šablona[y][xR] = x

2. Pro každé y šablony nakresli vodorovnou úsečku
   z šablona[y][xL] do šablona[y][xR]
```

Algoritmus 3.7: Upravená šablona pro konvexní oblasti

Odstranění třídění, inverze minimálního počtu hodnot a lineární závislost na počtu hraničních úseček znamenají, že metoda vyplňování šablony je velmi rychlá. Použití šablony je také ideální v případě vyplňování **konvexních oblastí**. Pro ně platí, že na rozkladovém řádku se vykresluje právě jediná úsečka. Šablonu pak úsporně reprezentujeme jako pole dvojic souřadnic (x_L, x_R), jehož rozměr je dán mezními rozměry y oblasti. U konvexní oblasti můžeme dále podle orientace hraničních úseček rozdělit hrany na levé a pravé, a podle toho při řádkovém rozkladu hrany určíme, zda nastavovat hodnoty x_L či x_R (alg.3.7).

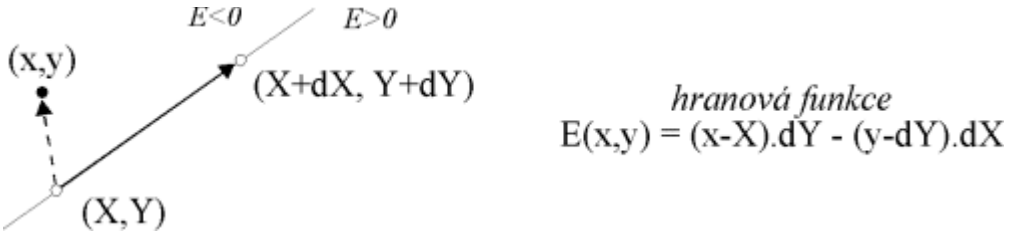
3.1.5 Pinedův algoritmus

V roce 1988 publikoval Juan Pineda [Pine88] jednoduchý algoritmus pro rasterizaci konvexních polygonů. Jeho princip si vysvětlíme na příkladu vyplnění trojúhelníku. Nechť je trojúhelník definován pomocí 3 orientovaných přímek jako průnik 3 polorovin. Výchozí situace je znázorněna na obr. 3.10.



Obr. 3.10: Trojúhelník jako průnik 3 polorovin

Znaménkem "-" označíme polorovinu vně trojúhelníku, "+" značí vnitřní polorovinu. Pro rozlišení vnitřní a vnější poloroviny použijeme hranovou funkci definovanou podle obr. 3.11 (následující stránka).



Obr. 3.11: Hranová funkce orientované hranice

Orientovaná přímka prochází bodem $[X,Y]$ ve směru určeném vektorem (dX,dY). Hranovou funkci $E(x,y)$ libovolného bodu $[x,y]$ v rovině definujeme pomocí vektorového součinu[2] jako

$$E(x,y) = (x - X).dY - (y - Y).dX$$

V případě konvexního polygonu s i orientovanými hraničními přímkami a odpovídajícími hranovými funkcemi $E_i(x,y)$ platí

Bod $[x,y]$ je uvnitř trojúhelníka : $\forall\ i\ E_i(x,y) \geq 0$

Hranová funkce je lineární pro obě souřadnice a triviálním dosazením získáme následující vztahy:

$$E_i(x + 1,y) = E_i(x,y) + dY_i$$

$$E_i(x,y + 1) = E_i(x,y) - dX_i$$

$$E_i(x - 1,y) = E_i(x,y) - dY_i$$

$$E_i(x,y - 1) = E_i(x,y) + dX_i$$

S využitím linearity hranových funkcí lze vyplňování realizovat jednoduchým postupem:

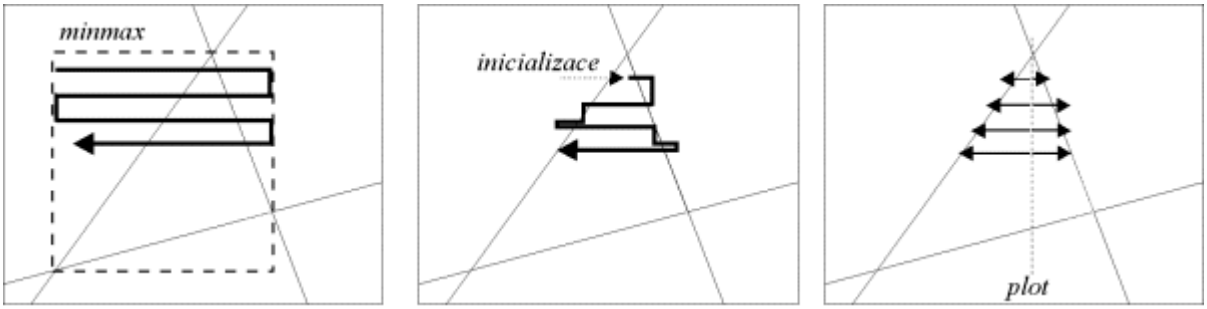
```
Ve výchozím bodě [x,y] inicializuj hranové funkce E_i(x,y).
Opakuj:
   Podle použité strategie prohlížení zvol krok.
   Oprav hranové funkce.
   Pokud ∀ i E_i(x,y) ≥ 0, pak vyplň bod [x,y]
```

Algoritmus 3.8: Princip Pinedova algoritmu

APPLET

applet 3.5

Na obr. 3.12 jsou uvedeny příklady strategií procházení rovinnou oblastí. V případě podle obr. 3.12a procházíme např. po řádcích obdélníkovou oblast "minmax" obalu trojúhelníka a hranové funkce vyhodnocujeme v každém bodě. Efektivnější, ale algoritmicky náročnější je procházení, při kterém měníme směr ihned po dosažení vnější poloroviny (obr. 3.12b). Přechod na další řádku může vést k návratu dovnitř trojúhelníku. V takovém případě je nutné nejprve nalézt v opačném směru hranici a teprve poté pokračovat směrem dovnitř.



a) b) c)

Obr. 3.12: Strategie vyhodnocení hranových funkcí

Prohlížení lze zefektivnit s použitím strategie "plotu" (obr. 3.12c). Hranové funkce počítáme nezávisle na sobě (a tedy i s možností paralelního výpočtu) na obě strany od zvoleného plotu.

Z vlastností Pinedova algoritmu je zřejmé, že

- vyplňování lze řešit paralelně, např. s použitím 3 sčítaček a jednoho logického součinu
- různé strategie umožňují algoritmus upravit pro specializované grafické akcelerátory
- vyplňovanou plochu můžeme rozdělit na více oblastí a řešit vyplňování paralelně s volitelným počtem vyplňovacích obvodů
- maximální paralelnosti lze dosáhnout tak, že každý pixel bude schopen vyhodnotit jednoduchou hranovou funkci a rychlost vyplňování pak nebude záviset na velikosti trojúhelníku

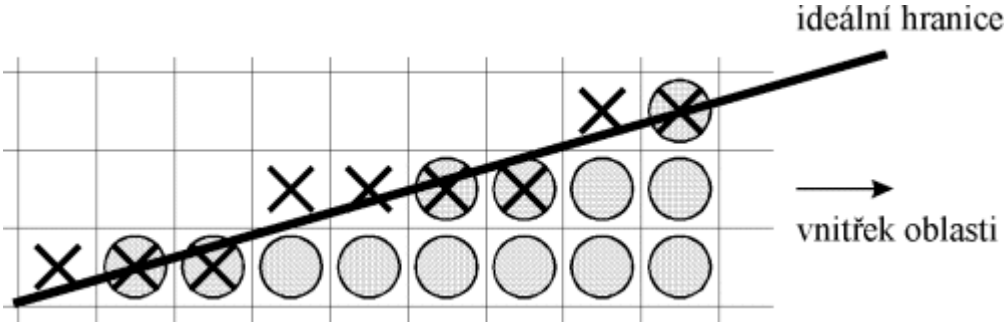
Zejména poslední z výše uvedených vlastností je velmi perspektivní a byla již s úspěchem využita při realizaci architektury PixelPlanes a PixelPowers.

3.1.6 Problémy s hranicí

Dosud jsme podrobněji nestudovali vztah mezi hranicí a vnitřkem oblasti z hlediska kresby v rastru. Hranice je z geometrického pohledu čára o nulové tloušťce, avšak pokud ji nejprve vykreslíme do obrazové paměti a poté oblast vyplníme (jinou barvou), mohou nás zajímat odpovědi na následující otázky:

1. Překryjí vnitřní pixely již nakreslené pixely hraniční?
2. Mohou výše uvedené algoritmy vyplnit oblast, která je omezena několika samostatnými hranicemi?
3. Budou se překrývat pixely dvou oblastí se společnou částí hranice?

Odpověď na první otázku dává obr. 3.13. Hraniční úsečky bývají vykreslovány Bresenhamovou metodou, a to s ohledem na směrnici (řídící osu). Při řádkovém vyplňování oblasti je však pro výpočet použit algoritmus s konstantním krokem v ose y (řídící osou je osa y bez ohledu na směrnici úsečky), takže hranice se směrnicí m, kde $|m| < 1$, nemohou být vnitřní pixely dokonale překryty.



Obr. 3.13: Pixely hranice oblasti, které nebyly překryty vnitřními pixely

Odpovědi na zbylé dvě otázky jsou již optimističtější. Všechny metody řádkového vyplňování jsou schopny zpracovat oblast s násobnou hranicí (např. plochu s otvory) bez jakýchkoliv zásahů do algoritmu.

Úvahy o možnosti překrývání sousedních oblastí na společné hranici nejsou zbytečné. Pokud by k překrývání docházelo, závisel by výsledný vzhled obrázku na pořadí vykreslování ploch, což není žádoucí. Řešením je dohoda, že například ta část hranice, která je **vlevo** a **nahoře**, bude patřit k vyplňované oblasti, zatímco zbylá část nikoliv (ta bude patřit případné sousední oblasti). Všimněme si, že právě zkracování hran o 1 pixel zdola automaticky "uvolní" dolní část hranice pro sousední oblast, ač tato úprava byla původně navržena pro řešení problému násobného průsečíku dvou navazujících hran. Uvolnění hranice vpravo je třeba realizovat zkrácením vykreslovaných vodorovných úseků, tedy kreslit úsečky (x_L,y) - $(x_R - 1,y)$ namísto (x_L,y) - (x_R,y).

Vyhlazování okrajů oblasti

Některé algoritmy pro vyplňování oblastí mohou být doplněny výpočty pro vyhlazení zubatého tvaru hranice. Podobně jako u úseček může být vyhlazování pomocí určování hodnot subpixelů nebo pomocí výpočtů plochy pixelu skutečně překryté vyplňovanou oblastí.

Jiné tvary hranice

Dosud jsme předpokládali, že vyplňovaná oblast je omezena mnohoúhelníkem. Někdy je požadováno, aby algoritmus dokázal vyplnit i oblasti, které jsou ohraničeny obecnější čárou kombinovanou například z úseček, kruhových oblouků, částí grafů polynomických funkcí nebo spline křivek. Metoda řádkového rozkladu je použitelná i pro tento případ, ovšem vyžaduje určitá doplnění. Pojem hrana je nahrazen pojmem křivka, což přináší nutnost složitějších výpočtů průsečíků s rozkladovými řádky. Aby byl dodržen princip vykreslování spojnic mezi dvojicemi průsečíků, musí mít každá křivka maximálně jeden průsečík s rozkladovým řádkem. Tato podmínka je porušena již u kruhového oblouku[3], který prochází "severním" či "jižním" pólem kružnice. Křivky je tedy nutno rozdělit na menší části tak, aby žádná část křivky neobsahovala lokální maximum nebo minimum souřadnic *y*.

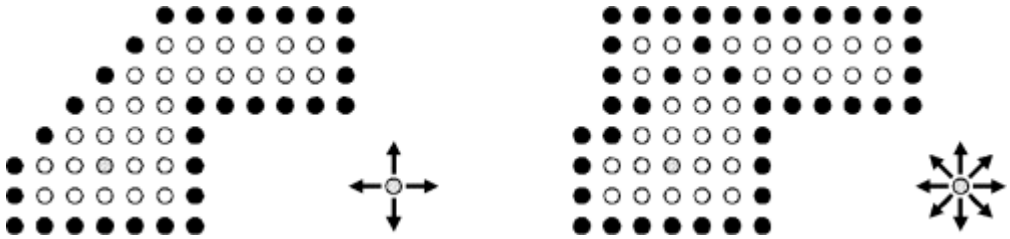
Výpočty průsečíků hraničních křivek s rozkladovými řádky mohou představovat značnou časovou zátěž. V mnoha případech je lepší nahradit nejprve křivky lomenou čárou a poté použít jednodušší metodu pro vyplňování mnohoúhelníku. Počet přímkových hran sice vzroste, ale například délka seznamu aktivních hran, která má největší vliv na rychlost metody, bude malá. Pokud máme k dispozici dostatečně velkou paměť hran, je toto řešení efektivní i při velmi jemné aproximaci křivek lomenými čárami.

3.2 Semínkové vyplňování

Grafické systémy umožňující interakční kreslení a malování od ruky obsahují další prostředky pro vyplňování oblastí. Uživatel pomocí myši, tabletu nebo jiného interakčního zařízení nakreslí obrys objektu, ukáže na jeho vnitřní bod a vybere z nabídky barev a vzorů. Systém pak vyplní vnitřek nakresleného objektu.

Hranice tedy není geometricky jasně určena, ale všechny informace o vyplňované oblasti se získávají čtením z obrazové, tedy rastrové paměti. Rastrový charakter úlohy má vliv i na to, co považujeme za vnitřek hranice. Vnitřní oblasti proto klasifikujeme do dvou tříd:

- **4-spojitá (4-souvislá) oblast** je tvořena diskrétními body v rastrové mřížce tak, že mezi libovolnými dvěma body existuje cesta složená z vodorovných a svislých kroků po vnitřních bodech oblasti.
- **8-spojitá (8-souvislá) oblast** je tvořena diskrétními body v rastrové mřížce tak, že mezi libovolnými dvěma body existuje cesta složená z vodorovných, svislých a diagonálních kroků po vnitřních bodech oblasti.



Obr. 3.14: 4-spojitá (vlevo) a 8-spojitá (vpravo) oblast se zvýrazněným semínkem

Charakteru oblasti odpovídá i charakter hranice (obr. 3.14). Například Bresenhamův algoritmus generuje úsečky, které mohou tvořit hranici 4-spojité oblasti. Pro ohraničení 8-spojité oblasti jsou však nedostačující a algoritmy pro vyplňování by mohly vést na vyplnění rastrových bodů i za takto nakreslenou hranicí. Ve většině systémů jsou zpracovávány 4-spojité oblasti.

Při semínkovém vyplňování postupujeme od zadaného semínka a zkoumáme, zda jeho sousední body patří k vnitřku oblasti. Existuje několik variant:

1. **Hraniční vyplňování**

   Testovaný bod je vnitřní, má-li jinou barvu, než je zadaná barva hranice.

2. **Záplavové vyplňování**

   Testovaný bod je vnitřní, má-li stejnou barvu jako zadané semínko. Této metodě se také říká lavinové vyplňování či přebarvování.

3. **Měkké vyplňování**

   Testovaný bod je vnitřní, má-li výrazně jinou barvu, než je zadaná barva hranice. To nastává například poté, co hranice byla vyhlazena a získala nestejný odstín.

Uvedené varianty se liší pouze v testu na příslušnost bodu k oblasti. Jako ukázku proto uvedeme jen jeden z algoritmů, a to jeho naivní, rekurzivní verzi. Obrazová paměť je v algoritmu 3.9 reprezentována dvourozměrným polem **rastr**.

```
void HranicniVyplnovani (char BarvaHranice, char NovaBarva,
                         int x, int y)
{
  if ((rast[x][y] != BarvaHranice) && (rastr[x][y] != NovaBarva))
  {
    rastr[x][y] = NovaBarva;
    HranicniVyplnovani (BarvaHranice, NovaBarva, x+1, y);
    HranicniVyplnovani (BarvaHranice, NovaBarva, x+1, y);
    HranicniVyplnovani (BarvaHranice, NovaBarva, x+1, y);
    HranicniVyplnovani (BarvaHranice, NovaBarva, x+1, y);
  }
  return;
}
```

Algoritmus 3.9: Hraniční semínkové vyplňování

Rekurzivní řešení je elegantní, avšak prakticky nepoužitelné. Dochází při něm často k vyčerpání zásobníku používaného pro předávání parametrů a návratových adres podprogramu. Další nevýhodou je značná neefektivita. Šíření semínek do všech směrů má za následek, že každý vnitřní pixel je testován několikrát i poté, co již byl vybarven. Poměr počtu rekurzivních volání k počtu skutečných obarvení pixelů je až 4:1. Každé vyvolání procedury je přitom spojeno s pomalou operací čtení obrazové paměti.

3.2.1 Řádkové semínkové vyplňování

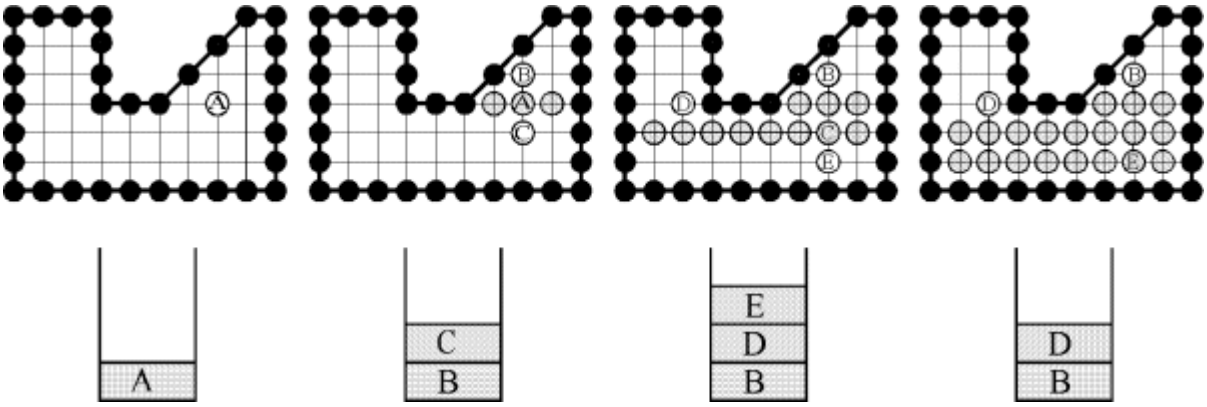
Metoda, která snižuje počet přístupů do obrazové paměti, se nazývá **řádkové semínkové vyplňování**, (angl. *scan line seed fill*). Rekurze je odstraněna použitím malého zásobníku, ve kterém jsou uchovávány souřadnice vždy jen několika vnitřních bodů vyplňované oblasti (algoritmus 3.10).

Na obr. 3.15 je nakreslen stav vyplňované oblasti a stav zásobníku při postupném provádění algoritmu. Startovním semínkem určeným uživatelem je bod *A*, který je vložen do zásobníku. Při prvém průchodu je ze zásobníku vybrán, je jím vedena úsečka až k hranicím oblasti a poté jsou testovány úseky nad a pod touto úsečkou. Výsledkem je zařazení bodů *B* a *C* do zásobníku. Při dalším průchodu je vedena úsečka bodem *C* a jsou nalezeny volné úseky s určujícími body *D* a *E*. Bod *A* není znovu do zásobníku zařazen, protože leží v již vyplněném úseku. Při zpracování bodu *D* se pouze kreslí vnitřní úsečka oblasti a zásobník se již dále nezvětšuje. Postupně je směrem vzhůru vyplněna celá levá část oblasti a nakonec i zbytek na pravé straně.

Čtení souvislého vodorovného úseku lze implementovat velmi efektivně vzhledem k uspořádání obrazové paměti po řádcích.

```
1. Vlož uživatelem zadané semínko do zásobníku
2. Dokud není zásobník prázdný
   (a) vyjmi semínko (se souřadnicemi [x,y]) ze zásobníku
   (b) nalezni hranici xL a xR na řádku y ve nejbližším okolí
       bodu [x,y]
   (c) nakresli úsečku [xL,y] - [xR,y]
   (d) na (vyšší) úsečce [xL,y + 1] - [xR,y + 1] hledej
       souvislé vnitřní úseky a pro každý z nich vlož
       do zásobníku souřadnice jednoho vnitřního bodu
   (e) na (nižší) úsečce [xL,y - 1] - [xR,y - 1] hledej
       souvislé vnitřní úseky a pro každý z nich vlož
       do zásobníku souřadnice jednoho vnitřního bodu
```

Algoritmus 3.10: Řádkové semínkové vyplňování



Obr. 3.15: Postup při řádkovém semínkovém vyplňování

APPLET

applet 3.8

3.2.2 Vyplňování vzorem a šrafování

Barevný vzor definovaný pomocí pole barev lze nanášet na obrazovku stejně jako při vyplňování geometricky určené hranice. Problém nastává tehdy, je li ve vzoru obsažena původní barva vnitřku oblasti. Tehdy dojde k zacyklení, neboť uvedené algoritmy opakovaně zjišťují, že určité místo uvnitř oblasti ještě nebylo vyplněno.

Zacyklení předejdeme tím, že odstraníme opakované čtení z obrazové paměti, neboť dosud popsané algoritmy přistupovaly na totéž místo v obrazové paměti několikrát (naivní semínkový algoritmus 4×, jeho řádková varianta až 3×). Tím se mimochodem také významně zvýší rychlost celého algoritmu.

Nejvhodnější se pro úpravu jeví algoritmus řádkového semínkového vyplňování. Rozšíříme údaje ukládané na zásobník o další položky ([Zemč93]). Pro každé nově ukládané semínko zapíšeme nejen jeho souřadnici y, ale také obě mezní souřadnice x_L,x_R souvislého vodorovného úseku, ve kterém leží semínko a dále pak příznak směru, odkud se řádkové vyplňování k semínku dostalo (zdola, shora).

Postup při vyplňování je popsán hrubým algoritmem 3.11 (následující stránka).

1. Vyjmeme semínko ze zásobníku a vyplníme jím určený úsek barevným vzorem. Toto vyplňování budeme šířit na obě strany až do dosažení hranice oblasti.
   Získáme tak meze vyplněného úseku x_{min} a x_{max}.
2. V zapamatovaném směru vyplňování (shora/zdola) otestujeme všechny volné úseky v rozsahu $<x_{min},x_{max}>$, pro každý z nich vložíme do zásobníku novou sadu údajů.
3. V opačném směru budeme testovat volné úseky pouze v intervalech $<x_{min},x_L)$ a $(x_R,x_{max}>$. Tím zabráníme opětnému čtení pixelů z úseku $<x_L,x_R>$, který byl zřejmě vyplněn již v některém z předchozích kroků.

Algoritmus 3.11: Zefektivnění řádkového semínkového vyplňování

Uvedenou úpravou zamezíme opakovanému čtení hodnot barev pixelů ze stejných míst obrazové paměti. Nebezpečí zacyklení zůstává pouze v případě, kdy se dva směry vyplňování sejdou proti sobě (případ vyplňování mezikruží dvou soustředných kružnic). Také tuto situaci vyřešíme, a to zvláštním testem při ukládání údajů o novém semínku na zásobník. Porovnáme nově zapisované údaje s obsahem celého zásobníku. Pokud je nalezen úsek se stejnou souřadnicí *y* a opačným směrem šíření vyplňování, je třeba takový úsek jednorázově vyplnit a vypustit ze zásobníku. Při shodě směru budeme pouze data nového úseku ignorovat.

Větší množství dat zapisovaných na zásobník vede k celkovému zrychlení metody. Jediným zpomalujícím prvkem oproti původnímu řádkovému semínkovému vyplňování je testování obsahu zásobníku. Délka zásobníku však obecně bývá malá (je rovna počtu "nekonvexností" vyplňované oblasti) a tak toto zpomalení je zanedbatelné. Uvedená úprava je dalším připomenutím, že zrychlení algoritmů se velmi často dosahu zvýšením objemu uchovávaných informací,a že čas a paměť jsou v počítačové grafice (a nejen v ní) spojité nádoby.

Šrafování

Šrafování je pro oblasti zadané v obrazové paměti poměrně složitou operací. V některých případech jej lze převést na vyplňování vzorem, a to tehdy, pokud šrafy v sousedních vzorcích na sebe navazují. Požadavek na obecný sklon šraf a šrafování přerušovanými čárami je však nutno realizovat jinak. Jednou možností je využití šablony. namísto obrazové paměti je nejprve běžným algoritmem vyplněna plnou barvou šablona. poté je na celou obrazovku aplikováno šrafování libovolným směrem a typem čáry, přičemž skutečný zápis pixelu je před vlastním vykreslením "klíčován" barvou v šabloně.

- [1]operace *modulo* určuje celočíselný zbytek po dělení
- [2]Jedná se o 3. složku vektoru, který je kolmý na rovinu trojúhelníka, ostatní složky jsou nulové.
- [3]vyplňování celých kružnic a elips je ovšem snadné - lze jej provádět přímo při rasterizaci těchto křivek po úpravě procedury, která generuje symetrické body; namísto dvojice bodů symetrických podle osy *y* je nakreslena vodorovná úsečka

---

Literatura:

- [Pine88] Pineda, J.: *A Parallel Algorithm for polygon Rasterization*. Computer Graphics, 22 (4), August 1988, 17-20
- [Zemč93] Zemčík, P.: *Scan-line algorithm for filling with pattern*. Machine GRAPHICS and VISION, 2 (2), 1993, pp. 143-147.

4. Křivky

4.1 Vyjádření a základní vlastnosti křivek

Křivku budeme chápat fyzikálně, jako dráhu pohybujícího se bodu a budeme používat její **parametrické** vyjádření

$$x = x(t)$$
$$y = y(t) \qquad (4.1)$$
$$z = z(t)$$

a parametr t je z intervalu $< t_{min}, t_{max} >$. Funkcemi 4.1 je určena **bodová rovnice** křivky:

$$Q(t) = [x(t),y(t),z(t)], \qquad (4.2)$$

nebo **vektorová rovnice**:

$$q(t) = (x(t),y(t),z(t)). \qquad (4.3)$$

Vektor q(t) = Q(t) - [0,0,0] se nazývá **polohový vektor**, jeho velikost je rovna vzdálenosti bodu Q(t) od počátku. Derivace takovéto křivky se provádí po jejích složkách. **Tečný vektor** v bodě $Q(t_0)$ je tvaru:

$$q'(t_0) = (x'(t_0),y'(t_0),z'(t_0)) = \left(\frac{dx(t_0)}{dt}, \frac{dy(t_0)}{dt}, \frac{dz(t_0)}{dt}\right). \qquad (4.4)$$

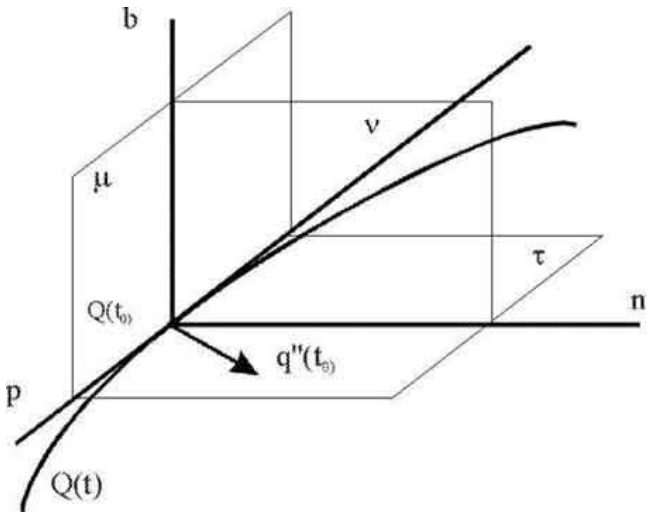
Rovnice tečny v tomto bodě má tvar:

$$p(t) = Q(t_0)+t\, q'(t_0) \qquad (4.5)$$

Vektor $q'(t)$ se nazývá **směrový vektor**. Ze vztahu 4.4 je patrné, že zásadní výhodou parametrické reprezentace křivek je snadné vyjádření tečny ke křivce.

Výhodou parametrického zápisu je též závislost na jediném parametru; fyzikální interpretací parametru t je čas

Změnou parametrizace rozumíme nahrazení parametru t jiným parametrem, který je zadán jako funkce $t^* = t^*(t)$, a která je **přípustná**, to jest ryze monotónní na definičním oboru t. Nové vyjádření $q(t) = (x(t^*), y(t^*), z(t^*))$ popisuje tutéž křivku s tím rozdílem, že tečný vektor v bodě $q(t^*)$ má stejný směr a jinou velikost (případně orientaci). Říkáme, že tečný vektor je závislý na parametrizaci. Ze vztahu 4.5 je patrné, že tečna na parametrizaci závislá není.



Obr. 4.2: Tečna p, binormála b, hlavní normála n, oskulační rovina τ, rektifikační rovina μ a normálová rovina ν ke křivce Q(t).

Oskulační rovinou křivky v bodě $Q(t_0)$ rozumíme rovinu τ určenou tečnou $p(t)$ a vektorem druhé derivace neboli vektorem zrychlení $q''(t_0)$ (obr.4.2). **Hlavní normála** n je přímka, která je kolmá k tečně $p(t)$ a leží v oskulační rovině τ. **Binormála** je přímka b, která prochází bodem $Q(t_0)$ a je kolmá k oskulační rovině τ. **Rektifikační rovina** μ jej určena binormálou b a tečným vektorem $q'(t)$ a konečně **normálová rovina** ν je určena přímkami b a n.

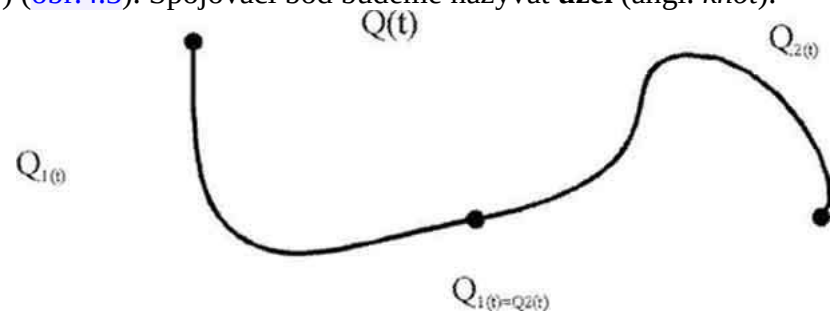
První křivost ${}^1k(t)$ křivky $Q(t)$ v bodě t_0 se vypočítá podle vztahu:

$$ {}^1k(t_0) = \sqrt{\frac{|q'(t_0) \times q''(t_0)|^2}{|q'(t_0) \cdot q'(t_0)|^3}} . \qquad (4.6) $$

Inflexní bod $Q(t_0)$ křivky $q(t)$ je takový bod, kde je

$$ q'(t_0) = k\, q''(t_0);\ k \quad 0 \qquad (4.7) $$

Předpokládejme, že $Q_1(t)$ a $Q_2(t)$ jsou dvě části (neboli **segmenty**) jediné křivky $Q(t)$ spojené v bodě $Q_1(1) = Q_2(0)$ (obr.4.3). Spojovací bod budeme nazývat **uzel** (angl. *knot*).

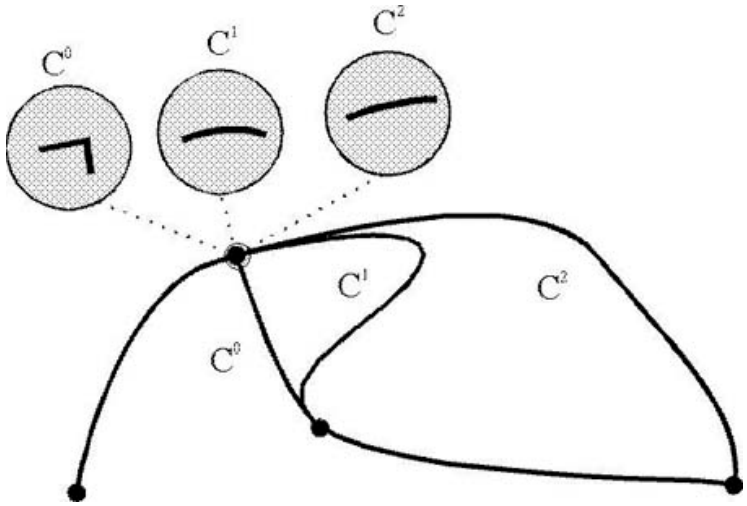


Obr. 4.3: Křivka $Q(t)$ vzniklá spojením dvou segmentů $Q_1(t)$ a $Q_2(t)$

Říkáme, že křivka $Q(t)$ je třídy C^n, má-li ve všech bodech spojité derivace do řádu n. C^n se nazývá **parametrická spojitost** (obr.4.4).

Dva segmenty jsou **spojitě** navázány (neboli mají spojení třídy C^0), pokud je koncový bod prvního segmentu počátečním bodem segmentu druhého. Dva segmenty mají spojení C^1, pokud je tečný vektor v koncovém bodě segmentu Q_1 roven tečnému vektoru segmentu Q_2 v jeho počátečním bodě. Analogicky rovnost vektoru první a druhé derivace je požadována pro C^2 (obr.4.4) atd. Zkráceně zapisujeme:

$$ q^{(i)}(1) = q^{(i)}(0);\ \forall\ i = 0,1,\ldots, n. \qquad (4.8) $$



Obr. 4.4: Spojitost C^0, C^1 a C^2

Čím vyšší spojitost je požadována, tím delší dobu (ve smyslu parametru t) se oba segmenty k sobě přimykají. Zjevně $C^{n+1} \Rightarrow C^n$.

Podmínka parametrické spojitosti je velice restriktivní, jak demonstruje i následující příklad ([Bars88]). Představme si křivku složenou ze dvou úseček (obr.4.5) zadaných:

$$Q_1(t) = [2t,2t];\ t \in < 0,1 >$$

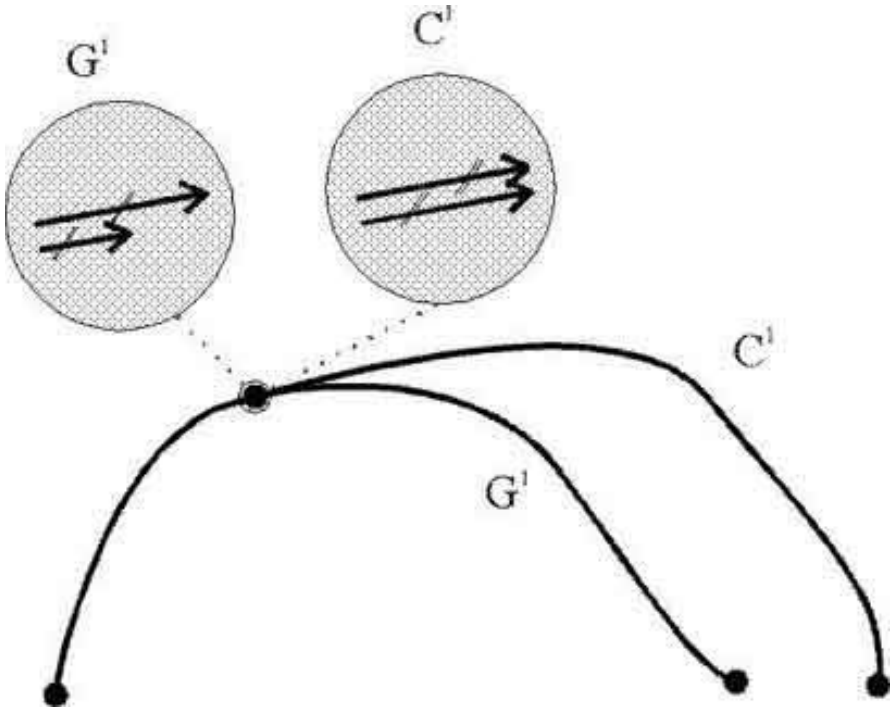
$$Q_2(t) = [t+2,t+2];\ t \in < 0,1 >$$

Tečné vektory mají hodnoty $q'_1(t) = (2,2)$ a $q'_2(t) = (1,1)$. Podmínka identity tečen není tedy splněna, křivka *není* C^1 spojitá, pohybující se objekt by v uzlu skokem změnil rychlost. Právě tyto případy byly motivující pro zavedení **geometrické spojitosti** označované G^n.

Řekneme, že dva segmenty křivky Q(t) jsou spojité G^0, pokud je koncový bod Q_1 počátečním bodem Q_2. Dva segmenty jsou G^1 spojité, pokud tečné vektory $q'_1(1)$ segmentu Q_1 a $q'_2(0)$ segmentu Q_2 jsou lineárně závislé, tj. pokud pro ně platí:

$$q'_1(1) = k\, q'_2(0);\ \ k > 0. \qquad (4.9)$$

Tato spojitost zaručuje totožnost tečen (nikoli tečných vektorů).



Obr. 4.6: Geometrická a parametrická spojitost. Lupa ukazuje tečné vektory

Ze subjektivního hlediska zaručuje G^1 spojitost hladkost jako C^1, z hlediska použití bývá daleko snazší zaručit spojitost G^1 nežli C^1.

Spojitost G^2 je definována jako shoda prvních křivostí ${}^1k$ v uzlu (viz vztah 4.6) obou segmentů:

$${}^1k_{Q1}(1) = {}^1k_{Q2}(0).$$

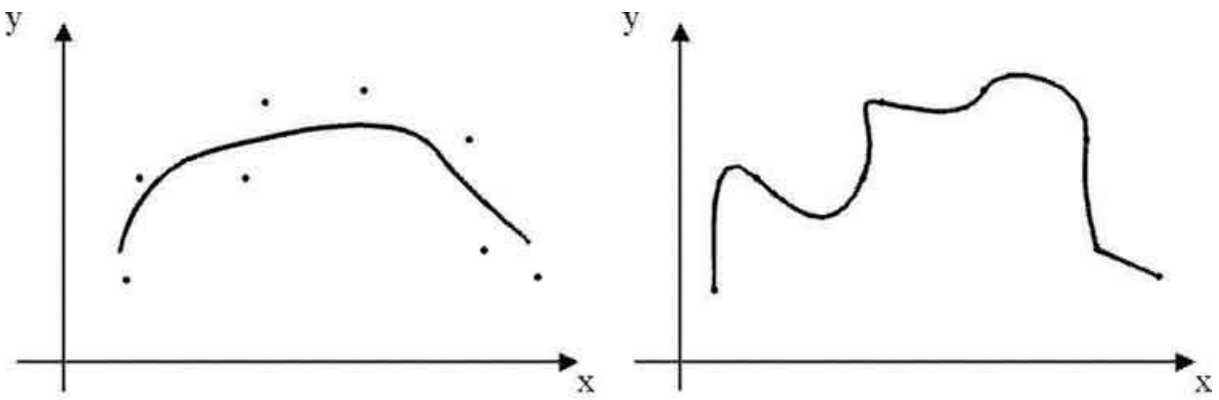
Spojitost C^1 implikuje G^1 (obráceně však nikoliv).

4.2 Modelování křivek

Základním prvkem teorie křivek v počítačové grafice jsou křivky **polynomiální** ($p_n(t) = a_0+a_1t+...+a_nt^n$). Z nich se skládají křivky **po částech polynomiální**, to jsou křivky jejichž segmenty jsou polynomiálními křivkami. Nejčastěji používané jsou křivky třetího stupně - **kubiky**, které poskytují dostatečně širokou škálu tvarů, jejich výpočet bývá nenáročný, jsou snadno manipulovatelné a je možné u nich zaručit spojitost C^2, požadovanou někdy při modelování v CAD systémech. Křivky vyššího stupně mohou způsobovat nežádoucí vlnění a oscilace a jsou náročnější na výpočet.

Modelování probíhá obyčejně tak, že je definováno několik **řídicích bodů** (**řídicí polygon**) a matematický aparát z jejich polohy určí průběh křivky. Některé metody umožňují zadávání křivek též pomocí tečných vektorů, je možno klást podmínky na hladkost navázání aj.

Existují dva základní druhy interpretace řídicích bodů a to **interpolace** a **aproximace**.



Obr. 4.7: Aproximační (vlevo) a interpolační křivka

Při interpolaci (obr.4.7) generovaná křivka probíhá danými body, zatímco při aproximaci je řídicími body tvar křivky určen, ta jimi však procházet nemusí.

Parametricky danou kubiku Q(t) ve tvaru:

$$\begin{aligned} x(t) &= a_x t^3 + b_x t^2 + c_x t + d_x \\ y(t) &= a_y t^3 + b_y t^2 + c_y t + d_y \\ z(t) &= a_z t^3 + b_z t^2 + c_z t + d_z \end{aligned} \qquad (4.10)$$

můžeme zapsat zkráceně v maticovém tvaru:

$$Q(t) = T\,C = [t^3, t^2, t, 1] \begin{bmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \\ d_x & d_y & d_z \end{bmatrix}$$

Derivaci $q'(t)$ získáme derivací vektoru T

$$q'(t) = \frac{d}{dt} q(t) = \frac{d}{dt} T\,C = [3t^2, 2t, 1, 0]\,C$$

Konstantní matici C můžeme rozepsat do součinu

$$C = M\,G,$$

kde matice M je typu 4 ×4 a nazývá se **bázová matice**. Čtyřprvkový vektor G se nazývá **geometrický vektor**. Geometrický vektor reprezentuje vliv vnějších parametrů. Obsahuje řídicí body nebo řídicí body a tečné vektory atp. Bázová matice, která je dána použitou metodou, pak určuje výpočet křivky podle vztahu:

$$Q(t) = [x(t), y(t), z(t)] = [t^3, t^2, t, 1]\begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix}\begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \end{bmatrix} \qquad (4.11)$$

Mezi často požadované vlastnosti křivek patří:

1. **Invariance** k afinním transformacím a projekcím, která zaručuje, že například rotace řídicího polygonu a následné generování křivky má stejný výsledek, jako rotace každého bodu z vygenerované křivky.
2. **Vlastnost konvexní obálky** (angl. *convex hull property*)
   a. **silná podmínka** - celá křivka leží v konvexní obálce všech svých řídicích bodů,
   b. **slabá podmínka** - část křivky leží v konvexní obálce některých řídicích bodů (typicky segment, v obálce svého generujícího polygonu).
3. **Lokalita změn** - změnou polohy (u racionálních křivek i váhy (viz odstavec)) řídicího bodu se mění jen část křivky, nikoli křivka celá.
4. Křivka může **procházet krajními body** svého řídicího polygonu.

4.3 Interpolační křivky

4.3.1 Fergusonovy kubiky

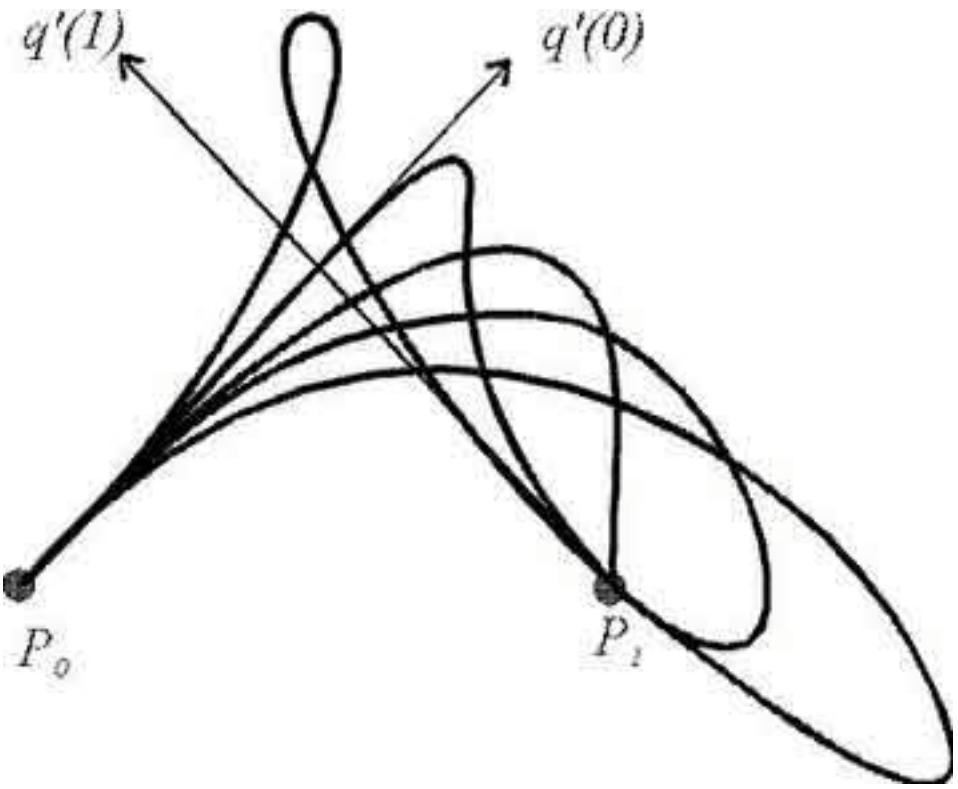
APPLET

applet 4.1

APPLET

applet 4.2

Mezi často používané křivky patří **Fergusonovy kubiky,** popsané roku 1964 *J. C. Fergusonem*. Tyto křivky jsou určeny dvěma řídicími body P_0, P_1 a dvěma tečnými vektory P'_0 a P'_1 v nich. Body P_0, P_1 určují polohu křivky (křivka jimi prochází). Směr a velikost vektorů pak určuje míru jejího vyklenutí. Čím je velikost vektoru větší, tím více se k němu křivka přimyká. Jsou-li oba vektory nulové pak křivka degeneruje na úsečku P_0P_1. Obrázek demonstruje změnu tvaru křivky, je-li vektor P'_0 konstantní a vektor P'_1 postupně rotuje.



Obr. 4.8: Fergusonovy kubiky s naznačenými tečnými vektory

Předpis pro výpočet Fergusonovy kubiky ve stylu 4.11 má tvar:

$$Q(t) = T \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_1 \\ P_2 \\ P'_1 \\ P'_2 \end{bmatrix} \qquad (4.12)$$

Rozepíšeme-li vztah 4.12 pro jednotlivé složky, dostaneme:

$$Q(t) = P_0\, F_1(t) + P_1\, F_2(t) + P_0'\, F_3(t) + P_1'\, F_4(t), \qquad (4.13)$$

kde F_1,F_2,F_3,F_4 jsou **kubické Hermitovské polynomy** tvaru:

$$\begin{aligned} F_1(t) &= 2t^3 - 3t^2 + 1, \\ F_2(t) &= -2t^3 + 3t^2, \\ F_3(t) &= t^3 - 2t^2 + t, \\ F_4(t) &= t^3 - t^2 \end{aligned} \qquad (4.14)$$

Pro hodnoty t = 0 resp. t = 1 prochází křivka body P_0 resp. P_1.

Největší přednost Fergusonových kubik se projeví při jejich navazování. Spojitost při navazování dvou Fergusonových kubik docílíme totožností posledního bodu segmentu Q_1 a prvního bodu segmentu Q_2 (obr.4.3). Spojitost C^1 je zaručíme identitou vektorů v uzlu. Spojitost G^1 docílíme jejich lineární závislostí, vyjádřeno pomocí geometrických vektorů *G* dvou po sobě jdoucích segmentů:

$$G_1 = \begin{bmatrix} P_1 \\ P_2 \\ P'_1 \\ P'_2 \end{bmatrix}_1 = \begin{bmatrix} P_2 \\ P_3 \\ k\,P'_2 \\ P'_3 \end{bmatrix}_2 = G_2 \quad k > 0 \qquad (4.15)$$

Detailně se Fergusonovým kubikám věnuje [Drs90].

4.3.2 Kochanek-Bartles

Interpolační křivky používané pro definici dráhy pohybujících se objektů v počítačové animaci a pojmenované podle svých autorů *Kochanek-Bartels* jsou založeny na interpolaci pomocí Fergusonových kubik (4.13). Vstupem algoritmu je posloupnost interpolovaných bodů P_i a pro každý bod také tři koeficienty a_i, b_i, c_i. Tyto koeficienty určují chování křivky v okolí příslušného bodu. Tečné vektory v i-tém a i+1-ním bodě interpolované posloupnosti se získají podle vztahu:

$$P'_i = \frac{(1-a_i)(1+b_i)(1+c_i)}{2}(P_i - P_{i-1}) + \frac{(1-a_i)(1-b_i)(1-c_i)}{2}(P_{i+1}-P_i) \qquad (4.16)$$

$$P'_{i+1} = \frac{(1-a_{i+1})(1+b_{i+1})(1-c_{i+1})}{2}(P_i - P_{i-1}) +$$

$$+ \frac{(1-a_{i+1})(1-b_{i+1})(1+c_{i+1})}{2}(P_{i+1}-P_i)$$

Význam koeficientů je následující:

- a_i - určuje jak ostře křivka v bodě P_i ,
- b_i - určuje šikmost křivky,
- c_i - řídí její spojitost.

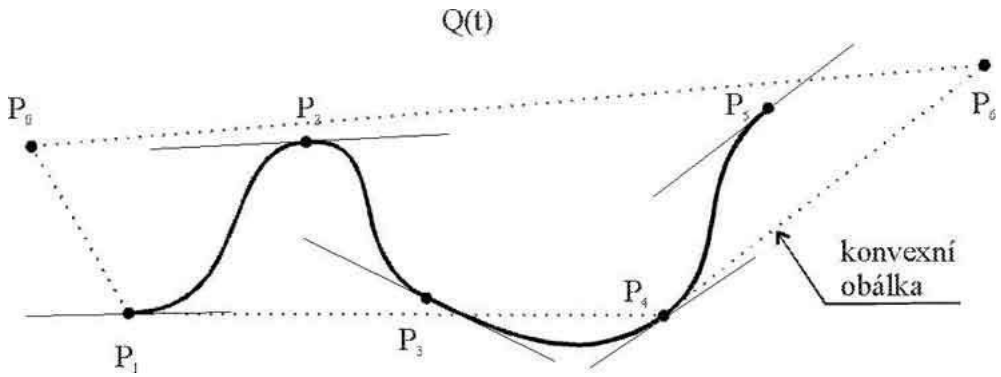
APPLET

applet 4.3

4.3.3 Catmul-Rom spline

Do široké rodiny **spline křivek** (s jejich popisem se setkáme v kapitole) patří interpolační *Catmul-Rom* spline, které jsou díky svým vlastnostem taktéž používány pro definování dráhy objektů v počítačové animaci.

Křivka je zadána posloupností bodů $P_0,P_1,...,P_n$. Vychází z bodu P_1 a končí v bodě P_{n-1}, neinterpoluje tedy první a poslední bod (obr.4.9).



Obr. 4.9: Catmul-Rom spline s vyznačenými tečnami v řídicích bodech

Křivka se vypočítá podle vztahu:

$$Q(t) = \frac{1}{2}[t^3,t^2,t,1]\begin{bmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}\begin{bmatrix} P_{i-3} \\ P_{i-2} \\ P_{i-1} \\ P_i \end{bmatrix} \qquad (4.17)$$

Výpočet probíhá postupným výpočtem křivky určené nejprve body P_0,P_1,P_2,P_3, poté pro body P_1,P_2,P_3,P_4 atd.

Catmul-Rom spliny jsou navrženy tak, že tečný vektor v bodě P_i je rovnoběžný s vektorem $P_{i-1}-P_{i+1}$ (obr.4.9). Touto vlastností je určen tvar křivky. Dvojnásobnost prvního a posledního bodu (tj. posloupnost $P_0,P_0,P_1,P_2,...,P_{n-1},P_n,P_n$) zaručí, že je křivka bude interpolovat.

Nevýhodou těchto křivek je, že výsledný spline obecně neleží v konvexní obálce svých řídicích bodů (obr.4.9).

APPLET

applet 4.4

4.4 Aproximační křivky

4.4.1 Bézierovy křivky

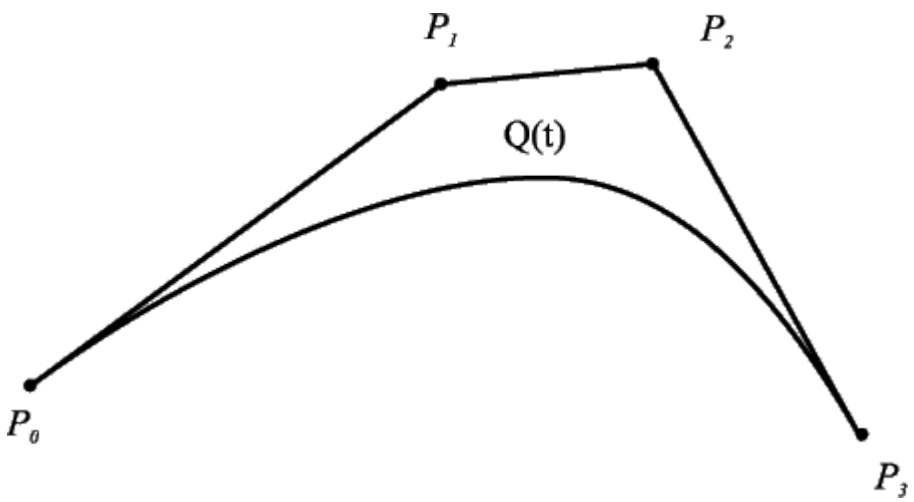
Bézierovy křivky n-tého stupně jsou určeny n+1 body řídicího polygonu a vztahem:

$$P(t) = \sum_{i=0}^{n} P_i B_i^n(t), \qquad (4.18)$$

kde B_i^n jsou **Bernsteinovy polynomy** n-tého stupně:

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i} \; ; t \in \langle 0,1 \rangle \; ; \; i = 0,1,... \; n, \qquad (4.19)$$

kde se definuje $\binom{n}{0} = 1$ a $0^0 = 1$.



Obr 4.10: Bézierova kubika

APPLET

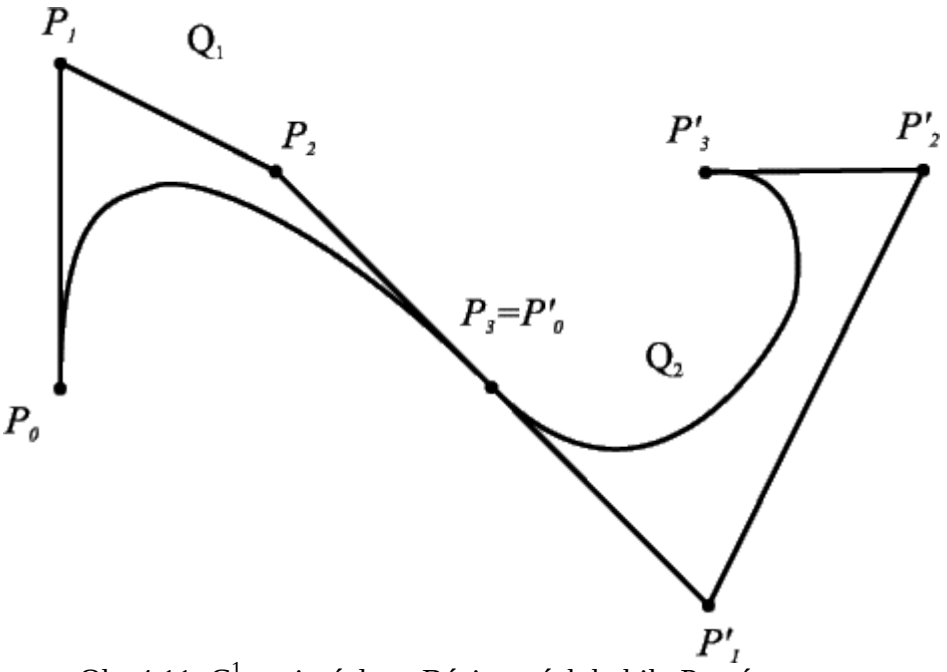
applet 4.5

Položíme-li ve vztahu 4.18 t = 0 resp. t = 1, snadno odvodíme, že křivka prochází prvním resp. posledním bodem řídicího polygonu (viz též obrázek 4.10).

Dosazením za t = 0 a t = 1 do derivace vztahu 4.18 dostaneme výrazy pro tečné vektory v krajních bodech:

$$P'(0) = n(P_1 - P_0),$$
$$P'(1) = n(P_n - P_{n-1}). \qquad (4.20)$$

Při navazování segmentů složených z Bézierových křivek zaručíme spojitost C^0 identitou posledního bodu řídicího polygonu segmentu Q_1 a prvního bodu segmentu Q_2. Spojitost C^1, identita a nenulovost tečných vektorů je garantována (jak je patrné ze vztahu 4.20), je-li bod $P^Q_{1n} = P^Q_{20}$ středem úsečky určené body P^Q_{1n-1} a P^Q_{22}, jak je vidět z obrázku . Spojitost G^1 zajistíme, pokud budou ležet tyto tři body na přímce, bude P^Q_{12} P^Q_{13} P^Q_{21} a bude zachováno jejich pořadí stejně jako na obrázku .



Obr 4.11: C^1 spojení dvou Bézierových kubik. První segment je označen P_0,P_1 až P_3, druhý P_0',P_1' až P_3'.

Bernsteinovy polynomy B_i^n (vztah 4.19) mají následující vlastnosti:

1. $\forall$ i,n $\in$ N$\cup${0} a t $\in$ < 0,1 > je $B_i^n(t) \geq 0$
2. $\sum_{i=0}^{n} B_i^n(t) = 1$ pro t $\in$ < 0,1 >
3. $B_i^n(t) = (1-t)B_i^{n-1}(t) + tB_{i-1}^{n-1}(t)$

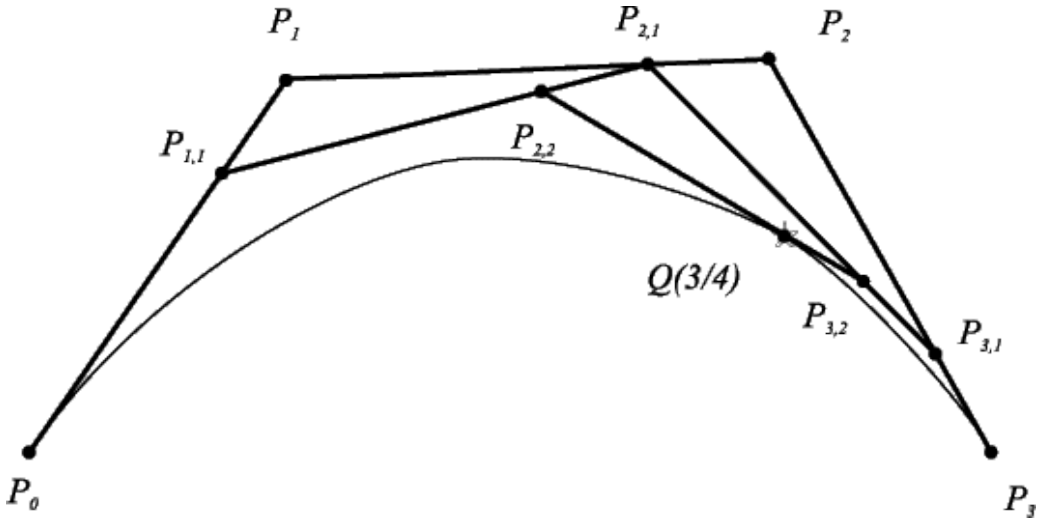
První vztah zaručuje nezápornost Bernsteinových polynomů. Druhý pak zaručuje, že výsledná křivka bude vždy ležet v konvexní obálce bodů řídicího polygonu.

Jinou metodou, jak vypočítat Bézierovu křivku stupně n, je použít rekurzívního **algoritmu de Casteljau**. Bod křivky o souřadnicích Q(t) se vypočítá podle vztahu:

$$P_{j,i}(t) = (1-t)P_{j-1,i-1} + tP_{j,i-1};\ i = 1,2,...,n;\ j = i,i+1,...,n \qquad (4.21)$$

Hodnota posledního kroku výpočtu $P_{n,n}(t)$ je rovna Q(t).

Obrázek ukazuje výpočet bodu Q(3/4).



Obr 4.12: Algoritmus de Casteljau

Zásadní aplikací tohoto algoritmu je **rozdělení Bézierovy křivky** na dvě části. Hodnota P(t), která je vstupem algoritmu, určuje bod, ve kterém dojde k jejímu rozdělení a rekurzívní výpočet postupně generuje nové body řídicích polygonů dvou nových křivek. Na obrázku 4.12 se jedná o polygony P_0,$P_{1,1}$,$P_{2,2}$,P(3/4) a P(3/4),$P_{2,2}$,$P_{2,1}$,P_3.

Další aplikací tohoto algoritmu je efektivní výpočet Bézierovy křivky. Intuitivní výpočet spočívá v generování křivky se zvětšujícím se parametrem t. Tento přístup však vede k nevhodně dlouhým částem křivky, tj. zbytečně řídkému dělení tam, kde je křivka rovná a stačilo by ji aproximovat delšími úsečkami a naopak málo podrobnému dělení u velkých oblouků. Efektivní výpočet spočívá v rekurzívním dělení křivky tak dlouho, dokud není . Tento postup demonstruje algoritmus:

```
INPUT: c - řídicí polygon křivky, e – přesnost
void DrawCurveUsingRecursiveSubdivision(c, e)
{
 if FlatEnough(c,e)  /* je-li splněna podmínka přímosti
                        části křivky */
 {
    DrawLine(Start(c),End(c));  /* namaluj ji jako úsečku */
    return;                     /* a skonči */
 }
 Subdivide(c,left,right);       /* rozděl křivku na levou
                                   a pravou část */
 DrawCurveUsingRecursiveSubdivision(left, e);
 DrawCurveUsingRecursiveSubdivision(right, e);
}
```

Algoritmus 4.1: Rekurzívní výpočet Bézierovy kubiky

Procedura pro výpočet dvou nových řídicích polygonů má podobu algoritmu:

```
void Subdivide(Polygon orig[], Polygon left[], Polygon right[])
{
   left[0]  = orig[0];
   left[1]  = (orig[0]+orig[1])/2.0;
   hlp      = (orig[1]+orig[2])/2.0;
   left[2]  = (left[1]+hlp)/2.0;
   right[3] = orig[3];
   right[2] = (orig[2]+orig[3])/2.0;
   right[1] = (hlp+orig[2])/2.0;
   left[3]  = right[0] = (left[2]+right[1])/2.0;
}
```

Algoritmus 4.2: Algoritmus de Casteljau dělící Bézierovu kubiku na dvě části

APPLET

applet 4.6

Jako podmínku dostatečné hladkosti je možné použít například minimální velikost plochy konvexní obálky.

Často potřebnou operací je zvýšení stupně křivky (to jest přidání bodu k řídicímu polygonu) tak, aby se tvar křivky nezměnil. Mějme $P_0,P_1,\ldots,P_n$, n+1 bodů řídicího polygonu. Nový polygon bude mít n+2 bodů $[\bar{P}]_0,[\bar{P}]_1,\ldots,[\bar{P}]_n,[\bar{P}]_{n+1}$. Aby byla zaručena identita křivek, musí pro body nového polygonu platit:

$$\bar{P}_i = \eta_i P_{i-1} + (1-\eta_i) P_i. \qquad (4.22)$$

kde $\eta_i = [i/(n+1)]$ a $i = 0,1,\ldots,n+1$.

Bézierovy křivky leží v konvexní obálce svého řídicího polygonu. Změna polohy bodu má vliv na změnu tvaru celé křivky a proto se křivky skládají z Bézierových segmentů.

Interpolace pomocí Bézierových křivek se provádí tak, že z interpolovaných bodů a tečných vektorů v nich se vypočítá odpovídající řídicí polygon.

Fergusonova kubika může být popsána jako Bézierova kubika a naopak, jedná se tedy o stejnou množinu křivek.

4.2.2 Bézierovy kubiky

Velmi často používanými křivkami jsou Bézierovy kubiky, a proto jsou uvedeny samostatně.

Bézierova kubika (obr. 4.10) je zadána čtyřmi body P_0,P_1,P_2 a P_3. Vychází z prvního bodu, končí v posledním a je určena vztahem:

$$Q(t) = P_0B_0(t) + P_1B_1(t) + P_2B_2(t) + P_3B_3(t) = \sum_{i=0}^{3} P_iB_i(t), \qquad (4.23)$$

Bernsteinovy polynomy třetího stupně mají tvar:

$$\begin{aligned} B_0(t) &= (1-t)^3, \\ B_1(t) &= 3t(1-t)^2, \\ B_2(t) &= 3t^2(1-t), \\ B_3(t) &= t^3 \end{aligned} \qquad (4.24)$$

a maticový zápis:

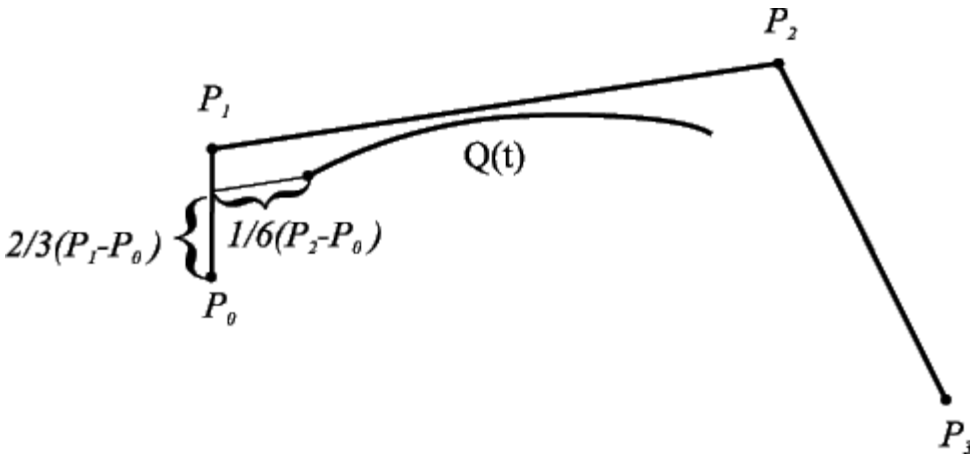
$$Q(t) = T \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \qquad (4.25)$$

Tečné vektory v prvním a posledním bodě mají tvar:

$$P'(0) = 3(P_0 - P_1)$$
$$P'(1) = 3(P_2 - P_3). \qquad (4.26)$$

4.4.3 Coonsovy kubiky

Jinou aproximační metodou jsou Coonsovy kubiky, které mají svůj význam především v teorii spline křivek.



Obr 4.13: Coonsova kubika

Coonsova kubika je určena čtyřmi body P_0,P_1,P_2 a P_3 a vztahem:

$$Q(t) = \frac{1}{6} T \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 0 & 3 & 0 \\ 1 & 4 & 1 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \qquad (4.27)$$

Coonsova kubika obecně **neprochází** svými krajními body. Dosazením t = 0 a t = 1 zjistíme (obr. 4.13), že začíná a končí v bodech:

$$Q(0) = \frac{P_0+4P_1+P_2}{6}$$
$$Q(1) = \frac{P_1+4P_2+P_3}{6} \qquad (4.28)$$

Derivací vztahu 4.27 a dosazením pro t = 0 resp. t = 1 dostaneme vztahy pro tečné vektory v krajních bodech:

$$q'(0) = \frac{P_2 - P_0}{2}$$
$$q'(1) = \frac{P_1 - P_3}{2}. \qquad (4.29)$$

Vektory druhých derivací mají podobu:

$$q''(0) = P_0 - 2P_1 + P_3$$
$$q''(1) = P_1 - 2P_2 + P_3 . \qquad (4.30)$$

Velký význam má **násobnost** bodů řídícího polygonu. Pokud položíme $P_0 = P_1$, vytvoříme **dvojnásobný bod** a ze vztahu 4.13 snadno odvodíme, že křivka začíná v jedné šestině úsečky mezi bodem P_0 a P_2.

Položíme-li $P_0 = P_1 = P_2$, bude výsledná křivka úsečkou, počínající v P_0 a končící v jedné šestině úsečky P_0P_3, Coonsova kubika tedy prochází trojnásobným bodem.

4.4.4 Spline křivky

Spline křivka [3] indexspline stupně n je po částech polynomiální křivka, která je třídy C^n.

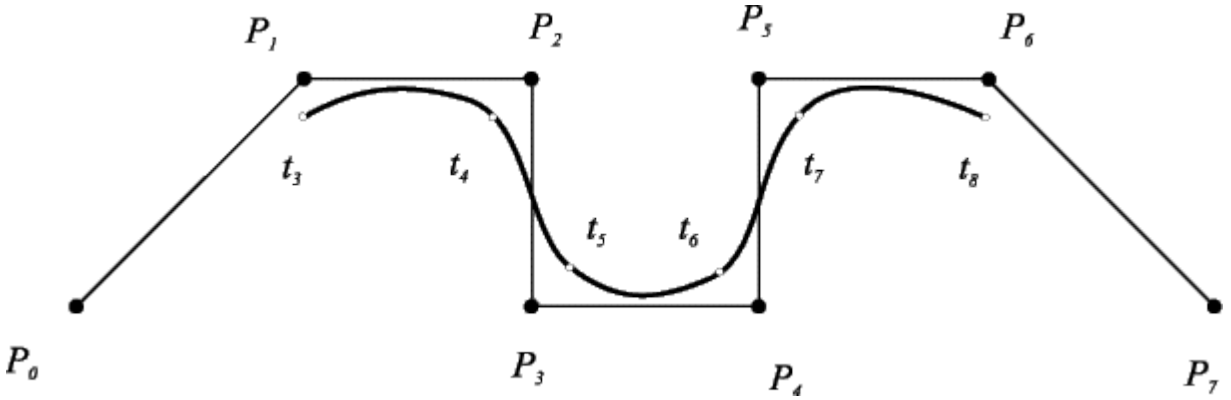
Přirozený spline je spline, který interpoluje své řídící body. Přirozený kubický spline je tedy interpolační křivka skládající se z polynomiálních oblouků stupně tři, která je ve svých uzlech C^2 spojitá. Výpočet přirozeného spline vede k úloze výpočtu inverzní matice k matici (m+1) × (m+1), kde m je počet bodů řídicího polygonu. Další nevýhodou těchto křivek je, že změnou polohy jediného definujícího bodu se změní tvar celé křivky. Výpočet těchto křivek je možno nalézt v [Fole90].

V počítačové grafice jsou nejčastěji používané **B-spline** kubiky.

Nejprve uvedeme nejjednodušší případ, neracionální a uniformní B-spline. Jeho zobecněním je neuniformní racionální B-spline, neboli **NURBS**.

Uniformní neracionální kubický B-spline

Uniformní neracionální kubický B-spline se také nazývá **kubický Coonsův B-spline**. Vznikne navázáním Coonsových kubik (4.27) takto: segment Q_i je určen body P_{i-3},P_{i-2},P_{i-1} a P_i. Následující segment Q_{i+1} je složen z bodů P_{i-2},P_{i-1},P_i a P_{i+1}, tedy ze tří posledních bodů segmentu Q_i a z jednoho bodu nového atd.

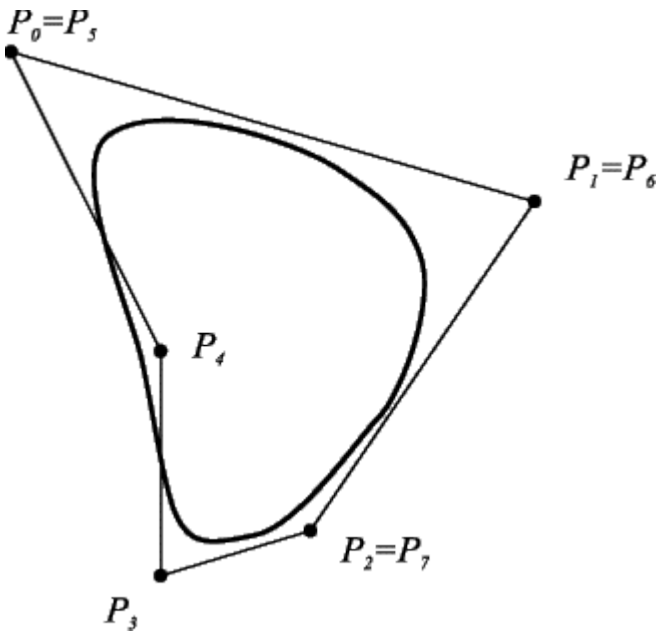


Obr 4.14: Coonsův kubický B-spline

Coonsův B-spline je určen n+1 body.

Porovnáme-li vztahy pro tečné vektory a vektory druhých derivací dvou po sobě následujících segmentů Q_i a Q_{i+1} (viz 4.28, 4.29 a 4.30), zjistíme, že segment Q_{i+1} vychází z posledního bodu segmentu Q_i a že jsou identické první a druhé derivace v tomto bodě. Křivka je tedy v uzlech C_2 spojitá.

Využijeme-li chování Coonsových kubik při násobnosti bodů, tak posloupností P_0, P_0, P_0, P_1,...,P_{n-1},P_n,P_n,P_n zajistíme, že výsledný kubický spline bude procházet krajní body svého řídicího polygonu, ovšem za tu cenu, že první segment bude úsečkou spojující bod P_0 s bodem ležícím na jedné šestině vzdálenosti P_0P_1 a analogicky na konci řídicího polygonu.



Obr 4.15: Uzavřený spline

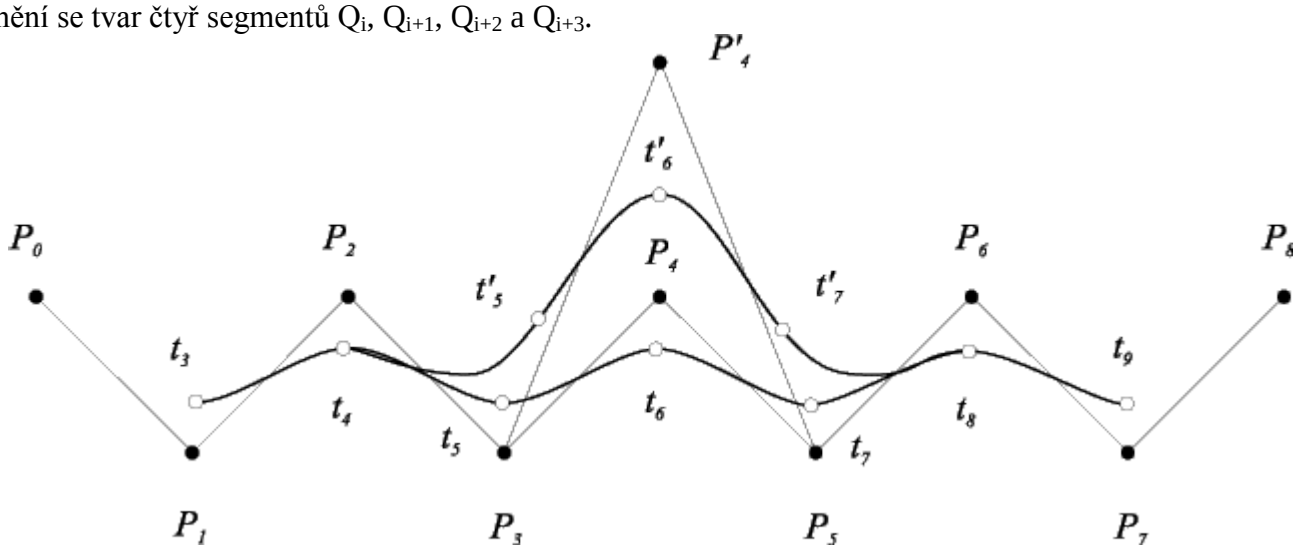
Uzavřený B-spline (obr. 4.15) získáme opakováním n prvních bodů řídicího polygonu na jeho konci. Pro kubický B-spline je tedy řídícím polygonem uzavřené křivky posloupnost bodů: $P_0,P_1,...,P_m,P_0,P_1,P_2$.

Coonsův kubický B-spline generovaný n+1 body řídicího polygonu $P_0,P_1,... P_n$, je složen z n-2 segmentů $Q_3,Q_4,...,Q_n$ (obr. 4.14). Parametr t probíhá interval $< 0,n+1 >$ a hodnoty parametru t v uzlech (označujeme je t_i) definují **uzlový vektor** (angl. *knot vector*). Tyto hodnoty mají konstantní vzdálenost $t_{i+1} - t_i = k$, uzlový vektor je **uniformní** (odtud i název těchto křivek).

Abychom zaručili, že pro segment Q_i je parametr t v intervalu $< 0,1)$, provedeme pro příslušný segment změnu parametrizace $t^* = t - t_i$. Pomocí vektoru $T = [(t-t_i)^3, (t-t_i)^2, (t-t_i), 1]$ a geometrického vektoru $G = [P_{i-3},P_{i-2},P_{i-1},P_i]^T$ můžeme potom pro výpočet neracionálního uniformního B-splinu použít vztah 4.27.

Mezi důležité vlastnosti B-spline křivek patří invariance vůči rotaci, translaci a změně měřítka, B-spline křivka leží celá ve své konvexní obálce a její segmenty leží v konvexních obálkách svých řídicích polygonů. Zvláště poslední vlastnost nám umožní pochopit následující text

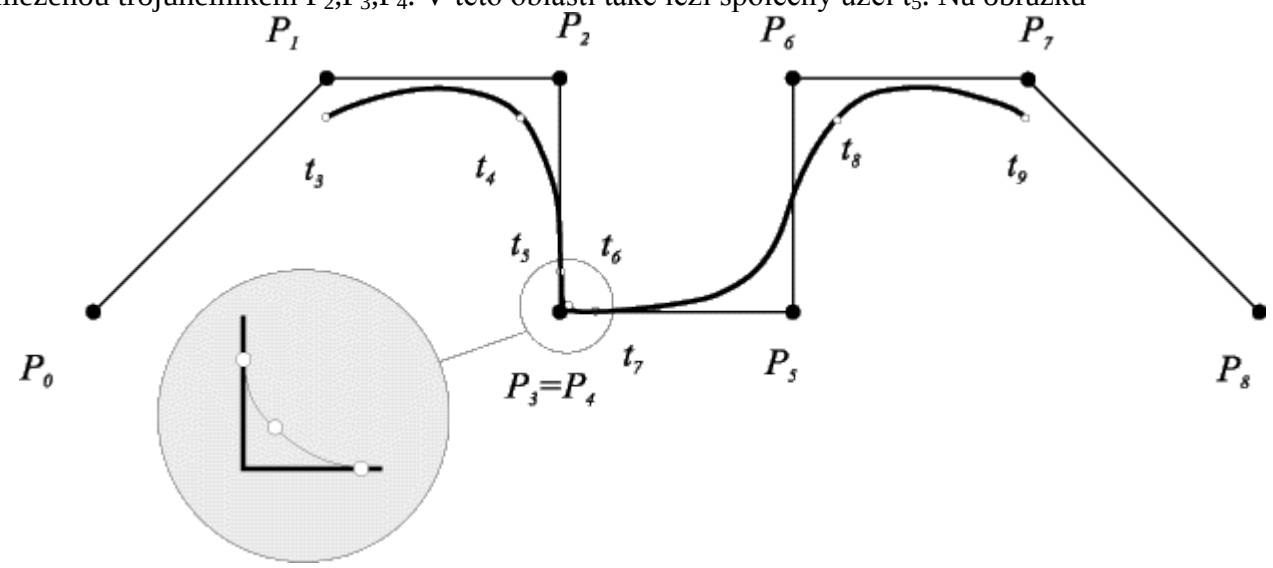
Lokalita změny znamená, že pokud posuneme některý z řídících bodů, změní se tvar pouze v té části Coonsova B-splinu, který je tímto bodem generován. Pokud posuneme bod P_i kubiky Q(t), změní se tvar čtyř segmentů Q_i, Q_{i+1}, Q_{i+2} a Q_{i+3}.



Obr 4.16: Lokalita změny tvaru křivky při změně polohy bodu řídicího polygonu. Uzlové vektory jsou označeny prázdnými kolečky.

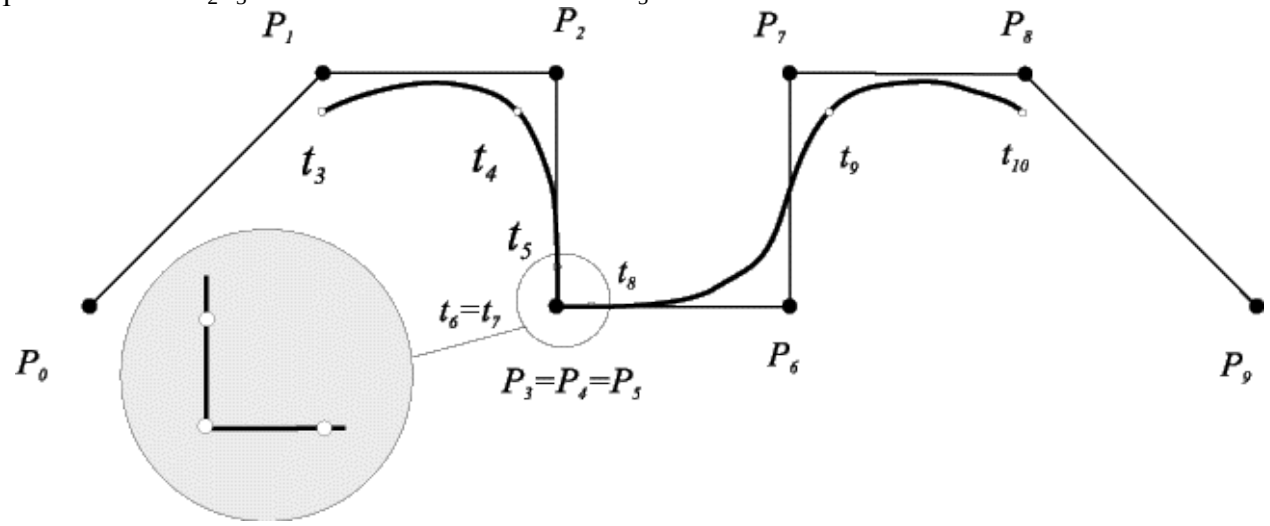
Na obrázku 4.16 jsou dvě křivky. První je určena řídicím polygonem $P_0,P_1,...,P_8$. Druhá se od první liší pouze tím, že bod P_4 je posunut do polohy P'_4. Uzly jsou označeny t_3 až t_9. Je patrné, že změnou polohy jediného bodu se změnila také poloha uzlů t_5 až t_7. Vždy dva uzly určují hranice segmentu, došlo tedy ke změně tvaru čtyř segmentů.

Díky tomu, že segment leží v konvexní obálce svého řídicího polygonu, je možné snadno pochopit co se stane, je-li některý bod řídicího polygonu vícenásobný. Podívejme se na obrázek 4.14 a sledujme, co se bude dít, budeme-li zvyšovat násobnost bodu P_3. Segment Q_4 je určen uzly t_4 a t_5 a řídicím polygonem P_1,P_2,P_3,P_4 a segment Q_5 uzly t_5 a t_6 a řídicím polygonem P_2,P_3,P_4,P_5. V případě, kdy není žádný bod několikanásobný, jsou všechny řídicí polygony čtyřúhelníky a konvexní obálky dvou po sobě následujících segmentů, pro náš případ řídicí polygony segmentů Q_4 a Q_5 mají společnou plochu vymezenou trojúhelníkem P_2,P_3,P_4. V této oblasti také leží společný uzel t_5. Na obrázku



Obr 4.17: Dvojnásobný bod

zdegenerovaly čtyřúhelníky P_1,P_2,P_3,P_4 a P_2,P_3,P_4,P_5 na trojúhelníky tím, že překrývající se oblastí je společná hrana P_2P_3. Na této hraně musí leží uzel t_5.



Obr 4.18: Trojnásobný bod

Na obrázku 4.18 je bod P_3 trojnásobným bodem, konvexní obálka P_2 až P_5 degeneruje na úsečku P_2P_3, stejně jako segment Q_5 degeneruje na úsečku ležící na P_2P_3.

NURBS

Neuniformní racionální B-spline křivky jsou dvojím zobecněním B-spline křivek uvedených v předchozí kapitole. Termín **neuniformní** je odvozen od vzdálenosti uzlů, která nemusí u těchto křivek

být konstantní. **Racionalita** pak znamená, že body jsou reprezentovány jejich **homogenními souřadnicemi** (viz vztah) [Polá91].

NURBS křivka je určena n+1 body řídicího polygonu, uzlovým vektorem U délky m+1 a stupněm křivky p. Uzlový vektor pro NURBS křivku procházející prvním a posledním bodem řídicího polygonu má tvar:

$$U = \{0,0,...0,t_{p+1},...,t_{m-p-1},\beta,\beta,...,\beta\}. \tag{4.31}$$

Hodnoty uzlů musí být neklesající, tedy: $t_{i+1} \geq t_i$. Sekvence nul na začátku, stejně jako sekvence posledních hodnot (bývá zvykem volit $\beta = 1$) má násobnost p+1. Zbývajících hodnot je m+1. Mezi počtem uzlů m+1, stupněm p křivky a počtem bodů řídicího polygonu n+1 platí vztah:

$$m = n+p+1.$$

NURBS křivka je potom určena vztahem:

$$Q(t) = \frac{\sum_{i=0}^{n} w_i P_i N_{i,p}(t)}{\sum_{i=0}^{n} w_i N_{i,p}(t)}, \tag{4.32}$$

kde w_i je váha i-tého bodu řídicího polygonu a $N_{i,p}(t)$ jsou **normalizované B-spline bázové funkce** určené rekurzívním vztahem:

$$N_{i,0}(t) = \begin{cases} 1 & \text{pro } t_i \leq t < t_{i+1} \\ 0 & \text{jinde} \end{cases}$$

$$N_{i,p}(t) = \frac{(t-t_i)}{t_{i+p}-t_i} N_{i,p-1}(t) + \frac{(t_{i+p+1}-t)}{t_{i+p+1}-t_{i+1}} N_{i+1,p-1}(t). \tag{4.33}$$

Během výpočtu se vyskytují i výrazy, které mají ve jmenovateli nulu. Jejich hodnota je definitoricky položena rovna nule.

Jiný zápis využívá **racionální B-spline báze**:

$$R_{i,p} = \frac{w_i N_{i,p}(t)}{\sum_{j=0}^{n} w_j N_{j,p}(t)} \tag{4.34}$$

Rovnici 4.32 lze potom zapsat jednodušeji:

$$Q(t) = \sum_{i=0}^{n} P_i R_{i,p}(t) \tag{4.35}$$

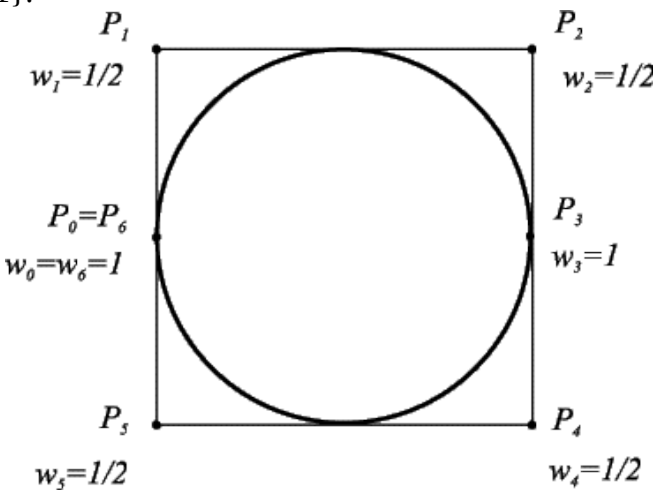
První derivace NURBS křivky má tvar ([Pieg91]):

$$q'(t) = \sum_{i=0}^{n} w_i P_i \frac{t}{t_{i+p}-u_i} N_{i,p-1}(t) + \sum_{i=0}^{n} w_i P_i \frac{t}{t_{i+p+1}-u_{i+1}} N_{i+1,p-1}(t). \qquad (4.36)$$

Základní vlastnosti NURBS křivek jsou:

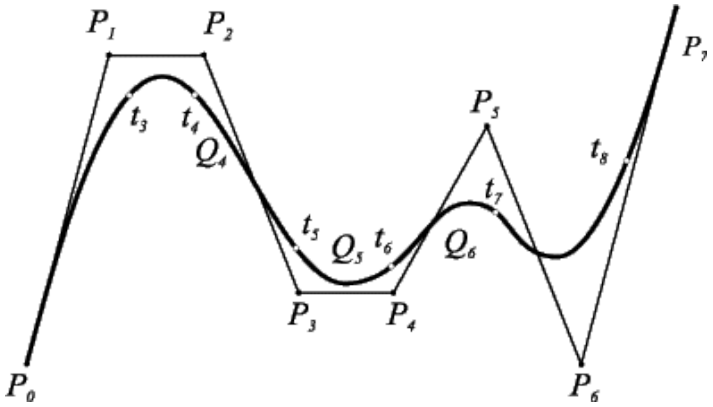
1. Při zadání uzlového vektoru podle vztahu 4.31 **procházejí prvním a posledním bodem řídicího polygonu**.
2. **Leží v konvexní obálce** svého řídicího polygonu, stejně jako jejich jednotlivé **segmenty leží v obálkách** svých řídicích polygonů.
3. Jsou **invariantní k rotaci, translaci, změně měřítka** a navíc i k **paralelnímu a středovému promítání**.
4. Umožňují **přesné vyjádření kuželoseček**.
5. Pro uzlový vektor U = {0,0,...,0,1,1,...,1} a pro stejný počet nul a jedniček jsou normalizované B-spline báze rovny Bernsteinovým polynomům a NURBS je **racionální Bézierovou křivkou**. Pro hodnoty $w_i = 1$ je NURBS Bézierovou křivkou tak jak bylo uvedeno v odstavci 4.4.1.

Zejména čtvrtá vlastnost je důležitá pro tvorbu jednotných datových struktur, které reprezentují jak tvary, jako jsou kružnice či parabola, tak volné (angl. *free-form*). Nevýhodou je, že reprezentace objektů není jednoduchá. Tak například kružnice je reprezentována sedmi body uzlovým vektorem U = {0,0,0,$^1/_4$,$^1/_2$,$^1/_2$,$^3/_4$,1,1,1}.



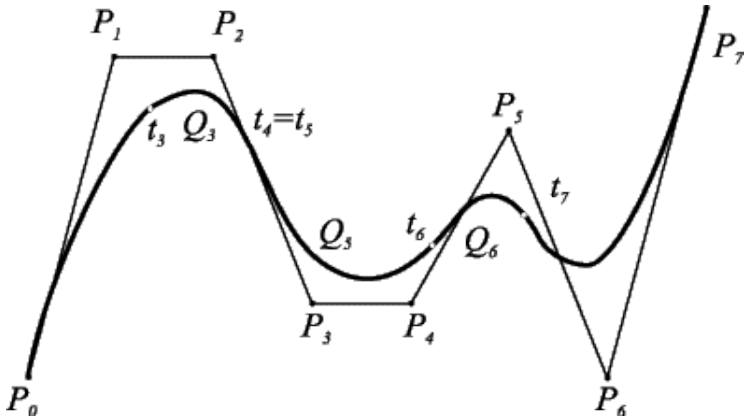
Obr 4.19: Kružnice definovaná jako NURBS

Jak bylo výše uvedeno, posloupnost uzlů musí být neklesající. Je tedy z principu možné, aby dva uzly měly stejnou hodnotu - hovoříme o **násobnosti uzlů**. Násobnost uzlu znamená zdegenerování segmentu do jediného bodu.



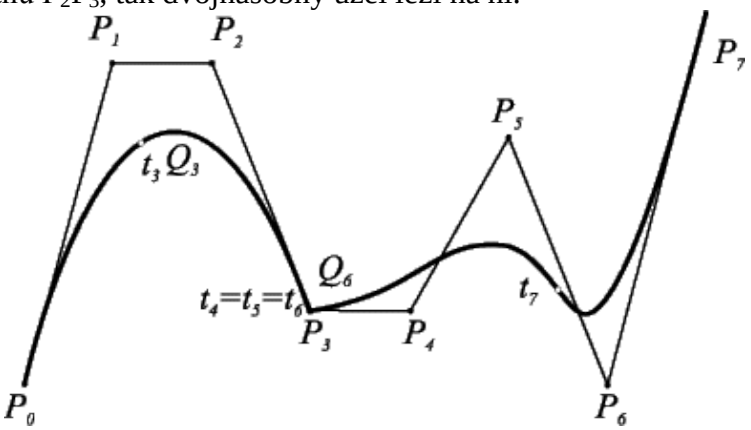
Obr 4.20: NURBS

Na obrázku 4.20 je NURBS křivka určená osmi body $P_0,...,P_7$ a uzlovým vektorem U = {0,0,0,0,1,2,3,4,4,4,4}, ze kterého je patrné, že se jedná o uniformně parametrizovaný kubický spline (p = 3,n = 7,m = 11).



Obr 4.21: Dvojnásobný uzel NURBS křivky

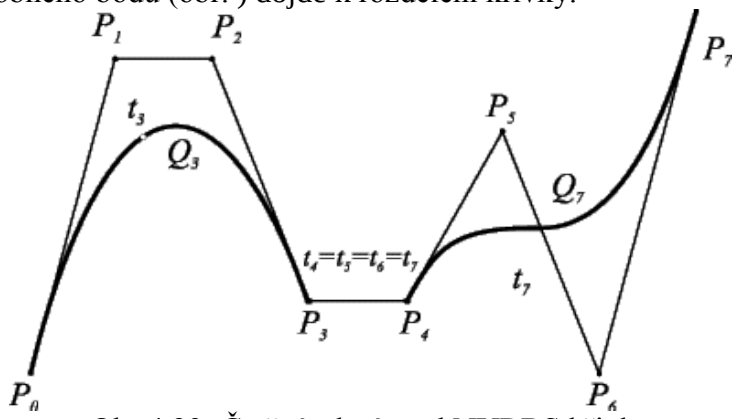
Na dalším obrázku 4.21 má uzlový vektor tvar U = {0,0,0,0,1,2,2,3,3,3,3}. Segment Q_4 degeneruje do jediného bodu $Q(t_4)$ (na obrázku 4.21 je označen jako t_4). Segment Q_3 leží v obálce bodů P_0,P_1,P_2,P_3 a Q_5 v obálce P_2,P_3,P_4,P_5. Oba segmenty se stýkají v jediném bodě - segmentu Q_4. Obálky segmentů Q_3 a Q_5 mají společnou hranu P_2P_3, tak dvojnásobný uzel leží na ní.



Obr 4.22: Trojnásobný uzel NURBS křivky

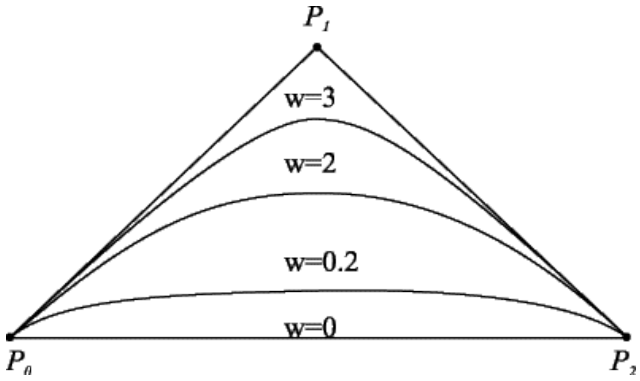
Analogická situace je v případě trojnásobného bodu na obr. 4.22. Nyní se setkávají dva segmenty Q_3 a Q_6 ve společném bodě, do kterého jsou zdegenerovány dva segmenty Q_4 a Q_5. Konvexní obálky obou segmentů P_0,P_1,P_2,P_3 a P_3,P_4,P_5,P_5 mají společný jediný bod P_3, ve kterém oba segmenty leží.

V případě čtyřnásobného bodu (obr.) dojde k rozdělení křivky.



Obr 4.23: Čtyřnásobný uzel NURBS křivky

Obrázek ukazuje vliv váhy bodu P_1 na tvar křivky. Pro hodnotu w = 0 neovlivňuje bod křivku žádným způsobem (srovnej vztah 4.32). Váha bodu w = 1 odpovídá případu neracionální křivky a se zvyšující se váhou se křivka k bodu přimyká, až konečně pro hodnotu w = … křivka bodem prochází a dochází ke ztrátě spojitosti. Podmínka nezápornosti vah garantuje umístění křivky v konvexní obálce.



Obr 4.24: Vliv váhy bodu na racionální B-spline

Důležitou operací je přidání uzlu ke křivce (a tedy i bodu řídicího polygonu). Mějme polygon určený body $P_0,P_1,...,P_n$ a uzlový vektor U. Mějme dále hodnotu nového uzlu $t = t'$. Algoritmus **de Boor** generuje body $Q_0 = P_0,Q_1,...,Q_{n+1} = P_n$

$$Q_i = (1-a_i)P_{i-1}+a_iP_i,\ 1 \le i \le n \qquad (4.37)$$

a a_i je dáno:

$$a_i = 1,\ 1 \le i \le j-3$$

$$a_i = \frac{t'-t_i}{t_{i+3}-t_i},\ j-2 \le i \le j \qquad (4.38)$$

$$a_i = 0,\ j+1 \le i \le n$$

Tento algoritmus je zvláštním případem tzv. **Oslo** algoritmu ([Pieg91]), který vkládá více nových uzlů najednou.

APPLET

applet 4.7

- [3] Termín spline pochází od pružného pravítka - křivítka, které tyto křivky modeluje.

Literatura:

- [Bars88] Barsky, B.: Computer graphics and geometric modeling using beta-splines. Springer-Verlag, 1988
- [Drs90] Drs, L.: Matematické metody v počítačové grafice, CVUT, Praha, 1990.
- [Fole90] Foley, J., van Dam, A., Feiner, S., Hughes, J.: Computer Graphics. Principles and Practice. Second Edition. Addison-Wesley Publ. Comp., Reading Massachusets, 1990.
- [Pieg91] Piegl,L.:On NURBS:A Survey. IEEE Computer Graphics and Applications, January 1991.
- [Polá91] Poláček, J., Ježek, F., Kopincová, E.: Počítačová grafika. ČVUT, Praha 1991.

5. Ořezávání

Ořezávací algoritmy umožňují vybrat z rozsáhlé kresby pouze ty části, které jsou viditelné ve stanoveném okně - výřezu obrazové plochy. Protože rozsah zobrazitelných souřadnic grafického zařízení je omezený a bývá podstatně menší, než rozsah souřadnic použitých v aplikační úloze, mohou některé části kresby ležet mimo zobrazovací plochu. V tomto případě je nutné provést ořezání kresby buď na úrovni vyššího programového vybavení (do zařízení jsou zaslány instrukce ořezané kresby), nebo zařízení provede ořezání vlastními technickými nebo programovými prostředky. Od začátku 80. let se při dialogu běžně používají virtuální obrazovky - okna. Ořezávací algoritmy umožňují jejich jednoduché používání v grafickém režimu výstupu. Další oblastí, ve které se používají ořezávací algoritmy, je řešení viditelnosti 3D scény. Zde hrají důležitou roli algoritmy pro ořezávání polygonů, které umožňují korektně vymezit části polygonů viditelné v okně ve formě uzavřených polygonálních tahů.

Algoritmy uvedené v této kapitole řeší ořezávání úseček v konvexním okně, jehož hranice je tvořena úsečkami. V praxi bývají čárové prvky často aproximovány úsečkami a uvedené algoritmy se tedy používají i při kresbě kruhových nebo křivkových elementů.

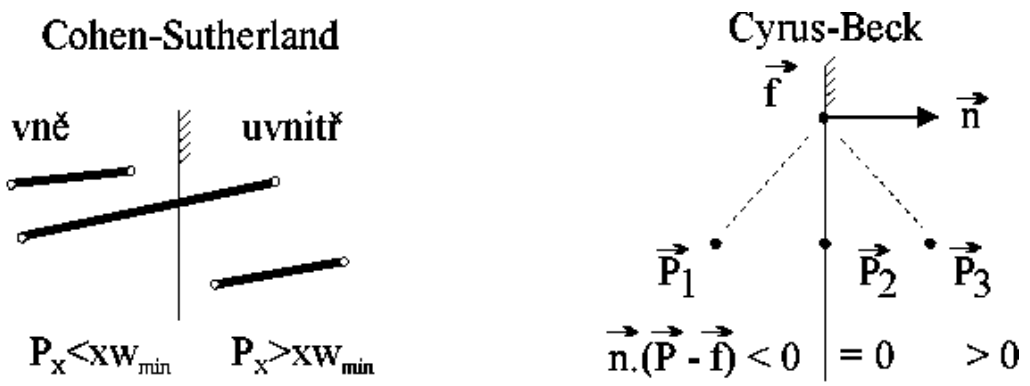
Weiler-Athertonův algoritmus je příkladem algoritmu, který řeší ořezávání nekonvexního polygonu v nekonvexním polygonálním okně.

5.1 Pozice bodu vzhledem k oknu

Při ořezávání se používá test, který zjišťuje polohu bodu vůči hraniční přímce. Výsledkem testu je určení, zda bod leží ve vnitřní nebo vnější polorovině vymezené hraniční přímkou. Osově orientované okno umožňuje použít jednoduché testy porovnáním příslušné souřadnice s hraniční hodnotou. Tento způsob používá algoritmus Cohen-Sutherland. U obecně orientované hranice je vhodné použít rovnici hraniční přímky ve tvaru $F(x,y) = a.x + b.y + c = 0$ a testovat pozici bodu $P = [xp, yp]$ pomocí testu $F(xp,yp) > 0, = 0, < 0$. Tento test lze formálními úpravami převést na test ve vektorovém tvaru, který je použit např. u algoritmu Cyrus-Beck.

U testu ve vektorovém tvaru prochází obecně orientovaná hranice bodem, který je určen pozičním vektorem f. Hraniční přímka je kolmá na normálový vektor n, který směřuje dovnitř okna. P je poziční vektor testovaného bodu.

Oba druhy testu jsou uvedeny na obr. 5.1.



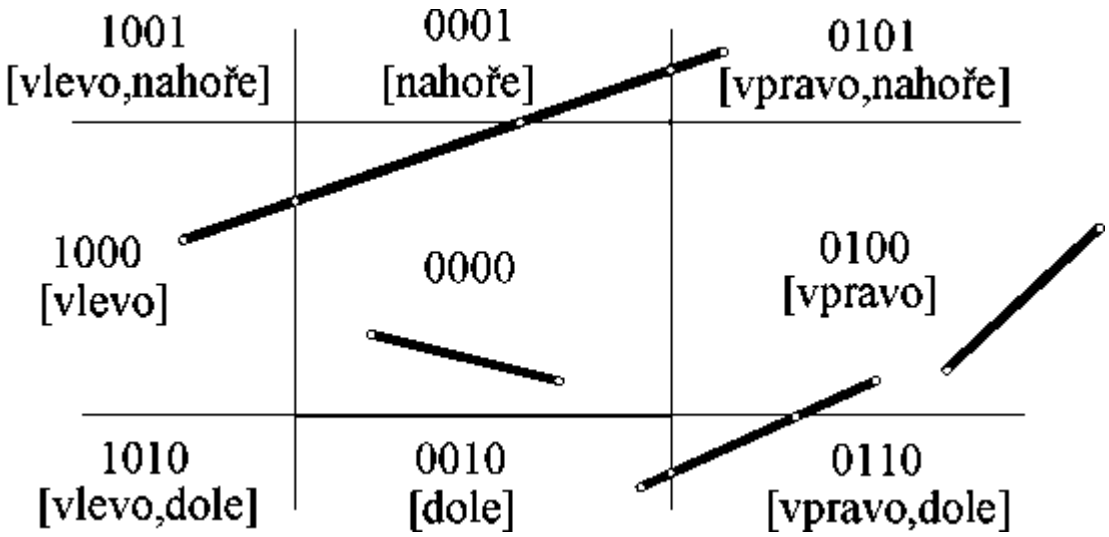
obr. 5.1: Test pozice bodu vzhledem k hranici

5.2 Ořezávaní úseček

5.2.1 Hraniční kódy - algoritmus Cohen-Sutherland

Pro ořezání úseček oknem, jehož hrany jsou rovnoběžné s osami souřadného systému, se v praxi používá několik algoritmů.

Cohen-Sutherlandův algoritmus (alg. 5.1) označuje krajní body úseček pomocí hraničních kódů (kódů oblastí) (obr. 5.2).



obr. 5.2: Kódy 9 oblastí určených okénkem

Kód bodu je určen pomocí testů jeho polohy vzhledem ke hranicím a je vyjádřen jako bitový kód nebo symbolická množina. Nechť KodA a KodB jsou kódy krajních bodů A a B ořezávané úsečky. Pokud jsou oba kódy prázdné množiny (KodA $\cup$ KodB = 0), pak celá úsečka leží uvnitř okna. Pokud je průnik obou množin neprázdný (KodA $\cap$ KodB $\neq$ 0), pak celá úsečka leží vně. Průsečíky vypočítává algoritmus násobením a dělením, pro implementaci vyžaduje použití aritmetiky necelých čísel.

V popisu alg. 5.1 je naznačena sekvenční implementace -- testy kódů -- střih podle hranice - testy kódů -- střih podle hranice...

```
void Orezani(float *xa, float *ya, float *xb, float *yb) {
  float x,y;
  KodA=KodBodu(xa,ya);
  KodB=KodBodu(xb,yb);                /* obr. 5.2 */
  /* vylučovací testy podle kódů */
  if ((KodA | KodB)==0) return(true)         /* úsečka celá uvnitř */
  else
    if ((KodA & KodB)!=0) return(false);     /* úsečka celá vně */
  if (((KodA | KodB) & 0001₂)) != 0) {       /* úsečka protíná horní hranici */
    x=xa+(ywmax-ya)*(xb-xa)/(yb-ya);
    if ((KodA & 0001₂)) != 0) { xa=x; ya=ywmax; KodA=KodBodu(xa,ya); }
    else
    { xb=x; yb=ywmax; KodB=KodBodu(xb,yb); }
    ... testy kódů
  }
  /* dolní hranice - kód 0010₂ */
  ... test na protínání a výpočet průsečíku
  ...testy kódů
  /* pravá hranice - kód 0100₂ */
  ... test na protínání a výpočet průsečíku
  ... testy kódů
  /* levá hranice - kód 1000₂ */
  ... test na protínání a výpočet průsečíku
  ... testy kódů
}
```

Algoritmus 5.1: Ořezávání úsečky - algoritmus Cohen-Sutherland

5.2.2 Parametrické ořezávání úseček - algoritmus Cyrus-Beck

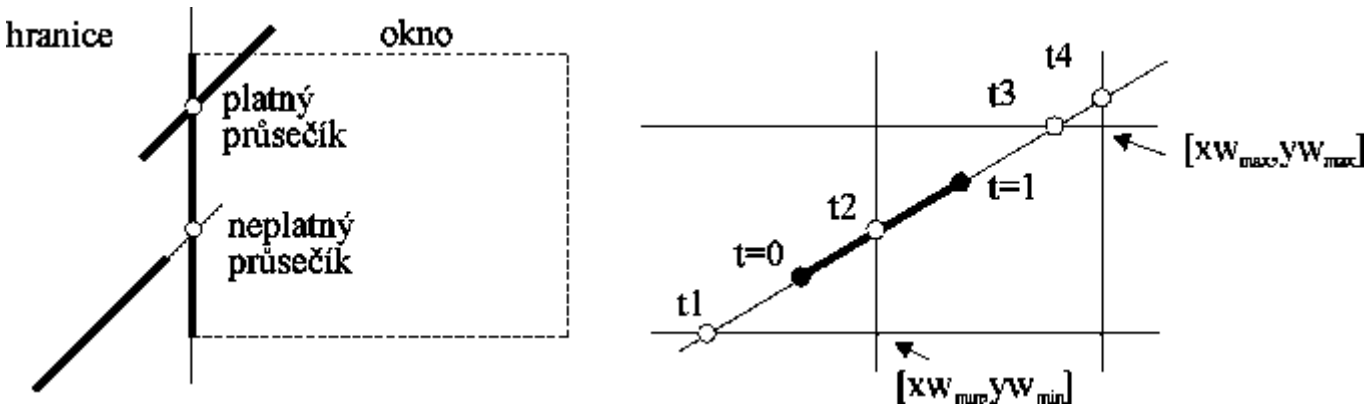
Algoritmus vychází z parametrické rovnice přímky $P(t) = P_1 \mid (P_2 - P_1)\cdot t$, procházející body P_1 a P_2. Pro $t \in < 0, 1 >$ je touto rovnicí definována úsečka P_1P_2. Při ořezávání potřebujeme určit, které body přímky P(t) (úsečky P_1P_2) leží vně a uvnitř hranice konvexní oblasti. C-B algoritmus používá vektorový test s vnitřním normálovým vektorem hranice, který je uveden na obr. 5.1. Pomocí skalárního součinu vektorů [P(t) - f] a n rozlišíme 3 případy:

$n\cdot[P(t) - f] < 0$ odpovídá bodu P_1, kdy vektor [P(t) - f] směřuje z bodu f ven z oblasti
$n\cdot[P(t) - f] = 0$ odpovídá bodu P_2, vektor [P(t) - f] je rovnoběžný s hranicí
$n\cdot[P(t) - f] > 0$ odpovídá bodu P_3, vektor [P(t) - f] směřuje z bodu f dovnitř oblasti.

Po dosazení za P(t) vypočteme průsečík pomocí rovnice $n\cdot[P_1 \mid (P_2 - P_1)\cdot t - f] = 0$
Po úpravě

$$t = -\frac{n\cdot(P_1 - f)}{n\cdot(P_2 - P_1)}$$

Označíme $w = P_1 - f$ jako váhový faktor a $D = P_2 - P_1$ jako směrový vektor. C-B algoritmus využívá dva rozhodovací členy. Pokud je skalární součin $D\cdot n_i \neq 0$, pak existuje průsečík s hraniční přímkou i-té hranice. V tomto případě je vypočten průsečík (parametr t) a následnými testy je tento průsečík přijat nebo odmítnut. Aby bylo možné nalézt koncové body ořezané úsečky při libovolném pořadí hraničních přímek, zahrnuje C-B algoritmus nalezení minimálního (t_l) a maximálního (t_u) parametru. V příkladu uvedeném na obr. 5.3 jsou vyloučeny vypočtené průsečíky t_3, t_4, zůstává koncový bod s t=1, v druhém směru je vyloučen průsečík t_1 a původní koncový bod t=0. Výsledkem je úsečka $P(t_2)P(1)$. Výpočetní postup je zachycen v alg.5.2.



obr. 5.3: Vyloučení neplatných průsečíků

```
Inicializace: tl = 0, tu = 1, D = P(1) - P(0)
Pro všechny hranice i s vnitřní normálou ni dělej:
  Pokud D·ni≠0
    pak t = - (wi·ni) / (D·ni);
      Pokud (D·ni > 0)& (t <= 1)
        pak tl = max(t, tl) /* oprav průsečík tl */
        jinak
      Pokud (D·ni < 0)&(t >= 0)
        pak tu = min(t, tu); /* oprav průsečík tu */
    jinak
      Pokud wi.ni < 0
        pak Ukonči výpočet; /* úsečka je bodem vně okna */
Pokud tl < tu
  pak OřezanáÚsečka(P(tl), P(tu)); /* jinak leží mimo okno */
```

Algoritmus 5.2: Parametrické ořezávání úsečky - alg. Cyrus-Beck

5.2.3 Parametrické ořezávání úseček - algoritmus Liang-Barsky

Alternativní řešení parametrického ořezávání představuje algoritmus Liang-Barsky. Tento algoritmus ořezává úsečky podle osově orientovaného obdélníkového okna. Rovnici úsečky mezi body (x_1, y_1) a (x_2, y_2) můžeme zapsat v parametrickém tvaru

$$x = x_1 \mid u\Delta x$$

$$y = y_1 \mid u\Delta y$$

kde $\Delta x = x_2 - x_1$ a $\Delta y = y_2 - y_1$. Parametr u nabývá hodnot od 0 do 1, souřadnice (x, y) představují body určené hodnotou u v tomto rozsahu. Pro u = 0 je $(x, y) = (x_1, y_1)$. Na druhém konci úsečky je u = 1 a $(x, y) = (x_2, y_2)$. Pokud bod (x, y) na úsečce leží uvnitř okénka určeného souřadnicemi (xw_{min}, yw_{min}) a (xw_{max}, yw_{max}), pak jsou splněny následující podmínky:

$$xw_{min} \le x_1 \mid u\Delta x \le xw_{max}$$

$$yw_{min} \le y_1 \mid u\Delta y \le yw_{max}$$

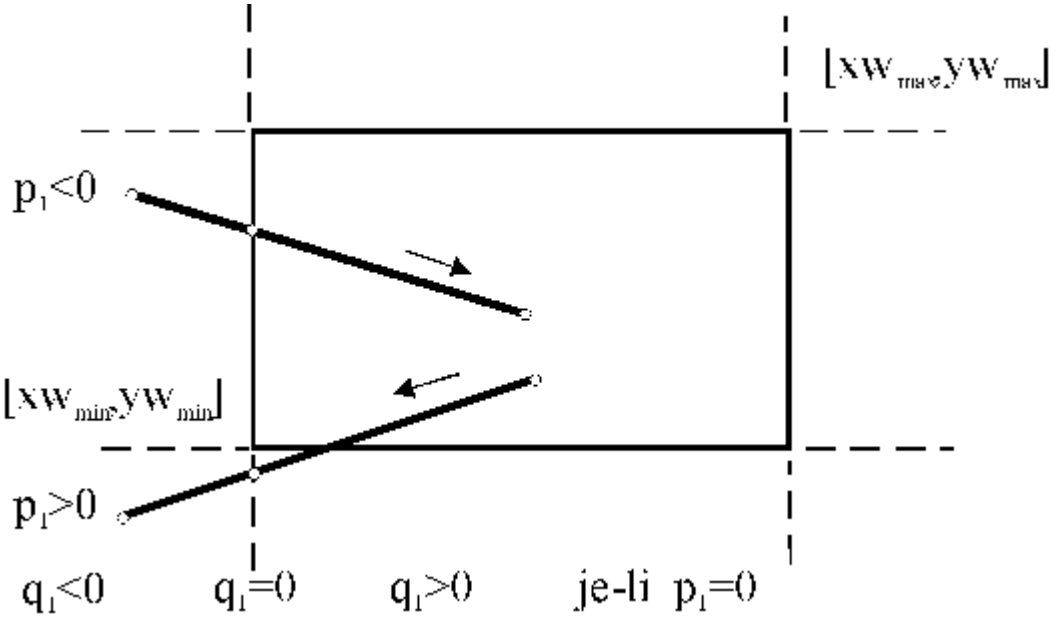
Tyto čtyři nerovnosti mohou být přepsány do tvaru

$$up_k \le q_k,\ k = 1, 2, 3, 4$$

v němž p_k a q_k jsou definovány jako

$p_1 = -\Delta x$	$q_1 = x_1 - xw_{min}$
$p_2 = \Delta x$	$q_2 = xw_{max} - x_1$
$p_3 = -\Delta y$	$q_3 = y_1 - yw_{min}$
$p_4 = \Delta y$	$q_4 = yw_{max} - y_1$

Geometrický význam hodnot p_k a q_k přibližuje obr. 5.4.



obr. 5.4: Orientované úsečky a jejich hodnoty p_1 a q_1

Každá úsečka, která je rovnoběžná s jednou z hraničních přímek, má $p_{k = 0}$ pro hodnotu k odpovídající této hranici (k = 1, 2, 3, 4) označuje po řadě levou, pravou, dolní a horní hranici). Pokud pro tuto hodnotu k je ještě $q_k < 0$, pak úsečka leží zcela mimo okénko a může být z dalších výpočtů vyloučena. Je-li $q_k \ge 0$, rovnoběžná úsečka leží uvnitř okénka. Znaménko p_k vyjadřuje směr kresby úsečky vzhledem k hranici k. Pro nenulové hodnoty p_k můžeme vypočítat velikost parametru u, která odpovídá průsečíku přímky, na které leží testovaná úsečka, s prodlouženou hraniční přímkou k jako

$$u = q_k / p_k$$

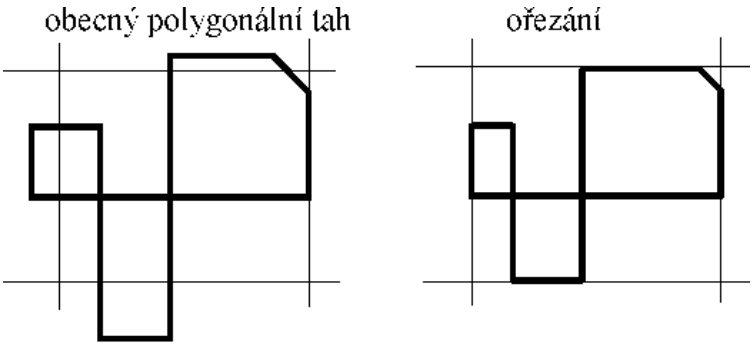
Pro každou úsečku lze spočítat hodnoty parametrů u_1 a u_2, které tak určují tu část úsečky, která leží uvnitř okénka. Hodnota u_1 je stanovena podle těch hranic, u kterých ořezávaná úsečka prochází z vnějšku dovnitř $p < 0$. Pro tyto hranice vypočítáme $r_k = q_k / p_k$. Jako hodnotu u_1 zvolíme maximum z množiny,

která obsahuje 0 a různé hodnoty r. Podobně získáme velikost u_2 z hranic, u kterých úsečka prochází zevnitř ven ($p > 0$). Hodnoty r_k jsou počítány pro každou takovou hraniční úsečku a hodnota u_2 je potom minimem z množiny, která obsahuje 1 a vypočítané hodnoty r. Každá úprava hodnot u_1 a u_2 vyžaduje pouze jedno dělení. Je-li $u_1 > u_2$, úsečka leží zcela mimo okénko a může být vyloučena. V opačném případě stanovíme v závěru algoritmu koncové body ořezané úsečky z změněných hodnot $u_1(> 0)$ a $u_2(< 1)$. Postup výpočtu je popsán v alg. 5.3.

```
int TestOrezani (float p, float q, float *u1, float *u2)
{ float r;
  if (p < 0)                                    /* úsečka prochází z vnějšku
dovnitř hranice */
  { r = q / p;
    if (r > *u2) return(0)                      /* úsečka zcela ořezána */
    else if (r > *u1) *u1 = r;                  /* aktualizace u1 */
  }
  else if (p > 0)                               /* úsečka prochází z vnitřku
hranice ven */
    { r = q / p;
      if (r < *u1) return(0)                    /* úsečka zcela ořezána */
      else if (r < *u2) \{ *u2 = r \}           /* aktualizace u2 */
    }
    else                                        /* p = 0, úsečka je rovnoběžná
s hranicí */
    if (q < 0) return(0);                       /* celá úsečka vně okna */
  return(1);
}
void ClipLiangBarsky (float *x1, float *y1, float *x2, float *y2)
{ float u1, u2, dx, dy;
  u1 = 0; u2 = 1; dx = *x2-*x1;
  if (TestOrezani(-dx,x1-xwmin,&u1,&u2)&TestOrezani(dx,xwmax-x1,&u1,&u2))
  { dy = y2-y1;
    if(TestOrezani(-dy,y1-ywmin,&u1,&u2)&TestOrezani(dy,ywmax-y1,&u1,&u2))
      { if (u1>0) {x1=x1+u1*dx; y1=y1+u1*dy;}    /* nový počátek */
        if (u2<1) {x2=x1+u2*dx; y2=y1+u2*dy;}    /* nový konec */
        PutLine((int)x1,(int)y1,(int)x2,(int)y2)
      }
  }
} /* ClipLiangBarsky */
```

Algoritmus 5.3: Ořezávání úsečky - algoritmus Liang-Barsky

Je-li za okénko, či zobrazovací pole zvolen natočený obdélník nebo mnohoúhelník obecného tvaru, uvedené algoritmy ořezávání úsečky musí být upraveny. Je vhodné provést nejprve předběžný výběr úseček. Natočené okénko nebo jiný mnohoúhelník lze uzavřít do většího obdélníku, jehož strany jsou rovnoběžné se souřadnicovými osami. Všechny úsečky, které leží mimo velký omezující obdélník, leží také mimo vlastní okénko. Testy úseček ležících uvnitř omezující oblasti jsou již složitější a průsečíky s hranicí okénka musí být vypočítávány z rovnic přímek.

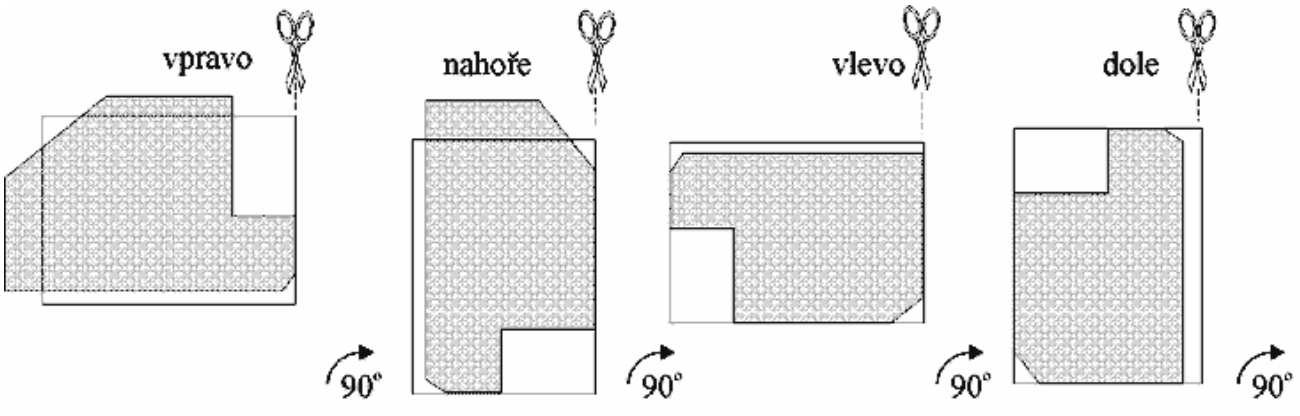


obr. 5.5: Ořezání polygonu

5.3 Ořezávání polygonů

Úloha ořezání polygonů v sobě zahrnuje úlohu ořezání úseček podle hranice. Navíc však je nutné správně vymezit oblasti, ohraničené uzavřeným polygonálním tahem, které jsou viditelné z původního polygonu v ořezávacím okně. Příklad ořezání polygonu je na obr. 5.5. Pro ořezávání polygonů podle okna, jehož hrany jsou rovnoběžné s osami souřadného systému, se používají dva nejznámější algoritmy. První z nich - Cohen-Sutherland - je založený na ořezání každé hrany polygonu podle celého okna (viz. algoritmus pro ořezání úsečky). Výsledky ořezání všech hran polygonu jsou zaznamenány do pomocné paměti ve formě řetězů úseček, které začínají a končí v průsečících s oknem. Krajní body řetězů jsou v pomocné paměti cyklicky testovány, řetězy jsou slučovány, zjednodušovány a v případě potřeby jsou do výstupního seznamu vkládány nové vrcholy totožné s vrcholy okna. Nevýhodou algoritmu je nutnost poměrně komplikované údržby dynamické datové struktury a opakování výpočtu, dokud nejsou sloučeny a zjednodušeny všechny řetězy, které tvoří polygony po ořezání.

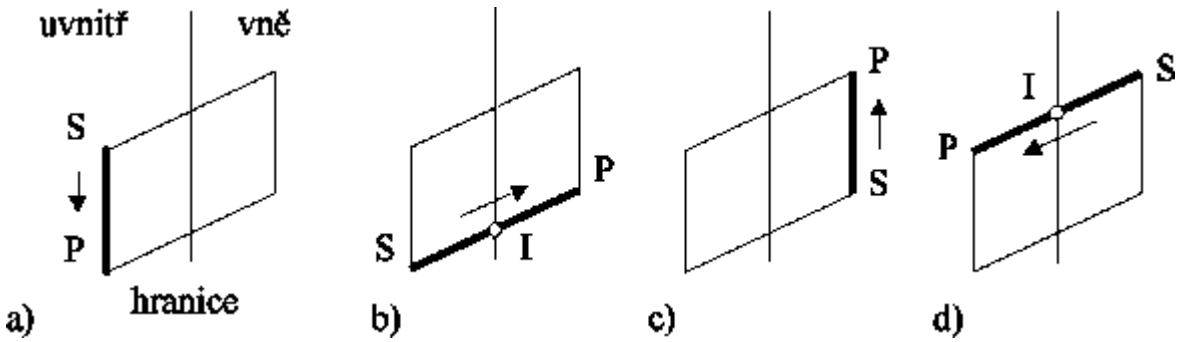
Následující algoritmus - Sutherland-Hodgman - postupně ořezává polygon podle jednotlivých hranic (obr. 5.6). Vstupem algoritmu je posloupnost bodů tvořících uzavřený polygonální tah. Algoritmus testuje dvojici po sobě jdoucích vrcholů a testuje jejich polohu vzhledem k hranici. Na výstup předává ve stejném pořadí body, které jsou vrcholy ořezaného polygonu.



obr. 5.6: Postupné ořezání polygonu

Na obr. 5.6 je nakresleno ořezávání, při němž současně otáčíme polygonem a okénkem tak, aby nůžky zůstaly ve stejné poloze. S využitím tohoto principu je algoritmus vhodný i pro zřetězené zpracování.

Na obr 5.7 je testovaná dvojice bodů označena S a P, průsečík s hranicí je bod I. V případě a) je výstupem nezměněná dvojice SP, v případě b) je koncový bod nahrazen průsečíkem (výstup SI), c) je bez výstupu (úsečka vně okna), v případě d) je výstupem dvojice IP.



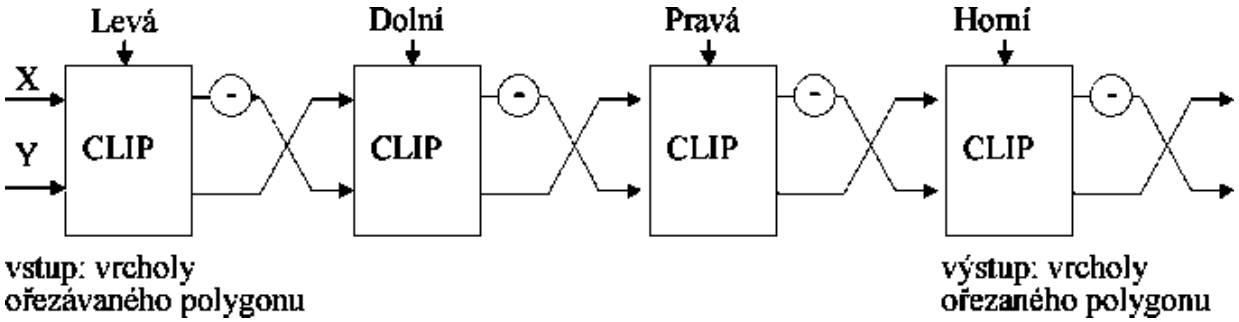
obr. 5.7: Test hrany SP

Zjednodušená verze algoritmu (pro levý okraj okna) je uvedena jako alg. 5.4. Úplný algoritmus aplikuje uvedený postup na všechny hranice okna při úpravě testů a vzorců pro výpočty

průsečíků. Uspořádání pro proudové ořezání je na obr. 5.8. Záměnou souřadnic a změnou znaménka realizujeme v každém kroku současné otočení polygonu a okna o 90°. Pak můžeme použít stejný kód pro všechny ořezávací bloky. Odlišují se pouze zadaným parametrem hranice.

```
Inicializace: S = Vn                                /* poslední vrchol polygonu; */
Pro vrcholy Vi(i = 1, ..., n) ořezávaného polygonu dělej:
  Pokud (Vix < Xmax)                              /* leží ve vnitřní polorovině */
    pak
      Pokud (Sx < Xmax)                           /* leží ve vnitřní polorovině */
        pak VystupBodu(P)
        jinak                                                  /* vypočti průsečík */
          Ix = Xmax;
          Iy = Sy | (Xmax - Sx) * (Viy - Sy) / (Vix - Sx);
          VystupBodu(I); VystupBodu(P);
      jinak
        Pokud Sx < Xmax                               /* leží ve vnitřní polorovině */
          pak                                                  /* vypočti průsečík */
            Ix = Xmax;
            Iy = Sy | (Xmax - Sx) * (Py - Sy) / (Px - Sx);
            VystupBodu(I);
  S=P;
```

Algoritmus 5.4: Ořezání polygonu -- Sutherland-Hodgman (pro jednu hranici)

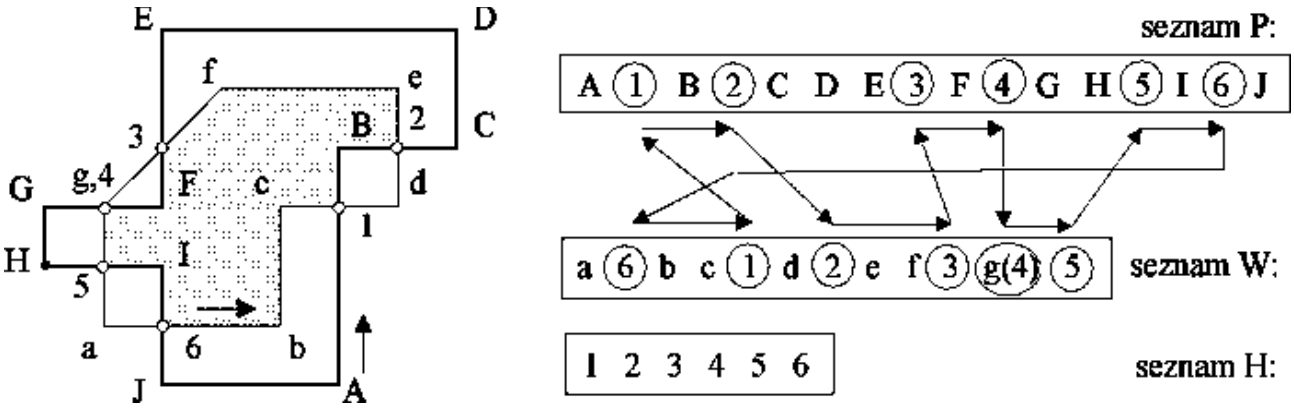


obr. 5.8: Proudové ořezávání

5.3.1 Algoritmus Weiler-Atherton

Pro obecné ořezávání nekonvexních polygonů s vnitřními dírami nekonvexním oknem se používá algoritmus Weiler-Atherton. Okno i polygon jsou reprezentovány seznamem smyček - orientovanými uzavřenými posloupnostmi hran tvořících hranice. Orientace určuje vnitřek ohraničené oblasti. Vnitřní smyčky, nakreslené s opačnou orientací než má vnější hranice, určují díry v oblasti. Základem algoritmu je nalezení všech průsečíků mezi 2 množinami úseček v rovině. Pro vysvětlení použijeme příklad (bez vnitřních smyček) na obr. 5.9.

Okno je tvořeno vrcholy a - b - c - d - e - f - g - a, polygon vrcholy A - B - C - D - E - F - G - H - I - J - A. Průsečíky obou polygonálních útvarů jsou body 1, 2, 3, 4, 5, 6. Algoritmus W-A využívá 3 seznamů - vrcholy ořezávaného polygonu, vrcholy okna a pomocný seznam průsečíků. Neformálně popsaný algoritmus (pro polygony bez vnitřních děr) je uveden jako alg. 5.5. Projdeme-li seznamy v příkladu na obr. 5.9 způsobem popsaným ve W-A algoritmu, nalezneme v seznamu průsečíků H první průsečík (např. 1) a ořezaný polygon bude určen vrcholy 1-B-2-e-f-3-F-4-5-I-6-b-c-1. Pokud vhodně změníme směr procházení seznamy, můžeme získat nejen všechny odstřižené části původního polygonu, ale i části okna, které polygon nevyplňuje. Začneme-li např. u průsečíku 6 procházet opačným směrem, pak obdržíme polygon 6-J-A-1-c-b.



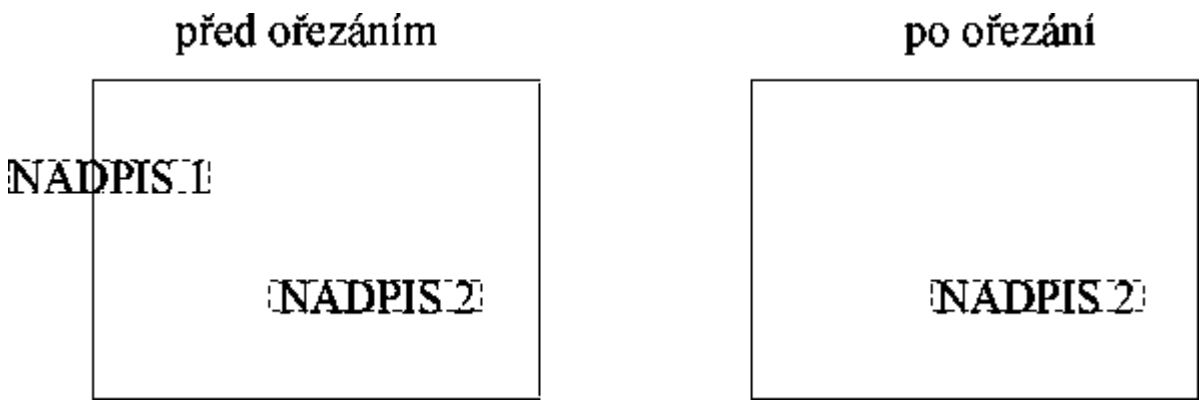
obr. 5.9: Obecné ořezávání polygonů

```
Vstup:
      Seznam P (vrcholy polygonu)
      Seznam W (vrcholy okna)
      Seznam H (pomocný - prázdný).

/* 1. Hledání a řazení průsečíků */
Urči všechny průsečíky mezi hraničními úsečkami polygonu a okna:
  Podle orientace příslušných hran polygonu a okna
    označ průsečík jako vstupní nebo výstupní.
  Průsečík zařaď do pomocného seznamu H.
  Průsečík zařaď mezi příslušné vrcholy do seznamů P a W.
  Místa zařazení v P a W spoj obousměrným ukazatelem.

/* 2. Výstup jednotlivých ořezaných částí */
Dokud H není prázdný dělej
  Vyber průsečík ze seznamu H.
  Opakuj
    Vyřaď průsečík ze seznamu H.
    Pokud je průsečík označen jako vstupní
      pak
        Přejdi do seznamu P;
        Opakuj
          Výstup vrcholu ze seznamu P
        do nalezení dalšího průsečíku;
      jinak
        Přejdi do seznamu W;
        Opakuj
          Výstup vrcholu ze seznamu W;
        do nalezení dalšího průsečíku.
  dokud není uzavřen tah návratem do výchozího průsečíku.
/* dokud H není prázdný */
```

Algoritmus 5.5: Ořezávání nekonvexních polygonů - algoritmus Weiler-Atherton

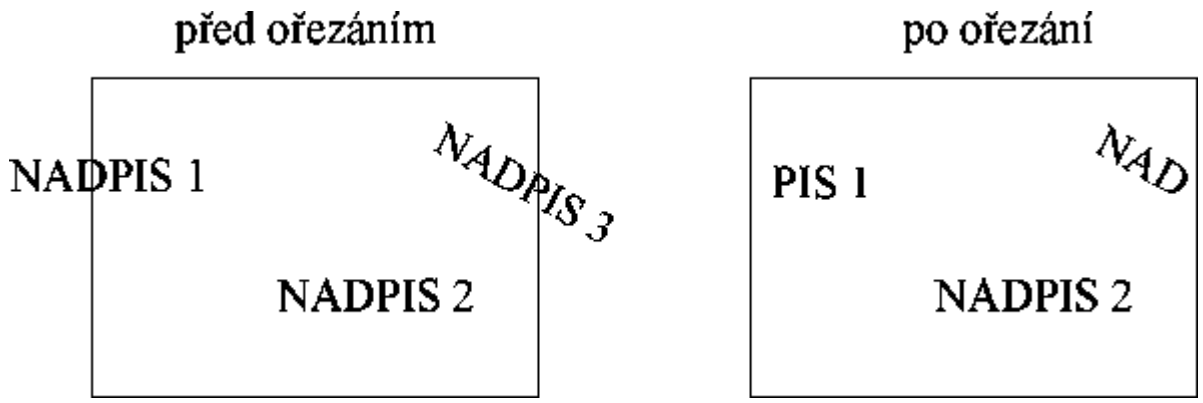
5.4 Ořezávání textu

Nejjednodušší způsob ořezávání kresleného textu okénkem je metoda vše nebo nic, naznačená na obr. 5.10. Porovnáním obdélníku, který ohraničuje řetězec znaků, s hranicemi okénka zjistíme, zda bude celý řetězec vynechán.



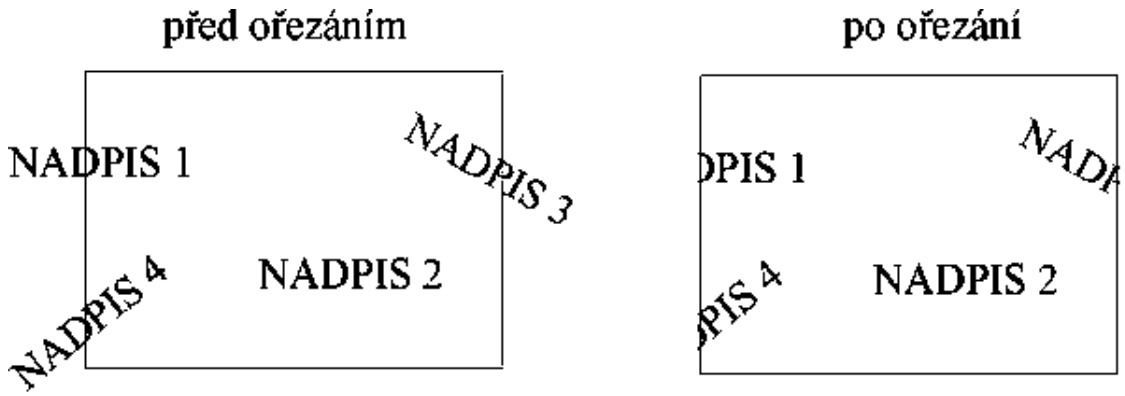
obr. 5.10: Ořezávání textu - test celé skupiny znaků

Nechceme-li vypustit celý řetězec, můžeme zvolit metodu celý nebo žádný znak. Pak jsou z řetězce vynechány jen ty znaky, které neleží celé uvnitř okénka (obr. 5.11). Při jednoduché implementaci stačí testovat polohu ohraničujících obdélníků jednotlivých znaků vůči okénku.



obr. 5.11: Ponechání znaků, které leží celé uvnitř okna

Poslední metodou je přesné ořezání jednotlivých znaků (obr. 5.12). Znak rozložíme na jednotlivé liniové části a použijeme některý z běžných ořezávacích algoritmů. Alternativní a u znaků s vyplněnými plochami zřejmě i efektivnější řešení je kresba znaku přes vhodně nastavenou bitovou masku.



obr. 5.12: Ořezání jednotlivých znaků

6. Zpracování barev

Rychlý vývoj rastrových displejů umožnil počítačové zpracování barevných informací ve velké šíři. V této kapitole si proto všimneme různých způsobů popisu barev a jejich skládání, uvedeme také algoritmy, které dovolí zobrazit mnohobarevný obraz na zařízení s omezenou velikostí barevné palety.

6.1 Světlo a barva

To, co chápeme jako světlo nebo různé barvy, je úzké frekvenční pásmo v oblasti 10^{14} Hz uvnitř elektromagnetického spektra. Každá frekvence uvnitř viditelného pásma odpovídá určité barvě. Na spodním konci rozsahu je červená barva (4,3 10^{14} Hz) a nejvyšší frekvenci má fialové světlo (7,5 10^{14} Hz). Mezi těmito krajními frekvencemi může lidské oko rozlišit až 400 000 různých barev. Tyto barvy pokrývají barevný rozsah od červené přes oranžovou a žlutou pro nižší frekvence až po zelenou, modrou a fialovou pro vyšší frekvence.

Světelný zdroj, jako je slunce nebo žárovka, vysílá všechny frekvence v daném pásmu, které se tak skládají ve výsledné bílé světlo. Takové světlo se nazývá **achromatické**.

Dopadá-li bílé světlo na objekt, jsou některé frekvence odraženy a některé jsou pohlceny objektem. Kombinace frekvencí přítomných v odraženém světle vytváří to, co vnímáme jako barvu objektu. Převládají-li v odraženém světle nízké frekvence, je objekt vnímán jako červený. Tehdy hovoříme o tom, že vnímané světlo má **dominantní frekvenci** (nebo dominantní vlnovou délku) na červeném konci spektra. Dominantní frekvence je také často nazývána barvou nebo zabarvením světla, případně tónem.

Pro popis charakteristik světla jsou užitečné kromě frekvence i další vlastnosti. Pozorujeme-li zdroj světla, naše oko reaguje nejen na barvu, ale i na další podněty:

Jas, případně **svítivost** (luminace) odpovídá intenzitě světla. Čím vyšší je intenzita světla, tím se jeví zdroj jasnější.

Sytost udává čistotu barvy světla. Je tím vyšší, čím užší je spektrum barevných frekvencí obsažených ve světle.

Světlost určuje velikost achromatické složky ve světle s určitou dominantní frekvencí.

Uvedené charakteristiky jsou používány k popisu různých vlastností, které vnímáme u daného zdroje světla. Můžeme se ještě setkat i s pojmem **barevnost** (angl. *chromaticity*), který označuje dvojici vlastností charakterizující světlo: sytost a dominantní frekvenci.

Pozorujeme-li světlo vytvořené kombinací dvou nebo více zdrojů, vnímáme výsledné světlo s charakteristikami, které jsou určeny původními zdroji. Použijeme-li dva světelné zdroje o různých barvách i intenzitách, můžeme vytvořit škálu dalších barev. Pokud kombinací dvou barevných zdrojů získáme bílé světlo, označujeme tyto dvě barvy jako **doplňkové** (komplementární). Je zřejmé, že různými kombinacemi barev lze vytvářet velké množství barevných odstínů. Nabízí se otázka, zda lze nalézt takové **základní barvy** (resp. dominantní frekvence), jejichž postupným skládáním by bylo možno vytvořit všechny existující barvy.

Z vlastností skládání vlnových dějů vyplývá, že neexistují takové dominantní frekvence, jejichž složením by vznikly všechny ostatní barevné frekvence. Přesto lze vybrat několik základních barev, které vytvoří základ pro velkou část existujících barevných odstínů. V technické praxi se používá tří základních barev. Jedna z možností je například červená, zelená a modrá. Kombinujeme-li základní barvy o různých intenzitách, dokážeme vytvořit barevnou škálu (barevný rozsah) pokrývající značnou část všech barev, na které je navyklé lidské oko.

Poznamenejme, že přidáním nové základní barvy ke stávajícímu souboru základních barev vznikne nová, širší barevná škála. K získání celého viditelného barevného spektra bychom však museli přidat nekonečně mnoho základních barev.

6.1.1 Chromatický diagram

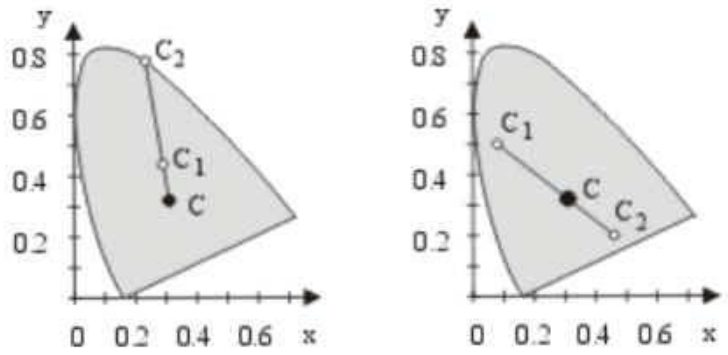
V roce 1931 byl vytvořen mezinárodní standard pro základní barvy. Jeho součástí je tzv. chromatický diagram CIE. CIE je zkratka Mezinárodní komise pro osvětlení (Commission Internationale de l'Éclairage) o kterém se zmíníme podrobněji. Ve standardu se předpokládá, že barvy

jsou definovány pomocí váženého součtu tří základních barev. Každá skutečná barva je ve standardu určena matematicky daným množstvím základních barev potřebných pro její vytvoření. Také každá základní barva je popsána vlastní křivkou rozdělení energie.

Předpokládejme, že určitý barevný odstín je vytvořen ze základních barev, jejichž množství označíme jako A, B a C. Tato množství zapíšeme v normalizovaném tvaru jako

$$x = \frac{A}{A+B+C}, \quad y = \frac{B}{A+B+C}, \quad z = \frac{C}{A+B+C}. \qquad (6.1)$$

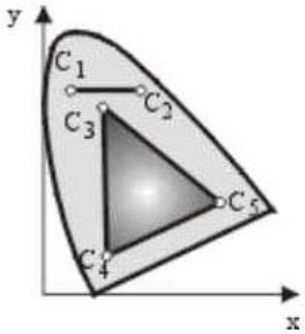
Protože platí, že $x + y + z = 1$, jsou kterékoliv dvě složky postačující pro úplné určení barvy. Vybereme-li například složky x a y, můžeme reprezentovat všechny barvy pomocí dvourozměrného diagramu. Křivka, která ohraničuje barvy viditelného spektra, se nazývá **chromatický diagram CIE**. Barevné body tvořící obalovou spektrální křivku jsou označeny vlnovou délkou v nanometrech počínaje červenou částí spektra a konče fialovou částí spektra. Bod C v diagramu odpovídá poloze bílého světla v angl. terminologii je označován jako *illuminant C* Je používán jako standard pro průměrné denní světlo.



Obr. 6.1: Určení sytosti, dominantní vlnové délky a doplňkových barev z chromatického diagramu CIE

Chromatický diagram poskytuje prostředky pro kvantitativní určení sytosti a dominantní vlnové délky, stejně jako z něho lze odvodit další barevné veličiny a vztahy:

1. Pro libovolný barevný bod, jako například C_1 na obr. 6.1 vlevo, definujeme sytost barvy jako relativní vzdálenost barevného bodu od standardního bílého světla C. Měření vzdálenosti se provádí na úsečce spojující body C a C_1 a končící na obalové křivce. Barva C_1 na obrázku je sytá asi z 25%, protože leží přibližně v jedné čtvrtině této úsečky.
2. Dominantní vlnová délka jakékoliv barvy je definována jako vlnová délka na spektrální křivce protínající úsečku spojující C a příslušný barevný bod C_1. Pro barevný bod C_1 na obr. 6.1 vlevo je dominantní vlnová délka v bodu C_2.
3. Doplňkové barvy jsou reprezentovány dvěma koncovými body úsečky, která prochází bodem C, jak je uvedeno na obr. 6.1 vpravo. Mají-li dvě doplňkové barvy stejnou sytost, vznikne jejich kombinací bílé světlo C.



Obr. 6.2: Chromatický diagram a barevné rozsahy

Barevné rozsahy jsou v chromatickém diagramu reprezentovány úsečkami, které spojují barevné body určené základními barvami. Na obrázku 6.2 je nakreslena spojnice mezi body C_1 a C_2. Všechny body na této úsečce lze získat kombinací různých intenzit základních barev C_1 a C_2. Podobně lze vytvořit všechny vnitřní body trojúhelníku C_3-C_4-C_5 kombinací základních barev z jeho vrcholů.

Jak vyplývá z geometrie chromatického diagramu, není možno nalézt takové tři základní barvy, které by určovaly trojúhelník, pokrývající celý diagram. To odpovídá dříve zmíněné skutečnosti, že z konečného počtu základních barev nelze vytvořit všechny existující barevné odstíny.

Chromatický diagram nestanovuje, jaké barvy volit za základní. Různá technická zařízení mívají definovány různé základní barvy a na diagramu lze vyznačit trojúhelníky odpovídajících barevných rozsahů. Porovnáním těchto trojúhelníků můžeme porovnávat barevné možnosti různých zařízení.

Detailnější informace o barvách lze nalézt například v [Skal93].

6.2 Barevné modely

Při práci s barvami je důležitá jednak volba základních barev, jednak způsob jejich kombinování (míšení). Představy o tom, jaká barva vznikne smícháním dvou různých barev, jsou u různých lidí poměrně odlišné. Je proto třeba nalézt prostředky, pomocí kterých mohou běžní uživatelé grafických systémů dobře popsat barvy a jejich složení.

Například pro malíře je běžné, že nové barvy vytváří míšením barevných pigmentů s bílým nebo černým pigmentem. Toto skládání barev se nazývá **subtraktivní**. Každé přidání barevného pigmentu vytvoří tmavší barvu. Složením všech barev vznikne černá, což je zcela opačná situace oproti **aditivnímu** skládání barevného světla, kde každé přidání barvy (světla) zesvětluje výsledek.

Soubor základních barev, pravidla jejich míchání a měněné barevné charakteristiky jsou v počítačové grafice definovány pomocí **barevných modelů**. Grafické systémy často obsahují několik barevných modelů a prostředky pro převody mezi nimi.

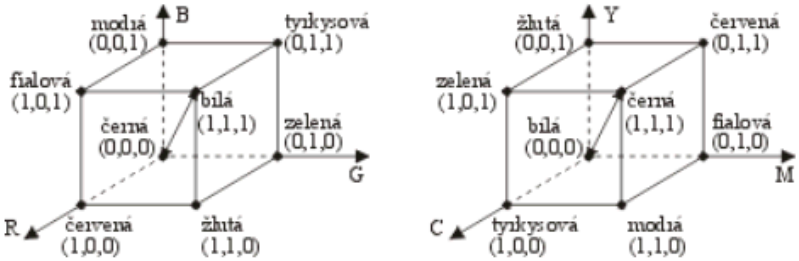
6.2.1 Model RGB

Naše oko vnímá barvu na základě podráždění tří barevných receptorů na sítnici. Receptory mají u většiny lidí nejvyšší citlivost pro vlnové délky v rozsahu přibližně 630 nm (červená), 530 nm (zelená) a 450 nm (modrá). Tomuto způsobu vnímání se nejvíce přibližuje barevný výstup na RGB monitorech, kde jsou barvy vytvářeny kombinací tří základních barev - červené (R), zelené (G) a modré (B). Jedním z nejznámějších modelů je proto model RGB.

Barevný rozsah můžeme v modelu RGB zobrazit prostorově pomocí jednotkové krychle umístěné v osách označených jako r, g a b (obr.). Počátek souřadnic odpovídá černé barvě, zatímco vrchol o souřadnicích (1,1,1) odpovídá bílé. Vrcholy krychle, které leží na osách, představují základní barvy a zbývající vrcholy reprezentují doplňkové barvy ke každé ze základních barev.

Model RGB je součtovým modelem. Intenzity základních barev se sčítají, čímž jsou vytvářeny nové barvy. Každý barevný bod uvnitř krychle může být reprezentován jako trojice (r, g, b), kde hodnoty r, g a b se pohybují v rozsahu 0-1. Fialový vrchol je získán součtem červené a modré, a má tedy souřadnice (1, 0, 1). Bílá ve vrcholu (1, 1, 1) je součtem hodnot v červeném, zeleném a modrém vrcholu. Odstíny šedi odpovídají bodům na diagonále krychle spojující černý a bílý vrchol. Každý bod na diagonále má stejný příspěvek každé základní barvy, takže 50% šedá je reprezentována jako (0.5, 0.5, 0.5).

Intenzity základních barev se v modelu RGB pohybují v intervalu $\langle 0, 1 \rangle$. Při popisu barev na počítači je tento rozsah převáděn do intervalu celých čísel. Vezmeme-li v úvahu citlivost lidského oka, stačí tento interval rozdělit přibližně na 100 dílů, mezi nimiž je již barevný rozdíl průměrným lidským zrakem nepostřehnutelný. V počítačové grafice se jako standard používá dělení intervalu na 256 dílů. Intenzitu potom můžeme kódovat pomocí osmi bitů. Libovolný bod uvnitř krychle RGB lze v takovém případě reprezentovat 24 (= 3*8) bity v anglické terminologii se barvy kódované pomocí 24 bitů označují jako *true colors.*



Obr. 6.3: Model RGB (vlevo) a CMY (vpravo)

6.2.2 Modely CMY, CMYK

Model RGB je technicky orientovaný model, vhodný pro displeje. Jeho aditivní charakter odpovídá skládání barevného světla. Lidská zkušenost se skládáním barev však vychází ze subtraktivního míšení barevných pigmentů. Míchání barviv je typické nejen pro malíře, ale i pro tiskařskou techniku. Těmto účelům vyhovuje model CMY, obsahující tři základní barvy - tyrkysovou, bleděmodrou (**C**yan), fialovou (**M**agenta) a žlutou (**Y**ellow). Také tento model lze popsat jednotkovou krychlí (obr. 6.3). Krychle CMY je doplňkem krychle RGB. Sčítání hodnot CMY odpovídá subtraktivnímu skládání barev, takže vrchol (1, 1, 1) reprezentuje černou barvu.

Převod mezi modely RGB a CMY je jednoduchý. Vyjádříme-li bod v prostoru RGB tříprvkovou maticí [r g b], určíme bod [c m y] v prostoru CMY pomocí odčítání matic:

$$\begin{bmatrix} c \\ m \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} r \\ g \\ b \end{bmatrix} . \qquad (6.2)$$

Model CMY je vhodný pro tiskařskou techniku. Barevné fotografie jsou reprodukovány jako soutisky tří obrazů, tvořených základními barvami - fialovou, zelenomodrou a žlutou. Tyto základní barevné pigmenty nesmí být dokonale krycí, neboť nové barvy vznikají vzájemným překrýváním. Složením všech tří barev proto v praxi nevznikne žádoucí černá, ale pouze hnědá. To je příčinou toho, že černá se tiskne jako samostatná barva. Lze ji použít i ke ztmavení ostatních barev. Z modelu CMY se tak v polygrafii stává model **CMYK** přidáním černé (blac**K**) jako čtvrté základní barvy. Její velikost pro daný pixel získáme jako minimální hodnotu ze složek c, m a y, které poté snížíme o k. Převody mezi RGB a CMY, resp. CMYK jsou jednoduché a nedochází při nich ke ztrátě informace. To ovšem neznamená, že obraz zapsaný v RGB modelu vytiskneme vždy dokonale na tiskárně v režimu CMYK. Trojúhelníky zobrazitelných barev (angl. *color gamut*) nebývají totožné.

APPLET

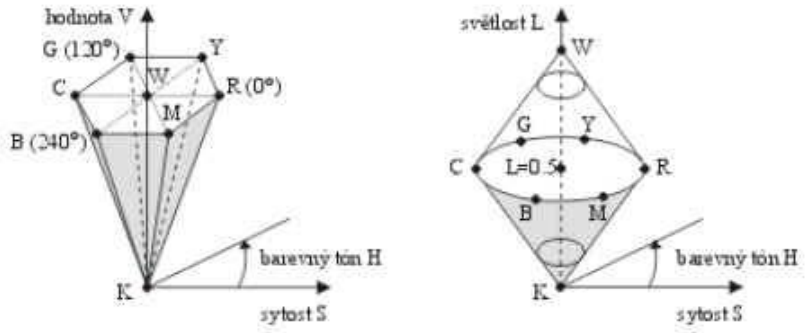
applet 6.1

6.2.3 Modely HSV a HLS

Modely RGB a CMY jsou přímo použitelné pro odpovídající technická zařízení. Pro většinu uživatelů je ovšem obtížné stanovit množství základních barev pro získání požadovaného odstínu. V uvedených modelech není například jednoduché změnit pro danou barvu její odstín tak, aby sytost zůstala zachována. Proto byly vyvinuty barevné modely blízké intuitivnímu (lidskému) popisu barev.

První z těchto modelů se nazývá **HSV** a jeho základními proměnnými jsou **barevný tón** (Hue), **sytost** (Saturation) a jasová **hodnota** (Value). Barevný tón označuje převládající spektrální barvu, sytost

určuje příměs jiných barev a jas je dán množstvím bílého (bezbarvého) světla. Pro prostorové zobrazení modelu se nepoužívá krychle, nýbrž šestiboký jehlan (obr. vlevo), jehož vrchol leží v počátku soustavy souřadnic HSV. Souřadnice h a v se mění od 0 do 1, souřadnice s je úhlová - nabývá hodnot od 0° do 360°. Vrchol jehlanu reprezentuje černou barvu. Jas roste směrem k podstavě, střed podstavy znamená bílou barvu. Sytost je znázorněna jako vzdálenost bodu od osy jehlanu. Dominantní barvy tedy leží na plášti, čisté barvy jsou potom na obvodu podstavy. Při pohybu po obvodu ve stejné výši od základny se postupně mění barevný tón, sytost a jas zůstávají nezměněny.



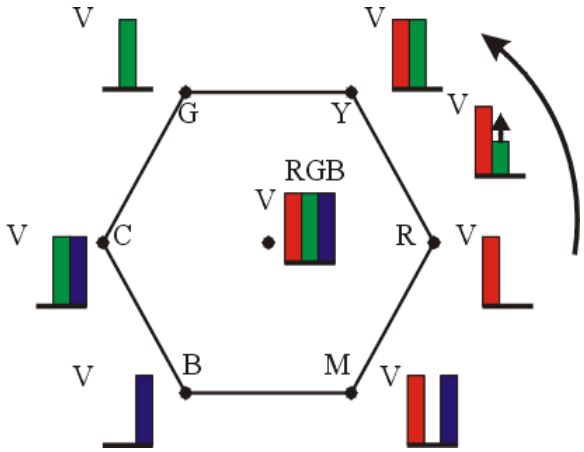
Obr. 6.4: Model HSV (vlevo) a HLS (vpravo)

Model HSV vykazuje některé nedostatky, které sice nejsou zásadního charakteru, nicméně mohou ztěžovat práci s definováním barvy v prostoru HSV. Jedním z nedostatků je jehlanovitý tvar, který způsobuje, že ve řezu se musí bod o konstantní hodnotě s pohybovat při změně h po dráze ve tvaru šestiúhelníku, nikoliv po kružnici, jak by bylo přirozené. Dalším záporným jevem je nesymetrie modelu z hlediska přechodů ve stupních šedi od černé k bílé. Tyto nedostatky odstraňuje model **HLS** zavedený firmou Tektronix jehož geometrie je uvedena na obr. 6.4 vpravo.

Název modelu je odvozen z pojmů **barevný tón** (Hue), **světlost** (Lightness) a **sytost** (Saturation). Model HLS je obdobou modelu HSV, v němž byl jehlan nahrazen kuželem, resp. dvojicí kuželů. Barevný tón je opět vyjádřen jako úhlová hodnota, světlost se mění od 0 (černá v dolním vrcholu) do 1 (bílá v horním vrcholu). Sytost nabývá na povrchu kuželů hodnoty 1 a klesá na 0 směrem k ose kuželů. Nejjasnější čisté barvy mají tedy souřadnice s = 1 a l = 0,5 a leží na obvodu podstav kuželů.

Tvar modelu HLS plně odpovídá skutečnosti, že nejvíce barev vnímáme při světlosti (oblast podstav). Schopnost rozlišit barvy klesá jak při velkém ztmavení, tak při přesvětlení (oblasti obou vrcholů kuželů). Další dobrá vlastnost modelu HLS spočívá v analogii míchání barev přidáváním černých a bílých pigmentů k základním spektrálním barvám. Zvýšení světlosti při nezměněné sytosti si lze představit jako přidání jistého množství bílých a ubrání stejného množství černých pigmentů. Samotné zvýšení sytosti znamená odebrání stejného množství bílých a černých pigmentů.

Modely HSV a HLS umožňují postupně měnit barevné charakteristiky při zachování ostatních typických vlastností barvy. To je příjemné pro uživatele, kteří chtějí definovat barvy pomocí přirozených pojmů, jako je sytost, světlost a dominantní barva. Modely jsou vhodné pro vzorníky barev, kde jsou zobrazovány řezy jehlanem, či kuželem pro různé hodnoty h.



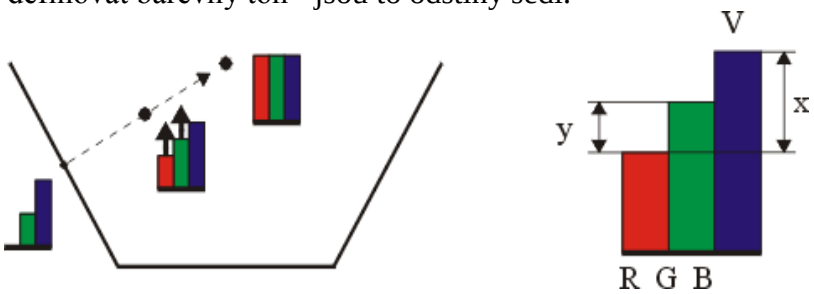
Obr. 6.5: Různé hodnoty (r, g, b) při konstantním v modelu HSV

Naprostá většina současných barevných zobrazovacích procesorů zpracovává barevné informace v prostoru RGB. Je tedy třeba umět převádět body z prostorů HLS a HSV do prostoru RGB. Jako ukázkový příklad zvolíme převod z modelu RGB do modelu HSV, který je velmi jednoduchý. Na něm si také uvědomíme některé vztahy mezi těmito barevnými soustavami. Zopakujme si obory hodnot jednotlivých proměnných:

$$r, g, b, s, v \in \langle 0, 1 \rangle,$$

$$h \in \langle 0, 360) + NEDEF$$

Zvláštní hodnota NEDEF je určena ty pro body v soustavě HSV, které leží na ose jehlanu. Pro ty skutečně nelze definovat barevný tón - jsou to odstíny šedi.



Obr. 6.6: Poměr hodnot (r, g, b) při změně s na jehlanu HSV

Na obrázku 6.5 si dále všimneme typických změn souřadnic RGB při změně polohy bodu v soustavě HSV:

- při cyklické změně tónu h se mění vždy jen jedna ze souřadnic r, g, b
- velikost největší ze souřadnic r, g, b je shodná s hodnotou v
- na hranách řezu (na plášti jehlanu) má vždy alespoň jedna ze souřadnic r, g, b hodnotu 0; v modelu HSV pro tyto body platí s = 1

Na obr. 6.6 potom můžeme sledovat situaci při změně s, když h i v zůstávají nezměněny:

- při změně s se mění pouze menší složky r, g, b, a to lineárně
- poměr rozdílů menších složek a maximální složky je při změně s konstantní; jedná se o poměr x/y na obr. 6.6 vpravo

```
#define NEDEF   -1.0
void RGB_to_HSV (float r, float g, float b, float *h, float *s, float *v)
{
  float max, min, delta;
  max = Maximum (r, g, b);     /* maximum ze tří čísel */
  min = Minimum (r, g, b);     /* minimum ze tří čísel */
  *v = max;
  if (max == 0.0) *s = 0.0;
  else *s = (max - min) / max;
  if (*s == 0.0) *h = NEDEF;   /* odstíny šedi */
  else
  {
    delta = max - min;
    if (r == max) *h = (g-b) / delta; /* dominantní R */
    else if (g == max) *h = 2.0 + (b - r) / delta;    /* dominantní G */
      else *h = 4.0 + (r - g) / delta;/* dominantní B */
    *h = *h * 60.0;     /* převod na stupně */
    if (*h < 0) *h += 360.0;   /* posun do kladných čísel */
  }
} /* RGB_to_HSV */
```

Algoritmus 6.1: Převod z barevného modelu RGB do modelu HSV

Z výše uvedených pravidel vychází algoritmus 6.1 převodu modelu RGB na model HSV. Ve funkci jsou podle největší a nejmenší hodnoty souřadnic r, g, b určeny souřadnice v a s. Podle největší ze souřadnic r, g, b je potom zvolen tón h.

```
# define NEDEF -1.0

void HSV_to_RGB (float h, float s, float v, float *r, float *g, float *b)
{
  int i;
  float f, p, q, t;    /* pomocné proměnné */
  if (h == NEDEF)      /* odstíny šedi */
  {  *r = v; *g = v; *b = v;
  }
  else
  {
    if ( s == 0.0) ;   /* CHYBA ! */
    else
    {          /* barvy */
      h = h / 60.0;
      i = floor(h);    /* celá část hodnoty h */
      f = h - i;               /* zbylá část hodnoty h */
      p = v * (1 - s);
      q = v * (1 - (s * f));
      t = v * (1 - (s * (1 - f)));
      switch (i) {
        case 0: *r = v; *g = t; *b = p; break;
        case 1: *r = q; *g = v; *b = p; break;
        case 2: *r = p; *g = v; *b = t; break;
        case 3: *r = p; *g = q; *b = v; break;
        case 4: *r = t; *g = p; *b = v; break;
        case 5: *r = v; *g = p; *b = q; break;
      } /* switch */
    } /* else */
  } /* if */
} /* HSV_to_RGB */
```

Algoritmus 6.2: Převod z barevného modelu HSV do modelu RGB

Obdobnými postupy se provádějí převody mezi modely RGB a HLS a obrácené převody zpět do modelu RGB (alg. 6.2).

APPLET

applet 6.2

APPLET

applet 6.3

6.2.4 Modely pro televizní a video techniku

S dosud uvedenými barevnými modely se setkáváme ve většině programů zpracovávajících barevné rastrové obrázky. Následující skupina modelů je pro běžného uživatele již vzácnější. Jsou používány

především tam, kde se stýká počítačová grafika a televizní technika, tj. v oblasti počítačových animací, filmu, triků, apod.

Model YUV

Základní model **YUV** je používán pro přenos televizních signálů v normě PAL. Do stejné rodiny patří další modely - **YIQ** pro americkou televizní normu NTSC a model $\mathbf{YC_BC_R}$ pro normu SECAM a zpracování videozáznamů. Jejich společným rysem je oddělení jasové (luminační) složky od barevných informací (chrominance). Toto uspořádání dovoluje přímo využít stejný jasový signál **Y** jak pro barevné, tak i černobílé televize. Zbývající dva signály nesou informace o velikosti barevných složek obrazu.

Model YUV je někdy označován též (Y, B-Y, R-Y), resp. číselně (Y, 0,493.(B-Y), 0,877.(R-Y)). Z modelu RGB získáme jeho hodnoty jednoduchým maticovým násobením:

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.141 & -0.289 & 0.437 \\ 0.615 & -0.515 & -0.100 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \qquad (6.3)$$

Model YC_BC_R

S tímto modelem se v počítačové grafice setkáme při zápisu rastrových obrázků ve formátu JPEG. Následující převod (vzorec) odpovídá počítačovému zpracování, kdy je každá barevná složka zapsána v 8 bitech, tedy v rozsahu 0-255. Podle standardu CCIR-601 má hodnota **Y** být v intervalu 0.0-1.0 a hodnoty $\mathbf{C_B}$, $\mathbf{C_R}$ v intervalech -0.5-0.5. Přepočet mezi těmito intervaly a rozsahem jednoho bytu je ovšem jednoduchý.

$$\begin{bmatrix} Y \\ C_B \\ C_R \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix} \qquad (6.4)$$

Zkráceně lze definovat hodnoty $\mathbf{C_B}$ a $\mathbf{C_R}$ jako

$$\begin{aligned} C_B &= 0.5643\,(B-Y) + 128 \\ C_R &= 0.7133\,(R-Y) + 128 \end{aligned} \qquad (6.5)$$

Zpětná transformace opět předpokládá rozsah všech složek v rámci jednoho bytu:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.402 \\ 1.0 & -0.34414 & -0.71414 \\ 1.0 & 1.772 & 0.0 \end{bmatrix} \left(\begin{bmatrix} Y \\ C_B \\ C_R \end{bmatrix} + \begin{bmatrix} 0 \\ -128 \\ -128 \end{bmatrix} \right) \qquad (6.6)$$

Model YIQ

Barevné složky předchozích dvou modelů obsahovaly informace o množství červeného, resp. modrého světla. V modelu YIQ je dávána přednost frekvencím oranžového a modrozeleného světla.

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.528 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \qquad (6.7)$$

6.3 Rozptylování barev

V počítačové grafice se používá mnoho zařízení, která mají jen omezenou škálu zobrazitelných odstínů. Jednoduché obrazové adaptéry VGA dokáží zobrazit pouze 256 barevných odstínů, modernější pak 32 tisíc až 16,7 milionů (true color). V případě jednoduššího adaptéru je nutno stanovit, které z mnoha tisíc barev vybereme do oné 256-ti prvkové palety a jak budeme zacházet s těmi barvami, na které se již nedostalo. Limitním případem z hlediska barev jsou černobílé tiskárny, kde je paleta tvořena pouze dvěma zástupci - bílou a černou.

Aby bylo možno i na takto jednoduchých zařízeních vykreslovat obrázky, které obsahují mnoho barevných odstínů nebo stupňů šedi, byly vyvinuty metody, které se snaží překlenout rozdíl mezi kvalitou zobrazovaných dat a omezenými možnostmi technických zařízení. Techniky, které dokáží z několika barev vytvořit iluzi bohaté barevné škály, se nazývají **polotónování** (angl. *halftoning*) a **rozptylování** (angl. *dithering*). Pojem polotónování je používán především u tiskáren, kde je jeden barevný pixel původního obrázku převeden na matici bodů s omezenou velikostí palety. Dochází tedy ke zvětšení obrázku. Rozptylovací metody jsou vhodné pro zobrazení obrázku, který byl např. vypočítán v kvalitě 24 bitů na pixel a který má být vykreslen na obrazovce v nezměněné velikosti. V této kapitole se zaměříme především na techniku rozptylování - polotónování budeme chápat jako její speciální případ.

Obě metody využívají schopnosti lidského oka vytvářet z několika blízkých barevných paprsků vjem jediného paprsku, jehož barva je dána **aditivním složením** barev paprsků původních. Tato vlastnost je základem všech metod počítačové grafiky pro generování obrázků o mnoha odstínech na zařízeních s omezenou velikostí palety. Ostatně je i principem barevné televizní obrazovky.

Rozptylovací techniky jsou používány pro zpracování jak černobílých, tak barevných obrázků. V dalším textu vysvětlíme nejprve základní principy na černobílých obrázcích. Vzhledem k oddělenému zpracování barevných obrazů po složkách lze barevné rozptylování chápat jako speciální případ rozptylování obrazu ve stupních šedi.

Předpokládejme nejprve, že chceme obrázek obsahující 16 (= 2^4) odstínů šedi vykreslit na displeji, který zobrazuje pouze černou a bílou. Původní 4 bity na pixel je nutno zredukovat na 1 bit na pixel obrázku tak, aby si lidské oko dokázalo z různých kombinací sousedních bílých a černých bodů vytvořit představu několika odstínů šedi.

6.3.1 Náhodné rozptýlení

Označme C_{in} vstupní intenzitu jasu (odstínu šedi) jednoho obrazového bodu. Její hodnota se pohybuje v intervalu celých čísel $\langle 0, 15 \rangle$. Obdobně označíme výstupní intenzitu $C_{out} \in \langle 0, 1 \rangle$. Při použití metody náhodného rozptýlení využijeme generátor náhodných čísel, jehož hodnoty vrací funkce.

Pro určení výsledné intenzity C_{out} černobílého obrazového bodu použijeme algoritmus:

```
Cout = 0;

if  (Cin > random (15)) Cout++;
```

přičemž předpokládáme, že funkce **random**(15) vrací celočíselné nezáporné hodnoty menší než 15. Je-li potom vstupní intenzita nulová, je výsledná intenzita také nulová. Má-li vstupní intenzita nejvyšší možnou hodnotu 15, nabude výstupní intenzita vždy své nejvyšší hodnoty jedna. V ostatních případech rozhodne o tom, zda výsledný pixel bude rozsvícen nebo ne, výsledek porovnání velikosti vstupní intenzity a náhodně vygenerovaného čísla. To se ve výsledném obrázku projeví tak, že například plocha o vstupní intenzitě 10 bude tvořena nepravidelně se střídajícími bílými a černými pixely. Poměr počtu

bílých pixelů k černým bude přitom dvě třetiny, jak to odpovídá poměru 10 / 15. Tímto způsobem zůstanou zachovány původní jasové poměry v celém obrázku.

Náhodné rozptýlení je rychlá a jednoduchá metoda zachovávající původní velikost obrázku. Dodává větším plochám vzhled drsného povrchu, výsledné obrázky působí dojmem fotografie s velkým zrněním.

APPLET

applet 6.4

6.3.2 Maticové rozptýlení

Maticová rozptylovací metoda používá pro generování různých odstínů šedi pravidelných, předem daných vzorků složených z bílých a černých bodů. Pokud jsou tyto vzorky dobře navrženy, působí výsledný obrázek jako kresba, ve které malíř tužkou vystínoval tmavší, či světlejší místa pravidelnými jemnými šrafami. Původně byla tato metoda navržena pro takový převod obrázků, při kterém je jeden vstupní pixel nahrazen skupinou (maticí) pixelů výsledných. Dojde tedy ke **zvětšení obrázku**.

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$C_{in} = 0 \quad C_{in} = 1 \quad C_{in} = 2 \quad C_{in} = 3 \quad C_{in} = 4$$

Obr. 6.7: Příklady rozptylovacích matic

Na obr. 6.8 vidíme příklad vzorků, které by byly použity v případě, že vstupní intenzita se pohybuje v rozsahu $\langle 0, 4 \rangle$. Pro $C_{in} = 0$ použijeme levý vzorek, pro $C_{in} = 4$ vzorek nejvíce vpravo. Pro větší škálu vstupních intenzit je nutno použít matice vyšších řádů.

$$\mathbf{M}_d = \begin{bmatrix} 0 & 12 & 3 & 15 \\ 8 & 4 & 11 & 7 \\ 2 & 14 & 1 & 13 \\ 10 & 6 & 9 & 5 \end{bmatrix}, \quad \mathbf{M}_p = \begin{bmatrix} 1 & 5 & 9 & 2 \\ 8 & 12 & 13 & 6 \\ 4 & 15 & 14 & 10 \\ 0 & 11 & 7 & 3 \end{bmatrix}$$

Obr. 6.8: Různé rozptylovací matice 4×4 pro $C_{in} \in \langle 0, 1 \rangle$

Vždy je třeba dodržet zásadu, že každá následující matice vznikla z předchozí přidáním jedničky na jednu novou pozici. Je tedy nutno vyvarovat se permutací uvnitř matice, které by v případě některých kombinací vstupních odstínů vedly k nesprávnému zesvětlení, či ztmavení obrázku. Při uvedené zásadě nemusíme ani udržovat v paměti všechny matice pro různé hodnoty C_{in} a můžeme použít matici jedinou (příklady jsou na obr. 6.8). V uvedeném případě dochází k nesouhlasu mezi rozsahem vstupní intenzity a počtem vzorků generovaných rozptylovací maticí. Pomocí matice 4×4 lze totiž vygenerovat 17 různých vzorků. V praxi se tato situace řeší například zopakováním čísla 8 v rozptylovací matici, čímž maximální hodnota v ní obsažená klesne na potřebných 16. Hodnoty v takovéto rozptylovací matici určují, při jaké velikosti vstupního odstínu šedi C_{in} se na výstupu objeví v odpovídajícím místě jednička. Pokud je C_{in} větší než daná hodnota v matici, bude výstupní pixel rozsvícen.

Je zřejmé, že jedničky a nuly lze v rozptylovacích maticích uspořádat mnoha způsoby. Pro různé typy zobrazovacích zařízení se skutečně používají různá uspořádání matic - jiné tvary matic jsou vhodné pro displeje, jiné pro tiskárny. V příkladu na obr. 6.8 vidíme matici $\mathbf{M}_d$ vhodnou pro displeje, která vytváří stínovací vzor. Jiné uspořádání je vhodné pro tiskárny, kde je často žádoucí vzor, známý z fotografií z novin, složených z větších a menších teček (matice $\mathbf{M}_p$ na obr. 6.8). Důvodem

pro sdružování černých bodů do puntíků je skutečnost, že tiskové body se navzájem částečně překrývají, takže například při tisku pravidelně se střídajících bílých a černých bodů získáme na tiskárně velmi tmavý vzorek, nikoliv . Sdružování bodů tento problém překlene.

Tvary matic bývají často předmětem firemního tajemství. Při barevném tisku se navíc používají různé matice pro různé barevné složky, přičemž jednotlivé vzory se ještě natáčejí, aby se potlačila opticky zřetelná pravidelnost vzorků v maticích. Pro černou barvu, která je výrazná, se často používá natočení jinak pravidelného vzorku o optimální úhel 45°. Pro fialovou se volí úhel natočení 75°, pro žlutou 90° a pro modrozelenou 105°.

V některých případech potřebujeme rozptylovací matice vyšších řádů. Ty lze generovat algoritmicky v případě, že řád rozptylovací matice **M** je mocnina 2:

Nechť $\mathbf{M}_{(2,2)} = \begin{bmatrix} 0 & 3 \\ 2 & 1 \end{bmatrix}$,

pak $\mathbf{M}_{(4,4)} = \begin{bmatrix} \mathbf{M}_0 & \mathbf{M}_3 \\ \mathbf{M}_2 & \mathbf{M}_1 \end{bmatrix}$, kde $\mathbf{M}_i = 4 * \mathbf{M}_{(2,2)} + i$.

Metodu maticového rozptýlení lze úspěšně použít i v případě, kdy **nezvětšujeme** výsledný obrázek. Tehdy použijeme místo celé matice pouze jeden její prvek. Pro výběr prvku poslouží souřadnice (x, y) zobrazovaného pixelu, které po použití operace *modulo* určí indexy v matici. Vzorec pro určení, zda v daném místě bude mít bod bílou nebo černou barvu, bude (v případě indexování rozptylovací matice od nuly) potom:

$C_{out} = 0$;

if (C_{in} > M[x % 4] [y % 4]) C_{out} ++;

Maticové rozptýlení dává dobré výsledky pro většinu počítačem generovaných obrázků. Pravidelné střídání odstínů nepůsobí rušivě ani na velkých plochách. Při zpracování fotografií však tato pravidelnost dělá dojem uměle vytvořeného obrazu. Právě pro vykreslování obrázků získaných fotografickou cestou byla vyvinuta následující metoda.

a) náhodné rozptýlení

b) rozptylovací matice typu $\mathbf{M}_d$

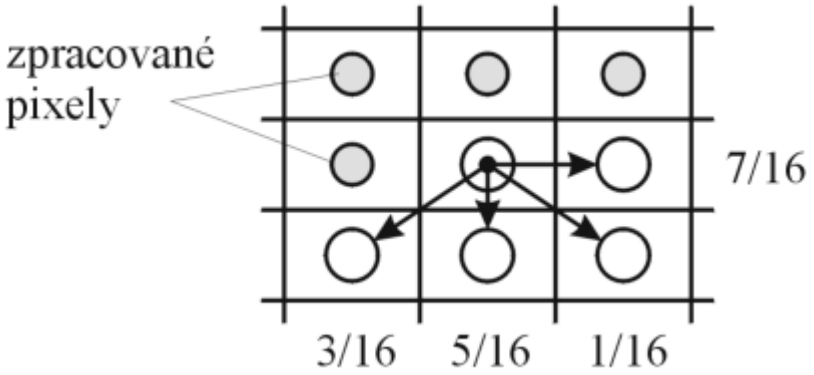
Obr. 6.9: Fotografie Sarah Bernhardtové zpracovaná různými rozptylovacími metodami

c) rozptylovací matice typu $\mathbf{M}_p$

d) distribuce chyby

6.3.3 Distribuce chyby

Při zachování stejné velikosti obrázku je intenzita vstupního pixelu nahrazena buď jedničkou, nebo nulou na výstupu, čímž nutně dochází ke ztrátě informace. Jeden ze způsobů, jak využít vstupní informaci v co nejvyšší míře, se nazývá **distribuce chyby**. V angl. literatuře se objevují pojmy *error diffusion* nebo *error distribution*. Vstupní intenzita C_{in} je zde použita nejen k nalezení nejbližší odpovídající intenzity C_{out}, ale zanedbaná hodnota vzniklá zaokrouhlením C_{in} z vyššího rozsahu do intervalu ⟨0, 1⟩ je použita k modifikaci hodnot sousedních pixelů.



Obr. 6.10: Distribuce chyby podle Floyd-Steinberga

Chceme-li například zpracovat pixel o vstupní intenzitě 5, pak výstupnímu pixelu přiřadíme intenzitu 0 a zanedbanou hodnotu 5 přičteme k nejbližším sousedním pixelům. Obdobně při vstupu intenzity 11 vykreslíme výstupní pixel intenzitou 1 a chybu 4 (= 15-11) odečteme od okolních pixelů. Distribuovaná chyba je pochopitelně mezi sousední pixely pravidelně rozdělena. Jedna z možností je uvedena na obr. 6.10 a nazývá se Floyd-Steinbergova metoda distribuce chyby. Pravidla pro rozdělení chyby do okolí pixelu byla vytvořena empiricky a jsou známy asi 4 různé způsoby distribuce chyby.

V tomto případě jsou na vstup dat kladeny omezující požadavky. Je nutno, aby pixely byly zpracovávány po řádcích (či po sloupcích), tedy nikoliv v libovolném pořadí, jak to dovolovaly předchozí metody. Pro jeden řádek je třeba vymezit jednorozměrné pole - **paměť chyby**. Před vykreslením každého pixelu je ke vstupní hodnotě C_{in} přičtena dosud získaná chyba z okolních (dříve zpracovaných) pixelů a teprve výsledná hodnota je převedena do výstupního rozsahu. Zanedbaná hodnota se opět rozdělí a přičte do **paměti chyby**.

Koeficienty udávající podíl chyby distribuovaný do okolních pixelů (resp. do dočasné paměti chyby) jsou pochopitelně neceločíselné. Vzhledem k celočíselnému charakteru zpracování intenzit je nutno dbát na to, aby nebyla celočíselným dělením zanedbána žádná část. Znamená to, že při rozdělování chyby z jednoho pixelu do jeho sousedů je třeba poslední díl distribuované chyby určit nikoliv uvedeným vzorcem, nýbrž jako rozdíl celé chyby a součtu všech dříve vypočítaných dílů této chyby.

APPLET

applet 6.5

Floyd-Steinbergova metoda dovoluje při výpočtech využít bitového posunu vpravo, a tím zjednodušit výpočty. Vedle tohoto způsobu distribuce chyby jsou uváděny ještě další metody (Burkes, Stucki), spojené se skutečným aritmetickým výpočtem distribuovaných dílů chyby, a tím i s většími časovými nároky ([Knut87]). Často se směr distribuce chyby mění po každém novém řádku a provádí se tedy (angl. *zig-zag*).

APPLET

applet 6.6

Na obrázcích 6.9a-d jsou ukázány různé rozptylovací metody použité na dobovou fotografii Sarah Bernhardtové. Vstupní digitalizovaný obraz měl 256 stupňů šedi.

6.4 Barevné palety

Pro výklad rozptylovacích metod v barevném prostoru použijeme velmi časté situace, kdy vstupní barevné informace jsou vypočítávány (nebo načítány) v kvalitě 24 bitů na pixel a mají být zobrazeny na zařízení s 8 bity na pixel. Předpokládejme dále, že oněch 8 bitů na pixel je indexem do palety, ve které je možné nastavit libovolných 256 odstínů z 16 miliónů možných barevných kombinací.

Kromě volby rozptylovací metody se nyní objevuje i otázka vhodného **nastavení barevné palety**. Je zřejmé, že barevná paleta by měla obsahovat ty barevné odstíny, které se v daném obrázku objevují nejčastěji. Každému obrázku by potom příslušela vlastní paleta barev.

Pokud jsou vstupní data postupně vytvářena nějakým algoritmem a není předem známo barevné složení celého obrázku, je nutno zapsat do palety pevně dané barevné odstíny, které budou později využívány pro kresbu. Jedno z nejjednodušších řešení představuje naplnění palety 256 odstíny šedi. Barevné odstíny na stupně šedi můžeme převést velmi přesně podle vzorce:

$$I = 0.299\ r + 0.587\ g + 0.114\ b, \qquad (6.8)$$

v němž r, g a b jsou základní barevné složky vstupní barvy a hodnota I je výsledná intenzita bílé. Obrázek takto vykreslíme průběžně pouze černobíle, úplné barevné informace můžeme uložit na disk a po důkladnějším rozboru barevného složení obrázku nastavíme barevnou paletu pro jeho závěrečné vykreslení v barvách.

Ukládání úplných barevných informací na disk ovšem zabírá značný diskový prostor a navíc musíme na barevný výsledek čekat až do konečného zpracování všech pixelů. Proto byly vyvíjeny metody, které po pevném (univerzálním) nastavení barevné palety umožní pomocí rozptylování barev dosáhnout kvalitního zobrazení již v průběhu generování obrázku.

6.4.1 Barevná paleta 3-3-2

Paleta, která má být použitelná pro obrázky s různým barevným složením, musí zřejmě obsahovat pravidelně vybrané zástupce z barevné krychle RGB. Krychli však nemůžeme rozdělit ve všech třech osách stejnoměrně, protože pro zakódování jednotlivých barevných zástupců máme jen 8 bitů, které nelze rozdělit na 3 stejné díly. Proto rozdělíme RGB krychli na 8 (= 2^3) řezů v ose R, 8 řezů v ose G a na 4 (= 2^2) řezů v ose B. Počet bitů pro kódování modré bude tedy nižší než pro ostatní dvě základní barvy. Toto omezení není příliš významné - lidské oko je na odstíny modré málo citlivé (viz vzorec 6.8), a tak redukce stupnice modré z 256 možných na 4 odstíny ve většině případů nevadí.

Vrcholy takto vytvořené sítě řezů v krychli RGB představují 256 zástupců do palety, která se standardně nazývá **paleta 3-3-2**. Mezi 8-bitovým obsahem pixelu a barevným složením položky palety je jednoznačný vztah. Má-li pixel hodnotu i, pak i-tá položka palety obsahuje následující velikosti složek r, g a b:

$$\begin{aligned} r &= (((i >> 5\ )\ )\ *\ 255)\ /\ 7, \\ g &= (((i >> 2\ )\ \&\ 7\ )\ *\ 255)\ /\ 7, \\ b &= (((i\ )\ \&\ 3\ )\ *\ 255)\ /\ 3, \end{aligned} \qquad (6.9)$$

Prvé tři bity každého pixelu obsahují kódy 0-7 pro 8 stupňů červené. Jim v paletě 3-3-2 odpovídají velikosti červené složky v osmi stupních, avšak v rozsahu 0-255.

Pro účely rychlého orientačního kreslení obrázku je takto zvolená paleta dostačující již sama o sobě. Vstupní barevnou informaci stačí převést z rozsahu 8-8-8 bitů na příslušné 3-3-2 bity. Tento převod lze provést pomocí celočíselné aritmetiky. Například složku r_8 z 8 bitů převedeme do rozsahu 3 bitů na hodnotu r_3 podle vzorce:

$$r_3 = (r_8 * 7) / 255 \qquad (6.10)$$

a výsledný index I v paletě získáme bitovým složením hodnot redukovaných barevných složek

$$I = (r_3 << 5) + (g_3 << 2) + b_2. \qquad (6.11)$$

Jedná se vlastně o jednoduché zaokrouhlení barvy do 256 předem zvolených hodnot. Kvalitnějšího vzhledu obrázku dosáhneme ovšem teprve tehdy, použijeme-li některé z rozptylovacích technik dříve uvedených pro černobílá zobrazovací zařízení. Tehdy vezmeme do úvahy i onu **zanedbanou hodnotu**, která se ztrácí při převodu z 8 bitů na nižší počet. Tuto hodnotu budeme značit r_{rest} (pro červenou složku). Jednotlivé barevné složky jsou rozptylovány samostatně a výsledná barva vznikne podle vzorce 6.11.

Postup lze rozdělit do několika kroků:

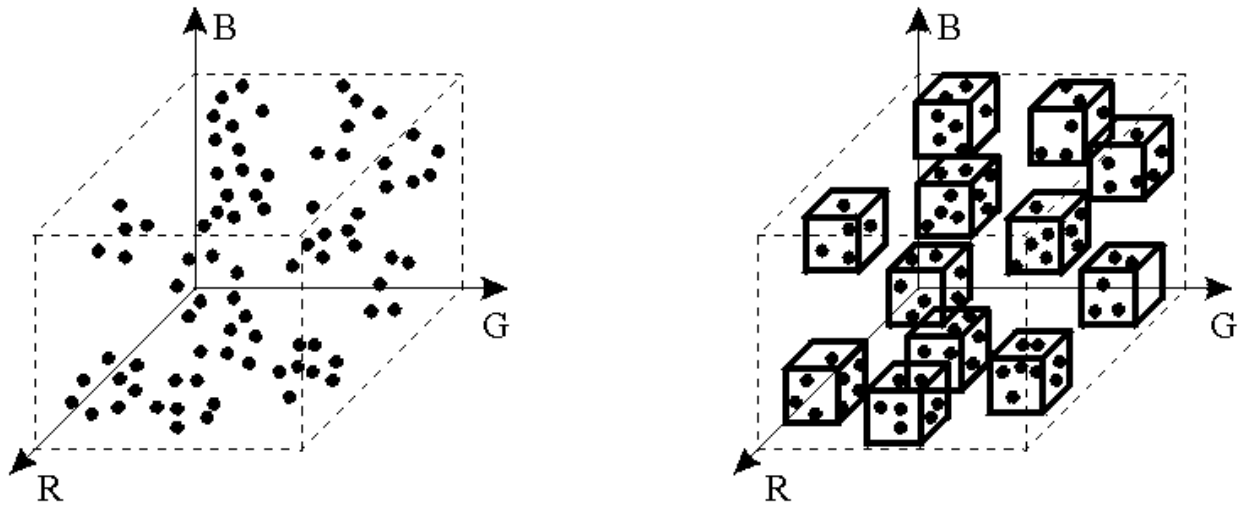
1. Pomocí vzorce 6.10 nejprve nalezneme nejbližší barevnou hodnotu existující v paletě 3-3-2. Ke vstupní intenzitě r_8 tedy najdeme intenzitu r_3.
2. Zároveň určíme zanedbanou hodnotu (např. $r_{rest} \in \langle 0, 31 \rangle$ pro červenou složku), která vznikla zaokrouhlením vstupní barvy do nižšího výsledného rozsahu.
3. Podle použité rozptylovací metody (náhodné nebo maticové rozptýlení) použijeme zanedbanou hodnotu r_{rest} k případnému zvýšení výsledné intenzity r_3 o jedničku.

U metody distribuce chyby je postup obdobný, využívá se však navíc tří pomocných pamětí chyby pro jednotlivé barevné složky.

6.4.2 Adaptivně vytvářená barevná paleta

Pevně daná paleta typu 3-3-2 dovoluje použít rozptylovacích technik, aniž by byla předem známa barevná struktura celého obrázku. Při požadavku na co nejvyšší kvalitu obrazu je takováto paleta nevýhodná, protože v řadě případů obsahuje i takové odstíny, které se na obrázku vůbec nevyskytují, případně jsou zastoupeny v minimálním počtu. Tehdy je na místě vytvořit paletu, která bude odpovídat rozložení barevných odstínů v konkrétním obrázku, a která bude tedy pro každý obrázek definována zvlášť. S tímto přístupem se setkáváme typicky u obrázků kódovaných ve formátu GIF.

Nalezení palety pro daný obrázek je náročné jak časově, tak paměťově. V prvé fázi je vytvářen **histogram** (viz odst. 7.7) četnosti barevných odstínů obsažených v celém obrázku. Pro další výpočty je důležitý nejen fakt, že se určitý odstín v obrázku objevuje, ale i to, jak často je v něm obsažen. Je zřejmé, že vytvoření histogramu představuje zejména na osobních počítačích pěkný oříšek - představme si např. objem paměti potřebný pro histogram všech 2^{18} barevných kombinací pro klasický adaptér VGA. Přitom již poměrně malý obrázek o rozměrech 256×256 pixelů může obsahovat okolo 30-40 tisíc různých odstínů. V praxi se naštěstí ukazuje, že jednotlivé barvy ve výsledné paletě nebývají těsně vedle sebe. To z hlediska hledání palety znamená, že histogram můžeme vytvořit jen podle 5 nejvýznamnějších bitů každé základní barevné složky.



a) b)

obr 6.11: Histogram vstupních odstínů (vlevo) a oblasti blízkých barev (vpravo) při vytváření adaptivní palety

V další části adaptivního vytváření palety je potřeba na základě histogramu nalézt 256 nejvhodnějších zástupců. Zde nám opět poslouží představa barevného prostoru jako krychle RGB (obr. 6.11 vlevo), ve které jsou pomocí teček znázorněny použité odstíny. Naším úkolem je nalézt 256 oblastí (obr. 6.11 vpravo), které mají pokud možno stejné vlastnosti [Heck82]. Požadavky kladené na tyto oblasti mohou být:

- oblasti se tvarem blíží krychli a mají všechny stejnou velikost
- oblasti obsahují stejný počet odstínů

Není jednoznačně dáno, kterou z podmínek vybrat. Algoritmicky jednodušší je první varianta. Při ní se nejprve uzavřou všechny odstíny obsažené v obrázku do omezujícího kvádru a poté se opakovaně provádí dvojice operací:

1. Nalezení oblasti s největším rozměrem v jedné z os R, G, B
2. Rozdělení této oblasti na polovinu v ose s největším rozměrem a vytvoření dvou nových omezujících kvádrů

Tento cyklus se opakuje až do vytvoření 256 oblastí. Každá oblast představuje jednu položku v paletě. Za hodnotu položky palety lze zvolit buď aritmetický průměr barevných odstínů v dané oblasti, nebo ten odstín, který se v oblasti vyskytuje nejčastěji.

Stanovením palety pro daný obrázek ovšem práce nekončí. Vstupní data obrázku je nyní nutno znovu přečíst a zpracovat, tj. pro jejich zobrazení většinou použít rozptylovací techniky. Na rozdíl od výše uvedených metod již nelze zpracovávat barvu pixelu po jednotlivých základních složkách, nýbrž jako jeden celek. Vždy je tedy třeba pro danou vstupní barvu nalézt nejbližší barvu existující v paletě. Nalezenou barvu použijeme k vykreslení pixelu na obrazovce. Žádáme-li co nejkvalitnější obraz, musíme dále zpracovat rozdíl mezi vstupní barvou a zobrazovanou barvou z palety. Jako jediná možná technika se jeví distribuce chyby.

Nalezení nejbližší existující barvy v paletě je časově náročné. Pokud udržujeme v paměti informace o barevných kvádrech vzniklých dělením barevného prostoru, spočívá hledání jen v porovnání složek R, G, B vstupní barvy s mezními rozměry oblastí. Nemáme-li údaje o dělení barevného prostoru zachovány, prohledává se všech 256 položek palety a hledá se euklidovsky nejbližší hodnota.

Samotná kombinace hledání palety pomocí stejně velkých oblastí a následná distribuce chyby nemusí vždy dávat dokonalé výsledky. Zástupci do palety jsou vybráni pomocí aritmetického průměru z každé oblasti a při následném zpracování vstupních odstínů může větší skupina stejných barev z okraje oblasti způsobit kumulaci chyby směrem k další oblasti, opticky i značně odlišné. Již z této úvahy je vidět, že charakter vstupních dat a střídání barevných odstínů má významný vliv na výsledný vzhled obrázku. Určitého zlepšení docílíme zařazením všech osmi vrcholů základní barevné krychle RGB mezi položky palety.

Významným faktorem, který jsme zatím do tvorby adaptivní palety nezařadili, je charakter vnímání barev lidským okem. Určitého zlepšení dosáhneme, vytváříme-li barevné oblasti nikoliv dělením v polovině, ale dělením, které odpovídá vztahu 6.8. Znamená to, že cílem dělení jsou kvádry s délkami stran R, G, B v poměru 3,3 : 1,7 : 10 (převrácené hodnoty koeficientů ze vztahu 6.8). V této souvislosti lze také přejít do jiného barevného prostoru, např. HSV nebo HLS a vytvářet oblasti vhodných vlastností v něm [Fole90].

APPLET

applet 6.7

Literatura:

- [Hack82] Hackbert, P.S.: Color Image Quantization for Frame Buffer Displays. Proceeding of SIGGRAPH 82, Computer Graphics, 16 (3), 297-307
- [Fole90] Foley, J., van Dam, A., Feiner, S., Hughes, J.: Computer Graphics. Principles and Practice. Second Edition. Addison-Wesley Publ. Comp., Reading Massachusets, 1990.
- [Knut87] Knut, D.: Digital Halftones by Dot Diffusion. ACM Transactions on Grapnics, 6 (4), October 1987, pp. 245-273.

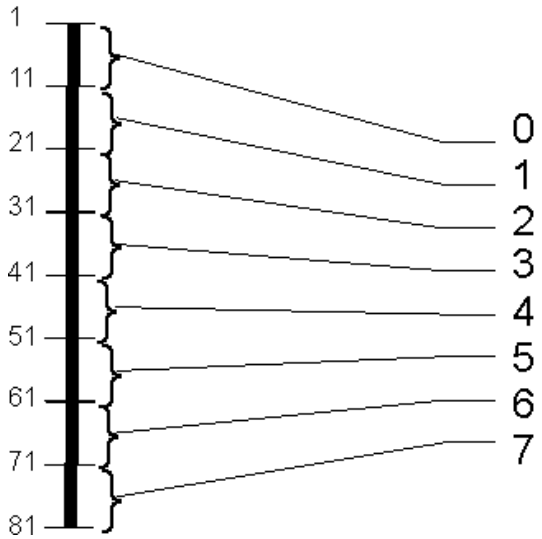
7. Úpravy obrazu

Chápejme obraz intuitivně, jako průmět reálného světa na sítnici oka, jako fotografii, obraz na obrazovce počítače, obrázek na papíře, či odraz vodní hladiny. Při formálním vymezení použijeme matematický model obrazu, kterým je spojitá funkce dvou proměnných - **obrazová funkce** ([Gonz87]).

$$f(x,y) \tag{7.1}$$

Definičním oborem funkce (7.1) je v idealizovaném případě spojitý interval $I \subset R^2$, x a y jsou souřadnice bodu ve 2D, v němž obecně nabývá funkce hodnoty z intervalu $< 0,+\infty)$. Způsobem získání obrazu je potom obor hodnot obrazové funkce určen.

Hodnota obrazové funkce bývá dána konkrétní potřebou, může jí být jediné reálné číslo (jas, červená barva, atp.), v počítačové grafice pak obvykle 3 hodnoty R-G-B, H-S-V, H-L-S, aj. (viz kapitola 6), některá hardwarová zařízení pracují s bodem, který je reprezentován čtyřmi souřadnicemi R-G-B-A. Symbol A bývá také označován α a reprezentuje procento průhlednosti bodu. Počítačové vidění pak často používá data, která reprezentují celé spektrum (například data z počítačového tomografu, či spektrální měření svitu hvězd). Někdy bývá výhodné reprezentovat bod jako komplexní číslo ([Lewi90]). V konkrétní realizaci je obor hodnot funkce (7.1) omezen fyzikálními vlastnostmi technického vybavení. V počítačové grafice se často pracuje s 256 stupni šedi, s 256 barvami z 16, 32 nebo 64 miliónů možných, nebo přímo s 64 milióny barev (hovoříme potom o true-color zobrazení), v DTP se často pracuje jen se dvěma barvami atp.



obr. 7.1: Kvantování

7.1 Digitalizace

Fyzikální realizaci obrazové funkce odpovídá nějaké zařízení, v našem případě stínítko obrazovky, papír tiskárny, fotografický papír, film atp. Ve všech těchto případech je definičním oborem obrazové funkce diskrétní a konečná množina bodů (například rozlišení obrazovky) a i obor hodnot je omezen na konečné číslo (množství současně zobrazitelných barev). Při získání obrazu tedy dochází ke ztrátě informace přechodem od spojité funkce (za předpokladu, že obraz skutečně je spojitou funkcí) k diskrétní a to jak v jejím definičním oboru tak v oboru hodnot. Tento proces se nazývá **digitalizace** a odehrává ve dvou krocích, jimiž jsou: **kvantování** a **vzorkování**. V některých případech se tento proces děje automaticky, fotografujeme-li například na černobílý film, ztrácíme barevnou informaci, jindy se o něj snažíme sami - chceme-li například zobrazit obrázek s velkým množstvím barev za pomoci počítače, který má počet barev omezen (viz kap. 6).

Kvantování spočívá v rozdělení oboru hodnot na konkrétní intervaly, jimž je pak přidělena jediná zástupná hodnota, viz obrázek obr.7.1.. Podle způsobu rozdělení kvantované veličiny hovoříme o kvantizaci **uniformní** a **neuniformní**. Uniformní kvantizace používá konstantní délku intervalu. V případě neuniformní kvantizace můžeme například využívat určitých lokálních vlastností v obraze, hovoříme potom o adaptivní kvantizaci.

Vzorkováním (viz obrázek obr.7.2.) (angl. *sampling*) rozumíme odebírání určitých hodnot - vzorků, které funkci reprezentují. Matematicky je vzorkování popsáno vztahem (7.2)

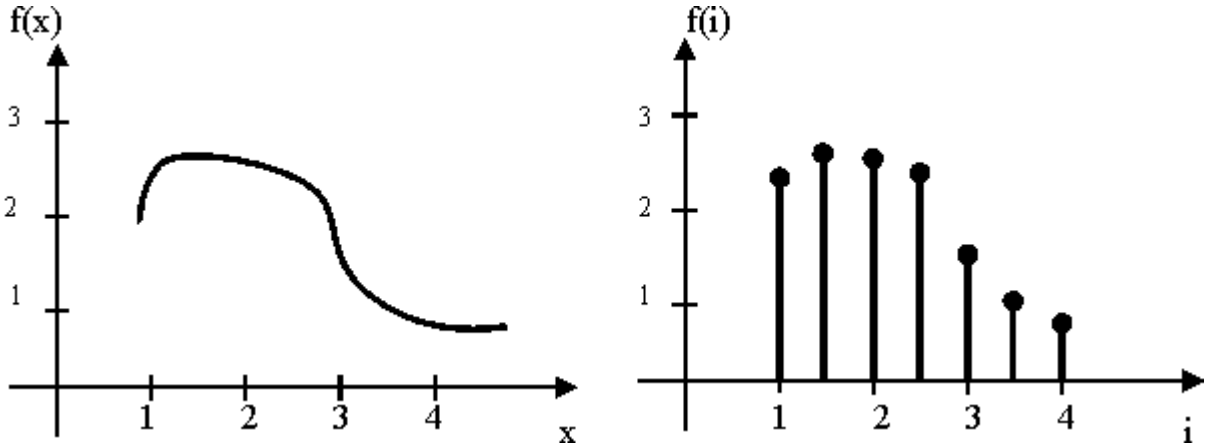
$$f_d(i) = f(x_0+i\Delta), \qquad (7.2)$$

nová funkce $f_d(i)$ nabývá v diskrétních hodnotách přirozených čísel $i = 0,1,...,n$ hodnot původní spojité funkce $f(x)$. Vzdálenost vzorků je konstantní a je určena hodnotou Δ, vzorkování začíná v bodě x_0. Výsledkem vzorkování do rastru obrazovky ve 2D je funkce $f_d(i,j)$, která aproximuje funkci (7.1) obdélníkovými oblastmi-pixely.

Příkladem vzorkování je digitální nahrávka na kompaktním disku. V určitém časovém intervalu, tj. s určitou frekvencí, je ze spojitého zvukového signálu zaznamenána jeho okamžitá hodnota. Čím je vzorkovací frekvence vyšší, tím je původní signál reprezentován přesněji.

Vzorkovací frekvenci a informaci nesenou v původní funkci uvádí do vztahu **Shannonova vzorkovací věta** ([Gonz87]):

Signál spojitý v čase je plně určen posloupností vzorků odebíraných ve stejných intervalech, je-li jejich frekvence větší než dvojnásobek nejvyšší frekvence v signálu.



obr. 7.2: Vzorkování

Při reprezentaci obrazu v počítačové grafice vzorkujeme nejčastěji do rozlišení obrazovky, jemnější vzorkování (angl. *oversampling*) bývá využito pro **antialiasing** - vyhlazení "zubatých" částí či hran v obraze. Každý pixel je pak vzorkem původní spojité funkce a Shannonův teorém v podstatě říká, že nejmenší geometrický detail z původního obrazu je zachován, pokud je velikost pixelu alespoň dvakrát menší.

7.2 Fourierova transformace

Názornou a často používanou metodou teorie signálů, je reprezentace signálu pomocí jeho frekvenční charakteristiky. Z akustiky jsou každému člověku zřejmé pojmy "hloubek" a "výšek", jejich potlačení, zvýraznění atp. Zcela stejné metodologie se používá při zpracování obrazu, který je také možno vyjádřit jako složení sinusových signálů různé fáze a amplitudy. K této metodě navíc přistupuje fakt, že se v případě zpracování obrazu nepracuje se spojitým, ale s diskrétním obrazem a některé postupy se tak zjednodušují. Na rozdíl od akustiky se však pracuje ve dvou rozměrech.

Při úvahách o funkci reprezentované její frekvenční charakteristikou máme tedy k disposici její dvě reprezentace; budeme říkat že se pohybujeme ve dvou oborech nebo také oblastech. V prvním případě, kdy pracujeme s obrazovými elementy, se pohybujeme v **prostorové oblasti**, v případě druhém hovoříme o **oblasti frekvenční**. Přechod od prostorové k frekvenční oblasti se nazývá **přímá Fourierova transformace**. Přímá Fourierova transformace spojité a integrovatelné komplexní funkce f(x), je definován vztahem:

$$F(u) = F\{f(x)\} = \int_{-\infty}^{+\infty} f(x)e^{-i2\pi ux}dx \qquad (7.3)$$

Přímou Fourierovu transformaci budeme zapisovat F{f(x)}, nebo zkráceně F(u).

Zpětná Fourierova transformace, přechod od frekvenční oblasti k prostorové, komplexní integrovatelné funkce F(u), se označuje $F^{-1}(u)$ a je definována vztahem

$$f(x) = F^{-1}(u) = \int_{-\infty}^{+\infty} F(u)e^{+i2\pi ux}du. \qquad (7.4)$$

Fakt, že f(x) a F(u) jsou dvojím vyjádřením téže funkce svázané relací Fourierovy transformace, bývá někdy zvykem označovat:

$$f(x) \Leftrightarrow F(u)$$

Funkce f(x) i její Fourierův obraz jsou obecně komplexní. V praxi bývá funkce f(x) reálnou funkcí (reprezentuje digitální obraz) a její protějšek funkcí komplexní (reprezentuje frekvence a amplitudy sinusových signálů). Použijeme-li pro vyjádření exponentu ve vztahu (7.3) a (7.4) *Eulerovu formuli*

$$e^{+/-i2\pi ux} = \cos 2\pi ux +/- i\sin 2\pi ux,$$

dostaneme vyjádření Fourierovy transformace v názornější podobě:

$$F(u) = \int_{-\infty}^{+\infty} f(x)\,(\cos 2\pi ux - i\sin 2\pi ux)\,dx$$

$$f(x) = \int_{-\infty}^{+\infty} F(u)(\cos 2\pi ux + i\sin 2\pi ux)\,dx. \qquad (7.5)$$

Každé komplexní číslo může být vyjádřeno svou velikostí (modulem) a úhlem. Funkci

$$|F(u)| = \sqrt{Re^2(u)+Im^2(u)} \qquad (7.6)$$

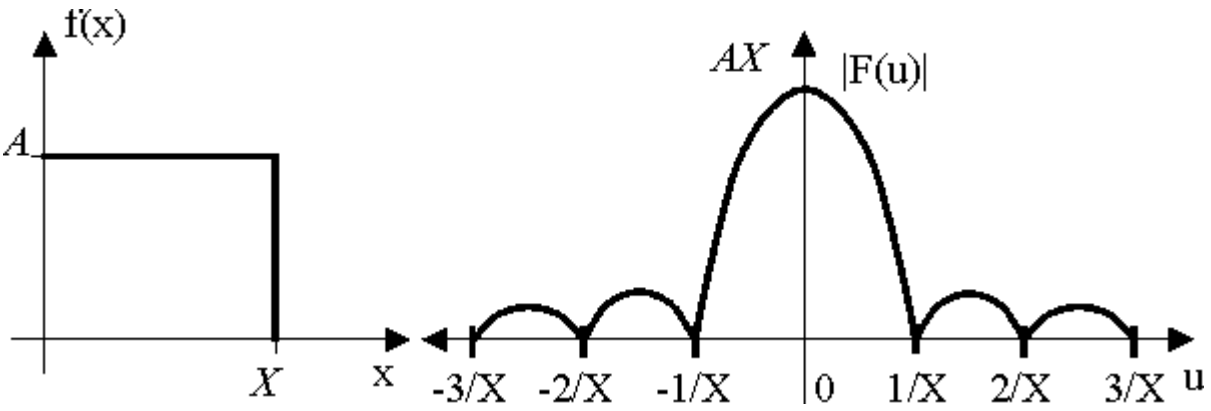
nazveme **Fourierovo spektrum** (nebo pouze spektrum) funkce f(x) viz obrázek obr.7.3.. **Fázovým úhlem** funkce f(x) nazveme funkci

$$\phi(u) = \tan^{-1}\left[\frac{Im(u)}{Re(u)}\right]. \qquad (7.7)$$

Pro zobrazování frekvenčních charakteristiky se také používá čtverec spektra:

$$P(u) = |F(u)|^2 = Re^2(u)+Im^2(u) \qquad (7.8)$$

Obrázek obr.7.3. ukazuje ohraničenou konstantní funkci a její spektrum.



obr. 7.3: Konstantní ohraničená funkce a její spektrum

V reálných datech dochází k rychlému poklesu amplitudy spektra, a proto se pro jejich zobrazování používá funkce kompenzující tento útlum (7.9)

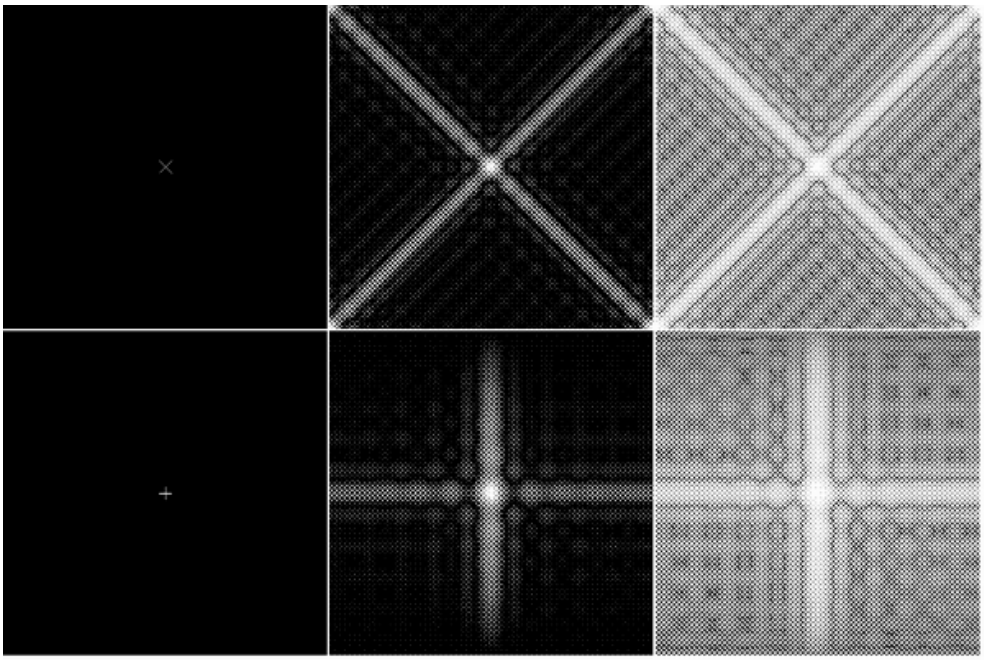
$$D(u) = \log(1+|F(u)|). \tag{7.9}$$

Zobecnění Fourierovy transformace pro funkce dvou proměnných f(x,y), má následující podobu:

$$F(u,v) = F\{f(x,y)\} = \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} f(x,y)e^{-i2\pi(ux+vy)}dxdy$$

$$f(x,y) = F^{-1}(u,v) = \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} F(u,v)e^{+i2\pi(ux+vy)}dudv. \tag{7.10}$$

Otočení předlohy ve 2D odpovídá i otočení jejího Fourierova obrazu viz obrázek obr.7.4.



obr. 7.4: Otočení předlohy odpovídá otočení Fourierova obrazu. Zleva je předloha, její Fourierův obraz a její spektrum s kompenzovaným útlumem D(u,v).

Některé další vlastnosti Fourierovy transformace shrnuje tabulka tab.7.1.

Vlastnost	*Prostorová oblast*	*Frekvenční oblast*
linearita	$\alpha_1 f_1(x) + \alpha_2 f_2(x)$	$\alpha_1 F_1(u) + \alpha_2 F_2(u)$
změna měřítka v prostorové oblasti	$f(ax)$	$^1/_a F(^u/_a)$
změna měřítka ve frekvenční oblasti	$^1/_a f(^x/_a)$	$F(au)$
posun v prostorové oblasti	$f(x-a)$	$F(u)e^{-i2\pi a}$
posun ve frekvenční oblasti	$f(x-a)e^{i2\pi ax}$	$F(u-a)$
konvoluce	$g(x) = f(x)*h(x)$	$G(u) = F(u)H(u)$
násobení	$g(x) = f(x)h(x)$	$G(u) = F(u) * H(u)$

tab. 7.1: Vlastnosti Fourierovy transformace. Symbol * označuje konvoluci.

Z tabulky je zřejmé, že **konvoluce** dvou funkcí v prostorové oblasti odpovídá součinu jejich obrazů v oblasti frekvenční a naopak.

$$f(x,y)*h(x,y) \Leftrightarrow F(u,v)H(u,v)$$

$$f(x,y)h(x,y) \Leftrightarrow F(u,v)*H(u,v)$$

Konvoluce funkcí f(x,y) a h(x,y) je definována vztahem (7.11)

$$f(x,y)*h(x,y) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f(k,l)h(x-k,y-l)dkdl. \qquad (7.11)$$

Konvoluci je možné chápat jako posunující se a zkreslující "okno" funkce h(x,y), kterým zkoumáme funkci f(x,y). Funkce h(x,y) se také označuje jako **jádro konvoluce**.

Při posuzování funkcí z hlediska jejich frekvenční charakteristiky platí že:

1. obrazem spojité a integrovatelné funkce je funkce spojitá,
2. obrazem periodické funkce je funkce diskrétní (například funkce

$$F(u) = \sum_{x=0}^{\infty} [(\sin(2i+1)u)/(2i+1)]$$

   je spojitá a je obrazem posloupnosti obdélníkových pulsů),
3. obrazem periodických rovnoměrných vzorků (zde v obdélníkovém rastru) jsou periodické rovnoměrně rozložené vzorky, a konečně
4. Fourierova transformace je **separabilní**, tj. 2D (a obecně n-D) Fourierovu transformaci je možno počítat postupně, použitím jednodimenzionální transformace, (viz vztah (7.13)), pro jednotlivé její proměnné.

Pro další text je nejdůležitější předposlední vlastnost.

7.2.1 Diskrétní Fourierova transformace

Předmětem **diskrétní Fourierovy transformace** (dále DFT) je, stejně jako ve spojitém případě, nalezení funkce určující amplitudy a fázové posuvy sinusových signálů, tentokrát však pro funkci

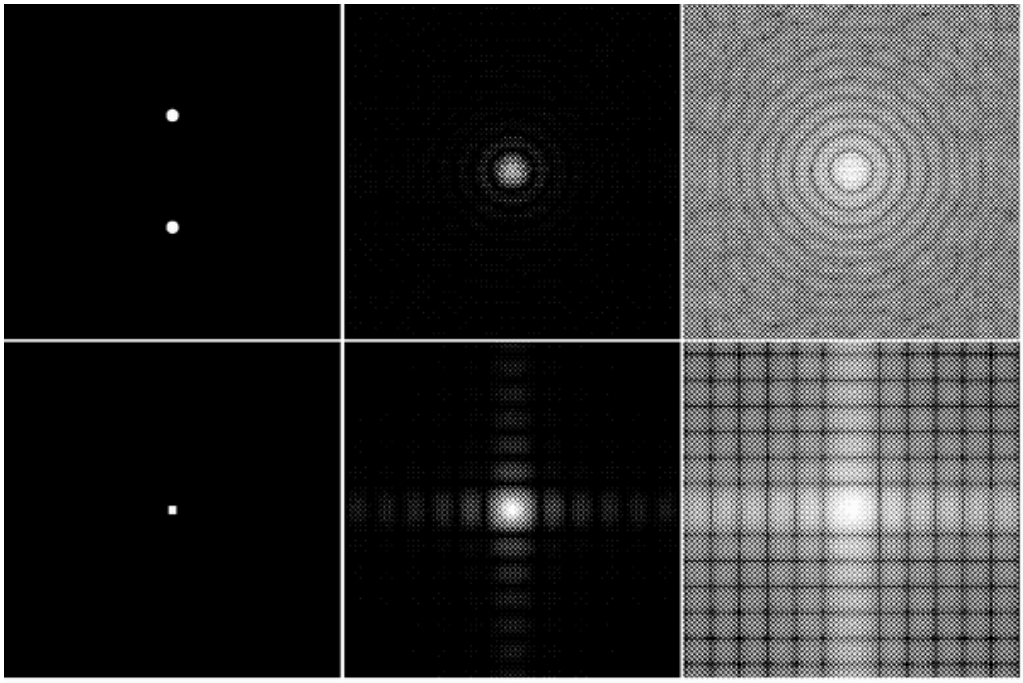
reprezentovanou jako samostatné hodnoty. Ze třetí vlastností Fourierovy transformace uvedené v předchozím odstavci a z faktu, že obraz v počítači vznikl uniformním vzorkováním nějaké (teoreticky) spojité funkce, navíc plyne, že je možno najít jeho Fourierův obraz jako konečné množství komplexních čísel, neboli amplitudy a fáze konečného množství harmonických. Jediný problém, neperiodicita obrazu, je vyřešen takzvaným periodickým prodloužením; funkce definována na konečném intervalu se chápe jako jediná perioda periodické funkce. Z Shannonova teorému navíc plyne, že takový obraz je určen plně.

Přímá a zpětná DFT pro dvojrozměrný případ a pro N×M vzorků funkce f(i,j) má tvar

$$F(u,v) = \frac{1}{MN} \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y) e^{-i2\pi([ux/N]+[vy/M])}$$

$$f(x,y) = \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} F(u,v) e^{+i2\pi([ux/N]+[vy/M])} \qquad (7.12)$$

Na obrázku obr.7.5. jsou některé základní funkce a jejich spektra.



obr. 7.5: Příklad 2D funkcí, jejich spekter a jejich spekter s kompenzovaným útlumem D(u,v)

Složitost DFT při výpočtu z definice, která je pro jednodimenzionální případ $O(n^2)$, ji činní pro praxi nepoužitelnou. V roce 1960 J. W. Cooley a J. W. Tukey publikovali algoritmus rychlé Fourierovy transformace (angl. *fast fourier transformation*), dále FFT, který způsobil velmi významné rozšíření Fourierovy transformace v oblasti digitálního zpracování obrazu, neboť FFT má složitost $O(N\log_2 N)$. Implementaci FFT je možné nalézt například v [Pres94].

APPLET

applet 7.1

Diskrétní Fourierova transformace je separabilní:

$$F(u,v) = \frac{1}{M} \sum_{y=0}^{M-1} \left[\frac{1}{N} \sum_{x=0}^{N-1} f(x,y) e^{-[(i2\pi ux)/N]} \right] e^{-[(i2\pi vy)/M]}$$

$$f(x,y) = \sum_{v=0}^{M-1} \left[\sum_{u=0}^{N-1} F(u,v) e^{+[(i2\pi ux)/N]} \right] e^{+[(i2\pi vy)/M]} \qquad (7.13)$$

Diskrétní konvoluce má tvar (7.14)

$$g(i,j)*h(i,j) = \sum_{n=-k}^{k} \sum_{m=-k}^{k} f(i-n,j-m)h(n,m). \qquad (7.14)$$

Funkce g(i,j) odpovídá obrazu, který je zpracováván, a h(n,m) je **konvoluční jádro**. V operacích s obrazem bývají používána malá jádra s lichým počtem prvků; 3×3, 5×5, 7×7 atd. Vztah (7.14) určuje novou hodnotu bodu o souřadnicích [i,j] jako lineární kombinací bodů z jeho okolí. Velikost okolí je určena právě koeficientem k, který zásadním způsobem ovlivňuje kvalitu a rychlost výpočtu. Je-li tato konstanta příliš velká, může být výhodné převést obraz i konvoluční jádro do frekvenční oblasti, vynásobit oba nové obrazy a převést zpět.

Výhodnou vlastností, kterou je možno využít stejně jako u DFT i zde, je separabilita. Pokud je možno konvoluční jádro h rozepsat podle vztahu (7.15)

$$h(i,j) = h_i(i)h_j(j), \qquad (7.15)$$

je možné počítat konvoluci po částech:

$$f(i,j)*h(i,j) = \sum_{n=-k}^{k} \sum_{m=-k}^{k} f(i-n,j-m)h_i(n)h_j(m). \qquad (7.16)$$

7.3 Převzorkování

Mezi základní operace aplikované v prostorové oblasti patří především geometrické transformace. Geometrické transformace je možno rozdělit na **lineární** a **nelineární**. Lineární transformace, mezi které patří především afinní transformace, jsou popsány v kapitole 1. V dalším textu si všimneme pouze změny měřítka, která s sebou nese díky diskrétním vlastnostem obrazu určité problémy. Mezi nelineární transformace patří digitální **morphing** a **warping**. Obě operace budou popsány v dalším textu.

7.3.1 Změna rozlišení

Velice častým požadavkem je změna rozlišení obrazu (angl. *zoom*), a to jak jeho zmenšení tak zvětšení. V literatuře často splývá pojem změna rozlišení s pojmem převzorkování (angl. *resampling*). Základní myšlenka této metody spočívá v rekonstrukci digitálního obrazu do jeho spojité podoby tak, že se původní diskrétní vzorky proloží nějakou spojitou funkcí - plochou, kterou je možno vzorkovat libovolně. Metody rekonstrukce se liší v podstatě jen tím, jakou funkcí se body digitálního obrazu interpolují či aproximují. Převzorkování se také používá při rotaci obrazu, posunu v diskrétní mřížce o velikost jinou než je velikost obrazového elementu atp. Protože všechny tyto operace jsou separabilní,

uvedeme si vesměs jejich jednodimenzionální podobu. Pro jejich vyjádření s výhodou použijeme konvoluce.

Patrně nejjednodušší metoda výpočtu interpolační plochy z diskrétních vzorků se nazývá **nearest neighbor interpolation** (nebo *point shift* nebo také *sample and hold*) a spočívá v duplikaci řádku či sloupce tak, aby došlo k žádoucí změně velikosti. Výpočet nové funkce g(x) má tvar (7.17)

$$g(x) = f(x_k); \quad \frac{x_{k-1}+x_k}{2} < x \le \frac{x_k+x_{k+1}}{2}, \tag{7.17}$$

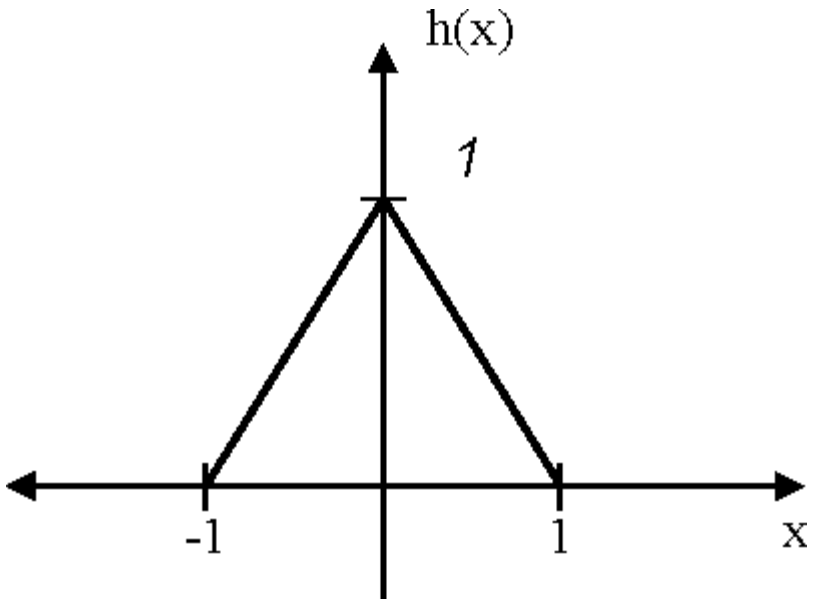
konvoluční jádro této operace má tvar (7.18)

$$h(x) = \begin{cases} 1 & \text{pro: } 0 \le |x| < 0.5 \\ 0 & \text{pro: } 0.5 \le |x| \end{cases} \tag{7.18}$$

a její frekvenční charakteristika je na obrázku obr.7.3.. Tato metoda je rychlá a jednoduchá, ale vede k nežádoucímu efektům například na hranách s malým sklonem, při zvětšení zvýrazňuje skoky a při zmenšení poškozuje tenké čáry. To tuto metodu předurčuje zejména k rychlému přehlédnutí (angl. *preview*) velkého obrázku. Bývá také implementována v technickém vybavení počítačů.

Přesnější metodou je **lineární interpolace**. Tato metoda spočívá v respektování vzdálenosti mezi dvěma sousedními vzorky f_0 a f_1 . Ty se proloží úsečkou a hodnota v bodě x se vypočítá prostou interpolací (7.19)

$$f(x) = f(0)+\left[\frac{x-x_0}{x_1-x_0}(f_1-f_0)\right]. \tag{7.19}$$



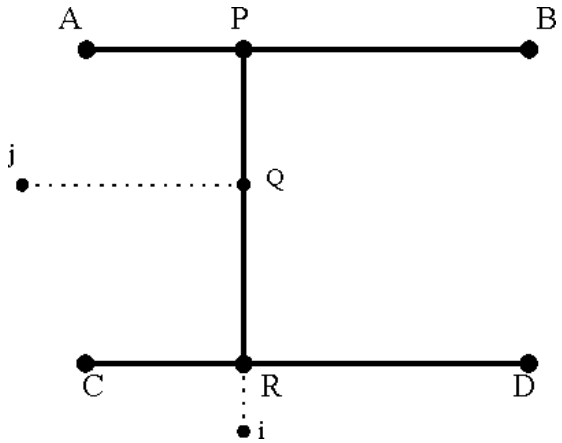
obr. 7.6: Konvoluční jádro lineární interpolace

Odpovídající konvoluční jádro má tvar (7.20) a bývá pro svůj tvar také nazýváno *tent function* (viz obrázek obr.7.6.).

$$h(x) = \begin{cases} 1-|x| & \text{pro: } 0 \le |x| < 1 \\ 0 & \text{pro: } 1 \le |x| \end{cases} \tag{7.20}$$

Jak bylo řečeno, tyto operace jsou separabilní. Uvedeme si proto alespoň v tomto případě dvojdimenzionální případ, tak zvanou **bilineární interpolaci**. Původní body obrazu jsou proloženy

po částech lomenou plochu. Hodnota v nových bodech je pak určena interpolací čtyř nejbližších sousedů. Princip této metody je patrný z obrázku obr.7.7.. Souřadnice bodů A,B,C,D stejně jako hodnoty funkce f(i,j) v těchto bodech jsou známy. Hledáme hodnotu v bodě $f_Q[i,j]$, která se spočítá z hodnot v bodech $f_P[i,y_A]$ a $f_R[i,y_C]$ podle vztahu (7.19). Funkční hodnoty funkce f(x,y) v bodě P a R jsou neznámé a spočítají se z hodnot v bodech A a B resp. C a D podle téhož vztahu. Tato metoda je rychlá, neboť se pro výpočet každého bodu používají pouze čtyři body z jeho okolí. Nevýhodou však je, že rozmazává původní ostré přechody.



obr. 7.7: Bilineární interpolace

Poměrně zajímavých výsledků je dosaženo, je-li jako interpolační plocha použita jiná plocha než plocha po částech lomená. Tyto metody dávají velmi kvalitní výsledky hlavně proto, že pro výpočet hodnoty v novém bodě využívají více informací z původní předlohy než metody uvedené výše. Jejich výpočetní složitost však obyčejně bývá výrazně vyšší. Některé postupy používají polynomiální funkce, konkrétně v počítačové grafice oblíbené kubiky. Používají se tzv. dvouparametrické filtry, kde dva koeficienty interpolace jsou na volbě uživatele (např. *Mitchelova interpolace*).

Jinou metodou je použití kubické interpolace či aproximace pomocí B-spline ploch. Konvoluční jádro aproximace okolí bodu B-spline plochou má tvar (7.21).

$$h(x) = \frac{1}{6}\begin{cases} 3|x|^3-6|x|^2+4 & \text{pro: } 0 \le |x| < 1 \\ -|x|^3+6|x|^2-12|x| & \text{pro: } 1 \le |x| < 2 \\ 0 & \text{pro: } 2 \le |x| \end{cases} \qquad (7.21)$$

Tato interpolace bývá také označována jako **Parzenovo okno**. Přirozeně, čím větší je počet bodů ze kterých se interpolace počítá, tím lepších výsledků bývá dosaženo a tím vyšší je i výpočetní složitost.

Existuje velké množství interpolací - uveďme například Kaiserovo okno, Hananovo, Gausovské či Lanczosovo okno ([Wool90]).

APPLET

applet 7.2

7.4 Odstraňování šumu

Pro popis šumu se používá teorie pravděpodobnosti. **Bílý šum** je šum, jehož pravděpodobnost výskytu je stejná pro všechny frekvence. **Gaussův šum** má dánu pravděpodobnost výskytu vztahem

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{[-(x-\mu)^2/(2\sigma^2)]}$$

kde μ je střední hodnota a σ je střední kvadratická odchylka ([Hlav92]).

Odstraňování šumu závisí na způsobu získání obrazu a na charakteristice šumu. Pokud máme k dispozici více exemplářů téhož obrazu, a není-li jejich zdroj zatížen systematickou chybou, je nejsnazším odstraněním šumu výpočet obrazu, ve kterém bude každý pixel průměrem nebo mediánem (viz alg.7.1.) z pixelů ležících na stejných souřadnicích ve všech obrazech předlohy.

Tento případ bohužel není příliš častý. Obyčejně máme k dispozici pouze jediný exemplář obrazu a odstraňujeme šum na základě lokální charakteristiky okolí každého pixelu, za předpokladu že se okolí pixelu příliš nemění. V lepším případě známe charakteristiku šumu.

7.4.1 Filtry pracující v prostorové oblasti

Nejjednodušší filtrovací metody vycházející z lokální statistiky okolí počítají hodnotu pixelu jako průměr z jeho okolí. Tím dochází k potlačení vysokých frekvencí (šumu), ale také k rozmazání hran. Tato technika se nazývá **obyčejné průměrování**. Konvoluční jádro pro okolí 3×3 má tvar

$$h = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}.$$

Snížením vah směrem od středu získáme filtr, který detekuje Gaussův šum:

$$h = \begin{bmatrix} 1/16 & 2/16 & 1/16 \\ 2/16 & 4/16 & 2/16 \\ 1/16 & 2/16 & 1/16 \end{bmatrix}.$$

Jinou možností je použít techniku pracující na základě lokální statistiky (angl. *order statistics*) pixelu. Příkladem je filtrace pomocí **mediánu**, viz obr.7.12.. Tato technika spočívá v reprezentaci okolí bodu f(i,j) jeho mediánem. Připomeňme, že medián z určitých čísel získáme jako hodnotu, ležící uprostřed setříděné posloupnosti viz alg.7.1.. Okolí bodu, které se používá nemusí být nutně obdélníkové, výhodné je okolí ve tvaru kříže ([Hlav92]). Tato metoda narušuje tenké čáry a ostré rohy, ale nerozmazává hrany.

APPLET

applet 7.3

7.4.2 Filtry pracující ve frekvenční oblasti

Filtry pracující ve frekvenční oblasti vycházejí z předpokladu, že šum je v obraze reprezentován vysokými frekvencemi. Tyto frekvence jsou buď jednoduše odřezány (*low-pass filter*) nebo potlačeny s nějakou váhou (*Butterworth-filter* na obr.7.9.).

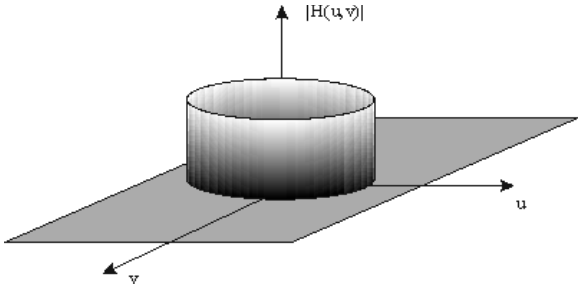
Konvoluční jádro ideálního nízkofrekvenčního filtru *ILPF* (angl. *ideal low-pass filter*), je popsán vztahem (7.22):

$$H(u,v) = \begin{cases} 1 \text{ pro: } \sqrt{(u^2+v^2)} \le D_0 \\ 0 \text{ pro: } \sqrt{(u^2+v^2)} > D_0 \end{cases} \qquad (7.22)$$

Vzdálenost D_0 bývá určována v procentech všech frekvencí v obraze. Slovo "ideální" akcentuje fakt, že filtr vůbec nepropouští frekvence vyšší než je hodnota D_0, viz obrázky obr.7.8. a obr.7.3..

```
for(i=0+k;i<resolutionX-k;i++)
for(j=0+k;j<resolutionY-k;j++)
{
 index=0;
 for(n=-k;n<k;n++)
 for(m=-k;m<k;m++)
 {
  hlp[index++]=g[i-n][j-m];  /* načti do pole okolí pixelu */
 }
 sort(hlp,index);  /* setřiď pomocné pole */
 f[i,j]=hlp[index/2];  /* vezmi jeho prostředek */
}
```

Algoritmus 7.1: Filtrace pomocí mediánu (n je liché číslo)

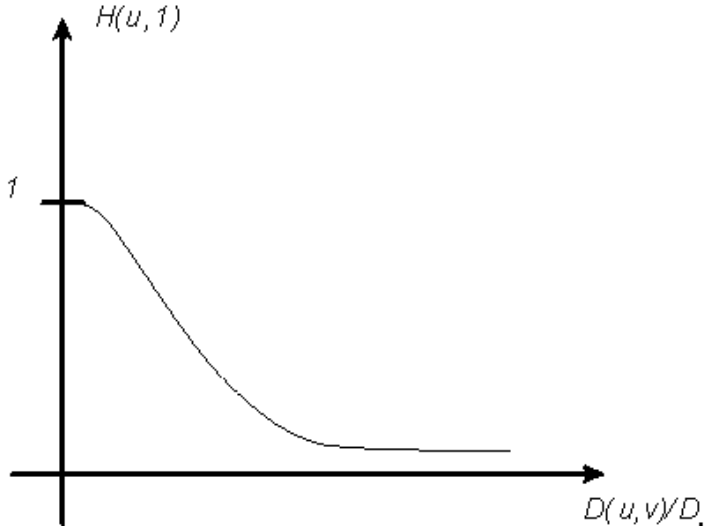


obr. 7.8: ILPF

Filtr *Butterworth* pracuje obdobně s tím, že vysoké frekvence nejsou odřezány, ale potlačeny s funkcí (7.23)

$$H(u,v) = \frac{1}{1+\left[\sqrt{(u^2+v^2)} / D_0\right]^{2n}} \qquad (7.23)$$

Při eliminaci šumu z obrazu při neznámé degradaci narazíme často na potřebu nalezení "nejlepšího" filtru. Existují optimalizační metody, které pro danou třídu obrazů hledají nejlepší filtr, některé z nich je možno vyhledat v [Gonz87].



obr. 7.9: Filtr typu Butterworth

APPLET

applet 7.4

7.5 Ostření obrazu

7.5.1 Ostření obrazu v prostorové oblasti

Ostření obrazu v prostorové oblasti využívá faktu, že lidské oko snáze porozumí obrazu ve kterém jsou výrazné hrany. Ostření obrazu je tedy založeno na detekci hran. Hrana (angl. *edge*) je určena **gradientem**, tj. velikostí a směrem. Gradient je určen vektorovým operátorem nabla

$$\nabla f(x,y) = \left(\frac{\partial f(x,y)}{\partial x}, \frac{\partial f(x,y)}{\partial y} \right) \tag{7.24}$$

a jeho velikost se spočítá podle známého vztahu:

$$|\nabla f(x,y)| = \sqrt{\left(\frac{\partial f(x,y)}{\partial x} \right)^2 + \left(\frac{\partial f(x,y)}{\partial y} \right)^2} \tag{7.25}$$

Označme s(i,j) funkci, která reprezentuje velikost gradientu v bodě [i,j] obrazu f(i,j). Obraz f(i,j), který bude ostřen koeficientem ostření c je definován:

$$g(i,j) = f(i,j) + c.s(i,j) \tag{7.26}$$

Ze vztahu 7.26 vidíme, že principem ostření je zvýraznění hran viz obr. 7.12. Funkce s(i,j) vrací velikost gradientu a o její patřičný násobek se zvýší intenzita pixelu v odpovídajícím bodě. Nejednoznačné je v digitálním obraze určení gradientu nabla f(i,j).

Nejjednodušší metodou určení velikosti gradientu pixelu je použití **Robertsova operátoru**, který používá ke svému výpočtu čtyř jeho nejbližších sousedů a má tvar:

$$|\nabla(f(i,j)| = |f(i,j) - f(i+1,j+1)| + |f(i,j+1) - f(i+1,j)| \qquad (7.27)$$

Vidíme, že výpočet je určením změny jasu ve směru hlavní a vedlejší diagonály.

Výpočet rozdílu intenzity ve směru kolmém na souřadné osy systému používá **Laplaceův operátor** Delta pro čtyřokolí. Jeho konvoluční maska má tvar:

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

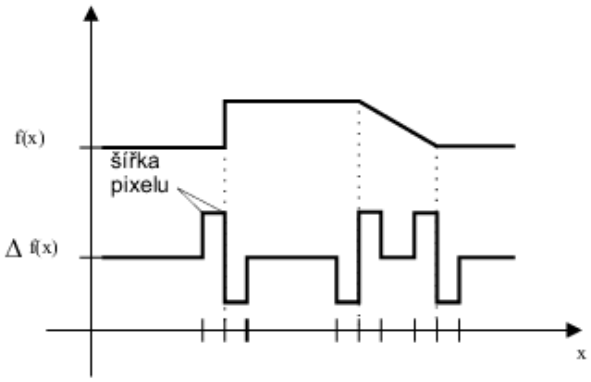
a varianta, jež používá k výpočtu velikosti gradientu hodnot z osmi sousedních bodů, má tvar:

$$h = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Hodnota, kterou tento operátor vrací po aplikaci ve vztahu 7.11, je

$$\nabla^2 f(i,j) = \Delta f(i,j) = \left(\frac{\partial f(i,j)}{\partial i}\right)^2 + \left(\frac{\partial f(i,j)}{\partial j}\right)^2.$$

Změnu v obraze, kterou lidské oko vnímá jako hranu může mít rozličnou šířku. Obrázek 7.10 ukazuje, jak Laplaceův operátor reaguje na různé typy hran.



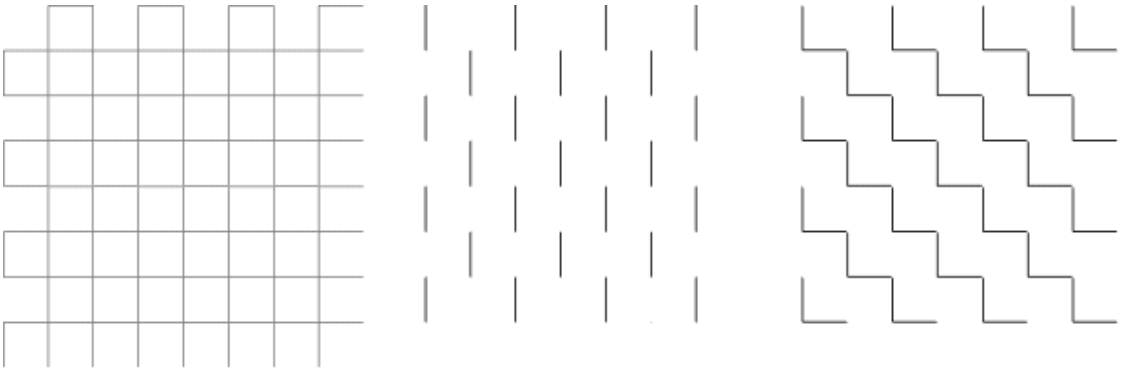
obr. 7.10:Úzká a ostrá pozitivní hrana, široká negativní hrana a odpovídající reakce Laplaceova operátoru Δ..

Směrově orientovaný je **Sobelův operátor**, který aproximuje první derivaci. Existuje celkem v osmi variantách, které vznikají postupnou rotací konvoluční masky

$$h = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Obrázek 7.11 vznikl filtrací černobílé šachovnice Laplaceovým operátorem, výše uvedeným operátorem Sobelovým a Sobelovým operátorem ve tvaru:

$$h = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$$

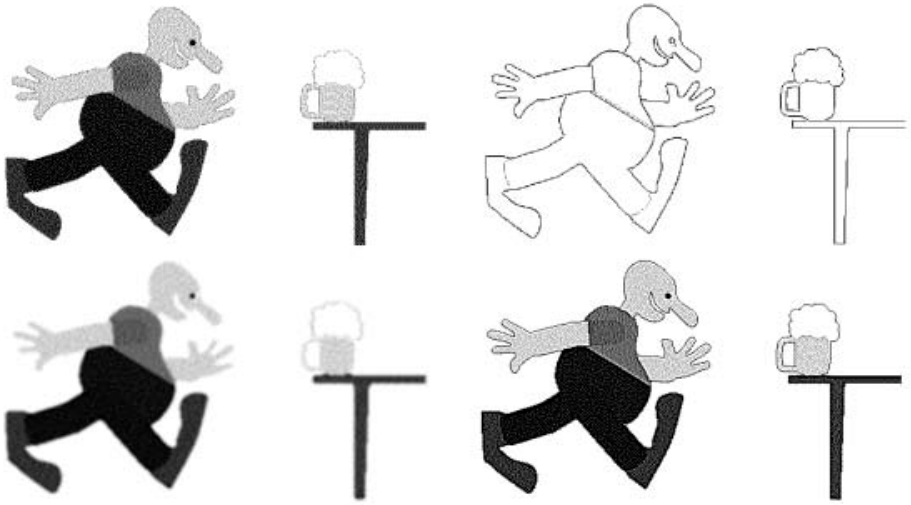


obr. 7.11: Šachovnice po aplikaci (zleva) Laplaceova operátoru, Sobelova operátoru detekujícího svislé hrany a Sobelova operátoru detekujícího diagonální hrany.

Poznamenejme, že tyto operátory jsou citlivé na šum. Méně citlivé jsou operátory pracující s větší konvoluční maskou.

7.5.2 Ostření obrazu ve frekvenční oblasti

Ostření obrazu ve frekvenční oblasti potlačuje nízké frekvence a zdůrazňuje vysoké. Pro zvýraznění vysokých frekvencí použijeme varianty filtrů uvedených pro odstranění šumu, tj. *high--pass filter* a *Butterworth--filter*.



obr. 7.12: Vlevo nahoře originál, vedle něj extrahované hrany pomocí Laplaceova operátoru pracujícím s osmiokolím. Pod originálem obrázek filtrovaný mediánem maticí 5x5 a vpravo dole obrázek ostřený pomocí hran.

Ideální vysokofrekvenční filtr *IHPF* (angl. *ideal high--pass filter*) propouští pouze vysoké frekvence a je popsán vztahem 7.28 (srovnej 7.22).

$$H(u,v) = \begin{cases} 0 & pro: \sqrt{(u^2+v^2)} \leq D_0 \\ 1 & pro: \sqrt{(u^2+v^2)} > D_0 \end{cases} \tag{7.28}$$

Vzdálenost D_0 bývá opět určována v procentech všech frekvencí v obraze.

Filtr *Butterworth* (*BHPF*) pracuje obdobně s tím, že nízké frekvence nejsou odřezány, ale potlačeny s funkcí 7.29

$$H(u,v) = \frac{1}{1 + D_0 / \left[\sqrt{(u^2+v^2)}\right]^{2n}} \tag{7.29}$$

Další operátory je možno vyhledat v [Gonz87], [Hlav92] či [Lewi90].

APPLET

applet 7.5

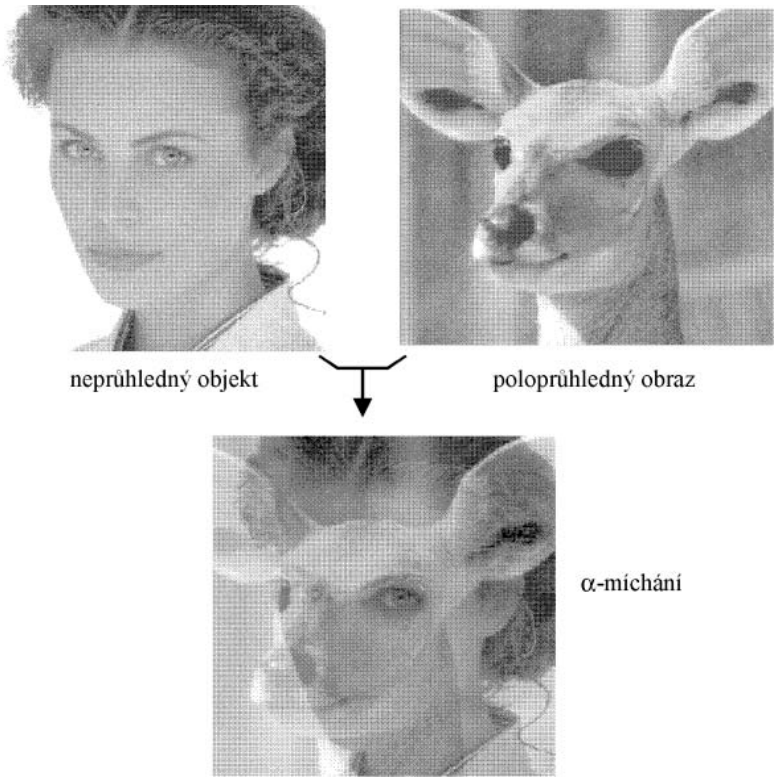
7.6 Lineární kombinace obrazů - α-míchání

Pomocí lineární kombinace můžeme realizovat různé přechody a přeměny mezi 2 a více původními obrazy. Proces lineární kombinace obrazů se často nazývá *alpha-míchání* (angl.α-*blending*, *alpha-blending*) vzhledem k formulaci výchozího jednoduchého vztahu

$$Obraz = [\alpha * ObrazA + (1.0 - \alpha) * ObrazB]$$

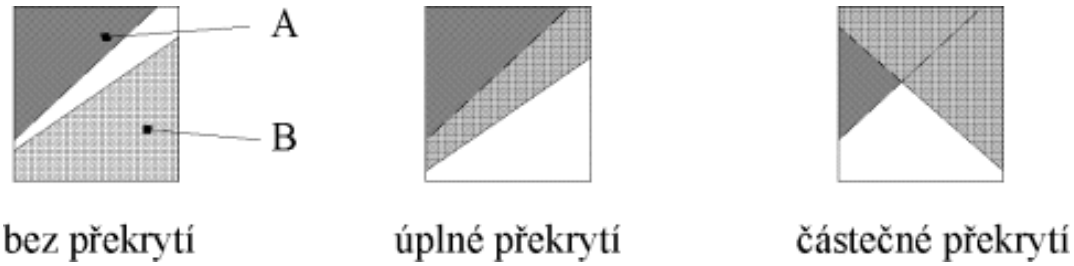
Faktor α určuje jeden prvek - obraz - z konvexní množiny vymezené vstupními obrazy A a B. Tato lineární kombinace je aplikována na všechny pixely, které tvoří obraz. Příklad takovéto kombinace je uveden na obr. 7.13.

Jednoduše formulovaná operace má hlubší význam, než je zřejmé na první pohled. Pokud α-faktor individualizujeme, tj. každý pixel obrazu bude mít svůj vlastní atribut α, pak může tento faktor vyjádřit např. transparentnost pixelu (celý pixel je pokrytý barvou s určitým stupněm průhlednosti), nebo procento pokrytí pixelu určitým objektem (plně pokryté pixely mají α = 1, pixely na hranách polygonů mohou mít α<1.



obr. 7.13: Příklad α-míchání obrazů

Při kombinaci 2 obrazů (nejčastěji však při postupném kreslení jednotlivých objektů do obrazové paměti) může opět vznikat zřetelný **alias**, tj. může dojít ke zřetelnému geometrickému nebo barevnému zkreslení. Na 7.14 je znázorněno několik možností pokrytí pixelu barvami 2 objektů. Uvažujeme-li pixel s jednotkovou plochou a pokrytí objektem A a B označíme αA,αB, pak odhadneme velikost plochy, na které není žádný z obou objektů jako (1 - αA)(1 -αB). Pravděpodobné velikosti ploch a možné barvy pro další kombinace jsou uvedeny v tab. 7.6



obr. 7.14: Vzájemná poloha 2 objektů v jednom pixelu

objekty na ploše	plocha	možné barvy
žádný objekt	$(1-\alpha_A)(1-\alpha_B)$	0
pouze A	$\alpha_A(1-\alpha_B)$	0,A
pouze B	$\alpha_B(1-\alpha_A)$	0,B
A a B	$\alpha_A\alpha_B$	0,A,B

Pokud tedy umíme uchovat ke každému pixelu atribut α, pak lze jednoduše realizovat efektní kombinace obrazů podle variant uvedených na obr. 7.15

operace	barvy	diagram	F_A	F_B
mazání	(0,0,0,0)		0	0
A	(0,A,0,A)		1	0
B	(0,0,B,B)		0	1
A přes B	(0,A,B,A)		1	$1-\alpha_A$
B přes A	(0,A,B,B)		$1-\alpha_B$	1
část A, která překrývá B	(0,0,0,A)		α_B	0
část B, která překrývá A	(0,0,0,B)		0	α_A
A kde není B	(0,A,0,0)		$1-\alpha_B$	0
B kde není A	(0,0,B,0)		0	$1-\alpha_A$
A nad B	(0,0,B,A)		α_B	$1-\alpha_A$
B nad A	(0,A,0,B)		$1-\alpha_B$	α_A
A xor B	(0,A,B,0)		$1-\alpha_B$	$1-\alpha_A$

obr. 7.15: Možné operace při α-míchání obrazů

Podle uvedených vztahů při α-míchání bude výsledná barva pixelu C určena jako lineární kombinace výchozích barev a C_B s použitím příslušných koeficientů F_A a F_B pro příslušnou operaci. Tedy $C = F_A C_A + F_B C_B$ a opravený atribut pokrytí pixelu bude $\alpha = F_A\alpha_A + F_B\alpha_B$.

Realizaci α-míchání podporuje technické řešení, tzv. alpha-buffer, je řešen obdobně jako Z-buffer (obvykle jako další rozšíření obrazového systému). Pixel je pak představován pěticí (R,G,B,Z,α). Alpha-buffer je nedílnou součástí špičkových trikových zařízení pro počítačovou animaci a tvorbu realistických zobrazení.

APPLET

applet 7.6

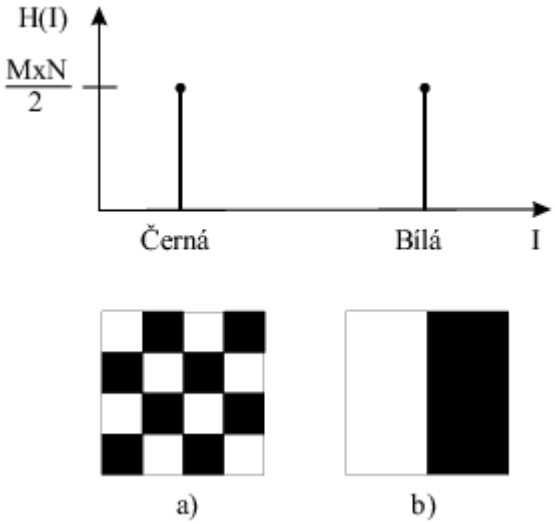
7.7 Histogram

Při snaze o kvantifikaci obrazu můžeme použít veličinu nazývanou **histogram**. Histogram je *vektor absolutních četností jednotlivých hodnot zastoupených v obraze*. Pokud je původní obraz složen z jasových složek, je histogram jednorozměrnou veličinou, v případě složek RGB je složen ze tří vektorů atd. Příklad histogramu je na obrázku 7.17.

Uvažujme dále pouze monochromatický histogram, zobecnění do vyšších dimenzí je prosté. Je-li rozlišení obrazu N x M, je i počet pixelů v něm N x M. Protože histogram je získán jako absolutní

četnosti jednotlivých složek, platí, že hodnota histogramu v bodě I, tj. v určité barvě šedi, je rovna počtu pixelů této hodnoty. Z této úvahy potom plyne vztah 7.30, který říká že součet všech hodnot histogramu je roven součtu bodů v obraze.

$$N \times M = \sum_{I=0}^{max} H(I) \tag{7.30}$$

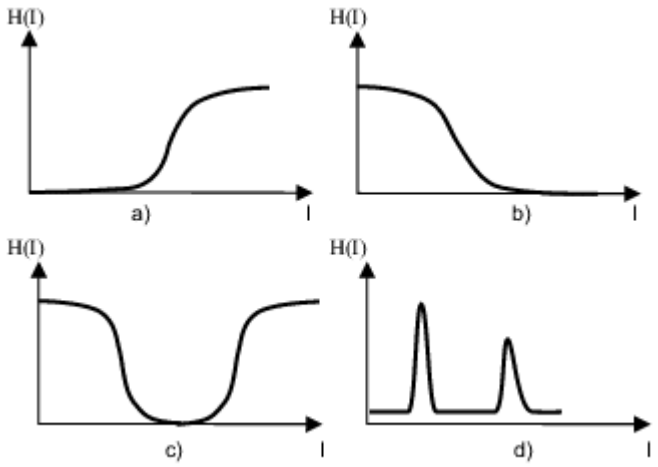


obr. 7.16: Dva černobílé obrázky se stejným histogramem

Histogram je statistickou veličinou odhadující hustotu pravděpodobnosti, která pouze kvantifikuje jasové poměry v obraze a nenese žádné informace o jejich plošném rozložení. Histogram na obrázku 7.16 je získán z obrazu skládajícího se ze dvou barev, z nichž obě jsou zastoupeny stejně, tj. padesáti procenty. Takový histogram však mají oba vzory a) i b). Tento histogram, skládající se ze dvou ostrých vrcholů, se nazývá **bimodální**.

Přestože histogram nenese informace o poloze pixelů v obraze, lze z něj vyčíst jisté důležité informace. Tak například z histogramu na obrázku 7.17b je na první pohled patrné, že se jedná o tmavý obraz, neboť nejvyšší hodnoty, to jest barvy s nejvyšší frekvencí výskytu, převládají v dolní části grafu.

Na obrázku 7.17 a je příklad histogramu, ve kterém převažují jasné složky, v případě b) je tomu naopak. Případ 7.17c odpovídá obrazu, ve kterém jsou stejně zastoupeny jasné i tmavé složky, předloha byla tedy kontrastní a konečně případ d) vznikl z obrazu, kde výrazně převažovaly dvě složky nad ostatními (opět bimodální histogram).

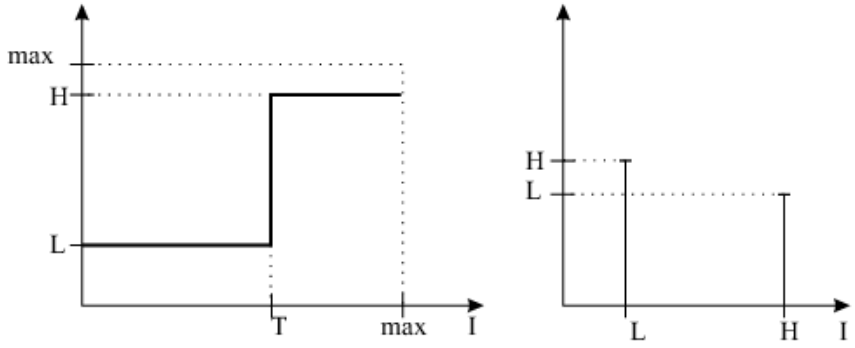


obr. 7.17: Příklady histogramů

Histogram je velmi důležitou informací pro některá, typicky optická zařízení, jako jsou kamery, filmové scannery, číslovací zařízení aj. Existují speciální obrázky - etalony, jejichž histogramy při správném sejmutí jsou známy a pomocí kterých se tato zařízení jasově kalibrují. Ze změny histogramu takovýchto kalibračních etalonů lze pak vyčíst, k jakým změnám v optice zařízení došlo a následně tyto změny eliminovat.

7.7.1 Operace s histogramem

Operace, které mají za svůj důsledek změnu histogramu, bývá zvykem označovat jako operace s histogramem. Ve většině případů se tedy nejedná o přímé zásahy do histogramu, ale o operace, které tyto změny způsobují. Vnutíme-li histogramu nějaký tvar přímo, může se stát, že se některé hodnoty jasu sloučí nebo, a to je horší případ, že se změní pouze určité procento z celkového množství bodů nějaké intenzity. Pro první případ se jedná o prostou změnu jasu *všech* příslušných bodů. V případě druhém musíme určit které z bodů změnit a které ponechat.



obr. 7.18: Vyhledávací tabulka prahování a příklad histogramu

Operace s vyhledávací tabulkou

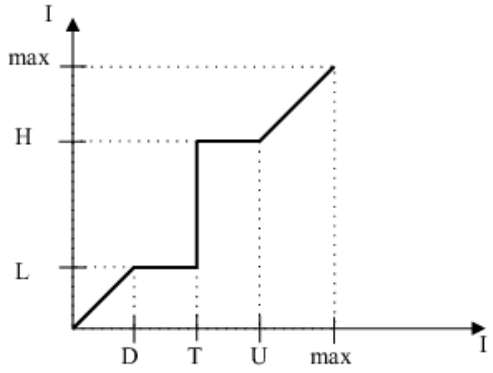
Nejčastější manipulace s barevnými složkami je prováděna pomocí **vyhledávací tabulky** (angl. *look-up table*). Vyhledávací tabulka je funkce 7.31, která určité úrovni jasu přiřazuje úroveň jinou.

$$I_{nov} = LUT(I) \tag{7.31}$$

Z definice funkce plyne, že každé hodnotě z definičního oboru může odpovídat jediná hodnota z oboru hodnot, není tedy možné vést rovnoběžku s osou y. Jelikož se však nemusí jednat o funkci prostou, je možno více hodnotám z definičního oboru přiřadit jedinou funkční hodnotu. Aplikování této operace na vyhledávací tabulku znamená, že některé jasové složky mohou být nahrazeny jedinou, v některých případech může dojít k omezení jasové stupnice a tím ke zvýšení hodnoty určité složky. **Prahování** spočívá v rozdělení jasové škály na dvě části a nahrazení každé z částí jedinou hodnotou tak, jak je uvedeno ve vztahu 7.32

$$I_{nov} = \begin{cases} L; & pro: \ I < T \\ H; & pro: \ I >= T \end{cases} \tag{7.32}$$

Odpovídající graf je uveden na obrázku 7.18. Všechny hodnoty s intenzitou menší než je práh T jsou nahrazeny (sloučeny) hodnotu L, intenzity vyšší jsou nahrazeny hodnotou H. Výsledný obraz obsahuje maximálně dva jasy. Poznamenejme, že volba parametru T má zásadní význam pro kvalitu celého obrazu. Pro automatickou volbu může být použita hodnota mediánu či průměru histogramu.



obr. 7.19: Ohraničené prahování

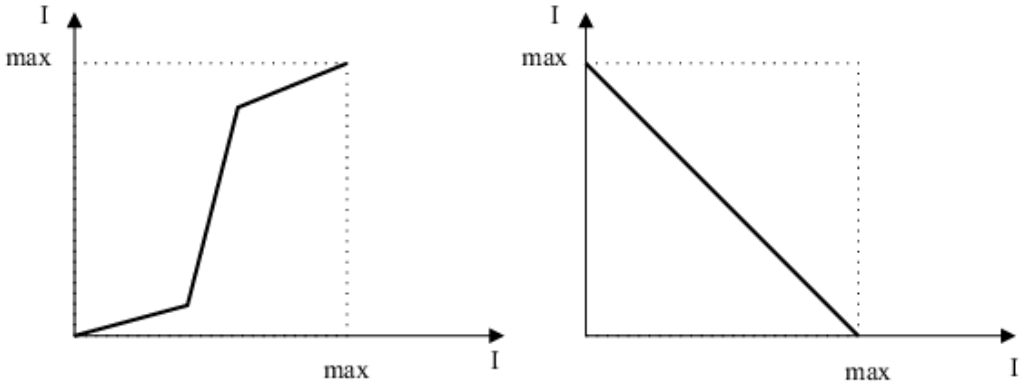
Obrázek 7.19 je grafem tzv. ohraničeného prahování 7.33, kde do hodnoty D a od hodnoty U zůstávají zachovány všechny jasové složky ve své původní hodnotě. Složky uvnitř tohoto intervalu jsou rozděleny do dvou hodnot L a H. Klíčem pro toto rozdělení je právě práh T.

$$I_{nov} = \begin{cases} I; & pro: \ 0 \leq I \ < D \\ L; & pro: \ D \leq I \ < T \\ H; & pro: \ T \leq I \ < U \\ I; & pro: \ U \ \leq I \ \leq max \end{cases} \tag{7.33}$$

Dalším příkladem operace s histogramem je zvýšení kontrastu. Grafické znázornění odpovídající vyhledávací tabulky je na obrázku 7.20 vlevo. V tomto případě dochází ke změně kontrastu po částech. Jinou možností, jak zvýšit kontrast je výpočet vyhledávací tabulky pomocí funkce modelující jas a kontrast na televizoru.

$$I_{nov} = jas * I_{old} + kontrast \tag{7.34}$$

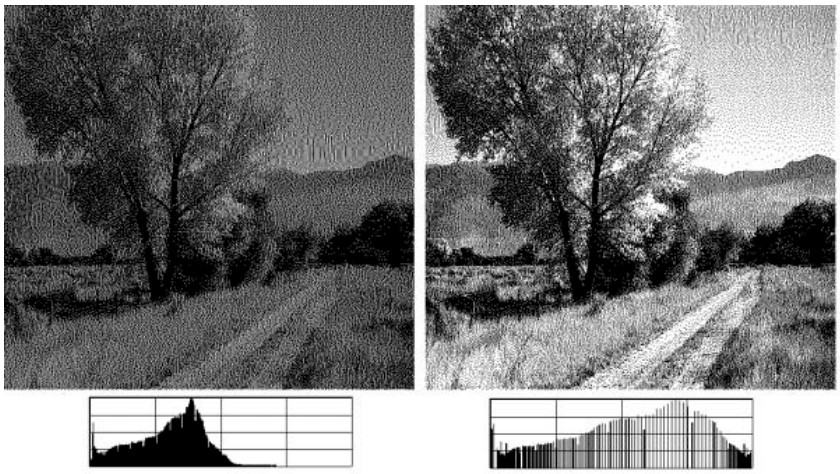
Konstanty jas a kontrast odpovídají kontrastu a jasu; jedná se o posouvání a změnu sklonu úsečky. Pro případ kontrast = I_{max} a jas = -1 dostaneme negativ obrázku (obr. 7.20 vpravo).



obr. 7.20: Zvýšení kontrastu a negativ

Ekvalizace histogramu

Jednou z nejčastěji používaných metod na vyrovnání jasových poměrů v obrázku je **vyrovnání** neboli **ekvalizace histogramu**.



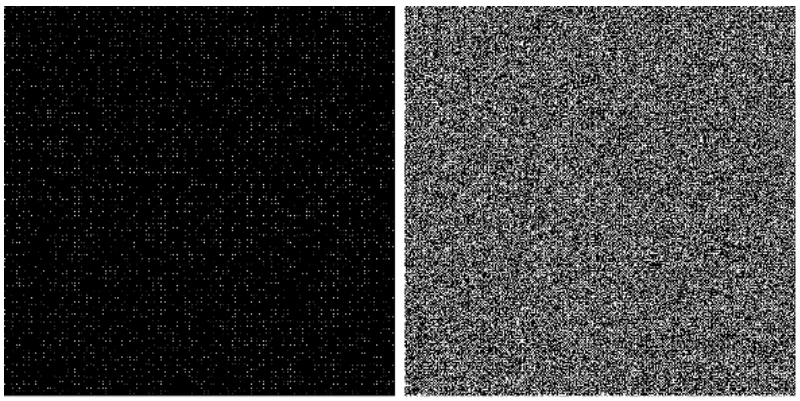
obr. 7.21: Ekvalizace histogramu

Typickou úlohou, která je řešitelná ekvalizací histogramu, je změna jasových poměrů ve fotografiích pořízených proti zdroji světla. V těchto obrázcích totiž převládá většina informace v pravé polovině histogramu. Přesunem některých jasových složek do zbývající části se stane obrázek kontrastnějším a vystupují detaily, viz obr. 7.21

Algoritmus ekvalizace histogramu spočívá v nalezení takové vyhledávací tabulky, pomocí které dosáhneme vyrovnaného histogramu. Předpokládejme, že máme obrázek v rozlišení N x M bodů maximální intenzity MAX. Ideálně vyrovnaný histogram by se skládal z přímky rovnoběžné s osou x ve výšce

$$D = \frac{N \times M}{MAX}, \tag{7.35}$$

což odpovídá stejnému množství bodů všech intenzit v obrázku. Připomeňme, že histogram neříká nic o tom, kde tyto body leží, ale pouze kvantifikuje jejich přítomnost. Jak bylo výše uvedeno, z vlastností vyhledávací tabulky vyplývá, že není možné některé intenzity rozdělit, tj. není možné "zlomit" některou z hodnot histogramu, zkrátit ji na úkor jiné. Jinými slovy řečeno, pokud některá jasová složka překračuje ideální hodnotu D, nemůžeme ji zkrátit. Základní ideou algoritmu, který vytváří vyhledávací tabulku je, že se chová jako by bylo možno takovou hodnotu distribuovat, prozkoumá, do jakého intervalu je možno ji rozdělit, a nerozdělenou hodnotu potom umístí do středu tohoto intervalu. Výpočet vyhledávací tabulky ekvalizující histogram je demonstrován algoritmem 7.2.



obr. 7.22: Černé pozadí se šumem před (vlevo) a po (vpravo) ekvalizaci histogramu

```
D = resolutionX * resolutionY / MAX;
outI = 0;
outHeight = 0;
for (inI= 0; inI < MAX; inI++)    /* pro všechny jasové složky*/
{
  firstI= outI;                  /* zde začíná interval */
  inHeight = histogram[inI];
  while(inHeight >= D - outHeight)      /* dělení toho, co je větší než D */
  {
    inHeight = inHeight - (D - outHeight);
    outI++;
    outHeight = 0;
  }
  outHeight+= inHeight;    /* toto zbývá */
  lut[inI]= (firstI + outI) / 2;    /* umístění do středu */
}
```

Algoritmus 7.2

K ekvalizaci histogramu, stejně jako ke všem operacím s obrázky, je zapotřebí přistupovat uváženě, neboť i tato metoda má v některých případech nežádoucí efekty. Předpokládejme, že máme obrázek, který má na zcela černém pozadí mírný, téměř černý šum. Ekvalizací histogramu docílíme toho, že se hodnoty šumu přesunou do jasnější oblasti a šum se tak zvýrazní jako je tomu na obrázku 7.22. Řešení tohoto problému na úrovni ekvalizace histogramu spočívá ve fixaci určitých hodnot, které při ekvalizaci nemohou změnit svou polohu. Řešení na jiné úrovni spočívá v odstranění nežádoucího šumu z obrázku.

V některých případech má smysl ekvalizovat pouze část obrázku (hovoříme potom o lokální ekvalizaci). Příkladem aplikace ve které se lokální ekvalizace využívá je astronomie. Na fotografiích vesmírných objektů zabírá podstatnou část obrazu obloha, která je úplně černá a nenese žádné informace, nemá tudíž smysl se zabývat její ekvalizací.

Ekvalizaci v barevném prostoru je možno provést mnoha způsoby. Nejjednodušší metodou je ekvalizovat **po částech**, tj. každou barevnou složku zvlášť. Tato metoda vede k barevným změnám v obrázku. Byla-li například předloha barevně zcela vyvážená v červené i zelené barvě a modrá převládala v dolní části histogramu, dojde ekvalizací k přidání modré barvy. To samozřejmě nemusí být vždy žádoucí.

```
/* spočítej histogram pro složku Y */
for (i = 0; i < resolutionX; i++)
  for (j = 0; j < resolutionY; j++)
    histogram[0.299 * picture[i][j].r + 0.587 * picture[i][j].g +
0.114 * picture[i][j].b]++;

Equalize(histogram, lut);    /* spočítej lut */

/* Vypočítej nové' hodnoty RGB pro každý pixel */
for (i = 0; i < resolutionX; i++)
  for (j = 0; j < resolutionY; j++)
  {
    Y = 0.299 * picture[i][j].r + 0.587 * picture[i][j].g + 0.114 *
picture[i][j].b;
    coef = lut[Y] / Y;
    picture[i][j].r = picture[i][j].r * coef;
    picture[i][j].g = picture[i][j].g * coef;
    picture[i][j].b = picture[i][j].b * coef;
  }
```

Algoritmus 7.3

Jinou možností je provést ekvalizaci v některém z alternativních barevných prostorů. Výhodný je model **YIQ** popsaný v kapitole 6. Hodnota Y nese informaci o jasu pixelu a je proto vhodným kandidátem pro ekvalizaci jasové složky, neboť ekvalizace jasu nemění barevné poměry v obrázku. Tyto hodnoty slouží k výpočtu histogramu, který reprezentuje pouze jasovou bilanci původní barevné předlohy. Aplikací algoritmu ekvalizace histogramu monochromatického obrázku 7.2, získáme vyhledávací tabulku *lut* pro překlad jasových složek. Hodnota nového jasu bude mít velikost Y_n. Zbývá přechod zpět do barevného prostoru. Z hodnot jasu Y a Y_n se vypočítají hodnoty R_n,G_n a B_n podle vztahu 7.36.

$$X_n = X\frac{Y}{Y_n}; X = R, G, B. \qquad (7.36)$$

APPLET

applet 7.7

7.8 Malování pomocí počítače

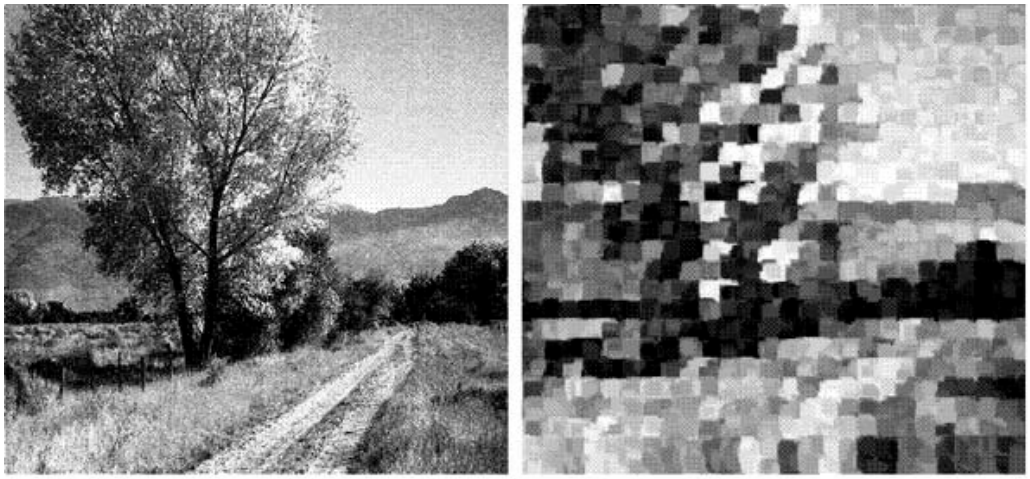
Přirozenou snahou člověka je automatizace rutinních postupů. Tato tendence vedla pozornost počítačových grafiků i do umění a výsledkem jsou smělé pokusy o "automatizaci" práce malíře ([Haeb90]). Samozřejmě, že princip, na kterém pracují současné počítače, něco podobného vylučuje, při plném vědomí tohoto faktu však můžeme tyto snahy označit za zajímavé.

Impresionismus vznikl v poslední polovině 19. století a klade si za cíl zaznamenávat přírodu v jistém časovém okamžiku. V malířství je pro něj typické nanášení čistých tónů kladením štětce tak, že polotóny vznikají až na sítnici diváka. Neoimpresionismus - pointilismus, klade barvy v bodech s důrazem na kontrast doplňkových barev. Tolik říká encyklopedický slovník [Ency93].

obr. 7.23: Originál a výsledek pokusu o pointilismus

Program simulující tuto činnost se nazývá *brush stroking*. Jeho vstupem bude:

- plátno - tím je obrazovka počítače
- štětec - má tloušťku a tvar, např. kruh, elipsu, či čtverec
- paleta - tou je předem definovaný a omezený počet barev
- směr, kterým malujeme - tím je vektor a konečně
- předloha - nějaký obrázek, nejlépe fotografie reálného světa.



obr. 7.24: Originál a výsledek pokusu o impresionismus

Vlastní program poté funguje tak, že inspirován předlohou, bere z ní čtverce o velikosti NxN pixelů, z nich počítá průměr a hledá nejbližší barvu v paletě. Vypočtenou barvou potom v předem vybraném směru namaluje (jako šmouhu třepící se na konci) do výsledného obrazu tah. Výsledkem bude plocha plně pokrytá takovýmito, vzájemně se překrývajícími tahy. Postup demonstruje algoritmus 7.4.

```
1. Zvol štětec,
2. zvol směr, kterým budeš malovat,
3. zvol délku čáry, kterou budeš malovat,
4. rozděl předlohu na čtverce NxN,
5. pro každý ze čtverců udělej:
     a) vypočti průměr z barev,
     b) vyber z palety barvu nejbližší barvě vypočtené,
     c) vezmi místo odpovídající středu čtverce a polož
        na něj  štětec
     d) namaluj čáru od místa položení štětce do délky
        ve zvoleném směru.
```

Algoritmus 7.4. Metoda *brush stroking*

Většina z uvedených kroků by měla být modulována náhodnými čísly. Směr tahu by se měl naklánět, při výběru barvy by se občas měla vybrat barva lišící se o nějaké náhodné číslo a délka tahu by měla být modulována náhodnou veličinou. Výsledek této metody je demonstrován na obrázku 7.24. Pokud nebudeme se štětcem nikam táhnout a budeme používat pouze kruhový štětec, bude se náš program pokoušet malovat jako pointilisté (viz obr. 7.23).

Snahy o napodobení uměleckých směrů nejsou v žádném případě ojedinělé. Byly například publikovány algoritmy, které napodobují techniku litografie. Tyto metody fungují na bázi detekce hran v obrázku a jejich dalších úprav.

APPLET

applet 7.8

7.9 Warping a morphing

V poslední době se středem zájmu firem pracujících ve filmovém průmyslu stává digitální **warping** a **morphing**. Warping znamená aplikaci (obyčejně nelineární) transformace na jediný obrázek nebo objekt, která se touto transformací mění jako v křivém zrcadle. Termínem morphing se označuje proces, kdy se jeden obrázek (objekt) postupným přechodem, přeměňuje v druhý.

Pohledem do historie zjistíme, že morphing ve filmu se zprvu dělával metodou "chytrého střihu". V okamžiku, kdy mělo dojít ke změně objektu bylo zastaveno natáčení, herec (objekt) se pozměnil, další kousek filmu se natočil atd. Jedná se o poměrně jednoduchou metodu v případě, že je deformovaný objekt v klidu (angl. *stop-motion*), výrazně složitější je morphing pohybujících se objektů (angl. *go-motion*).

Počítačem podporovaný morphing je možno logicky rozdělit do 3D a 2D. Třírozměrný morphing znamená práci v modelovém prostoru a je jednoduchý u homomorfních modelů ve smyslu identity reprezentací. Máme-li objekt reprezentovaný například pomocí okřídlených hran (angl. *windged edge*), je pro oba objekty požadován stejný počet uzlů, hran a plošek a jejich homomorfismus. Není problematická změna například krychle na kvádr, nebo koule na elipsoid. Tento intuitivní postup bývá komplikován v případě nekonvexních objektů, objektů s dírou, či v případě změny tvaru. Poměrně elegantní řešení warpingu 3D objektů poskytuje metoda zvaná **free form deformation** ([Sede86]) a všechny její varianty, např. **NURBS based free form deformation** ([Lamo94]). Podstatnou nevýhodou těchto metod zůstává nutnost tvorby 3D modelu, který musí být realistický (orosený a právě probuzený huňatý medvěd). Z mnoha důvodů (v neposlední řadě komerčních) je jednodušší použít (ulovit) reálný model, nafilmovat ho a pro morphing použít 2D počítačových metod. Vždy je snadnější modifikovat realitu, než při jejím napodobování začínat s prázdnýma rukama.

Metody pracující ve 2D patří tedy mezi úpravy obrazu (angl. *image processing*). Jejich produktem je vždy sekvence obrázků, které jsou v případě morphingu přechodem od jednoho vstupního obrázku k druhému a v případě warpingu přechodem od jediného vstupního obrázku do jeho požadované změny.

Úplně nejjednodušším postupem je interpolace barvy pixelu (angl. *cross-dissolve*). Tato metoda interpoluje pixely na stejných souřadnicích podle vztahu 7.37.

$$X = X_1 + \frac{i-1}{n-1}(X_n - X_1). \qquad (7.37)$$

Pixel v počátečním resp. koncovém obrázku je označen X_1 resp. X_n a jsou vstupy uvedeného algoritmu, n je číslo posledního obrázku a celkový počet vygenerovaných obrázků je n-3. Index i je číslo obrázku. Tato metoda odpovídá postupnému prolnutí druhého obrázku do prvního. Při zpracovávání *RGB* obrázků se podle vztahu 7.37 upravují postupně jednotlivé barevné kanály.

Jinou možností je aplikace algoritmů **particle systems** ([Fole90]). Určitým bodům, nebo skupinám bodů se explicitně určí, kam se budou pohybovat. Směr pohybu se definuje vytvořením dvojic bodů či skupin bodů v předloze a výstupu a aplikací algoritmů modelujících particle systems na jejich pohyb. Výsledkem je potom rozplynutí vstupního obrázku složením výstupního.

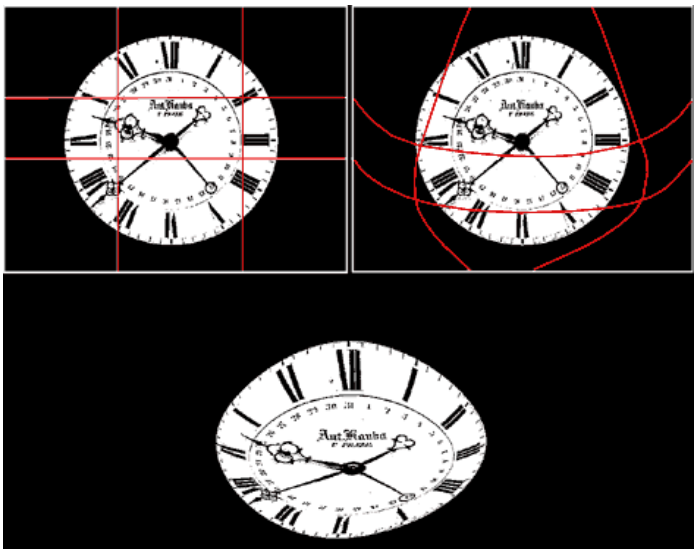
7.9.1 Dvouprůchodový síťový warping

Dvouprůchodový síťový warping byl publikován Douglasem Smythem pracujícím ve speciální divizi *Lucasfilm Ltd.* pro počítačové triky *Industrial Light Magic* známé spíše pod zkratkou *ILM* v roce 1990 ([Smyt90]). Tato metoda byla s úspěchem použita ve filmech *The Abbys, Indiana Jones and Last Crusade, Terminator*, ale i v českém *Akumulátor I* aj.

Algoritmus dvouprůchodového zpracování (angl. *two-pass algorithm*) byl publikován v roce 1980 Edwinem Catmullem a Alvy Ray Smithem ([Catm80]). Jeho idea spočívá v rozdělení plošné obrazové transformace na dvě, které jsou prováděny jedna po druhé odděleně pro sloupce a řádky. Tento postup je vhodný zejména pro rotaci, perspektivu atp. ([Wool90]).

Předpokládejme vstupní obraz A a výstupní B, oba v odstínech šedi. Pro *RGB* obrazy se jedná o výpočet všech tří kanálů separovaně. Indexujme obraz A souřadnicemi $[u,v]$ a obraz B souřadnicemi $[x,y]$. První průchod pak znamená vytvoření meziobrázku I, který má souřadnice $[x,v]$ viz obrázek vlevo nahoře. Ve druhém průchodu se vytvoří obraz B. Označení $[u,v]$ se používá proto, že tyto metody

se používají také pro mapování textur. První průchod probíhá bez výrazných problémů, neboť spočívá ve vzorkování předlohy s konstantním krokem u. Po prvním průchodu jsou tedy ve správných pozicích pixely ležící na řádcích (viz obr.7.26). Druhý průchod v obecném případě spočívá v nalezení transformace převádějící I na výstup B. Tato operace nemusí být zcela triviální a její řešení může vést k numerickým metodám. U separabilních operací (viz 7.13), o kterých bude řeč dále, tyto problémy nevznikají.



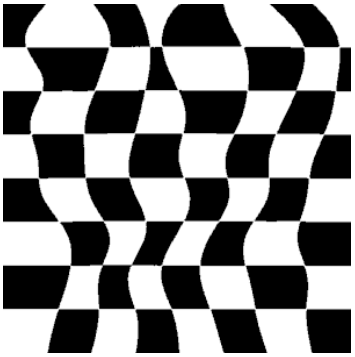
Obr. 7.25: Síťový warping. Nahoře pokrytí sítí ve zdrojovém a cílovém obrazu, dole výsledný obrázek

Algoritmus síťového warpingu spočívá v pokrytí vstupního obrazu A sítí řídících bodů S (viz obr. 7.25), určující hranice oblastí zkroucení. Ve výstupním obrazu se vytvoří síť řídících bodů D, která je topologicky ekvivalentní se sítí S. Obě sítě navíc nesmí překrývat sebe samotné. Sítě S a D určují vzájemně korespondující oblasti. Datovou strukturou reprezentující obě sítě je dvojrozměrná matice o rozlišení $h \times w$. Aby nedošlo k nežádoucím modifikacím obrazu, musí řídící body tvořící okraj obou obrazů v S a D být nepohyblivé, nesmějí se účastnit warpingu. Polohy bodů sítě jsou reálná čísla. Další podmínkou kladenou na vstupní data je stejné rozlišení všech tří obrazů: A, B i meziobrazu I. V dalším textu budeme toto rozlišení označovat $h_{in} \times w_{in}$.

Sítě řídících bodů určují spline křivky v horizontálním a vertikálním směru. V původním algoritmu byly použity interpolační Catmul-Rom spliny, předmětem pozdějšího výzkumu bylo posuzování různých jiných metod. Pro další text není určení konkrétní metody relevantní.

První průchod

První průchod se skládá ze dvou fází. V *první fázi* se určí průsečíky řádků s vertikálním splinem, a to jak v obraze S tak v obraze I. Tím se vytvoří dvě tabulky T_S a T_I o rozměrech $h_{in} \times w$. Tabulka T_S obsahuje index průsečíku splinu S ve sloupci w s řádkem h_{in} vstupního obrazu A. Druhá tabulka obsahuje index průsečíku splinu D s řádky obrazu I.



Obr. 7.26: Šachovnice po prvním průchodu síťového warpingu

Ve *druhé fázi* prvního průchodu, kdy jsou již vytvořeny tabulky T_S a T_I, dojde k interpolaci *řádků*. Protože počet splinů ve vstupním obraze i v meziobraze je shodný, identický je i počet úseků, které jsou jimi vyznačeny na jednotlivých řádcích. Jednotlivé oblasti se převzorkují. Protože se pracuje s reálnými čísly, dochází při převzorkování k částečnému pokrytí některých bodů, které se výhodně využije pro antialiasing. Hodnota pixelu P v meziobraze I se spočítá jako poměrné pokrytí pixelů z obrazu S na příslušném řádku (obraz I je vlastně projekcí S). Intenzita pixelu P se určí z korespondujících bodů ze zdrojového obrazu podle vztahu :

$$P = \frac{\sum_{x = x_0}^{x1} k_x S_x}{x_1 - x_0} \qquad (7.38)$$

Intenzity pixelů zdrojového obrazu mají indexy S_x. Váha k_x je mírou pokrytí jednotlivých pixelů a je určena vztahem:

$$P = \begin{cases} \lceil x \rceil - x_0 & pro: \lfloor x \rfloor < x_0 \\ 1 & pro: x_0 \le x < x_1 \\ x_1 - \lfloor x \rfloor & pro: \lceil x \rceil > x_1 \end{cases} \qquad (7.39)$$

Je vidět, že úplně pokryté pixely se respektují celé, zatímco okraje jsou reprezentovány procentem pokrytí. Tato metoda je de-facto antialiasingem metodou *oversampling*, a proto jsou výsledné obrázky velice kvalitní.

Po prvním průchodu jsou na svých pozicích pixely ležící na řádcích (viz obr. 7.26).

Druhý průchod

Druhý průchod je téměř identický s prvním s tím rozdílem, že vstupním obrazem je I a výstupním je D. V *první fázi* se vypočítají tabulky T_I a T_D jako průsečíky horizontálních splinů. Pro T_I se použijí spliny určené řídící maticí S. *Druhá fáze* spočívá v interpolaci *sloupců*.

Podstatnou nevýhodou výše uvedené metody je její globalita. Je-li počet oblastí příliš malý, jsou tyto velké a dochází k modifikaci velkých částí obrazu. Tomu lze zamezit zvýšením složitosti sítě, tím dojde sice ke zvýšení detailu, ale také ke zvětšení objemu dat, snížení přehlednosti a v konečném důsledku ke snížení rychlosti celého algoritmu. Není také snadné docílit tak jednoduše jevících se operací, jako je zavření oka, neboť vyžadují detailní obklopení objektu, a to je možné pouze za cenu zvýšení složitosti sítě. Většinu těchto problémů řeší algoritmus **feature based morphing**, o kterém bude řeč dále.

7.9.2 Feature based morphing

Feature based morphing a **feature based warping** jsou nelineární operace s obrazem, které pracují na zcela odlišném principu než předchozí algoritmus. V dalším textu uvedeme nejprve warping a poté morphing.

Principem této metody je modifikace obrazu na základě korespondujících úseček, které jsou vkládány do dvojice obrazů a které svou velikostí a polohou definují lokální změnu. Úsečky můžeme pro zjednodušení chápat jako magnety, které ve vstupním obraze vytvářejí vyrovnané magnetické pole (proto se této metodě také říká **field morphing**) a ve druhém obraze změnou polohy vytvářejí pole modifikované, které tak mění tvar obrazu.

Stejně jako v předchozím případě budeme předpokládat vstupní obraz A a výstupní B, oba v odstínech šedi (zobecnění do *RGB* spočívá opět v separovaném zpracování všech tří barevných kanálů).

Pro výpočet pixelu v obraze *B* použijeme metodu dopředného mapování, tj. budeme řešit otázku, jaký pixel (pixely) z *A* budeme mapovat do pixelu o souřadnicích [*i*,*j*] v obraze *B*.

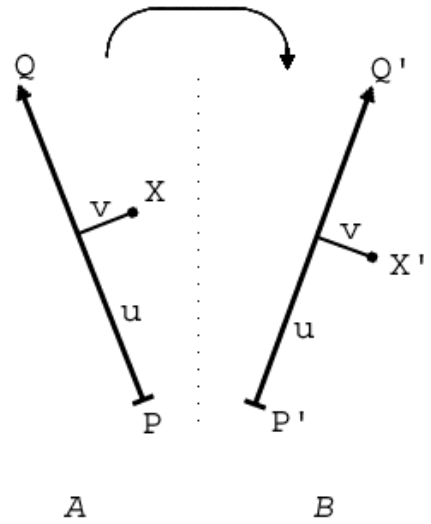
Transformace jedním párem úseček

Předpokládejme v obraze *A* úsečku definovanou dvěma body P' a Q' a v obraze *B* úsečku *PQ* (viz obr.). Transformace

$$PQ \square P'Q'$$

definuje zobrazení celého obrazu *A* na *B*.

$$A \square B$$



Obr. 7.27: Princip mapování pixelů metodou *feature based morphing*

Výpočet pixelu, který je zobrazen do výstupního obrazu *B* z obrazu *A*, se provede podle následujícího postupu. Pravá část obrázku 7.27 představuje obraz *B*, do něj se body zobrazují a levá pak obraz *A* v němž se vyhledávají. Algoritmus prochází obraz *B* bod po bodu a vypočítává, který bod z obrazu *A* použít. Pro výpočet polohy bodu je nutné nejprve určit konstanty *u* a *v*. Konstanta *u* se určí podle následujícího vztahu:

$$u = \frac{(X-P)\,.\,(Q-P)}{|Q-P|^2}\,. \qquad (7.40)$$

Vztah v čitateli zlomku (7.40) je skalární součin dvou vektorů (je tedy číslem). Jeho velikost určuje projekci vektoru [(*XP*)\vec] do vektoru [(*QP*)\vec]. Jmenovatel zlomku je čtverec velikosti vektoru [(*PQ*)\vec], podle známého vztahu $(P_X\text{-}Q_X)^2 + (P_Y\text{-}Q_Y)^2$. Hodnota *u* je v intervalu $< 0,1 >$, je-li bod *X* v pásu určeném body *P* a *Q* (obr.).

Konstanta *v* je určena vztahem

$$v = \frac{(X-P).\perp(Q-P)}{|Q-P|}\,, \qquad (7.41)$$

kde ⊥ znamená kolmici a má kladné nebo záporné znaménko podle libovolné, avšak pevně stanovené orientace. Poloha pixelu, který je použit pro mapování do obrazu *B* je určena vztahem:

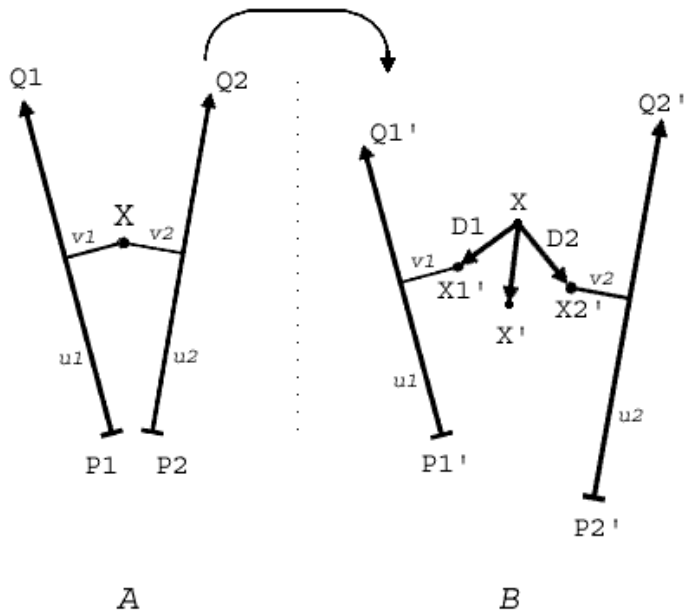
$$X' = P' + u(Q'\text{-}P') + \frac{v.\perp(Q' - P')}{|Q-P|}\,, \qquad (7.42)$$

První symbol na pravé straně je bod, další dva pak určují vektory, vztah je tedy formálně v pořádku, v afinním prostoru je součet bodu a vektoru bod ([Budí83]).

Poznamenejme, že tento jednoduchý algoritmus definuje pro celý obraz rotaci, posun a změnu měřítka ve směru úsečky.

Transformace s více páry úseček

Mějme v obraze A úsečky zadané dvojicemi bodů P_i' a Q_i' a v B body P_i a Q_i. Každá ze dvojic úseček pak definuje mapovaní jiného bodu X'_i podle vztahu 7.42 (viz též obr.7.28).



Obr. 7.28: K výpočtu warpingu s více páry úseček (viz text)

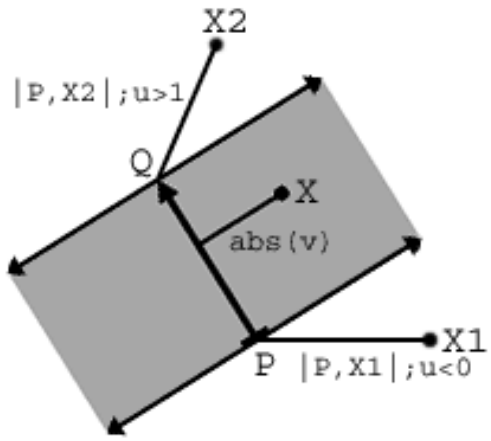
Výsledný bod X', je spočítán jako vážený průměr všech složek X'_i. Jedinou nejednoznačností je určení **váhy** složky. Váha se určí podle vztahu :

$$w_i = \left(\frac{|P_iQ_i|^b}{(a+dist)} \right)^2 \qquad (7.43)$$

Konstanta *dist* je vzdáleností bodu X od úsečky P_iQ_i a je určena:

$$dist = \begin{cases} abs(v) & pro: u \in \langle 0,1 \rangle \\ |P,X| & pro: u < 0 \\ |Q,X| & pro: u > 1 \end{cases}$$

Obrázek demonstruje tento výpočet.



Obr. 7.29: K výpočtu *dist* (viz text). Oblast, ve které je $u \in < 0,1 >$, je zvýrazněna tmavým pásem.

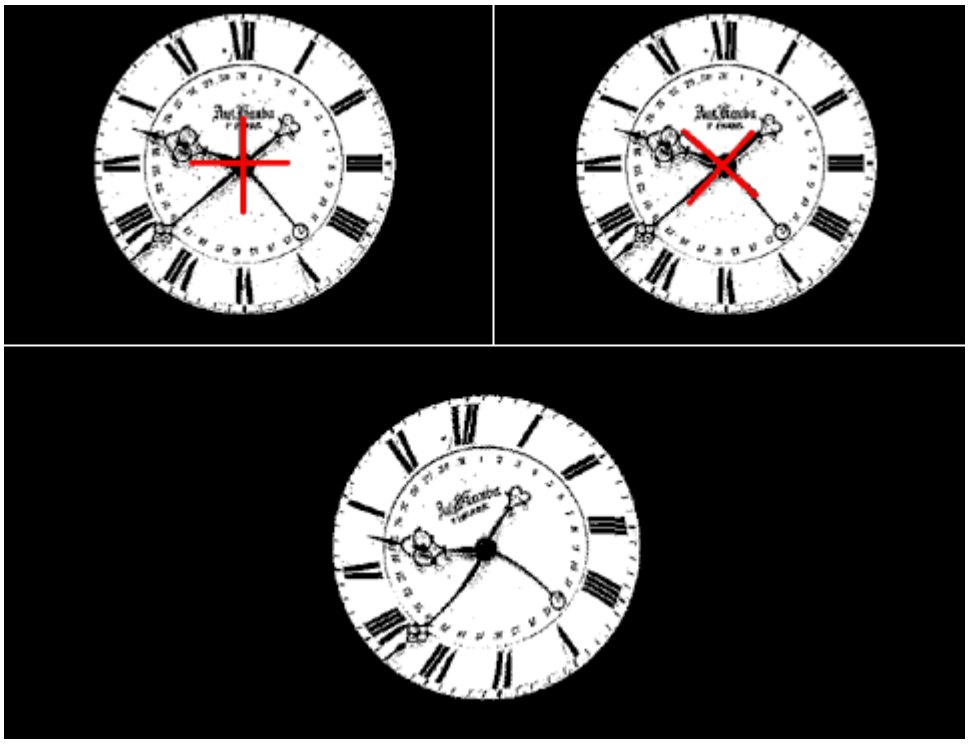
Konstanty a, b a p mají následující význam:

- $a \approx 0$: čím blíže je bod úsečky, tím blíže je i ve výsledném obraze,
- $b >> 0$: pixel je ovlivněn převážně svým okolím,
- $b = 0$: $w_i = 0$ pixel je významně všemi úsečkami stejně,
- $p \in < 0,1 >$
  - $p = 0$ mají všechny úsečky stejné váhy,
  - $p = 1$ čím je úsečka delší, tím má větší vliv.

```
dsum = (0,0); /* poloha bodu*/
wsum = 0;     /* soucet vah*/
  for (i=0;i<pocet usecek; i++) /* pro vsechny dvojice usecek */
     {
       vypocti u a v jako funkci P[i] a Q[i]
       vypocitej X'[i] jako funkci P[i], Q[i] u a v
       D[i] = X'[i] - X[i]  /* poloha bodu pro slozku P[i] a Q[i] */
       dist = vzdalenost X od (P[i] Q[i])
       spocitej vahu w podle vztahu 7.43
       dsum = dsum + D[i]*w;
       wsum = wsum + w;
     }
    X' = X + dsum/wsum;
    B[X] = A[X']  /* namapuj pixel */
```

Algoritmus 7.5: Feature based warping

Algoritmus 7.5 pak demonstruje postupný výpočet bodu v novém obraze vážením vstupních složek X_i' podle vztahu 7.43.



Obr. 7.30: Feature based warping. Nahoře vstupní obrázky se zadanými řídícími úsečkami ve tvaru kříže ve středu hodin, dole výsledný obrázek

Feature based warping je intuitivní metodou a vede k poměrně rychlým výsledkům. Jeho nevýhodou je špatně pochopitelný význam konstant a explicitní zákaz křížení magnetů, který vede ke vzniku "bublin"- *Ghostbustering effect*[4]. Další nevýhodou této metody je nejednoznačnost operací při morphingu. Chceme-li otočit úsečku o 180 stupňů a zadáme-li v obraze *A* body *P* a *Q* zaměněné s jejich obrazy v *B*, může se stát, že se při interpolaci spojí v jediný bod a poté se vymění.

7.9.3 Morphing

Morphing je operací, při které dochází nelineární transformací vstupního obrazu k jeho modifikaci do obrazu výstupního. Jeho výstupem je tedy sekvence obrazů, přechod od vstupního obrazu ke koncovému. Popíšeme morphing jako variantu warpingu a to jak síťového, tak feature based.

Předpokládejme vstupní obraz *A* a výstupní *B*. Mějme dále definováno pokrytí obou obrazů (sítí či dvojicemi úseček). Mějme dále zadán počet mezifází, obrazů *I* jako *n*. Každý meziobraz vznikne v následujících krocích:

1. Lineární interpolací se spočítá pokrytí meziobrazu *I* v kroku *k*. Vznikne tak nová síť (či nové páry úseček).
2. Z nově vzniklé sítě se spočítá warpovaný vstupní obraz *T*.
3. Každý pixel v obraze *I* je spočítán jako vážený průměr z pixelu v koncovém obrazu a meziobrazu podle vztahu:

$$I_k = \frac{n-k}{n} T + \frac{k-1}{n} A \qquad (7.44)$$

Je vidět, že morphing je de facto α-mícháním warpovaného meziobrázku a jednoho z obrazů předlohy.

---

[4]Název tohoto jevu pochází ze stejnojmenného filmu Ghostbusters (Krotitelé duchů), kde se odehrává následující dialog:
"Don't cross the streams!"
"Why?!"
"It would be bad."
Tento dialog je i jediným autorovým zdůvodněním tohoto jevu v uvedeném algoritmu...

Literatura:

- [Gonz87] Gonzales R. C., Wintz P. : *Digital Image Processing,*, Second edition, Addison-Wesley Publishing Company, 1987.
- [Hlav92] Hlaváč V., Šonka M. : *Počítačové vidění*, Grada, 1992.
- [Lewi90] Lewis R. : *Practical digital image processing*, Ellis Horwood, 1990.
- [Pres94] Press W. H., Vetterling W. T., Teukolsky S. A., Flanner B. P. : *Numerical recipies in C*, Second edition, Cambridge University Press, 1994.
- [Wool90] Woolberg G. : *Digital image warping*, IEEE Computer Society Press Monograph, Los Alamitos, California, 1990.
- [Budí83] Budínský, B.: *Analytická a diferenciální geometrie* SNTL, 1983.
- [Catm80] Catmull, E., Smith, A.R.: *3D Transformations of Images in Scanline Mode.* Proceedings SIGGRAPH, 1980, 14 (3), pp 279 - 285, 1980.
- [Fole90] Foley, J., van Dam, A., Feiner, S., Hughes, J.: *Computer Graphics. Principles and Practice.* Second Edition. Addison-Wesley Publ. Comp., Reading Massachusets, 1990.
- [Lamo94] Lamousin, H.J., Waggenspack, W.N.: *NURBS-Based Free-Form Deformation.* Computer Graphics and Applications, November 1994.
- [Sede86] Sederberg, T.W., Parry, S.R.: *Free-Form Deformation of Solid Models.* ACM Computer Graphics, 20 (4), August 1986.
- [Wool90] Woolberg, G.: *Digital Image Warping.* IEEE Computer Society Press Monograph, Los Alamitos, California, 1990.

8. Rastrové grafické formáty

V okamžiku, kdy se začneme podrobněji zabývat formáty pro úschovu obrazu, zjistíme, že jejich počet je překvapivě vysoký, ačkoliv jejich společným cílem je ve většině případů pouze úschova bitové mapy představující obdélníkový (barevný) obrázek. Existence mnoha formátů má několik příčin:

historické důvody
formáty odrážejí technický vývoj, zejména postupně se zvyšující barevné možnosti grafických adaptérů.
vazba na program
podle druhu aplikace vznikly specializované formáty, například pro úschovu skic a kreseb (PCX), černobílých dokumentů (TIFF) či pro přenos barevných fotografií (GIF, JPEG).
technické důvody
mnoho formátů bere ohled na rozlišení scannerů, pomocí kterých jsou obrazy zaznamenávány, na obrazová rozlišení v různých osách (aspect ratio), na zvláštní architekturu obrazové paměti v počítačích třídy PC (planes) či na uspořádání bytů v 16-bitovém slově:

adresy bitů	i	i+1	
little endian	Lo	Hi	procesory Intel
big endian	Hi	Lo	procesory Motorola,Sparc, MIPS

komprese
vzhledem k velkému paměťovému objemu barevných obrazů je žádoucí uchovávat obraz v komprimované podobě. Volba vhodné kompresní metody je často závislá na charakteru obrazu.

Uvedený přehled možných odlišností obrazových formátů není pochopitelně úplný. V řadě případů se setkáváme i s dalšími, speciálními rysy, jakými jsou například požadavky na uložení více (různých) obrázků v jednom souboru, zápis sekvence obrazů pro animaci, multimediální obrázky rozšířené o informace potřebné pro interakci (tzv. ROI - regions of interest) nebo vícerozměrné obrázky z medicínských aplikací (prostorové řezy MRI, CT).

V dalším textu se proto omezíme jen na popis základních metod pro úschovu obrazů, hlavními kritérii budou barevné rozlišení a způsob komprese. V závěru kapitoly pak uvedeme jen stručně popis několika nejznámějších formátů.

Barevné rozlišení

Většina formátů je schopna uchovávat obrázky jak černo bílé, tak barevné. Zvyšující se počet barev vede pochopitelně na větší paměťové nároky:

Obrázek		bit/pixel	Poznámka
černobílý	B/W	1	délka řádku zaokrouhlována na celé byty, (angl. *padding*)
v odstínech šedi	gray scale	8	i 6 bitů na PC platformě
s paletou	paletted	4	16-řádková paleta EGA
		8	256-řádková paleta VGA
plně barevný	color	24	RGB, CMY, YUV, YC_bC_r
		32	RGBA, CMYK

Mezi další varianty patří tzv. *high color* obrázky s přímým barevným kódováním barevných složek po 5 bitech (16-bitové slovo na pixel) či paletované obrázky s kódováním CMYK.

8.1 Komprese rastrových obrázků

Velký objem dat a zároveň specifický tvar obrazových informací jsou podnětem pro používání různých druhů kompresí. Na rozdíl od **bezztrátové** (angl. *lossless*) komprese běžných datových souborů, při které nesmí být žádná data ztracena či změněna, je u rastrových obraz ů mnohdy žádoucí docílit co nejvyššího kompresního poměru i tím, že změníme barevné hodnoty pixelů a zhoršíme tak výsledný obraz použitím **ztrátové** (angl. *lossy*) komprese.

V dalších částech popíšeme principy v grafice nejčastěji používaných kompresních metod. Detailní informace může čtenář nalézt v [Murr94] a další literatuře zabývající se kódováním. Následující tabulka obsahuje jednoduchý přehled vlastností kompresních metod:

Kompresní metoda	Zkratka	Ztrátová	Příklad formátu
Run length encoding	RLE	ne	PCX (ZSoft PaintBrush)
Huffmanovo kódování	CCITT	ne	TIFF
Lempel - Ziv - Welch	LZW	ne	GIF (Compuserv), ZIP, ARJ
Diskrétní kosinová transformace	DCT	ano	JPEG
Fraktální komprese	FIF	ano	FIF

Poznamenejme, že u všech obrazových formátů se komprese týká pouze vlastních obrazových dat. Hlavička souboru, definice palety a další doplňující informace se zapisují v nekomprimovaném tvaru.

APPLET

applet 8.1

8.1.1 Run length encoding

Jednoduchá a pro velkou třídu obrázků i efektivní metoda vychází z předpokladu, že v rastrovém obrázku, vzniklém jako kresba či skica, se opakují hodnoty sousedních pixelů. Tehdy do souboru zapíšeme nejprve počet opakujících se totožných hodnot a poté hodnotu samotnou.

Například při kódování pixelů definovaných jedním bytem rozlišíme příznak opakování hodnotou nejvyššího bitu:

1	čítač	hodnota	**hodnota** se opakuje **(1+čítač)** x
0	hodnota		přímý zápis jedné 7-bitové neopakující se hodnoty
1000 0000		hodnota	zápis neopakující se hodnoty větší než binárně 1000 0000

Efektivnější varianta RLE je schopna zachytit posloupnost různých hodnot a opakovací bit využít ve významu příznaku zápisu posloupnosti:

0	čítač N	hodnota 0	hodnota 1	.....	hodnota N

Základní varianta kódovacího algoritmu RLE je uvedena v alg.

Efektivita kódování klesá při zpracování hodnot s nenulovým nejvyšším bitem. Pokud zapisujeme obrázky definované pomocí palety, je vhodné přerovnat paletu (tj. přečíslovat hodnoty pixelů) tak, aby nejméně často používané odstíny byly umístěny v nejvyšších řádcích palety.

Metoda RLE je vhodná pro barevné rozlišení 1 či 8 bitů na pixel. Nepoužívá se pro kódování pixelů definovaných přímo hodnotami RGB, kdy není zajištěna fyzická sousednost bytů opakujících se pixelů. Výjimkou je kódování po jednotlivých barevných rovinách (angl. *color planes*).

Ve většině případů je kódování RLE prováděno v rámci jednoho řádku. Všimněme si, že zatímco obrázek tvořený mnoha vodorovnými čárami je kódován velmi efektivně, tentýž obrázek otočený o 90° je zapsán téměř beze změny. Proto se v některých aplikacích objevuje použití speciálního kódu ve významu opakovače řádků.

V případě, že kódovaný obrázek obsahuje neopakující se hodnoty v sousedních pixelech, dochází u RLE k **záporné kompresi** (angl. *negative compression*), tj. ke zvětšení výsledného souboru. Kódování RLE je proto vhodné jen pro obrázky kreslené nebo pro tzv. *cartoons* - ilustrace s většími stejnobarevnými plochami.

Vzácněji se můžeme setkat i se ztrátovou variantou metody RLE, při níž se nejprve testují sousední (barevné) pixely a pokud se liší jen v jednom či dvou bitech, nahradí se hodnotou jedinou.

1. Vynuluj *Čítač*
2. Do proměnné *BarvaA* přiřaď hodnotu prvního pixelu.
3. Dokud jsou na vstupu pixely, opakuj:
    a. Do proměnné *BarvaB* přiřaď hodnotu dalšího pixelu.
    b. Pokud je *BarvaA* = *BarvaB* a zároveň hodnota *Čítače* nepřesáhla maximální hranici (danou velikostí bytu či slova), pak zvyš *Čítač* o jedna, jinak:
        i. Zapiš *Čítač* a *BarvuA*
        ii. Do proměnné *BarvaA* přiřaď hodnotu *BarvaB*
        iii. Vynuluj *Čítač*
4. Zapiš *Čítač* a *BarvuA*

algoritmus 8.1: Princip kódování

8.1.2 Huffmanovo kódování

Toto kódování bylo původně navrženo komisí CCITT (International Telegraph & Telephone Consultative Committee) pro přenos černobílých dokumentů faxem. Myšlenka kódování pochází z r. 1952 od Davida Huffmana a je založena na použití různě dlouhých bitových kódů pro symboly s různou frekvencí výskytu. Často používané symboly mají kratší kódy, přičemž frekvence se nestanovuje individuálně pro každý dokument, nýbrž je brána z tabulek CCITT, vzniklých statistickým zpracováním mnoha typických dokumentů.

Postupně vzniklo několik variant kódování, lišících se stupněm komprese i určením:

skupina	kód	kompresní poměr	použití
Group 3	**G31D**	5:1	odolné proti poruchám
	G32D	8:1	citlivé na poruchy
Group 4	**G42D**	15:1	(bezchybný) zápis na disk

G31D

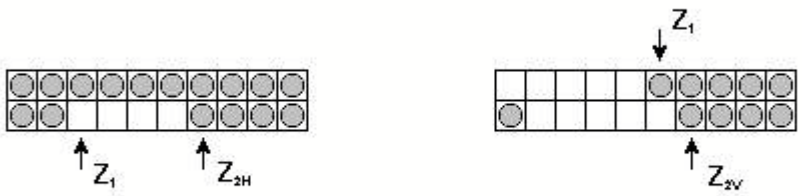
Úseky opaku jících se bílých, resp. černých pixelů na jednom řádku jsou nejprve zakódovány metodou RLE, avšak opakovače jsou nahrazeny Huffmanovými kódy. Často se vyskytující skupiny 2, 3 a 4 bitů se shodnou barvou jsou tedy zapsány pomocí kratších výsledných kódů. Další krátké kódy, většinou pro bílé pixely, jsou určeny pro střední úseky (63 bitů) a velmi dlouhé úseky (až do 2623 bitů).

Svůj vlastní kód mají i konce řádků (EOL), takže výsledný obraz lze poměrně dobře rekonstruovat i při ztrátě dat při přenosu. K synchronizaci dojde vždy po nalezení kódu EOL. Dalším efektivním kódem je FILL - vyplnění všech pixelů stejnou barvou až do konce řádku. Protože při práci s faxem není předem znám počet přenášených řádků, je dalším kódem RTC (Return to control) označen konec dat.

Zaměření na faxy je patrné nejen z odolnosti proti poruchám, ale i z toho, že dekódování lze provádět okamžitě bez použití vyrovnávací paměti (bufferu).

G32D

Tato varianta bere do úvahy dvourozměrný charakter obrazových dat (viz 2D v názvu). Hlavní myšlenkou je zapisování pozic pixelů, ve kterých dochází ke změně barev. V případě černobílého obrázku tedy není nutno uchovávat žádné barevné informace. Změna barvy z se detekuje jak v rámci jednoho řádku, tak vzhledem ke změnám z_i na řádku předchozím:



Obr.8.1: Kódování černobílého obrazu metodou G32D

Zvláštní krátké kódy jsou určeny pro změny v rozmezí do 4 pixelů, ať již na témže řádku (z_{2H} na obrázku vlevo), nebo na předchozím řádku (z_{2V}na obrázku vpravo).

Změny barev jsou zapisovány relativně vůči předchozím pozicím, takže při ztrátě dat dochází k výrazné deformaci obrazu. Proto bývá většinou každý čtvrtý řádek kódován bezpečnou metodou G31D.

G42D

Při zápisu na disk se implicitně nepředpokládá ztráta dat při přenosu, takže není nutno do obrázku dodávat synchronizační informace. Metoda G42D je přímo odvozena z metody G32D odebráním ne potřebných kódů a je používána například ve formátu TIFF.

Vyznačuje se velmi vysokým stupněm komprese, který je ovšem dán omezeným použitím na černobílé kancelářské dokumenty.

8.1.3 Lempel-Ziv-Welch

Na rozdíl od předchozích kompresních schémat specializovaných na obrazová data, je tato metoda zcela obecná a setkáme se s ní ve většině komerčních kompresních programů (ZIP, ARJ, LHARC, compress, ZOO). Původní myšlenka z r. 1977 (Abraham Lempel, Jakob Ziv) byla v r. 1984 doplněna Terrym Welchem ([Welch84]). Metoda je někdy nazývána **Dictionary based encoding**.

Princip spočívá v nahrazení vzorků vstupních dat binárními kódy proměnné (postupně rostoucí) délky. Vstupní vzorky se překládají pomocí slovníku (tabulky), který je průběžně doplňován o nové

vzorky. Délka slovníku je dána aktuálním počtem bitů použitých pro kódování. Po zaplnění slovníku se zvýší počet bitů určených pro výstupní kód o jedničku, takže délka slovníku se zdvojnásobí.

Předpokládejme zpracování obrázku s barevným rozlišením 1 byte na pixel. Pak

- základní slovník je určen 8 bity (má délku 256)
- základní počet bitů pro kódování vzorků je 9
- příznak vyčištění slovníku (*Clear code*) je binárně `100000000`

Nově nalezené vzorky jsou zaznamenávány do slovníku pomocí kódů vyšších, než je *clear code*. Při kódování větších souborů se slovník může poměrně rychle zaplňovat a pro praktické účely bývá jeho délka omezena na 4 096 položek (tedy na max. 12-bitové výstupní kódy). Vyslání dohodnutého symbolu *clear code* znamená vyčištění slovníku a jeho zkrácení do základní podoby.

Sympatickou vlastností kódování LZW je to, že slovník nemusí být zapisován do výstupních dat. Prvních 256 položek slovníku se nepřenáší, protože jsou totožné s hodnotami od 0 do 255, další položky pak lze odvodit z aktuálně zpracovávaných dat. Předpokládá se, že každý nový byte je počátečním symbolem nějakého vzorku. Máme-li na počátku na vstupu posloupnost bytů 5-3-5-3, je první číslo 5 bráno jako počátek nového vzorku 5-3. Na výstup je sice zapsán pouze kód čísla 5, ale slovník je automaticky rozšířen o vzorek 5-3. Obdobně je tomu při zpracování čísla 3. Opakující se dvojice 5-3 je pak nejen zapsána pomocí vlastního kódu (většího než *clear code*), ale spolu s následujícím číslem (či případným vzorkem) je uložena do slovníku.

Dekódování je prováděno zrcadlově vůči kódování. Z přicházejících kódů je dynamicky budován slovník a z něj vytvářen výsledný obraz. Slovník vlastně představuje zřetězené vzorky původních hodnot. S výjimkou nejnižších 256 kódů jsou všechny kódy reprezentovány jako dvojice ukazatelů na již nalezené vzorky s nižším kódovým označením.

Metoda LZW se vyskytuje v několika obměnách. Ze základních variant uveďme:

1. namísto rušení celého slovníku lze symbol *clear code* považovat za příznak, po kterém následují informace potřebné pro nahrazení málo používaných vzorků častěji používanými skupinami (například zrušení kódu pro dvojici 3-5 v předchozím příkladu)
2. především při kódování obrázků je vhodné vstupní data nejprve vyjádřit pomocí rozdílů mezi sousedními pixely (angl. *differencing*), čímž většinou nalezneme mnoho stejných vzorků. Příkladem je posloupnost pixelů 1-5-9-13-17, která se v přírůstkovém (rozdílovém) tvaru zjednoduší na 1-4-4-4-4.

8.1.4 Diskrétní kosinová transformace a JPEG

APPLET

applet 8.2

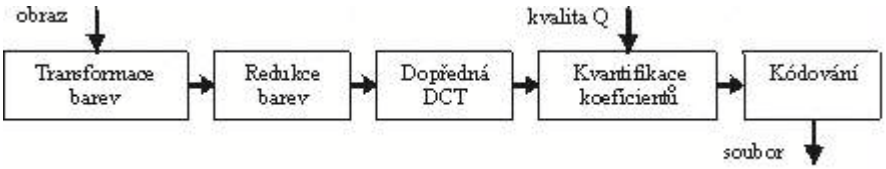
Metody RLE a LZW nejsou efektivní při kompresi plně barevných obrázků s mnoha barevnými přechody. Naneštěstí právě nejkvalitnější obrázky (ať již jde o reálné fotografie či počítačem generované obrazy) se vyznačují tím, že jen málokteré sousední pixely mají totožné hodnoty. Pro takové obrazy byla navržena metoda, při níž je kompresní poměr řízen požadavkem na výši kvality dekomprimovaného obrazu. V praxi se ukazuje, že snížení kvality na 75% je pro většinu uživatelů nepozorovatelné, přitom kompresní poměr v takovém případě může být 20:1 až 25:1.

Metoda řízené ztrátové komprese využívající diskrétní kosinové transformace byla vyvinuta skupinou **JPEG** (vyslov Joint Photographic Experts Group) v rámci mezinárodní standardizační organizace ISO v roce 1991 ([ISO94J], [ISO95J]). Je vhodná především pro kódování fotografií,

u nichž sousední pixely (na řádku či ve sloupci) mají sice odlišné, ale přesto blízké barvy. Snižování kvality obrazu se projeví potlačováním rozdílů v blízkých barvách.

Metoda není vhodná pro obrazy s nižším barevným rozlišením (s paletou, upravené ditheringem), také na velkých jednobarevných plochách vytváří artefakty. Optimálním vstupem je obraz ve 24 bitech na pixel.

Samotná diskrétní kosinová transformace je formou diskrétní Fourierovy transformace. Obrazová data jsou považována za barevné vzorky spojitých barevných signálů naměřené v diskrétní síti pixelů. Výsledkem kosinové transformace je pak nalezení takové sady koeficientů různých vlnění, jejichž složením lze rekonstruovat původní obraz. Rozklad na jednotlivé signály je výsledkem **dopředné transformace** (FDCT), která je vlastně harmonickým analyzátorem. **Zpětná transformace** (IDCT) pak představuje harmonický syntezátor.



Obr.8.2: Posloupnost operací při kompresi JPEG

Postup při kompresi JPEG je poměrně složitý a sestává z několika kroků, jak ukazuje obr. 8.2 . Při dekompresi jsou tyto kroky prováděny analogicky v obráceném pořadí. Podívejme se na jednotlivé etapy blíže:

1. **Transformace barev**
   Ať již jsou vstupem barvy pixelů v prostoru RGB, CMY či CMYK, pro další zpracování je třeba je převést do barevného modelu $\mathbf{YC_bC_r}$ se třemi byty na pixel. V další fázi jsou totiž jasové složky Y a barevné složky C_b, C_r zpracovávány odděleně.
2. **Redukce barev**
   Lidské oko je mnohem citlivější na rozdíly v úrovních jasu, než na změny barevných odstínů. V tomto kroku je proto možno snížit objem dat zprůměrováním barevných složek několika sousedních pixelů a nahrazením jedinou hodnotou. Používají se dvě varianty:

   2h1v: průměrování sousedních dvojic $\boxed{C}\boxed{C}$ redukce 6 ⇒ 4 byty

   2h2v: průměrování čtveřic $\begin{array}{|c|c|}\hline C & C \\ \hline C & C \\ \hline\end{array}$ redukce 12 ⇒ 6 bytů

   Tento krok neznamená zásadní zhoršení obrazu, neboť jeho kvalita je přímo ovlivňována až ve 4. fázi. Jasové složky Y všech pixelů zůstávají zachovány.

3. **Dopředná diskrétní kosinová transformace**
   Obrazová data jsou rozdělena do čtverců 8×8. Každý čtverec je podroben diskrétní kosinové transformaci. Výsledkem je 8×8 koeficientů F(u,v) získaných výpočtem z hodnot pixelů f(x,y) podle vzorce:

$$F(u,v) = \frac{1}{4} C(u) . C(v) . \left[\sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y). \cos \frac{(2x+1)u\pi}{16} . \cos \frac{(2x+1)v\pi}{16} \right] \quad (8.1)$$

$$C(u), C(v) = \frac{1}{\sqrt{2}} \quad \text{pro} \quad u,v = 0, \quad \text{resp. } 1 \text{ jinde.}$$

Zpětná (inverzní) transformace má tvar:

$$f(x,y) = \frac{1}{4} . \left[\sum_{u=0}^{7} \sum_{v=0}^{7} C(u) . C(v) . F(u,v). \cos \frac{(2x+1)u\pi}{16} . \cos \frac{(2x+1)v\pi}{16} \right] \quad (8.2)$$

Hodnoty získané dopřednou transformací jsou neceločíselné. Jejich zaokrouhlením vzniká další (nevýznamné) zhoršení kvality obrázku.

Na obrázku můžeme sledovat postupné změny hodnot kódované oblasti 8×8 pixelů (posloupnost obrázků a-d) a jejich dekódování (e-f).

139	144	149	153	155	155	155	155
144	151	153	156	159	156	156	156
150	155	160	163	158	156	156	156
159	161	162	160	160	159	159	159
159	160	161	162	162	155	155	155
161	161	161	161	160	157	157	157
162	162	161	163	162	157	157	157
162	162	161	161	163	158	158	158

a) vstupní data f(x,y)

235.6	-1.0	-12.1	-5.2	2.1	-1.7	-2.7	1.3
-22.6	-17.5	-6.2	-3.2	2.9	-0.1	0.4	-1.2
-10.9	-9.3	-1.6	1.5	0.2	-0.9	-0.6	-0.1
-7.1	-1.9	0.2	1.5	0.9	-0.1	0.0	0.3
-0.6	-0.8	1.5	1.6	-0.1	-0.7	0.6	1.3
1.8	-0.2	1.6	-0.3	-0.8	1.5	1.0	-1.0
-1.3	-0.4	-0.3	-1.5	-0.5	1.7	1.1	-0.8
-2.6	1.6	-3.8	-1.8	1.9	1.2	-0.6	-0.4

b) koeficienty F(u,v) po FDCT

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	61
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

c) kvantifikační tabulka

15	0	-1	0	0	0	0	0
-2	-1	0	0	0	0	0	0
-1	-1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

d) kvantifikované koeficienty

240	0	-10	0	0	0	0	0
-24	-12	0	0	0	0	0	0
-14	-13	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

e) expandované koeficienty před IDCT

144	146	149	152	154	156	156	156
148	150	152	154	156	156	156	156
155	156	157	158	158	157	156	155
160	161	161	162	161	159	157	155
163	163	164	163	162	160	158	156
163	164	164	164	162	160	158	157
160	161	162	162	162	161	159	158
158	159	161	161	162	161	159	158

f) rekonstruovaná data

4. **Kvantifikace koeficientů DCT**

Právě v této fázi může uživatel stanovit kvalitu (a tedy i stupeň komprese), s jakou má být obrázek komprimován. Koeficienty F(u,v) jsou snižovány vydělením určitým kvantifikačním činitelem a posléze zaokrouhleny.

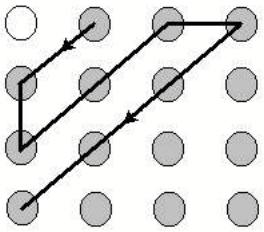
Zvýrazněný koeficient F(0,0) v levém horním rohu čtverce v tab. b je nazýván **DC člen** a představuje stejnosměrnou složku harmonického rozkladu. Má největší celkový vliv na veškerá obrazová data ve čtverci. Ostatní koeficienty se nazývají **AC členy** a ovlivňují vyšší frekvence, tedy změny mezi jednotlivými pixely. Jejich vliv klesá se vzdáleností od DC členu.

Kvantifikační činitelé byly stanoveny komisí JPEG a jsou uloženy v několika tabulkách (tab. c) pro složky Y, C_b a C_r. Každý koeficient ve zpracovávaném čtverci je dělen jedním odpovídajícím činitelem, velikost činitelů se opět zvětšuje se vzdáleností od levého horního rohu.

Uživatel stanovuje koeficient kvality Q v rozsahu od 1 do 100. Kvantifikační tabulka byla definována pro průměrnou kvalitu obrazu (cca 75%) a po zadání Q jsou její hodnoty přepočítávány v nepřímé úměře na Q. Takto upravenými kvantifikačními činiteli jsou dělena zpracovávaná data. Kvantifikační tabulky jsou stejné pro daný obrázek a jsou ukládány do výstupního souboru kvůli rekonstrukci obrazu.

5. **Kódování**
Výsledkem předchozích kroků byly numerické úpravy čtvercové podoblasti vstupního obrazu, které zjednodušily tvar dat až do podoby řídké matice s dominantním levým horním rohem (tab. d). Tato matice je nyní kódována a zapisována do výstupního souboru, přičemž je využíváno toho, že většina koeficientů v okolí pravého dolního rohu má nulovou hodnotu. Matice čísel je převedena na posloupnost tak, jak ukazuje obr.8.3. , a tato posloupnost je zakódována Huffmanovým nebo aritmetickým kódováním.

Členy DC se zapisují samostatně. Nejprve se sdruží DC členy z celého obrázku a poté jsou kódovány jejich rozdílové hodnoty (relativní změny vůči předchozí hodnotě).



Obr.8.3:Kódování AC členů

Kódování JPEG se vyznačuje nejen vysokým stupněm komprese, ale i tím, že umožňuje extrémně rychlý náhled (angl. *preview*) na zakódovaný obrázek. Využívá se přitom toho, že jednotlivé složky Y, C_b a C_r jsou ukládány samostatně, také jejich DC členy jsou sdružovány do skupin a ukládány samostatně.

Chceme-li si tedy rychle prohlédnout obrázek ve formátu JPEG, stačí v souboru nalézt sekci se všemi DC členy jasové složky Y a zobrazit je jako odstíny šedi. Získaný obrázek bude mít 8× menší rozměry oproti původnímu, ale dodá nám dostatečnou představu o vzhledu celkového obrázku.

8.1.5 Fraktální komprese

Tato moderní a teprve rozvíjející se metoda ztrátové komprese patří mezi **nesymetrické** kompresní postupy, tedy mezi ty, u nichž se výrazně liší čas komprese a dekomprese.

Metodu navrhl Michael F. Barnsley ([Barn93]) na základě svých výzkumů v oblasti fraktálních algoritmů a systémů IFS. Je založena na principu vyhledávání podobností (tvarových, barevných) v různě velkých částech obrazu. Podobnost je formálně zapsána pomocí afinní transformace:

$$W\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by + e \\ cx + dy + f \end{bmatrix}, \tag{8.3}$$

která vyjadřuje otočení i zmenšení nalezeného podobného motivu, stejně jako jeho posunutí (koeficienty e, f). Opakování téhož motivu je pak definováno pouhými šesti koeficienty.

Do výsledného souboru formátu FIF (Fractal Image Format) se zapisují koeficienty podobnosti v komprimovaném tvaru. Je zřejmé, že nacházení podobných oblastí a motivů představuje klíčový problém metody. Na obrázcích skutečného okolního světa i na počítačem generovaných obrazech však lze nalézt překvapivě vysoké množství opakujících se detailů a motivů. Algoritmus FIF se snaží nejprve vhodně rozdělit obraz na menší, nestejně veliké části (domény), a poté z nich pomocí různých transformací poskládat (vyjádřit) celý obraz.

Velikost komprimovaného obrázku závisí nepřímo úměrně na velikosti domén. Také třída použitých transformací ovlivňuje stupeň komprese. Zadá-li uživatel cílovou velikost souboru FIF například 1:10 vzhledem k původnímu obrázku, může řídit kvalitu obrazu stanovením časového limitu, ve kterém mají být nacházeny podobnosti. Práce s fraktální obrazovou kompresí tak vyžaduje zkušenost s odhadem rozumné velikosti souboru a povoleného času vyhledávání podobností, protože každý obrázek má z hlediska podobnosti jiné vlastnosti.

Ačkoliv doba komprese obrázku je velmi dlouhá (pohybuje se řádově v minutách až desítkách minut podle požadavků na stupeň komprese a kvalitu obrazu), fraktální charakter dat umožňuje při dekompresi vytvořit obrázek s mnohem většími rozměry než měl původní obraz. Nalezne-li například kódovací algoritmus ve vstupním obrázku úsečku, definuje ji jako

$$y = ax + b,$$

což je vlastně afinní transformace (viz rov. 8.3). Je zřejmé, že iteračním postupem lze takto zadanou přímku do libovolné přesnosti a bodového rozlišení.

Celkově je kvalita výsledných obrázků lepší než při použití metody JPEG, u které v některých případech může dojít k optickému zvýraznění dělení obrázku na čtverce 8×8. Navíc doba dekomprese je u formátu FIF kratší. Pokud se podaří výrazně snížit dobu komprese obrázku (objevují se i hardwarové akcelerátory), lze v blízké budoucnosti očekávat příchod formátu FIF do praktických aplikací.

8.2 Příklady rastrových formátů

Literatura [Murr94] uvádí, že existuje okolo 40 rozšířených formátů pro úschovu rastrových obrázků. Nemá jistě smysl je zde uvádět, stejně tak není třeba detailně rozebírat vnitřní strukturu vybraných formátů. V této části proto popíšeme jen několik nejznámějších formátů a všimneme si jejich případných omezení.

PCX

Jeden z nejstarších formátů vyvinutý firmou ZSoft je dodnes značně rozšířen. Existuje v několika verzích a dokáže uchovávat obrázky od černobílých či 16-barevných ve verzi 2.5 až po barevné rozlišení 8 a 24 bitů/pixel ve verzi 3.0.

Používá kompresi RLE, z historických důvodů je příznak opakování detekován nikoliv prvním nejvyšším bitem, nýbrž nastavením prvních dvou bitů na 1. Je však také možno ukládat nekomprimovaná data.

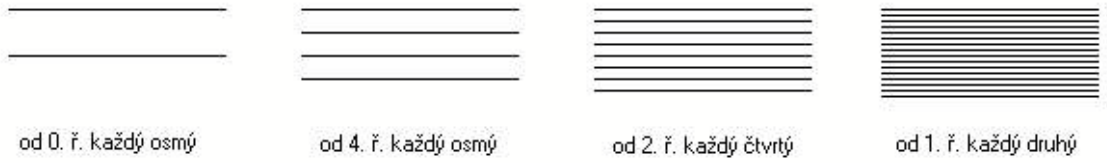
Formát PCX je typickým příkladem formátu, sledujícího postupný vývoj technických prostředků. Jeho hlavička má prostor pouze pro 16-barevnou paletu EGA, takže případná 256-prvková paleta je ukládána na konec souboru. Zobrazovací programy, které nejprve nastaví paletu a poté zapisují do obrazové paměti dekódovaný obrázek, musí hledat příznak přítomnosti palety 768 bytů (3×256) od konce souboru. Ani toto místo není jednoznačné, neboť paleta může být kratší. Jsou povoleny délky 256, ..., 2.

Historicky původní uspořádání dat v obrazové paměti po barevných rovinách se odráží v zápisu dat do souboru, takže barevně bohatší obrázky (24 bitů/pixel) musí být zapisovány do tří samostatných bloků pro jednotlivé složky R, G a B.

I přes tyto komplikace je formát PCX oblíben, dekódování RLE je snadné a rychlé. Je jedním z formátů podporovaných MS Windows.

Graphics Interchange Format (GIF)

Jeden z nejoblíbenějších formátů z doby prvních obrazových adaptérů VGA, specializovaný na obrázky s paletou a s jedním bytem na pixel, případně i s menším barevným rozlišením, byl vyvinut firmou CompuServ.



Obr.8.4: Prokládání řádků ve čtyřech krocích

Použitá kompresní metoda LZW přináší pro většinu obrázků velké snížení objemu dat. Původní určení pro přenos obrázků po telefonních linkách se projevuje ve složitější struktuře formátu, je však vítané pro síťové multimediální aplikace, např. World Wide Web.

Mezi základní charakteristiky a možnosti patří:

- **více obrázků** v jednom souboru, každý z nich může mít vlastní barevnou paletu. Většina aplikací však používá GIF pro uložení právě jednoho obrázku.
- možnost **prokládání řádků** je vhodná pro on-line přenos obrázků po síti. Uživatel je schopen již po získání 1/4 či 1/2 objemu obrazových dat rozpoznat vzhled obrázku a přenos dat případně ukončit.

- ukládání **textových informací**, a to buď jako součást zobrazovaných dat nebo v podobě komentáře čitelného při prohlížení souboru běžným editorem.
- **řídicí prvky** pro interaktivní práci s obrazem, např. časové prodlevy při zobrazování nebo aktivní oblasti určené pro identifikaci (*pick*).
- blíže neurčená **aplikační data**.

Základní verze **GIF87a** je stále používána. Nové prvky ve verzi **GIF89a** (označeny ∗ v předchozím přehledu), které jsou zaměřeny na multimedia, jsou však většinou aplikací nevyužívány.

Počítačové obrázkové archivy na celé světě obsahují velké množství souborů zapsaných právě ve formátu GIF. Omezení na max. počet 256 barev je však kritickým faktorem pro jeho používání v bud.

Targa (TGA)

Tento formát byl vyvinut firmou Truevision specializovanou na výrobu obrazových adaptérů. Používá kompresi RLE, mezi programátory je však oblíbena především jeho varianta bez komprese, umožňující např. po uložení jednoduché 18-bytové hlavičky zapsat obrázek v 24 bitech/pixel do souboru přímo pixel po pixelu. Oproti formátu PCX nedovoluje TGA ukládat obrázky po jednotlivých obrazových rovinách (angl. *color planes*).

Formát TGA byl první 24-bitový obrazový formát na platformě počítačů PC a je podporován i v operačním systému UNIX. Dokáže uložit obrázky v barevném rozlišení 1, 8, 16, 24 a dokonce i 32 bitů/pixel. Poslední případ je určen pro barevný model RGBA.

Původní verze 1 z r. 1987 je nejrozšířenější. Novější verze 2 zachovává původní strukturu formátu a pouze přidává na konec souboru blok aplikačních dat.

Následující přehled obsahuje čtyři nejčastěji používané nekomprimované zápisy obrázků ve formátu Targa. Obsah bytů je uveden hexadecimálně, pořadí je dáno uspořádáním *little endian*.

typ 1: 8 bitů/pixel + paleta

hodnoty v bytech	význam (dekadická hodnota)
`00 01 01`	základní typ hlavičky (0), paleta (1), typ obrazu (1)
`00-00`	počáteční index palety (zde 0)
`00-01 18`	délka palety (zde 256), počet bitů na položku palety (zde 24)
`xx-xx yy-yy`	počátek `x,y` umístění obrázku
`XX-XX YY-YY`	rozměry `X` a `Y` obrázku
`08 20`	počet bitů/pixel (8), kód umístění na obrazovce (vlevo nahoře)
...	položky palety v pořadí B-G-R
...	obrázek po řádcích (1 byte/pixel)

typ 2: 24 bitů/pixel

hodnoty v bytech	význam (dekadická hodnota)
`00 00 02`	základní typ hlavičky (0), žádná paleta (0), typ obrazu (2)
`00-00 00-00 00`	nulové údaje o paletě
`xx-xx yy-yy`	počátek `x,y` umístění obrázku
`XX-XX YY-YY`	rozměry `X` a `Y` obrázku
`18 20`	počet bitů/pixel (24), kód umístění na obrazovce (vlevo nahoře)
...	obrázek po řádcích a pixelech v pořadí B-G-R

typ 3: 256 odstínů šedi

hodnoty v bytech	význam (dekadická hodnota)
`00 00 03`	základní typ hlavičky (0), žádná paleta (0), typ obrazu (3)
`00-00 00-00 00`	nulové údaje o paletě
`xx-xx yy-yy`	počátek `x,y` umístění obrázku
`XX-XX YY-YY`	rozměry `X` a `Y` obrázku
`08 20`	počet bitů/pixel (8), kód umístění na obrazovce (vlevo nahoře)
...	obrázek po řádcích (1 byte/pixel)

typ 3: černobílý

hodnoty v bytech	význam (dekadická hodnota)
`00 00 03`	základní typ hlavičky (0), žádná paleta (0), typ obrazu (3)
`00-00 00-00 00`	nulové údaje o paletě
`xx-xx yy-yy`	počátek `x,y` umístění obrázku
`XX-XX YY-YY`	rozměry `X` a `Y` obrázku
`01 20`	počet bitů/pixel (1), kód umístění na obrazovce (vlevo nahoře)
...	obrázek po řádcích (1 bit/pixel)

Tag Image File Format (TIFF)

Tento formát je dokladem toho, že velká univerzálnost přináší velkou složitost a rozmanitost struktur. Je spolu s formátem JPEG jediným mezinárodním standardem pro kódování statických obrazů [ISO96] a je tedy podporován všemi výpočetními platformami (UNIX, MS-DOS, Macintosh), takže se objevuje ve variantách *little* i *big endian*. Podobně jako GIF dokáže uložit více obrázků do jednoho souboru. Prošel poměrně složitým historický vývojem a je tudíž schopen zapsat obrazy v nejširší škále barevných rozlišení a modelů:

verze	rok	komprese	poznámka
3.0	1986	CCITT	černobílé obrazy a odstíny šedi
4.0	1987	RLE	RGB
5.0	1988	LZW	palety
6.0	1992	JPEG	CM YK, YC_bC_r, standard ISO

Ačkoliv je jasně a dobře dokumentovaný, mezi běžnými uživateli panuje nedůvěra ke kompatibilitě souborů TIFF, byť jsou stejné verze. Tento fakt je způsoben tím, že škála možností zápisu do formátu TIFF je tak široká, že jen málokterý program dokáže interpretovat všechny povolené varianty. Soudí se, že: Z hlediska vnitřní struktury je soubor TIFF rozdělen do tří tříd:

H	*Header*	hlavička souboru (jediná)
IFD	*Image File Directory*	popis významu dat
I	*Image data*	vlastní data specifikovaná pomocí IFD

První nepříjemností je několik variant zápisu uvedených tříd. Za hlavičkou mohou následovat třídy IFD a I buď pravidelně se střídající, nebo nejprve zcela všechny IFD následované všemi I, případně opačně.

Speciálním rysem formátu je právě třída IFD, která je tabulkou proměnlivé délky s položkami o délce 12 bytů. Každá taková položka se nazývá **tag** (česky např. *visačka*) a obsahuje krom svého identifikátoru i specifikaci datového typu (byte, short, long, apod.), počet příslušných dat a adresu počátku dat v odpovídající sekci I (Image data).

Existuje více než 70 typů visaček ve visaček verzi 6.0 byl pojem **tag** nahrazen pojmem **field**, takže lze očekávat i změnu *visačky* na obyčejné *pole* aplikačním programům je dokonce povoleno definovat svoje vlastní. Ačkoliv mohou být visačky v sekci IFD libovolně prostřídány, formát TIFF definuje čtyři druhy povinných skupin, u kterých je nutno dodržet pořadí. Podle barevného rozlišení je třeba vždy začít do souboru zapisovat visačky z patřičné povinné skupiny B (černobílé), G (odstíny šedi), P (s paletou) nebo R (plně barevné).

Dalším stupněm volnosti u formátu TIFF je rozdělení vlastních obrazových dat. Krom běžného uložení obrazu vcelku je sympatická možnost ukládat obraz po **pruzích** (angl. *strips*), které jsou menší než 64kB. Krom zadání šířky pruhů lze definovat i jejich prokládání. V poslední verzi TIFF 6.0 je pak podporováno uspořádání obrazových dat do **dlaždic** (angl. *tiles*), které například umožní aplikačnímu programu zpracovat jen část velkého obrazu (plakátu), na běžné obrazovce jinak naprosto nezobrazitelného.

8.3 Formáty pro animované sekvence

Animace je v současné počítačové grafice téměř samostatným odvětvím, zabývajícím se jak popisem vhodných modelů a jejich pohybu, tak efektivním prováděním samotné animace. Proto se o formátech pro animované sekvence zmíníme jen krátce, abychom ukázali na rozdíly mezi kódováním

statických obrazů (angl. *still images*) a animovaných sekvencí či přímo video záznamu (angl. *motion pictures*). Jako ukázky jsme vybrali na platformě PC značně rozšířený formát FLI a mezinárodně standardizovaný formát MPEG pro zápis video i audio informací.

Flic Autodesk Animator (FLI)

Formát, nazývaný někdy také **Flic**, byl původně určen pouze pro animované sekvence v rozlišení 320×200 pixelů, a to vždy s použitím palety (16 či 256 barev). Novější verze **FLC** umí zpracovat obrázky až do velikosti 64K×64K pixelů.

Každý jednotlivý obrázek, resp. snímek (angl. *frame*) může mí t svoji vlastní paletu. K dosažení vyšší komprese se používá kódování rozdílového obrázku, tzv. **delta komprese**, mezi dvěma po sobě následujícími snímky. Rozdílový obrázek většinou obsahuje mnoho nulových hodnot, takže do souboru zaznamenáme pouze pozice nenulových hodnot a jejich velikosti. Jen v případě náhlé změny snímku je rozdílový obrázek větší než samotný snímek - tehdy je povoleno uložit do souboru nekomprimovaný obraz.

Soubor FLI, resp. FLC, je tvořen posloupností bloků, které se nazývají *chunks*. Jsou to různě velké datové struktury, jejichž záhlaví určuje jejich velikost i význam. Tyto bloky mohou obsahovat další bloky. Z definovaných osmi typů jmenujme:

FLI_COPY - nekomprimovaný snímek,
BYTE_RUN - snímek s RLE kompresí, nejčastěji první,
COLOR_64, COLOR_256 - definice palety,
DELTA_FLI, DELTA_FLC - delta komprese snímku; obsahuje posloupnost údajů o přeskočených (nezměněných) řádcích a pixelech, jakož i hodnoty změněných pixelů,

MPEG

Obdobně jako JPEG, také formát MPEG vyslov je výsledkem oficiální standardizační aktivity, tentokrát skupiny Motion Picture Experts Group ([ISO93M1], [ISO93M2]). Jeho první verze je již běžně používána, druhá se připravuje. V první verzi MPEG-1 je formát MPEG určen pro úschovu a přenos video a audio informací ve vysoké kvalitě, tj. až do rozměru 4095×4095 pixelů, 30 snímků za sekundu při přenosových rychlostech do 1.5 MBit/s. Obraz je kódován v barevném modelu YUV.

Další verze, MPEG-2, je schopna při přenosových rychlostech do 15 MBit/s zvládnout on-line přenos TV obrazu v diskrétní podobě a v různých rozlišeních, např. pro HDTV. Pracuje se i na hardwarových kodérech a dekodérech MPEG.

Pomineme otázky kódování zvukových informací a všimneme si záznamu obrazu. Formát MPEG používá diskrétní kosinovou transformaci obdobně jako JPEG. Nepleťme si MPEG s formátem MJPEG (Motion JPEG), který je pouhým záznamem posloupnosti samostatně zpracovaných JPEG obrázků, avšak vzhledem k charakteru zpracovávaných dat dosahuje běžně mnohem vyššího kompresního poměru, typicky 20-40:1 (teoreticky až 200:1).

Posloupnost snímků je zpracována a rozdělena do tří skupin:

I-frame (intraframe) - běžný snímek
P-frame (predictive) - rozdílový obrázek mezi běžným snímkem a předchozím I nebo P-snímkem
B-frame (bi-directional) - rozdílový obrázek mezi dvěma nejbližšími I či P-snímky

Každý druh snímku je samostatně komprimován obdobnou metodou jako JPEG. Nejmenší komprese se dosahu je u I-snímků, nejlepší u B-snímků. Typicky jsou objemy snímků I:P:B po kompresi v poměru 15:5:2. Ačkoliv by bylo výhodné vytvořit co nejvíce B-snímků, z praktických důvodů jsou snímky kódovány v přesně definovaných posloupnostech s periodou 12 snímků

I B B P B B P B B P B B I

tak, aby se každé 0.4 sekundy objevil jeden I-snímek.

Vzhledem k tomu, že B-snímky jsou vztaženy k okolním snímkům, není při přenosu či zápisu dat do souboru dodrženo uvedené pořadí snímků.

První část posloupnosti

$I_1\ B_2\ B_3\ P_4\ B_5\ B_6\ P_7\ B_8\ B_9\ P_{10}\ B_{11}\ B_{12}\ I_{13}$

je zapsána jako

$I_1\ P_4\ B_2\ B_3\ P_7\ B_5$ ...,

protože pro zobrazení mezilehlých snímků B_2 a B_3 je potřeba znát snímek P_4. Při dekódování je tedy nutno mít dostatek paměti na několik snímků, typicky na posloupnost $I_1\ B_2\ B_3\ P_4$, která je již vhodná pro zobrazování. Na displeji je nejprve zobrazen snímek I_1. Z něj je za pomoci P_4 a B_2 vytvořen snímek I_2 (nahrazující B_2). Totéž se opakuje pro B_3. Čtvrtý obrázek vznikne přičtením rozdílového snímku P_4 k aktuálnímu snímku I_3.

Literatura:

- [Barn93] M. Barnsley and L. Hurd. *Fractal Image Compression.* A. K. Peters, 1993
- [ISO93M1] ISO, 11172-1: *Information technology - Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to about 1.5Mbit/s-Part1:Systems.* International standart ISO/IEC, 1993
- [ISO93M2] ISO, 11172-2: *Information technology - Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to about 1.5Mbit/s-Part2:Video.* International standart ISO/IEC, 1993
- [ISO94J] ISO, 10918-1:*Information technology-Digital Compression and Coding of Continuoustone Still Images, Part 1: Requirements and Guidelines.* International standart ISO/IEC, 1994
- [ISO95J] ISO, 10918-5:*Information technology-Digital Compression and Coding of Continuoustone Still Images, Part 2: Compliance Testing.* International standart ISO/IEC, 1995
- [ISO96] ISO, 12639: *Prepress Digital Data Exchange - Tag Image File Format for Image Technology (TIFF/IT). International standart ISO/IEC , 1996*
- [Murr94] J.D. Murray and W. van Ryper. *Encyklopedie grafick7ch form8t;*. Computer Press, 1995.
- [Welch84] T. A. Welch A Technique for High Performance Data Compression, IEE *Copmuter*, 17(6), June 1990

9. Počítačová geometrie v počítačové grafice

Počítačová geometrie se zabývá studiem algoritmického řešení úloh týkajících se geometrických objektů, jako jsou např. body, přímky, kružnice a polygony v rovině, nebo obdobné objekty v prostorech s vyšší dimenzí. Tuto oblast otevřel v roce 1976 M. I. Shamos a od té doby se rychle rozvíjí ([PrSh88]).

Velké množství problémů a úloh v počítačové grafice se týká přímo geometrických objektů. Algoritmy počítačové geometrie pracují a dávají výsledky v prostoru objektů. Objekty jsou popsány ve strukturální formě a na této strukturální úrovni jsou řešeny příslušné úlohy. Tento přístup je zcela odlišný od řešení podobných nebo shodných úloh v obrazovém prostoru, kdy je strukturální forma převedena na obrazově bližší formu, např. na pixely a při vlastním řešení není původní struktura geometrických objektů využita.

Práce se strukturálními modely objektů přináší některé významné výhody. Hlavním přínosem je skutečnost, že časová a paměťová složitost algoritmů nezávisí na velikosti obrazu a jednotlivých těles, ale na jejich počtu a struktuře. Některé algoritmy v prostoru objektů jsou výrazně jednodušší, nebo, jako v případě transformací objektů, podstatně přesnější. Jako příklad aplikace počítačové geometrie si uvedeme přístupy k řešení několika základních úloh v počítačové grafice. Jedná se o následující úlohy:

- přibližné určení/vymezení tvaru objektu (konvexní obal, triangulace)
- nalezení blízkých objektů a stanovení vzdálenosti, nalezení nejbližšího souseda (použití Voronoiových diagramů)
- nalezení objektů viditelných v okénku (použití k-d stromů, intervalových a segmentových stromů)
- hledání průsečíků mezi více objekty ve scéně (prostorové třídění objektů)
- třídění scény při řešení viditelnosti

Dále uvedené postupy se nezabývají vlastním výpočtem zjišťovaných údajů (neuvádíme zde žádné analytické vzorce), ale orientují se na snížení výpočtové složitosti v prostředí s velkým množstvím objektů. Pro všechny metody je společné použití určité formy třídění, buď statické (před vlastním výpočtem) nebo dynamické, tj. třídění v během výpočtu. Předpokládáme, že čtenář je obeznámen s pojmy strom, vyvážený binární vyhledávací strom, třídění a zatřiďování apod.

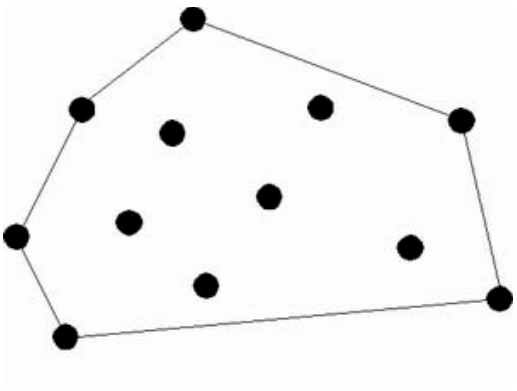
Cílem počítačové geometrie je nalezení takových algoritmů, které vyřeší optimálním způsobem paměťově a časově náročné úlohy v prostoru objektů. Výsledkem řešení je (ve většině případů) určení vlastností objektů, které závisí na jiných objektech ve scéně, nebo analýza vlastností objektu se složitou strukturou.

9.1 Tvar objektů

U zadaného objektu nebo u množiny objektů, např. bodů, je často důležité získat informaci o tvaru tohoto objektu nebo množiny. Chceme např. vymezit část prostoru, ve které se objekty nachází, nebo chceme rozdělit objekt na menší a jednodušší objekty. V této části se věnujeme dvěma problémům. Nejprve se zaměříme na konvexní obaly množiny bodů a polygonů. Poté si uvedeme některé přístupy k řešení triangulace, která dělí bodové množiny nebo polygony na více nepřekrývajících se trojúhelníků.

9.1.1 Konvexní obaly

Nechť V je množina bodů nebo jiných objektů v rovině. Konvexní obal (angl. *convex hull*) CH(V) je nejmenší konvexní objekt, který obsahuje všechny body nebo objekty V. Objekt je konvexní právě tehdy, když pro každé dva body objektu úsečka, která tyto body spojuje, leží celá uvnitř objektu. Konvexní obal ohraničuje množinu objektů z vnějšku. Na obr. 9.1 je znázorněn konvexní obal množiny bodů.



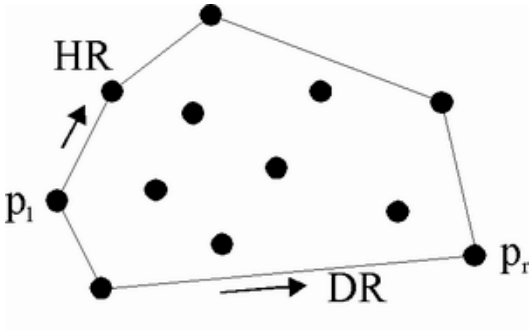
obr. 9.1: Konvexní obal množiny bodů

Konvexní obaly hrají významnou roli v mnoha aplikacích. Poskytují základní informaci o tvaru objektu. Protože se s konvexními objekty pracuje jednodušeji, než s obecnými tvary, bývá v mnoha aplikacích použit konvexní obal jako první dobrá aproximace objektu. Pokud tedy např. přímka neprotíná konvexní obal, pak určitě neprotíná ani těleso uvnitř konvexního obalu. Konvexní obal je vhodnou alternativou při volbě omezujících objemů, které se používají v zobrazovacích algoritmech.

Nejprve se omezíme na objekt tvořený množinou izolovaných bodů v rovině. Pro nalezení konvexního obalu existuje několik metod. Uvedeme si příklady metod, které používají odlišné přístupy.

Grahamův algoritmus - konvexní obal množiny bodů

Body setřídíme podle x-souřadnice v čase O(n . log n) do seznamu P. Zřejmě nejlevější a nejpravější body p_l a p_r patří ke konvexnímu obalu. Konvexní obal bude tvořen dvěma řetězy (posloupnostmi) bodů mezi p_l a p_r tak, že všechny body leží mezi horním řetězem HR a dolním řetězem DR. Uvedeme postup výpočtu HR, DR se řeší obdobně. Řetěz tvoříme zleva doprava z p_l do p_r a opakujeme následující kroky. Použité značení je na obr. 9.2.

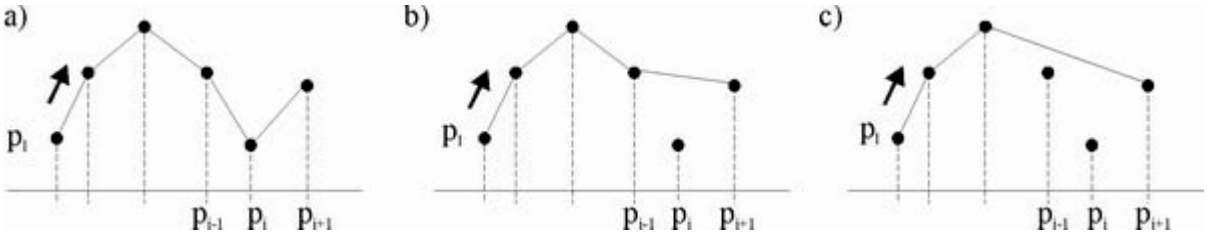


obr. 9.2: Konstrukce konvexního obalu Grahamovým algoritmem

V k-tém kroku je vytvořen řetěz HR = $\{p_1, p_2, \dots, p_{j-1}\}$. Na konec řetězu připojíme pod označením p_j další bod ze setříděného seznamu P. Ověříme, zda poslední 3 body řetězu p_{j-2}, p_{j-1}, p_j splňují podmínku konvexního tvaru. Vyhodnotíme vektorový součin dvou po sobě jdoucích vektorů určených vrcholy a zjistíme znaménko z-souřadnice výsledku vektorového součinu. Souřadnice těchto bodů v rovině označíme $[x_{j-2}, y_{j-2}], [x_{j-1}, y_{j-1}], [x_j, y_j]$. Pro horní řetěz musí být

$(x_{j-1}-x_{j-2}).(y_j-y_{j-1})- (x_j - x_{j-1}) . (y_{j-1} - y_{j-2}) \geq 0$, (u dolního řetězu ≤ 0).

Pokud ne, vynecháme z řetězu předposlední bod p_{j-1} a test opakujeme zpětně vždy se třemi posledními body řetězu do té doby, než vyloučíme z HR body, které leží pod horní hranicí konvexního obalu. Příklad opravy HR s vyloučením dříve zařazených bodů je uveden na obr. 9.3.



obr. 9.3: Vytváření horního řetězu na konvexním obalu

Protože při tvorbě výstupních řetězů HR a DR v každém kroku buď připojujeme nový bod do řetězu, nebo vylučujeme již zařazený bod, je pro setříděnou množinu bodů časová složitost algoritmu O(n).

Konvexní obal množiny n bodů v rovině je řešitelný v čase O(n . log n). Pokud jsou body setříděné ve zvoleném směru, je problém řešitelný v čase O(n) .

APPLET

applet 9.1

Jarvisův algoritmus

Tento algoritmus má sice časovou složitost nejhoršího případu O (n^2), ale pracuje velmi dobře při malém počtu bodů na konvexním obalu. Implementačně je velmi jednoduchý.

Nechť p_0 je nejnižší bod $p_0 \in V$ (s minimální y-souřadnicí). Tento bod je zřejmě součástí konvexního obalu. Nechť l je vodorovná přímka vedená bodem p_0. l otáčíme v kladném směru okolo p_0, dokud neprochází bodem $p_1 \in V$. p_1 nalezneme např. tak, že testujeme všechny body $p_i \in V$ a hledáme nejmenší úhel mezi p_0 p_i a l. Nalezený bod p_1 bude také součástí konvexního obalu. Nyní otáčíme přímkou l ve stejném směru kolem bodu p_1 a nalezneme obdobně bod p_2 atd. Algoritmus ukončíme po návratu přímky l do bodu p_0. Pro svoji podobnost s balením se tento algoritmus označuje také jako **zabalovací algoritmus** (angl. *gift wraping*).

Konvexní obal množiny n bodů lze vypočítat v čase O(k . n), kde k je počet bodů na konvexním obalu.

Důkaz: Podle Jarvisova algoritmu testujeme pro každý bod konvexního obalu n bodů množiny v čase O(n). Celkem tedy potřebujeme čas O(k . n).

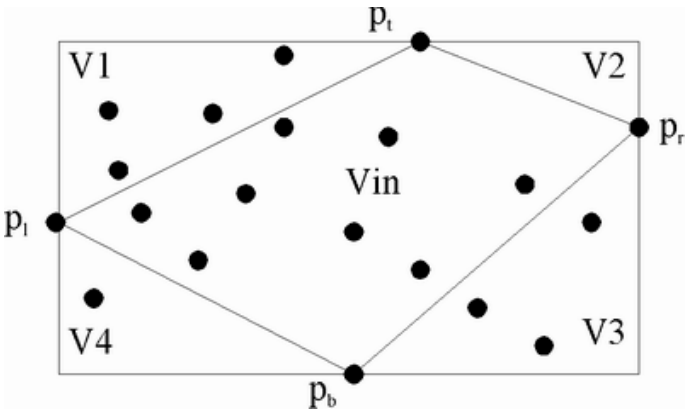
APPLET

applet 9.2

Pokud jsou body rozloženy v rovinné oblasti rovnoměrně, je vhodný např. algoritmus Overmars & van Leeuven.

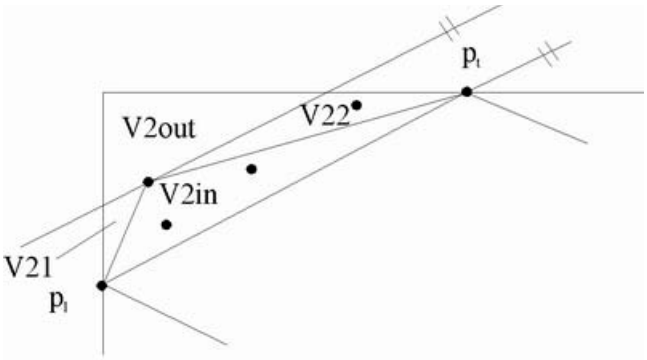
Overmars & van Leeuvenův algoritmus

Určíme nejlevější, nejpravější, nejvyšší a nejnižší bod množiny V s označením p_l, p_r, p_t, p_b podle obr. 9.4.



obr. 9.4: Oblasti vymezené obdélníkovým obalem

Tyto body zřejmě patří do konvexního obalu. Všechny body z V leží uvnitř obdélníku rovnoběžného s osami x,y. Úsečky spojující mezní body na obr. 9.4 dělí obdélníkovou oblast na 5 podoblastí, označených V1, V2, V3, V4 a Vin. Zřejmě vnitřní body Vin nepatří do konvexního obalu a mohou být z dalších testů vynechány. Výpočet pokračuje testováním bodů z oblastí V1, V2, V3, V4. Postup si ukážeme na trojúhelníkové oblasti V1.



obr. 9.5: Dělení rohové oblasti

Podle obr. 9.5 nalezneme bod s největší vzdáleností od přímky, která vymezila trojúhelníkovou oblast V1. Tímto bodem vedená rovnoběžná přímka vymezí další oblasti označené V21, V22, V2in, V2out. Body z V2in opět vyloučíme, neboť neleží na konvexním obalu, oblast V2out je prázdná a pokračujeme tedy rekurzivně v řešení trojúhelníků V21 a V22. Výpočet končí v kroku i, pokud jsou trojúhelníky Vi1 a Vi2 prázdné. V každém kroku algoritmu je nalezen nový vrchol konvexního obalu. Tento krok trvá v nejhorším případě čas O(n), takže celkově k bodů konvexního obalu nalezneme v čase O(k . n). Nejdelší čas odpovídá situaci, kdy po každém kroku leží všechny testované body v jediném ze zbývajících trojúhelníků. Střední očekávaná doba výpočtu je podstatně lepší. Při rovnoměrném rozmístění bodů v oblasti se v každém kroku algoritmu redukuje jejich počet průměrně o polovinu. Za tohoto předpokladu je pak očekávaný čas výpočtu O(n).

APPLET

applet 9.3

Konvexní obal množiny rovnoměrně rozložených bodů v rovině lze nalézt v průměrném čase O(n).

V počítačové grafice se častěji počítají konvexní obaly objektů. Významné je zvláště nalezení konvexního obalu nekonvexního polygonu. Lze snadno dokázat, že konvexní obal polygonu je roven konvexnímu obalu vrcholů polygonu. Lze tedy aplikovat všechny předchozí algoritmy a vypočítat konvexní obal polygonu v čase O(n . logn) . Existují však lepší řešení uvedené úlohy. Používají vhodného postupu po hranách polygonu. Postup je zahájen ve vrcholu, jež je součástí konvexního obalu. Pak lze konvexní obal nalézt v lineárním čase. Je to důsledek skutečnosti, že třídění bodů, které řešíme u bodových algoritmů, nahrazujeme již vyřešeným tříděním bodů podle jejich pořadí na obvodu polygonu. Tyto algoritmy však pracují správně pouze tehdy, pokud polygonální tah neobsahuje smyčky, tj. pokud se hrany polygonu neprotínají.

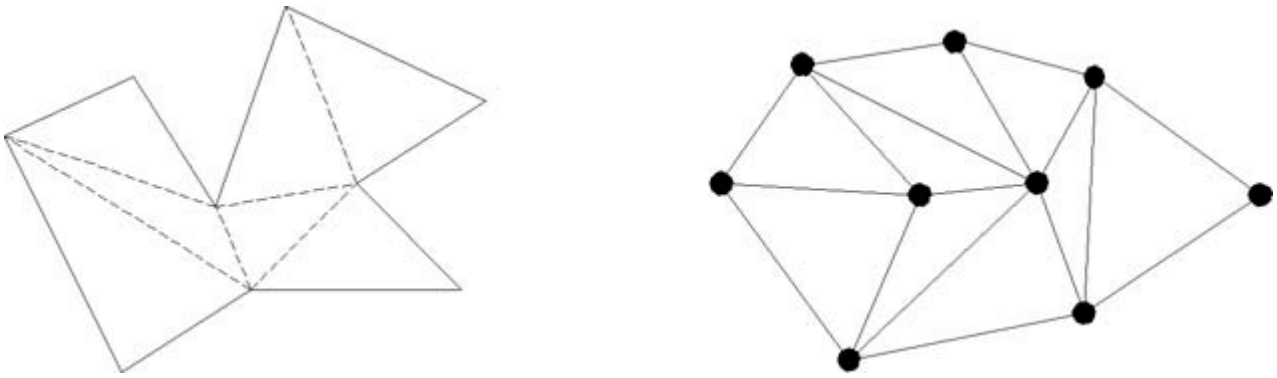
Konvexní obal jednoduchého polygonu s n vrcholy v rovině lze vypočítat v čase O(n).

APPLET

applet 9.4

9.1.2 Triangulace

Za druhý tvarový problém je považována úloha dekompozice objektu na tvarově jednodušší disjunktní části, většinou trojúhelníky. V případě polygonu chceme rozdělit polygonem vymezenou oblast na trojúhelníky, které jsou tvořeny pouze vrcholy původního polygonu. Při triangulaci množiny izolovaných bodů chceme dělit konvexní obal na trojúhelníky tak, že všechny body jsou použity jako vrcholy souvislé trojúhelníkové sítě. Příklady triangulace jsou na obr.9.6.

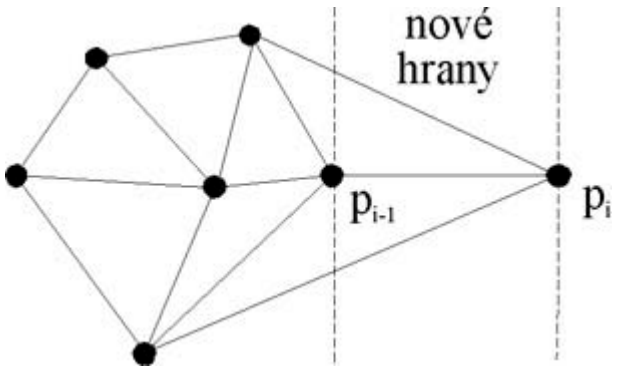


Obr. 9.6: Příklady triangulace

Triangulace není jednoznačná. Stejný polygon nebo stejná množina bodů mohou být rozloženy na trojúhelníkovou síť více způsoby. Triangulace hraje významnou roli v mnoha aplikacích. Po triangulaci polygonu lze např. řešit mnoho úloh podstatně jednodušším způsobem. Pomocí trojúhelníkové sítě lze snadno určit, zda bod leží uvnitř nebo vně polygonu. Před interpolací určí triangulace vrcholy jednotlivých plošek, hodnoty ve vrcholech jsou použity jako okrajové podmínky interpolace trojúhelníkové oblasti.

Triangulace množiny bodů

Pro triangulaci množiny bodů použijeme řádkový algoritmus. Přímka rovnoběžná se zvolenou osou souřadného systému se pohybuje ve směru druhé osy a dělí množinu bodů na dvě podmnožiny. U první podmnožiny je triangulace dokončená a je vyřešen konvexní obal zpracované podmnožiny bodů. Druhá podmnožina obsahuje ostatní dosud nezpracované body. Předpokládejme, že dělící přímka prochází bodem p_i . p_{i-1} je předchozí bod řešení, který leží zřejmě na dosud určeném konvexním obalu. Propojíme p_i, p_{i-1}. V obou směrech od bodu p_{i-1} až do nalezení tečen konvexního obalu spojujeme body konvexního obalu s bodem p_i (obr.9.7).



Obr. 9.7: Postupná triangulace s využitím konvexního obalu

Takto vyřešíme připojení bodu p_i k triangulační síti. Současně opravíme konvexní obal tak, že z něj vypustíme hrany, které spojují oba tečné vrcholy ve směru přes vrchol p_{i-1} a nahradíme je dvojicí nově přidaných hran. Pro třídění bodů v daném směru je potřeba čas $O(\ n\ .\ \log n\ )$. V každém kroku přidáváme nové hrany do planárního grafu, celkový počet těchto hran je přímo úměrný počtu uzlů (Přesněji: je-li $G = \langle H,U \rangle$ planární graf s hranami H a uzly U, pak $card(H) \leq 3\ .\ card(U)-6$). Triangulaci množiny setříděné v potřebném směru lze tedy vypočítat v čase $O(\ n\ )$.

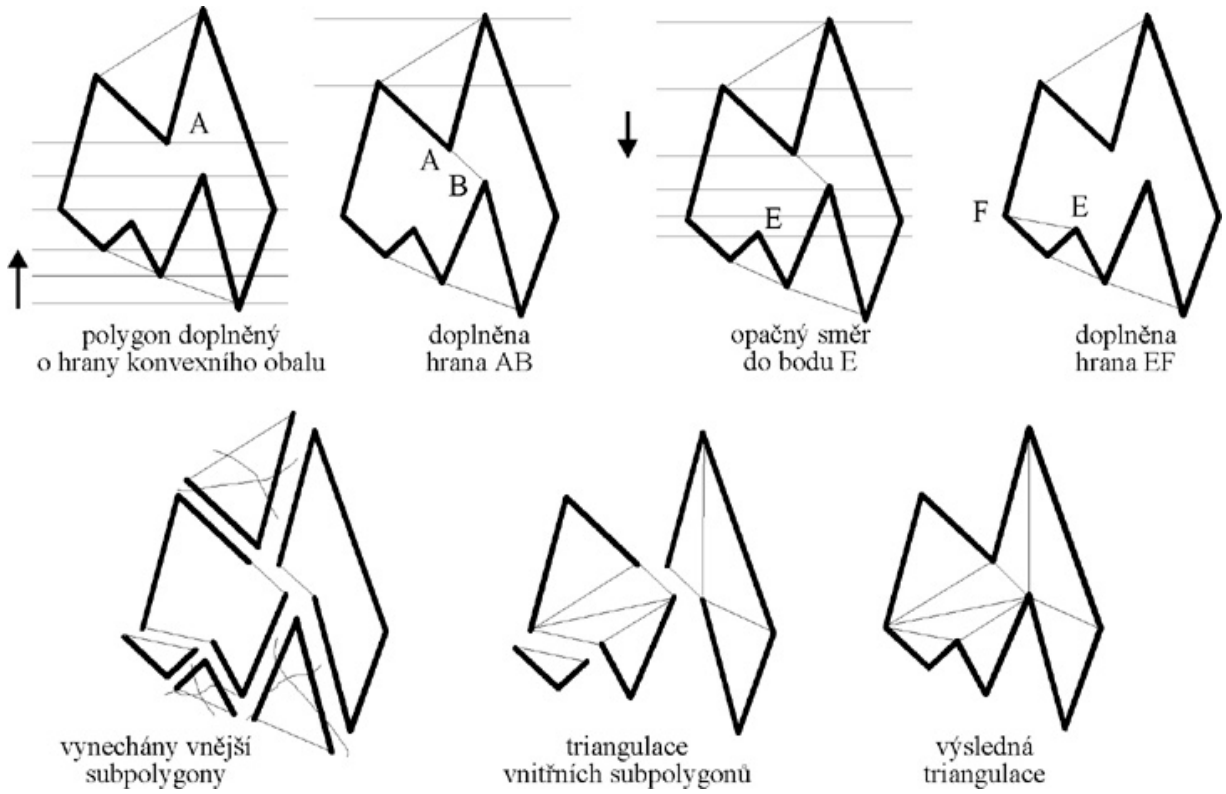
Věta 9.5 *Pro danou množinu n bodů v rovině lze triangulaci spočítat v čase O(n . logn). Pokud jsou body ve zvoleném směru setříděny, triangulace tímto směrem trvá pouze čas O(n).*

Tato metoda představuje také alternativní řešení konvexního obalu množiny bodů.

Triangulace polygonu

Triangulace polygonu je obtížnější. Jednou z možností, jak úlohu vyřešit, je použití obecnější metody, která vytvoří tzv. triangulaci s omezením. Jedná se o úlohu, kdy je dána množina neprotínajících se úseček v rovině. Cílem úlohy je vyřešení triangulace tak, aby vytvořená triangulační síť propojila koncové body úseček a použila zadané úsečky. Tuto obecnou metodu použijeme pro triangulaci polygonu. Metoda vytvoří u nekonvexních polygonů hrany, které leží vně polygonu, tyto lze však jednoduše detekovat a odstranit.

Polygon rozdělíme na jednoduché subpolygony, omezené dvěma vertikálně monotónními řetězy hran. Vertikálně monotónní řetězy mají nejvýše jeden průsečík s horizontální řádkou. U těchto monotónních polygonů snadno vyřešíme triangulaci postupným krokem po obou monotónních řetězech v čase $O(n)$. Pro vytvoření monotónních polygonů nejprve vypočteme konvexní obal všech vrcholů. Jeho hrany budou zároveň patřit do triangulační sítě obecné úlohy pro množinu úseček. Pak použijeme řádkovou metodu a hledáme ve směru zdola nahoru vrcholy, které jsou lokální minima (jejich hrany směřují nahoru). Nalezený bod propojíme hranou s nejbližším níže položeným vrcholem již prohlédnutého úseku. Po dosažení nejvyššího vrcholu obrátíme směr prohledávání, směrem dolů hledáme vrcholy, které jsou lokální maxima. Tyto spojujeme s nejbližšími výše položenými body (obr.9.8).



Obr. 9.8: Triangulace nekonvexního polygonu

Tento proces trvá čas $O(n \cdot \log n)$. Po jeho skončení dostaneme monotónní polygony.

Věta 9.6 *Pro polygon P s n hranami lze vypočítat triangulaci jeho vnitřku v čase O(n . logn). Jestliže P je monotónní, výpočet trvá O(n).*

Zatím časově nejlepší algoritmus publikovali R. E. Tarjan a C. J. Van Wijk v r. 1988. Dosahuje časové složitosti $O(n \cdot \log n)$. Otázka, zda existuje řešení v čase $O(n)$, dosud není zodpovězena.

Doba řešení uvedených algoritmů závisí na počtu zadaných bodů. Při návrhu a volbě algoritmů je vhodné respektovat i očekávanou složitost a tvar výstupu. Za nejefektivnější algoritmus hledání konvexního obalu je v současnosti považován algoritmus Kirkpatrick & Seidel, který pracuje v čase $O(n \cdot \log k)$, kde k je počet bodů na konvexním obalu.

APPLET

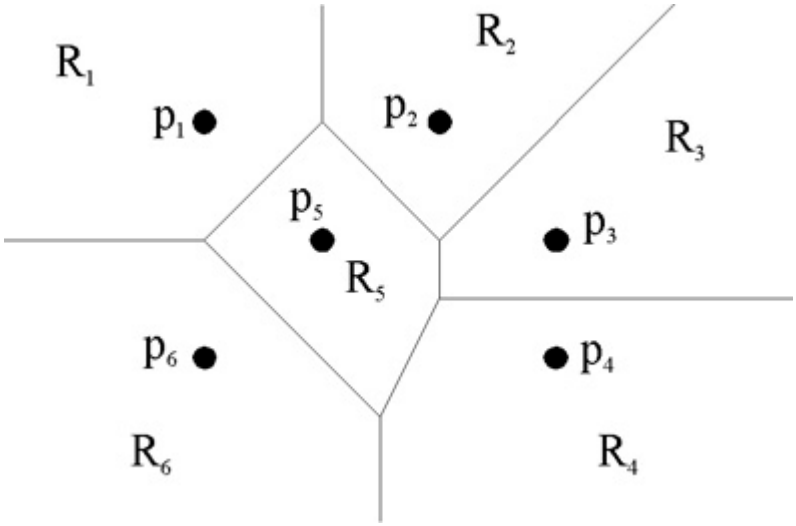
applet 9.5

9.2 Blízkost

Množství problémů v počítačové geometrii souvisí s hodnocením vzdálenosti nebo blízkosti objektů. Často např. hledáme dva nejbližší body v množině. Zajímáme se o způsob uložení množiny bodů, objektů tak, že pro daný dotazovací bod nalezneme efektivně nejbližší bod z množiny. Tato úloha hraje významnou roli např. v robotice při plánování nejkratší cesty, při výběru grafických objektů kurzorem nebo i nalezení určitého objektu v databázi.

9.2.1 Voronoiův diagram

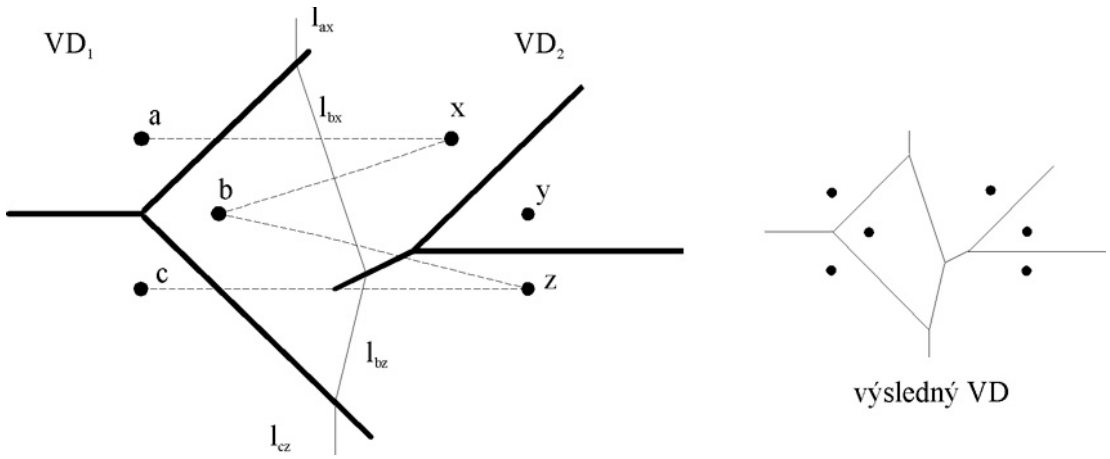
Věta 9.7 *Nechť V je množina n bodů v rovině. Rovinu rozdělíme na n oblastí $R_1, R_2, \ldots R_n$ takových, že R_i obsahuje všechny body z E^2, pro něž je bod $p_i \in V$ nejbližší soused. Toto rozdělení roviny se nazývá Voronoiův diagram (VD) množiny bodů.* Příklad je na obr.9.9 .



Obr. 9.9: Voronoiův diagram

VD má následující vlastnosti. Všechny oblasti jsou konvexní a oblast R_i obsahuje jediný bod $p_i \in V$. Některé oblasti jsou neohraničené. Tyto obsahují body $p_i \in CH(V)$. Počet hran a uzlů VD je přímo úměrný počtu bodů v množině V. Pokud žádné 4 body neleží na kružnici, uzly mají stupeň 3. Uzel VD leží ve středu kružnice určené 3 body z V, které leží v přilehlých oblastech VD a neleží na přímce.

M. I. Shamos a D. Hoey si jako první uvědomili, že VD je významným nástrojem v počítačové geometrii. Publikovali algoritmus pro výpočet (konstrukci) VD množiny n bodů v čase $O(n \cdot \log n)$. Je to algoritmus typu *rozděl a panuj*. Pro vysvětlení jeho činnosti použijeme příklad na obr.9.10 .



Obr. 9.10: Spojení Voronoiových diagramů

Algoritmus konstrukce Voronoiova diagramu

Množinu n bodů v rovině rozdělíme svislou přímkou na dvě stejně velké podmnožiny $V1$ a $V2$. Při dosažení malého počtu bodů zkonstruujeme VD1 a VD2 rozdělených množin, jinak dělení rekurzivně opakujeme. Po vyřešení dílčích VD spojujeme dvojice VD následujícím způsobem:

Nechť VD1 a VD2 jsou dva vypočítané Voronoiovy diagramy množin oddělených hraniční svislou přímkou. Pro jejich spojení využijeme skutečnosti, že hrany výsledného VD patří buď zcela do VD1 nebo VD2, nebo leží na osách úseček, spojujících body z V1 a V2 (tzv. bisektory). Tyto hrany tvoří řetěz, který je monotónní ve směru osy y.

Nalezneme konvexní obal sjednocených množin $V1$ a $V2$. Nový konvexní obal obsahuje úsečky tečné k původním konvexním obalům. Sestrojíme bisektory těchto tečných úseček.

Nechť l je bisektor horní úsečky, určené vrcholy konvexního obalu (body a,x na obr.9.10. Přímka l leží současně v oblastech $R_i \subset VD1$ a $R_j \subset VD2$. Určíme místo, ve kterém l opustí buď oblast R_i nebo R_j a vstoupí do oblasti R_k. Oblast R_k patří buď k VD1, nebo k VD2 a obsahuje bod p_k. Na hranici ukončíme hranu l a vytvoříme novou hranu, která leží na bisektoru bodů p_k a zbývajícího bodu p_i nebo p_j (pokud opustíme oblast R_j nebo R_i). Na obr.9.10 je v druhém kroku zkonstruován bisektor $l_b x$ mezi body b,x.

Směrem dolů vytváříme další hrany VD a algoritmus ukončíme, pokud dosáhneme spodní polopřímkové hrany.

Tato konstrukce dokazuje následující tvrzení.

Věta 9.8 *Voronoiův diagram množiny n bodů v rovině lze vypočítat v čase O(n . logn).*

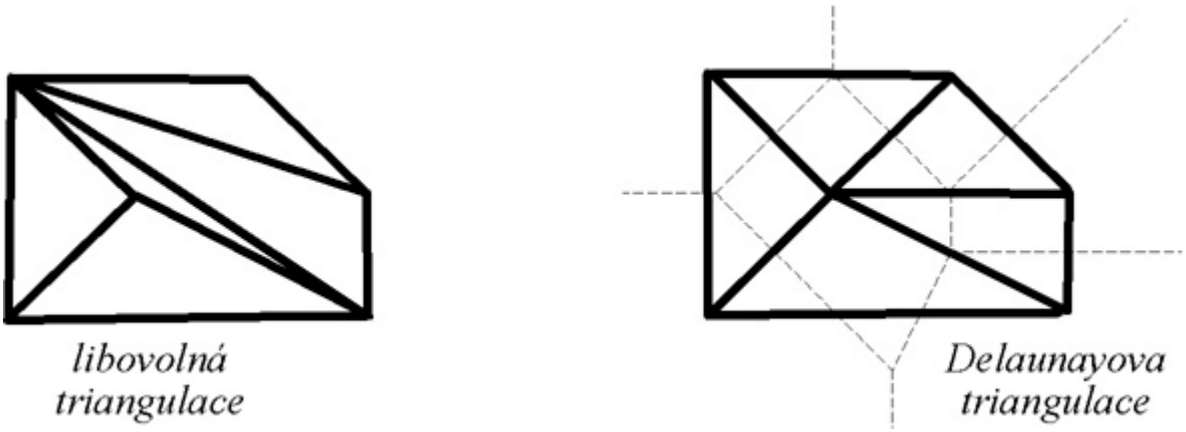
Jako příklad aplikace uvedeme řešení problému *nalezení nejbližších sousedů*. Chceme nalézt dvojici nejbližších bodů v množině V izolovaných bodů. Je zřejmé, že tato dvojice musí sdílet hranu VD(V). Stačí tedy projít všechny hrany VD a nalézt tu, jež má nejmenší vzdálenost k bodům množiny umístěných v sousedních oblastech. Tyto body mají minimální vzdálenost. Stejně tak můžeme nalézt ke zvolenému bodu v rovině nejbližší bod z množiny V.

Nechť V je množina n bodů v rovině. Ke každému bodu z V lze nalézt nejbližší bod z V v čase $O(\, n \,.\, \log n \,)$.

Delaunayova triangulace

Nechť V je množina n bodů. Předpokládáme, že žádné 4 body z V neleží na kružnici. Pokud propojíme všechny sousední body určené Voronoiovým diagramem úsečkami, dostaneme triangulační síť, kterou studoval Delaunay v roce 1935. Je zřejmé, že Delaunayovu triangulaci lze spočítat v čase $O(\, n \,.\, \log n \,)$, tedy v čase odpovídajícím konstrukci Voronoiova diagramu. Tato triangulace je výhodná např. pro interpolaci, neboť minimalizuje délky hran trojúhelníků a zpřesňuje tak výpočty přírůstků.

Na obr.9.11 jsou ukázány dva možné způsoby triangulace 6 bodů, ze kterých může pozorný čtenář odhadnout výhodnost Delaunayovy triangulace.



Obr. 9.11: Dvě řešení triangulace dané množiny bodů

11. Stínování a viditelnost

Prostorové objekty je vhodné zobrazovat na rovinné ploše obrazovky tak, aby nezanikal dojem hloubky, tedy třetího rozměru. Toho docílíme využitím několika postupů:

perspektivní promítání dodá dojem prostoru i při jednoduché čárové kresbě, například hran objektů,

zobrazení odraženého světla, známé pod pojmem **stínování**, vytváří plastický vjem povrchu těles, a to i v případě, že povrch je tvořen rovinnými ploškami,

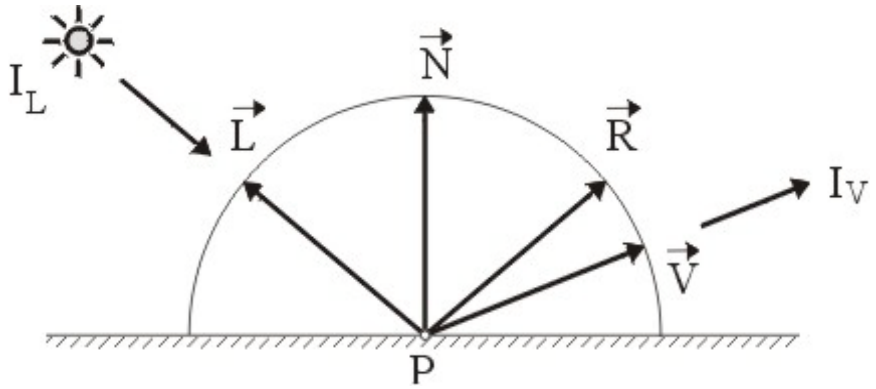
odstranění zakrytých částí objektů je přirozenou součásti algoritmů pro zobrazení prostorových scén.

Uvedené postupy se používají současně. V následujících částech se zaměříme na některé z nich, zejména na metody zpracování odraženého světla a stínování a zmíníme se o vybraných algoritmech pro řešení viditelnosti.

11.1 Odraz světla od povrchu tělesa

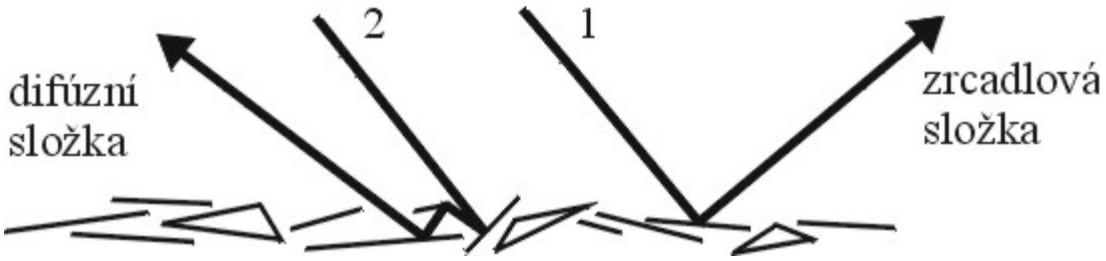
Hovoříme-li o vzhledu povrchu, obvykle používáme slov jako červený, zelený, lesklý, matný, drsný, kovového vzhledu apod. Tyto pojmy se vesměs vztahují ke způsobu, jakým povrch odráží dopadající světlo - neboli k jeho optickým vlastnostem.

Dopadne-li světelný paprsek do bodu na povrchu tělesa, po odrazu se rozptýlí obecně do všech směrů. Matematická funkce vyjadřující intenzitu paprsku rozptýleného světla v závislosti na jeho směru a na směru, intenzitě a vlnové délce dopadajícího paprsku se nazývá **osvětlovací model**. Pomocí něho můžeme postihnout vlastnosti jako barva, lesklost, apod.



obr. 11.1: Odraz paprsku od povrchu tělesa

Uvažujme situaci na obr. 11.1. Světelný paprsek ze zdroje dopadá do bodu *P* a po odrazu je část rozptýleného světla zachycena pozorovatelem. Jednotkové vektory ***L***, ***V*** a ***N*** leží na přímkách dopadajícího paprsku, směr pohledu a normály povrchu. Intenzitu dopadajícího světla označme I_L (L - light), intenzitu odraženého světla I_V (V - viewer).



obr. 11.2: Odraz na mikroskopické úrovni

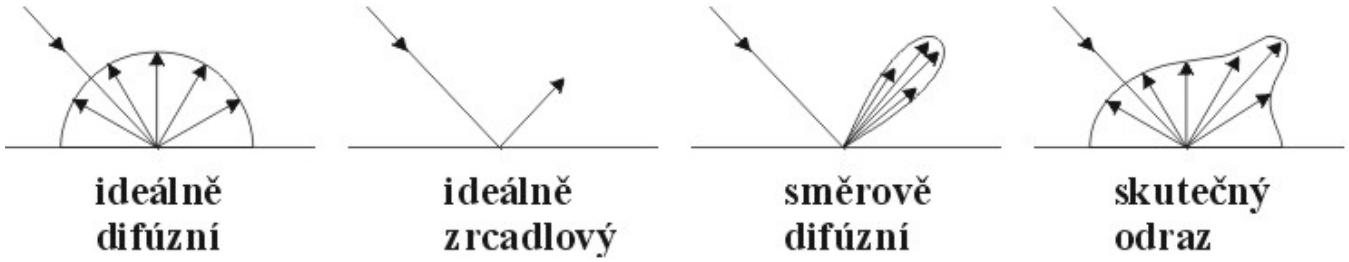
Podívejme se, co se děje s paprskem dopadajícím do bodu na povrchu. Povrch tělesa nikdy není dokonale hladký a při velkém zvětšení vypadá podobně jako na obrázku 11.2 - je tvořen drobnými ploškami a krystaly. Paprsek může po dopadu opustit povrch dvěma způsoby. Buď se zrcadlově odrazí

od některé z plošek (na obrázku označen 1), nebo je podroben vícenásobnému odrazu a lomu (označen 2). Intenzitu odraženého světla proto vyjádříme jako součet dvou složek:

$$I_V = I_s + I_d. \qquad (11.1)$$

Složka I_s se nazývá **zrcadlová** (s - specular) a odpovídá paprsku označenému 1. Její charakteristickou vlastností je směrovost. Směrovost je tím vyšší, čím jsou plošky na povrchu méně rozházené, neboli čím je povrch hladší. Světlo odražené od dokonalého zrcadla má pouze tuto složku. Zrcadlová složka je příčinou vzniku odlesků na zobrazovaném tělese, přičemž tyto odlesky mohou mít jinou barvu, než má samotný povrch tělesa.

Složka I_d se nazývá **difúzní** (d - diffuse) a odpovídá paprsku 2. Pro ní je charakteristické naopak to, že nezávisí na směru pohledu. Po vícenásobném odrazu a lomu bude totiž směr paprsku náhodný se stejnou pravděpodobností všech směrů. Intenzita této složky bude tedy záviset pouze na úhlu dopadu - bude přímo úměrná jeho kosinu.



obr. 11.3: Difúzní a zrcadlové složky odraženého světla

Na obr. 11.3 vidíme, že skutečné povrchy odrážejí jak difúzně, tak zrcadlově, přičemž zastoupení jednotlivých složek je odlišné u různých druhů materiálů.

11.1.1 Phongův osvětlovací model

Nejčastěji používané vztahy pro intenzitu odraženého světla navrhl již v roce 1977 Bui-Tuong Phong[1] ([Phon75]), původně pouze pro monochromatické světlo. Pro zrcadlovou složku použil vztah

$$\mathbf{I_d} = \mathbf{I_L}\, \mathbf{r_d} \left(\overset{(\rho)}{V} \cdot \overset{(\rho)}{R} \right)^h, \qquad (11.2)$$

kde $\mathbf{I_L}$ je intenzita dopadajícího paprsku, $\boldsymbol{V}$ je jednotkový vektor pohledu (obr. 11.1). Vektor $\boldsymbol{R}$ je symetrický k vektoru $\boldsymbol{L}$ podle normály - lze jej vypočítat ze vztahu

$$\overset{(\rho)}{R} = 2\left(\overset{(\rho)}{L} \cdot \overset{(\rho)}{N} \right) \overset{(\rho)}{N} - \overset{(\rho)}{L}$$

Připomeňme, že skalární součin jednotkových vektorů je roven cosinu úhlu jimi svíraného.

Koeficient zrcadlového odrazu $\mathbf{r_s}$ určuje zastoupení zrcadlové složky v celkové odražené intenzitě. Pracujeme-li nikoliv pouze se skalární intenzitou, ale s barvou se složkami R, G a B, chápeme jak intenzitu světla $\mathbf{I_L}$, tak koeficient $\mathbf{r_s}$ jako třísložkové barevné vektory a příslušné rovnice vypočítáváme třikrát pro jednotlivé složky R, G a B. V rovnicích osvětlovacího modelu se tak objevují jak **vektory geometrické** (směry paprsků), tak **barevné** (složení barev paprsků).

Koeficient h je skalární a znamená ostrost zrcadlového odrazu (v intervalu $\langle 1, \infty \rangle$). Pokud je $\boldsymbol{V}.\boldsymbol{R} < 0$ (pozorovatel je vzhledem k zrcadlu ve stejné části prostoru jako zdroj světla, takže nemůže vidět odraz), pokládáme $\mathbf{I_s} = 0$ (nulovému barevnému vektoru s významem černé barvy). Důležité je, že intenzita nabývá maxima tehdy, když směr dopadu a směr pohledu splňují zákon odrazu. Toto maximum je tím výraznější, čím je parametr h větší. S rostoucím h se odlesky na zobrazovaném tělese stávají menší a ostřejší, dokonalé zrcadlo má $h = \infty$.

Vztah pro difúzní složku má tvar

$$\mathbf{I_d} = \mathbf{I_L}\, \mathbf{r_d} \left(\overset{(\rho)}{L} \cdot \overset{(\rho)}{N} \right), \qquad (11.3)$$

kde $\mathbf{r_d}$ je koeficient difúzního odrazu, udává zastoupení difúzní složky a do značné míry vyjadřuje to, co vnímáme jako barvu tělesa. Intenzita $\mathbf{I_d}$ je tím větší, čím je směr dopadu bližší normále. Vztah má smysl pouze pro $\boldsymbol{V.N} > 0$, neboť v opačném případě je povrch odvrácen od světla a $\mathbf{I_d} = 0$.

Odražená intenzita se z praktických důvodů obohacuje o třetí složku - vzniká tedy aditivním složením tří intenzit:

$$\mathbf{I_V} = \mathbf{I_s} + \mathbf{I_d} + \mathbf{I_a}. \qquad (11.4)$$

$\mathbf{I_a}$ je složka příslušná **okolnímu světlu** (a - ambient). Předpokládá se totiž, že do bodu *P* kromě paprsku přímo od zdroje dopadá ještě tzv. okolní světlo vzniklé mnohonásobnými odrazy a rozptyly od ostatních těles. Toto světlo je příčinou složky $\mathbf{I_a}$. Je

$$\mathbf{I_a} = \mathbf{I_A}\, \mathbf{r_a}, \qquad (11.5)$$

kde $\mathbf{I_A}$ je intenzita okolního světla stejná pro **celou scénu**, $\mathbf{r_a}$ je koeficient odrazu okolního světla. Z hlediska barevných poměrů bývá barevný vektor $\mathbf{r_a}$ totožný s koeficientem $\mathbf{r_d}$, jeho absolutní hodnota určuje, zda těleso je tmavé či světlé. Složka $\mathbf{I_a}$ zabraňuje tomu, aby povrchy odvrácené od světelných zdrojů byly zcela černé. I_A volíme většinou zkusmo, velikost je nepřímo úměrná počtu světelných zdrojů ve scéně.

Složka $\mathbf{I_a}$ se pro bod povrchu přičte pouze jednou - na rozdíl od $\mathbf{I_s}$, $\mathbf{I_d}$, které se přičtou pro každý světelný zdroj. Dopadají-li tedy do bodu *P* paprsky z *M* světelných zdrojů, bude platit

$$\mathbf{I_V} = \mathbf{I_A}\, \mathbf{r_a} + \sum_{k=1}^{M} \mathbf{I_{Lk}} \left[\mathbf{r_s} \left(\overset{(\rho)}{V} \cdot \overset{(\rho)}{R_k} \right)^h + \mathbf{r_d} \left(\overset{(\rho)}{L}_k \cdot \overset{(\rho)}{N} \right) \right]. \qquad (11.6)$$

Je třeba zdůraznit, že Phongův osvětlovací model je pouze **empirický** a nemá přímý vztah k fyzikální podstatě šíření a odrážení světla. Přesto je díky své jednoduchosti široce používán v počítačové grafice. Z něj jsou odvozovány i další osvětlovací modely, které se více blíží skutečnému popisu chování odraženého světla. Jednou z možných cest je nahrazení konstantních materiálových koeficientů $\mathbf{r_s}$ a $\mathbf{r_d}$ funkcemi úhlu dopadu a zavedení závislosti na vlnové délce (barevného složení) odráženého světla.

Detailnější výklad o fyzikálně založených osvětlovacích modelech lze nalézt v [Torr67] a [Cook82].

Všimněme si také, že předpokladem pro většinu osvětlovacích modelů je existence pouze **bodových zdrojů světla**. Ty je sice možno snadno rozšířit na směrové zdroje světla (např. světlo z reflektoru), ale reprezentovat skutečné, tedy **plošné zdroje** je velmi obtížné. Nahrazení plošného zdroje (zářivky či okna) skupinou bodových zdrojů zpomaluje výpočty výrazným způsobem. Se zpracováním plošných světel se setkáme až u dalších zobrazovacích metod v kapitole .

11.2 Stínování

Pod pojmem stínování (angl. *shading*) rozumíme vykreslování barevných objektů různými odstíny barev. Pomocí stínování lze odlišit případné křivosti a zaoblení ploch, a docílit tak přirozeného vzhledu prostorových objektů. Ačkoliv je stínování spjato se zpracováním světla, nezabývá se nalezením a kresbou vržených stínů (angl. *shadow*). Ty lze zpracovat speciálními metodami (viz kap. 12).

Z předchozího textu již víme, jakým způsobem lze vypočítat barvu bodu na povrchu plochy na základě znalostí směru promítacího paprsku (poloze pozorovatele), normály plochy a umístění zdrojů

světla a dalších faktorů. Vyhodnocování světelného modelu v každém bodě, který je vykreslován na obrazovce, je zdlouhavé a proto bylo pro rastrové displeje vyvinuto několik metod, které umožňují provést podrobný výpočet světelného modelu pouze pro několik bodů na povrchu tělesa a odvodit z nich barevné odstíny ostatních zobrazovaných bodů.

11.2.1 Konstantní stínování

Tato nejjednodušší metoda je velmi rychlá. Je určena pro zobrazování rovinných ploch nebo obecných ploch aproximovaných rovinnými záplatami. Předpokládá, že každá plocha má jen jedinou normálu. Podle této normály je vypočítán jeden barevný odstín, který je přiřazen všem pixelům plochy. Do světelného modelu není zahrnuto zrcadlové osvětlení.

Pro kresby mnohostěnů je tento způsob stínování postačující a úspěšně znázorňuje umístění a natočení těles v prostoru. U obecnějších těles je konstantní odstín plošek negativním jevem, protože místo zkvalitnění obrázku zdůrazňuje, že oblý povrch je ve skutečnosti jen aproximován skupinou plošek (obr. 11.6 vlevo).

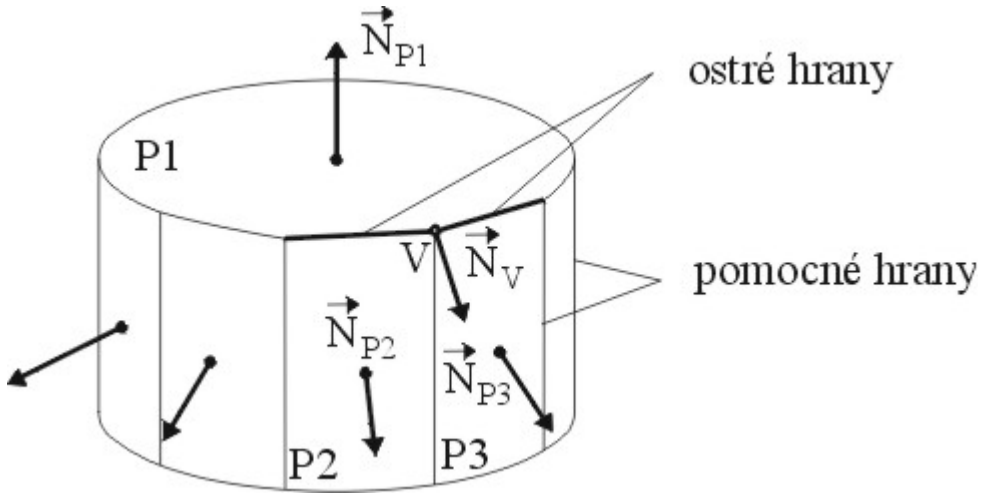
APPLET

applet 11.1

Pro obecná tělesa byly proto vyvíjeny další metody pro určení spojitého barevného stínování. Tyto metody můžeme nalézt pod názvem **inkrementální stínovací metody**. Jsou založeny na určování odstínu barvy bodu v ploše ze znalosti hodnot ve vrcholech této plochy. Inkrementální stínovací metody dokáží vyplnit plochu po řádcích a na každém řádku řešit lineární interpolaci odstínu barvy nebo normály povrchu, ze které je barva vypočítána.

11.2.2 Interpolace barvy

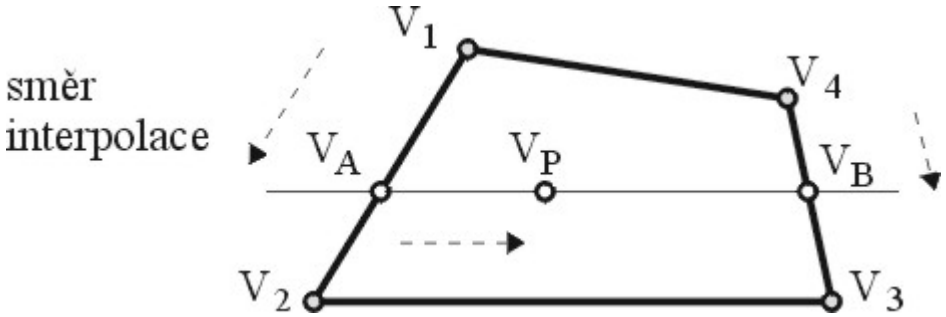
Metoda byla navržena pro tělesa, jejichž povrch je tvořen množinou rovinných ploch. V literatuře se často setkáme s názvem **Gouraudovo stínování**, protože poprvé byla metoda uvedena H. Gouraudem[2] v [Gour71].



obr. 11.4: Ostré a pomocné hrany tělesa určují plochy podílející se na výpočtu normály ve vrcholu

Pro činnost algoritmu je důležitá znalost barevného odstínu všech vrcholů zpracovávané plochy, tedy především normál, ze kterých se ve vrcholech stanovuje barva podle daného osvětlovacího modelu. Někdy jsou údaje o normálách ve vrcholech součástí geometrického popisu (reprezentace těles), někdy je třeba je stanovit pomocí vektorového součtu (s následným normováním) z normál ploch incidujících s daným vrcholem. Je třeba dbát na správný výběr ploch. Na obrázku 11.4 je naznačen případ, kdy vrchol *V* sousedí se skutečnou hraniční plochou tělesa (horní podstavou válce) a se dvěma záplatami válcového

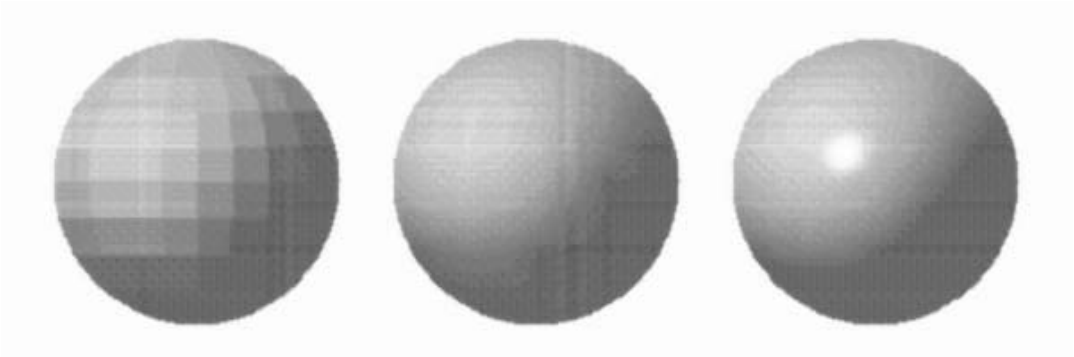
pláště. Při stínování plochy $P2$ musíme normálu ve vrcholu V určit pouze ze součtu normál ploch $P2$ a $P3$. Kdybychom do výpočtu zahrnuli i normálu podstavy $P1$, vznikl by optický dojem, že válec má zaoblenou podstavu. K rozhodnutí, které normály ploch zařadíme do výpočtu, nám pomůže znalost typů hran obsažených v modelu tělesa. Správná hraniční reprezentace by měla u každé hrany udržovat informaci o tom, zda je hrana **ostrá** (tedy skutečná) nebo pouze **pomocná** (tedy spojující dvě záplaty). Při výpočtu normály pomocí součtu vektorů budeme zpracovávat jen ty plochy, které jsou spolu spojeny **pomocnými** hranami.



obr. 11.5: Interpolace barvy v bodě V_P

Po určení barevných odstínů ve vrcholech ploch, přikročíme k vyplnění plochy odstíny, získanými bilineární interpolací (viz též obr. 7.7). Pro každý rozkladový řádek rovnoběžný s osou y stanovíme barvy v koncových bodech úseku (na obr. 11.5 v bodech V_A a V_B) interpolací z barev vrcholů plochy. Potom celý vodorovný úsek vybarvíme odstíny získanými lineární interpolací barev mezi krajními body V_A a V_B.

Při implementaci lineární interpolace podle uvedených vztahů se používá modifikovaný algoritmus DDA. Pokud intenzity nabývají celočíselných hodnot například v rozsahu 0-255, lze použít celočíselný Bresenhamův algoritmus. Metodu interpolace barvy tak lze snadno implementovat pomocí technických prostředků. V současné době je Gouraudovo stínování standardní metodou používanou v grafických akcelerátorech.



obr. 11.6: Konstantní, Gouraudovo a Phongovo stínování téhož modelu definovaného pomocí rovinných plošek

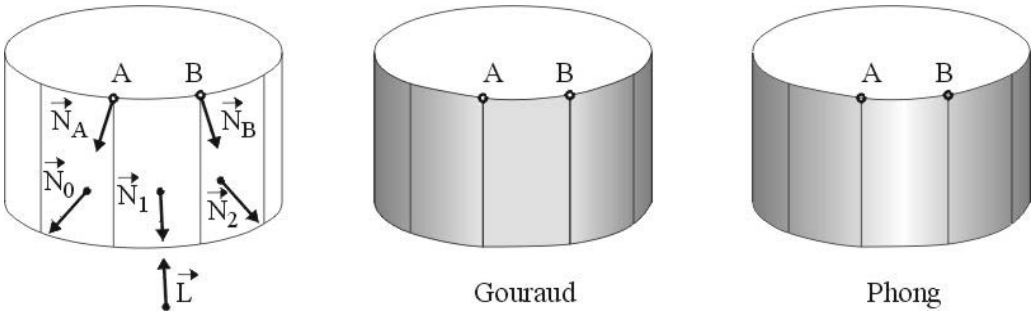
Gouraudova metoda zajišťuje plynulé stínování křivých povrchů tak, že aproximace povrchů ploškami není zřetelná. Přesto ani tyto obrázky ještě neposkytují zcela věrný obraz reálných objektů - interpolace samotného odstínu barvy totiž nemůže způsobit místní zvýšení jasu na plošce, která nahrazuje oblinu kolmou na dopadající světelný paprsek, stejně jako nemůže kvalitně vytvořit odlesky způsobené odraženým světlem. Lze říci, že tato metoda zahlazuje barevné rozdíly u místních nerovností povrchu.

11.2.3 Interpolace normály

Také tato metoda je určena ke spojitému stínování těles, jejichž povrch je tvořen množinou rovinných ploch. V literatuře je nazývána **Phongovým stínováním** podle autora Bui-Tuong Phonga ([Phon75]). Jeho jméno se tak objevuje v počítačové literatuře ve dvou souvislostech - jako jedna z variant empirického světelného modelu a jako metoda plynulého stínování povrchů těles.

Stejně jako u interpolace barvy jsou nejprve určeny normálové vektory ve vrcholech stínované plochy. Nejsou z nich však vypočteny barevné odstíny ve vrcholech, nýbrž jsou použity k interpolaci normálových vektorů ve vnitřních bodech plochy. Současně s rozkladem plochy na pixely jsou vypočítávány normály ve vnitřních bodech plochy a pomocí nich je ze světelného modelu určen odstín barvy každého pixelu.

Výpočet lze opět implementovat některým z inkrementálních interpolačních algoritmů, nyní však v prostoru normálových vektorů. Časové nároky algoritmu ve srovnání s Gouraudovým stínováním vzrostou, neboť v každém bodě plochy je počítáno osvětlení, přičemž světelný model obsahuje jak difúzní, tak zrcadlovou složku. Vzorce použité pro vyhodnocení světelného modelu byly uvedeny v předchozí kapitole.



obr. 11.7: Porovnání Gouraudova a Phongova stínování

Na obr. 11.7 je ukázáno, jak bilineární interpolace normály i bez použití zrcadlového odrazu zvýší jas části aproximovaného válce v oblasti kolmé na světelné paprsky, zatímco pouhou interpolací barvy ke zvýšení jasu z principu nemůže dojít. Ačkoliv na obrázku normály v bodech *A* a *B* mají odlišné směry

$$N_A^{(\rho)} = N_0^{(\rho)} + N_1^{(\rho)}$$

$$N_B^{(\rho)} = N_1^{(\rho)} + N_2^{(\rho)}$$

u Gouraudova stínování jsou z nich odvozeny tytéž intenzity osvětlení, protože úhly svírané normálami $\boldsymbol{N}_A$, $\boldsymbol{N}_B$ s vektorem $\boldsymbol{L}$ jsou totožné. Celá oblast mezi vrcholy *A* a *B* je pak vyplněna barvou téže intenzity. U Phongova stínování je patrné postupné naklánění normál a s ním i vzrůst velikosti jasu uprostřed oblasti mezi vrcholy *A* a *B*.

APPLET

applet 11.2

11.3 Problém viditelnosti

Při projekci prostorových objektů na obrazovku je třeba určit, které grafické prvky (hrany či plochy těles) budou vykresleny celé a které jen zčásti, protože jsou zakryty jinými prostorovými objekty. Z historického hlediska byly nejprve vyvíjeny algoritmy, které zpracovávaly údaje o tělesech popsaných hraniční reprezentací, ať již šlo pouze o hrany[3] nebo o plochy[4] S nástupem barevných obrazovek došlo k rychlému rozvoji rastrových algoritmů, které řešily viditelnost těles popsaných rozličnými způsoby. Přesto nelze říci, že by původní klasické metody byly zcela opuštěny, protože na rozdíl od rastrových algoritmů jsou jejich výsledky zobrazitelné i na plotrech a tiskárnách s vysokým rozlišením.

Metody určování viditelnosti tak podle výsledného obrázku dělíme na:

liniové (čárové), jejichž výstupem je soubor úseček, které lze libovolně zvětšovat

rastrové, jejichž výstupem je obrázek v rastru, který má pevně dané rozměry odvozené nejčastěji z rozměrů obrazovky

Liniové algoritmy jsou nenahraditelné v technické praxi, kdy různé pohledy na tělesa jsou s jejich pomocí kresleny na plotrech ve vysoké přesnosti. Často jsou vykreslovány i zakryté části, např. jinou barvou nebo jiným typem čáry.

Rastrové algoritmy naopak umožňují práci s barvou a jejími odstíny, dokáží vybarvit viditelné plochy, a poskytnout tak přesvědčivý obrázek prostorové scény.

Algoritmy pro řešení viditelnosti můžeme rozdělit i podle jiného kritéria. Důležitým faktorem při zpracování viditelnosti je čas. Ten podle použité metody může záviset buď pouze na počtu těles ve scéně, nebo i na velikosti zobrazovací plochy. Z tohoto pohledu dělíme algoritmy na

1. **Objektově orientované**, jejichž složitost je úměrná n^2, kde n je počet objektů ve scéně. Základem algoritmů je porovnávání vzájemných poloh těles, resp. jejich částí, jako jsou plochy a hrany.
2. **Obrazově orientované**, jejichž složitost je úměrná součinu $n.p$, kde n je počet objektů a p je počet pixelů rastru. Základem metod jsou testy určující pro každý pixel obrazovky, která část promítnutého objektu je v tomto pixelu nejblíže pozorovateli, a tedy viditelná.

Toto dělení není úplně jednoznačné, protože některé obrazově orientované algoritmy využívají pro zrychlení také postupy typické pro objektově orientované algoritmy. My se zde zaměříme na nejčastěji používané rastrové algoritmy pro určování viditelnosti. Jejich charakteristickým rysem je to, že výpočty stínování jsou prováděny současně s řešením viditelnosti.

Při výkladu následujících algoritmů budeme vycházet z těchto předpokladů:

- vstupem jsou objekty popsané plošnou hraniční reprezentací, resp. množina ploch,
- každá plocha nese informaci o své barvě, případně o normálách ve vrcholech,
- plochy byly před vstupem do algoritmu transformovány v souladu se zvoleným směrem pohledu tak, aby průmětna ležela v rovině obrazovky.

11.3.1 Paměť hloubky

Tato metoda patří k nejznámějším a také nejrychlejším. Klade sice vysoké nároky na paměť, ale současně dosahuje nejvyšší rychlosti zpracování.

Základním prostředkem pro určení viditelnosti je **paměť hloubky**, známá také pod anglickým názvem **Z-buffer** nebo **Depth-buffer**. Je to paměťová oblast reprezentující dvourozměrné pole, jehož rozměry jsou totožné s rozměry obrazovky, resp. s rozměry obrazové paměti. Položka paměti hloubky však neobsahuje barvu pixelu, nýbrž souřadnici z toho bodu, který leží nejblíže k pozorovateli a jehož průmět leží v odpovídajícím pixelu v obrazové paměti.

Předpokládejme, že pozorovatel se nachází nad rovinou xy a objekty ve scéně leží pod ním. Určení menší vzdálenosti dvou bodů od pozorovatele se v takovém případě převede na nalezení větší souřadnice z těchto bodů. Postup je uveden v algoritmu .

1. Vyplň obrazovou paměť barvou pozadí
2. Vyplň paměť hloubky (depth_buffer) hodnotou -…
3. Každou plochu rozlož na pixely a pro každý její pixel o souřadnicích $[x_i, y_i]$ stanov hloubku z_i
4. Je-li z_i > depth_buffer $[x_i][y_i]$, pak
    a. obarvi pixel $[x_i, y_i]$ v obrazové paměti barvou této plochy
    b. depth_buffer $[x_i][y_i] = z_i$

Algoritmus 11.1: Řešení viditelnosti v paměti hloubky

Rozklad ploch na pixely je prováděn obdobným způsobem jako při vyplňování ploch barvou. Oproti vyplňování ve dvourozměrném prostoru jsou zde zpracovány všechny 3 souřadnice. Rozklad lze provádět v celočíselné aritmetice a pouze souřadnice *z* vypočítávat v reálných číslech.

Použití paměti hloubky je výhodné v tom, že každá plocha je zpracovávána pouze jednou. Doba zpracování tedy roste s počtem ploch lineárně. Některé prameny uvádějí i menší složitost než lineární! V praxi se totiž ukazuje, že složitější scény sice obsahují více ploch, ale rozměry těchto ploch jsou obecně menší. Počet zpracovávaných pixelů při rozkladu ploch tak zůstává téměř konstantní, což je ojedinělá vlastnost tohoto algoritmu. Z tohoto důvodu je algoritmus prováděn technickými prostředky v grafických procesorech, kde umožňuje zobrazovat prostorové scény v reálném čase.

11.3.2 Řádkový rozklad

Tento algoritmus je kompromisem mezi velkými paměťovými nároky metody paměti hloubky a rychlým časem zpracování. Princip je obdobný metodě paměti hloubky s tím rozdílem, že pro určení pixelů nejbližších k pozorovateli není vymezena rozsáhlá obdélníková paměť hloubky, ale pouze jeden řádek (řádek hloubky, angl. *depth line*). Viditelnost je vyhodnocována vždy jen v jediném řádku. Po zpracování všech ploch z hlediska viditelnosti je řádek vykreslen a přechází se na další.

1. Vyplň daný řádek v obrazové paměti barvou pozadí
2. Vyplň řádek hloubky (depth_line) hodnotou -…
3. Každou plochu, která je protínána daným řádkem, rozlož na pixely a pro každý její pixel o souřadnici x_i stanov hloubku z_i
4. Je-li z_i > depth_line [x_i], pak
    a. obarvi pixel o souřadnici x_i na odpovídajícím řádku v obrazové paměti barvou této plochy
    b. depth_line [x_i] = z_i

Algoritmus 11.2: Řešení viditelnosti řádkovým rozkladem

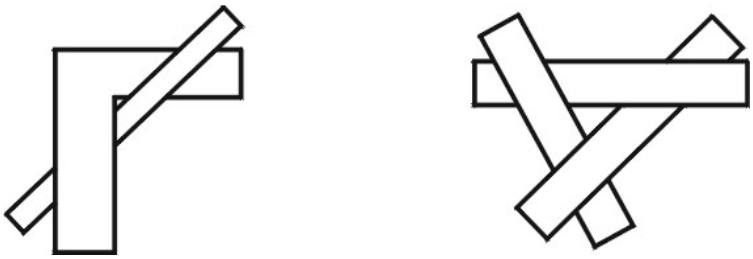
Algoritmus 11.2, do velké míry podobný algoritmu paměti hloubky, se tedy provádí postupně pro všechny řádky obrazovky.

Každá plocha je v tomto algoritmu zpracovávána několikrát, což snižuje jeho rychlost. Metodu optimalizujeme využitím **koherence** obdobně, jako při vyplňování rovinné oblasti barvou (viz kap. 3). Je výhodné plochy předtřídit podle nejvyšší souřadnice *y* a pamatovat si vzájemnou polohu ploch na zpracovávaném řádku s tím, že na následujícím zůstane ve většině případů nezměněna. Lze se také zaměřit pouze na místa, ve kterých na řádku dochází ke změně viditelnosti (překryv ploch) a mezilehlé úseky přímo vykreslovat bez jakýchkoliv dalších výpočtů.

11.3.3 Malířův algoritmus

Princip této poměrně rozšířené metody[5] spočívá v přímém vykreslování ploch na obrazovku, a to v pořadí od nejvzdálenějších k nejbližším vůči pozorovateli. Bližší plochy překryjí zadní, čímž je viditelnost vyřešena přirozeným způsobem. Název algoritmu připomíná malířské postupy, při nichž malíř nanáší na obraz nejprve barvy na pozadí a přes ně kreslí objekty v popředí.

Při realizaci této jednoduché myšlenky nejprve zjistíme mezní souřadnice *z* každé plochy a podle menší z nich plochy setřídíme. Naivní verze algoritmu takto setříděný seznam přímo vykreslí, důkladnější varianta však ještě musí provádět několik testů, zda plocha určená k vykreslení nezakrývá jinou plochu. Pokud ano, dojde k záměně ploch v seznamu. Takový případ nastává, obsahují-li data dlouhou plochu umístěnou zepředu dozadu přes celou scénu. Ačkoliv má taková plocha svojí nejmenší souřadnici *z* nejdále ze všech ostatních ploch (ve směru od pozorovatele), musí být správně vykreslena až mezi posledními.



obr. 11.8: Složitější vzájemná poloha ploch při malířově algoritmu

Ani záměna ploch v setříděném seznamu nevede vždy ke správnému řešení. Mohou nastat i složitější případy vzájemného překrývání více ploch (obr. 11.8). Tento stav můžeme snadno odhalit, protože vlivem cyklického překryvu se na místo testované plochy dostane v seznamu ploch taková plocha, která na tomto místě již jednou byla. Situaci řešíme rozdělením ploch na menší části, čímž se snižuje efektivita malířova algoritmu.

Mnoho implementací malířova algoritmu se omezuje na provádění jen těch nejjednodušších testů, což vede k téměř lineární závislosti času na počtu ploch. Časově nejnáročnějším zůstává samotné vykreslení (stínování) ploch. Je-li scéna tvořena několika stovkami, či tisíci plošek, lze připustit i určité chyby za cenu vysoké rychlosti řešení viditelnosti celé scény.

- [1] čti **[fong]**
- [2] čti **[guró, guródem]**
- [3] **HLE** - *Hidden Line Elimination*
- [4] **HSE** - *Hidden Surface Elimination*
- [5] v anglické literatuře se vedle názvu *Painter's algorithm* objevuje i pojem *Priority list*

Literatura:

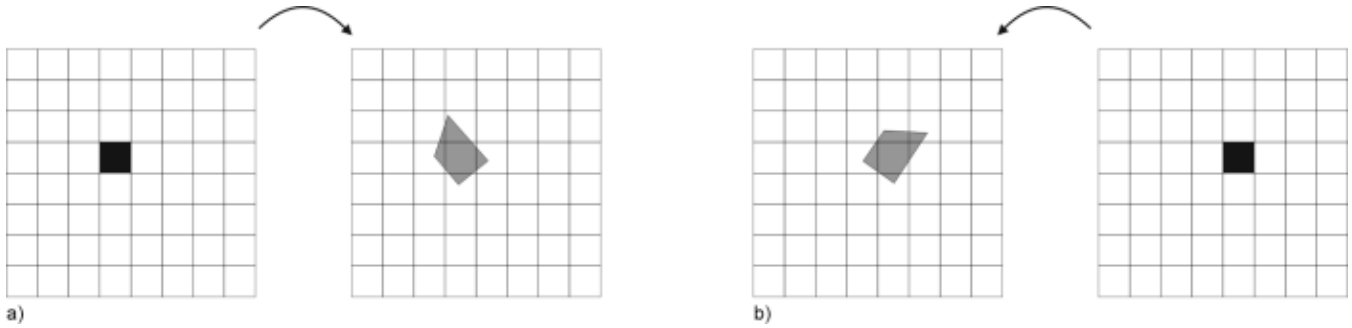
- [Phon75] Phong, B. T.: *Illumination for computer generated images.* CACM, 18 (6), June 1975
- [Torr67] Torrance, K. E., Sparrow, E. M.:*Theory for Off-Specular Reflection from Roughened Gems III.* J. Opt. Soc. Am., 57 (9), July 1991, pp. 309-318
- [Cook82] Cook, R. L., Torrance, K. E:*A Reflectance Model for Computer Graphics.* ACM Transactions on Graphics, 1 (1), 1982, pp. 7-24
- [Gour71] Gouraud, H.:*Continuous shading of curved surfaces* IEEE Transaction on Computers, 24 (4), August 1990

18. Geometrické transformace diskrétního obrazu

Obvyklé geometrické transformace pracují se spojitě zadaným objektem. Při práci s diskrétním obrazem však narážíme na problémy, které pramení z toho, že transformace obecně nepřiřadí diskrétním celočíselným souřadnicím pixelu ve vstupním obraze diskrétní celočíselné souřadnice v obraze výstupním, ale mohou ho mapovat na libovolné místo. V obraze získaném transformací, ve které by byl každý pixel transformován zvlášť, by tímto postupem vznikaly "díry", nebo naopak několik pixelů by bylo mapováno na totéž místo.

V dalším textu budeme předpokládat, že máme vstupní obraz *A*, který budeme transformovat. Výstupem bude obraz *B*, nebo v některých případech ještě pomocný obraz (meziobraz) *I*.

Nejprve uvedeme některé pojmy, které budeme v dalším textu používat. Mapování určuje způsob přiřazení pixelů z jednoho obrazu do druhého a můžeme ho rozdělit na *dopředné mapování* (*forward mapping*) a *zpětné mapování* (*backward mapping*). Oba tyto případy demonstruje obrázek 18.1.



obr. 18.1: a) Dopředné a b) zpětné mapování. Pixel z jednoho obrazu se mapuje na oblast ve druhém obrazu, některé pixely může pokrývat zcela, jiné pouze z části. Při dopředném mapování může dojít ke vzniku "bílých míst".

Při dopředném mapování procházíme pixely vstupního obrazu *A* a hledáme jejich umístění ve výstupním obraze *B*. Při zpětném mapování procházíme pixely výstupního obrazu *B* a hledáme odpovídající oblasti v obraze *A*, které se do procházených pixelů mapují. Zjevným problémem dopředného mapování je, že ve výsledném obraze *B* mohou vznikat volná místa, můžeme nalézt takovou transformaci, která jednoduše do určitého místa žádný pixel nenamapuje. V případě zpětného mapování k tomuto jevu nedochází.

Další důležitý pojem je separabilní operace. Mějme dvojrozměrný obraz složený z bodů [*x*,*y*] a transformaci

$$T(u,v) = [x(u,v)\ ,\ y(u,v)]. \qquad (18.1)$$

Funkce $x(u,v)$ a $y(u,v)$ jsou indexovány souřadnicemi u a v a vrací novou polohu pixelu.

Transformaci označíme jako separabilní, pokud ji lze zapsat jako posloupnost dvou transformací, z nichž každá je funkcí pouze jediné proměnné, tj.

$$T(u,v) = F(u,G(v)). \qquad (18.2)$$

V našem případě pracuje první transformace *G* pouze s řádky obrazu, zatímco druhá transformace *F* transformuje sloupce. Je důležité si uvědomit, že transformace *G* generuje meziobraz *I*, se kterým pracuje transformace *F*. Typickým reprezentantem separabilních operací jsou dvouprůchodové algoritmy otáčení a warping. Jejich schematický postup je patrný z obrázku 18.2. Separabilitu využijeme s výhodou u proudového zpracování grafické informace.

Příkladem separabilní transformace je dvourozměrná (a obecně *n*-rozměrná) Fourierova transformace. Fourierův obraz vznikne aplikací jednorozměrné transformace nejprve na všechny řádky

vstupního obrazu A, tím získáme mezivýsledek I. Na něj je aplikována stejná transformace, tentokrát však po sloupcích. Pro tento konkrétní případ je ve vztahu 18.2 $F = G$, obecně tomu tak být nemusí.

Algoritmy uvedené v dalším textu pracují na principu zpětného mapování. Pokud tomu bude jinak, bude to výslovně uvedeno.

18.1 Změna rozlišení

Častým požadavkem je změna rozlišení obrazu (*zoom*), a to jak jeho zmenšení, tak jeho zvětšení. Různé algoritmy pro změnu velikosti obrazu se liší především tím, jak kvalitně rekonstruují spojitou funkci z diskrétního obrazu. Přirozeně, čím přesnější tato rekonstrukce je, tím je časově náročnější a naopak. Většina algoritmů, které se pro tyto operace používají, je separabilní. K jejich vyjádření s výhodou použijeme jednorozměrné konvoluce.

Interpolace nejbližším sousedem:

Patrně nejjednodušší metoda výpočtu interpolační plochy z diskrétních vzorků se nazývá *interpolace nejbližším sousedem* (*nearest neighbour interpolation*) nebo též *point shift* nebo také *sample and hold*. Výpočet spojité funkce $g(x)$ z diskrétní funkce $f(x_k)$ definované v diskrétních bodech x_k, $k = 1,...,n$ má tvar

$$g(x) = f(x_k); \quad \frac{x_{k-1} + x_k}{2} < x \leq \frac{x_k + x_{k+1}}{2} \tag{18.3}$$

konvoluční jádro této operace má tvar

$$h(x) = \begin{cases} 1 & pro: 0 \leq |x| < 0.5 \\ 0 & pro: 0.5 \leq |x| \end{cases} \tag{18.4}$$

Jednoduše řečeno, tato metoda na požadovanou pozici okopíruje nejbližšího souseda. Vzhledem k tomu, že se jedná o separabilní operaci, okopíruje nejbližší řádek, resp. sloupec, podle pořadí prováděných operací.

Tato metoda je zjevně velice rychlá a jednoduchá, na druhou stranu vede k nežádoucím efektům například na hranách s malým sklonem při zvětšení zvýrazňuje skoky a při zmenšení poškozuje tenké čáry. To tuto metodu předurčuje zejména k rychlému náhledu (*preview*) velkého obrázku. Bývá také implementována v technickém vybavení počítačů.

Bilineární interpolace:

Bilineární interpolace je separabilní operace, kterou získáme postupnou aplikací lineární interpolace.

Mějme dva sousední vzorky v bodech x_0 a x_1 s hodnotami f_0 a f_1. Tyto body se proloží úsečkou a hledaná hodnota v bodě x se vypočítá jako

$$f(x) = f_0 + \frac{x - x_0}{x_1 - x_0}(f_1 - f_0) \tag{18.5}$$

Přirozeně, že mezi tyto dvě hodnoty může padnout libovolné množství vzorků.

Odpovídající konvoluční jádro bývá pro svůj tvar nazýváno *tent function* a má tvar

$$h(x) = \begin{cases} 1 - |x| & pro: 0 \leq |x| < 1 \\ 0 & pro: 1 \leq |x| \end{cases} \tag{18.6}$$

Bilineární interpolace je rychlá, neboť se pro výpočet každého bodu používají pouze čtyři body z jeho okolí. Její nevýhodou je, že rozmazává původní ostré přechody. Tato metoda bývá též snadno implementovatelná v technickém vybavení počítačů.

18.2 Otáčení diskrétního obrazu

S otáčením obrazu o celočíselné násobky devadesáti stupňů nejsou žádné problémy. Dají se implementovat jako jednoduchá změna pixelů na příslušných pozicích v matici reprezentující obraz, navíc je možno tyto operace implementovat bez nutnosti alokovat další prostor pro výstupní obraz. Daleko složitější je to s otáčením diskrétního obrazu o obecný úhel. Pro tento případ se často používá algoritmus, který patří mezi tzv. *dvouprůchodové algoritmy* (*two-pass algorithm*). Jejich podstatou není nic jiného, nežli převedení transformace otáčení na dvě transformace. Zatímco v případě Fourierovy transformace či lineární interpolace byly operace pro první i druhý průchod identické, to znamená, že se stejná transformace aplikovala jednou na řádky a podruhé na sloupce, v případě otáčení tomu tak již není.

Dále budeme používat zpětné mapování. Mějme transformaci otáčení, a přepišme ji tak, aby vyhovovala výše uvedené notaci s u a v

$$\begin{aligned} x &= u \cos \alpha - v \sin \alpha \\ y &= u \sin \alpha - v \cos \alpha \end{aligned} \tag{18.7}$$

V tomto vztahu mají body ve vstupním obraze A souřadnice $[u,v]$ a pixely výstupního obrazu B jsou označeny indexy $[x,y]$.

V prvním průchodu algoritmu se aplikuje na všechny řádky, tj. provede se transformace

$$y = u \sin \alpha + v \cos \alpha. \tag{18.8}$$

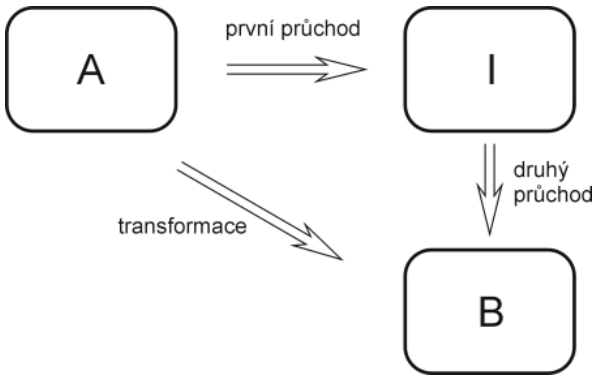
Touto operací získáme meziobraz I indexovaný $[x,v]$. Nyní již však nemůžeme aplikovat transformaci

$$x = u \cos \alpha + v \sin \alpha. \tag{18.9}$$

protože ta je funkcí u, se kterým již bylo počítáno v prvním průchodu. Vyjádříme proto u jako funkci x a v ze vztahu 18.9 a dosadíme do 18.8. Odtud získáme vztah pro druhý průchod

$$y = \frac{x \sin \alpha + v}{\cos \alpha}. \tag{18.10}$$

Souřadnice bodu ve vstupním obraze A získáme výpočtem z celočíselných souřadnic pixelu v obraze B. Bod v obraze A nemusí samozřejmě ležet přesně uprostřed pixelu. Pro výpočet hodnoty, která se bude mapovat do obrazu B, můžeme použít libovolnou metodu převzorkování s tím, že se nevypočítává spojitá funkce pro celý obraz, ale pouze pro to okolí bodu $[u,v]$, které je pro výpočet důležité.



obr. 18.2: Schéma funkce dvouprůchodového algoritmu. Transformace vstupního obrazu A na obraz B je dosaženo výpočtem meziobrazu I.

Při aplikaci prvního průchodu může dojít k akumulaci mnoha pixelů ze vstupního obrazu na jediném místě (*many-to-one mapping*), nebo naopak k aplikaci malého množství pixelů na místě jiném. Tento jev je označován jako *bottleneck problem* a nastává typicky pro hodnoty blízké 90°. Autoři algoritmu proto doporučují například otáčení o 87° provést jako otočení o 90° a poté o -3°.

Obraz vzniklý otočením má větší rozlišení, nežli jeho předloha. Nové rozlišení vypočítáme snadno otočením rohů předlohy a jejich promítnutím do souřadnicových os. Volná místa, která v otočeném obraze vznikají, jsou obyčejně vyplněna barvou, kterou zvolí uživatel.

APPLET

applet 18.1

18.3 Matematická morfologie

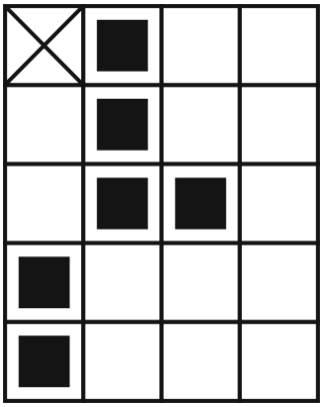
Matematická morfologie vychází z vlastností bodových množin. Lze zpracovávat jak obrazy s více úrovněmi jasu, tak i binární obrazy. Spojitou obrazovou funkci s více úrovněmi jasu v rovině lze chápat jako bodovou množinu v trojrozměrném prostoru E_3, třetí rozměr odpovídá jasu.

Poznamenejme, že reprezentace bodovými množinami je analogická jako u spojitých funkcí. Ke každému bodu v prostoru E_3 lze najít libovolně blízký bod. Nadále se budeme věnovat pouze zpracování binárního obrazu, tj. bodovým množinám X reprezentujícím objekty a množinám X^c reprezentujícím pozadí. K pozadí zde řadíme i díry v objektech.

Přirozeným nástrojem pro matematický popis objektů v rovině je *Euklidův dvojrozměrný prostor* E_2 (dále jen E) se systémem všech podmnožin. Jsou definovány relace inkluze $\subset$, resp. $\supset$, průnik $\cap$ a sjednocení $\cup$, existence prázdné množiny $\varnothing$ a množinového doplňku. Jednostranný rozdíl množin $X \mid Y$ je definován pomocí průniku: $X \mid Y = X \cap Y^c$.

Binární obraz je reprezentován *bodovou množinou*. V digitálním zpracování obrazu se používá diskrétní vyjádření bodových množin. Body příslušející objektům (označeny 1) v rovině jsou popsány dvojicí svých celočíselných souřadnic - (řádek,sloupec). Body doplňku bodové množiny jsou označeny 0. Počátek má souřadnice (0,0). Počátek budeme v našich příkladech označovat dvojitým přeškrtnutím z rohu do rohu. Na obr. 18.3 je vidět příklad bodové množiny

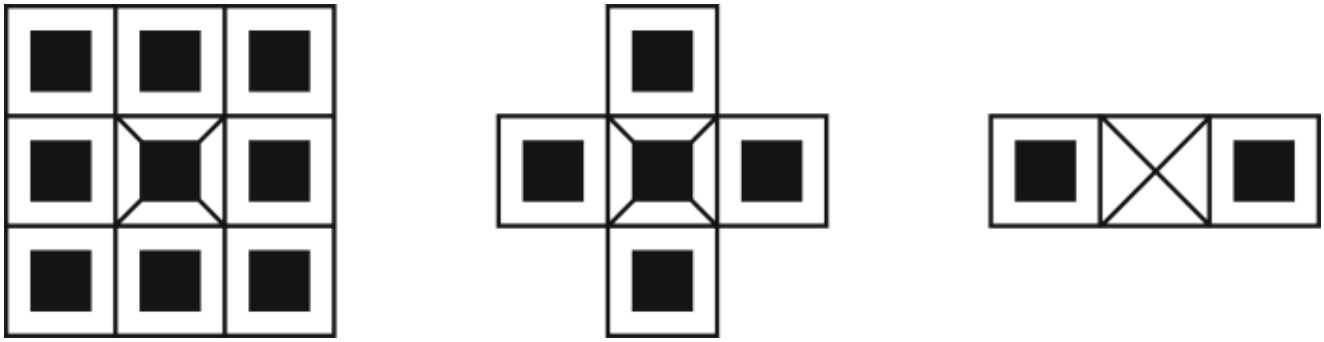
$X = \{(0,1),(1,1),(2,1),(2,2),(3,0),(4,0)\}$



obr. 18.3: Příklad bodové množiny

Prakticky používané morfologické transformace jsou realizovány jako relace obrazu (bodové množiny X) s jinou menší bodovou množinou B, které se říká *strukturní element*. Některé často používané strukturní elementy jsou na obr. 18.4. Všimněte si, na obr. 18.4 vpravo, že počátek nemusí být bodem strukturního elementu. Elementům, které mají stejné vlastnosti pro různé směry, říkáme *izotropické*.

Morfologickou transformaci si představíme, jako bychom strukturním elementem *B* systematicky pohybovali po obraze. Bodu obrazu, který se shoduje s počátkem souřadnic strukturního elementu, říkáme *okamžitý bod*. Výsledek relace mezi obrazem a strukturním elementem (může nabýt hodnoty 0 nebo 1) je zapsán do okamžitého bodu obrazu.



obr. 18.4: Často užívané strukturní elementy

Dilatace a eroze:

Dilatace skládá body dvou množin pomocí vektorového součtu (též *Minkowkého množinový součet*)

$$X \oplus B = \{ d \in E^2 : d = x + b, x \in X, b \in B \}. \qquad (18.11)$$

obr. 18.5: Původní obraz (vlevo) a dilatace (vpravo)

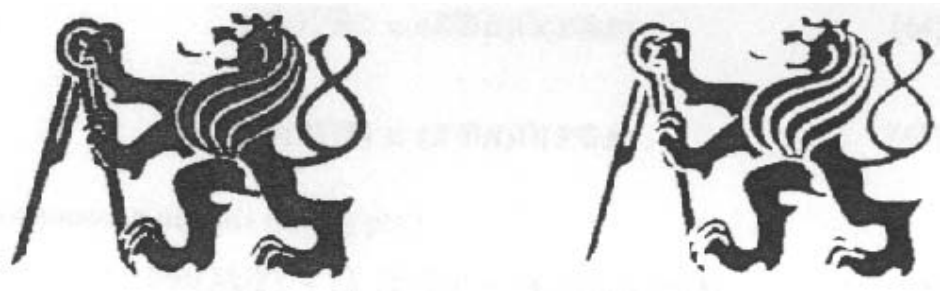
Na obr. 18.5 je původní obraz rozměru 256x256 bodů a výsledek dilatace strukturním elementem rozměru 3x3, který představuje kruh v oktagonálním rastru. Z obr. 18.5 je vidět, že objekt se rozrostl o jednu slupku na úkor pozadí. Také díry, které měly tloušťku jeden bod, se zaplnily. Dilatace se používá k zaplnění malých děr a úzkých zálivů. Pokud dilatací zvětšené objekty chceme zmenšit do původního rozměru, použijeme erozi.

APPLET

applet 18.2

Eroze skládá dvě bodové množiny s využitím rozdílu vektorů. Je duální transformací k dilataci. Připomeňme, že eroze není inverzní transformací. Eroze je definována

$$X \ominus B = \{ d \in E^2 : d + b \in X \text{ pro } \forall\, b \in B \}. \qquad (18.12)$$



obr. 18.6: Původní obraz (vlevo) a eroze (vpravo)

Na obr. 18.6 je výchozí obraz a jeho eroze strukturním elementem rozměru 3x3. Všimněme si, že objekty tloušťky 1 zmizely. Eroze se používá pro zjednodušení struktury objektů. Složité objekty spojené čárami tloušťky 1 se rozloží na několik jednodušších objektů.

Máme-li k dispozici operaci eroze, můžeme snadno a rychle najít obrysy objektů v obraze. Jednou možností je odečíst od původního obrazu jeho erozi. Takto byly nalezeny obrysy na obr. 18.7.

obr. 18.7: Obrysy

APPLET

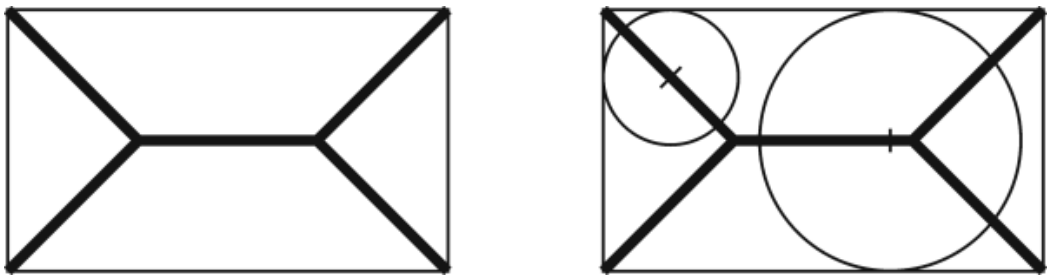
applet 18.3

Topologické vlastnosti a morfologie, skelet:

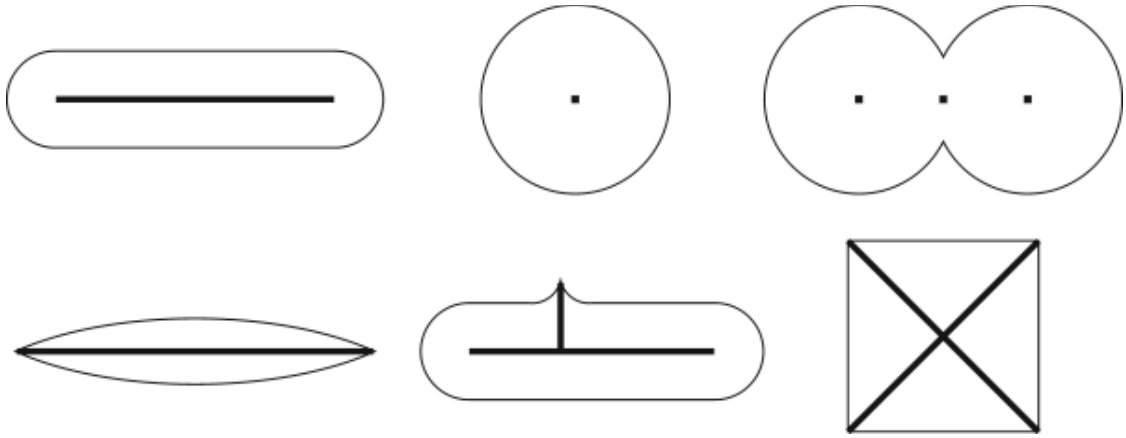
Matematickou morfologii lze použít pro studium topologických vlastností obrazů. Víme již, že topologické vlastnosti souvisejí s pojmem spojitost.

Často je výhodné popsat objekty v obraze objektem složených z tenkých čar, kterému se říká *skelet* (kostra) objektu. Vysvětleme si, co je skelet. Pro jednoduchost nejprve pro spojité množiny. Neformálně lze říct, že skelet $S(Y)$ množiny Y je sjednocením středů x kružnic $D(x)$, které jsou obsaženy v Y a dotýkají se hranice množiny Y alespoň ve dvou bodech.

Ukažme si příklady skeletů pro některé množiny. Na obr. 18.8 je skelet pro obdélník a na obr. 18.9 je několik dalších objektů s jejich kostrami. Všimněme si, že skelet dvou dotýkajících se kružnic je dán třemi izolovanými body. Z toho můžeme vyvodit, že skelet spojité množiny nemusí být vždy spojitá množina.



obr. 18.8: Skelet pro obdélník.



obr. 18.9: Příklady skeletů dalších objektů.

Z hlediska matematické morfologie je zajímavé, že skelet lze vytvořit pro množiny na diskrétním rastru pomocí erozí a dilatací. Takto získaný skelet má nepraktickou vlastnost, může být složen z čar tlustších než jeden obrazový bod. Proto se skelet často nahrazuje množinou zpracovanou sekvenčním ztenčováním.

Nejprve zaveďme transformaci *hit or miss* (též transformace tref či miň nebo Serrova). Budeme potřebovat složený strukturní element $B = \{B_1, B_2\}$. B_1 je podmnožinou množiny X (objekty v obraze) a B_2 je podmnožinou X^C (pozadí). Transformace *hit or miss* se označuje pomocí $\otimes$ a je definována:

$$X \otimes B = \{ x\colon B_1 \subset X \text{ and } B_2 \subset X^C\}. \qquad (18.13)$$

Transformace *hit or miss* se chová jako hledání shody mezi binárním obrazem X a složeným strukturním elementem. Transformaci *hit or miss* můžeme také vyjádřit pomocí dilatací a erozí:

$$X \otimes B = (X \ominus B_1) \cap (X^C \ominus B_2) = (X \ominus B_1) \mid (X \oplus B_2). \qquad (18.14)$$

Jedním možným použitím transformace hit or miss je ztenčování a zesilování bodových množin.

Ztenčování obrazu X složeným strukturním elementem $B = \{B_1, B_2\}$ je definováno:

$$X \oplus B = X \mid (X \otimes B). \qquad (18.15)$$

Podobně **zesilování** je definováno:

$$X \odot B = X \cup (X \otimes B). \qquad (18.16)$$

Ztenčován a zesilování jsou duální transformace

$$(X \odot B)^C = X^C \oplus B^*, \text{ kde } B^* = (B_2, B_1). \qquad (18.17)$$

Ztenčování a zesilování se velmi často používají opakovaně. Nechť je dána posloupnost strukturních elementů $\{B_{(i)}\} = \{B_{(1)}, B_{(2)}, ...\}$, kde $B_{(i)} = (B_{i1}, B_{i2})$ jsou složené strukturní elementy.

Sekvenční ztenčování lze vyjádřit pomocí

$$X \oplus \{B_{(i)}\} = ((((X \oplus B_{(1)}) \oplus B_{(2)} ... \oplus B_{(i)})...). \qquad (18.18)$$

Sekvenční zesilování lze vyjádřit analogicky

$$X \odot \{B_{(i)}\} = ((((X \odot B_{(1)}) \odot B_{(2)} \ldots \odot B_{(i)})\ldots). \qquad (18.19)$$

Existuje několik posloupností strukturních elementů, které jsou v praxi významné. Obvykle jsou označovány jako prvky **Golayovy abecedy**. Zde ukážeme jen jednu významnou posloupnost strukturních elementů rozměru 3x3, a to pro čtvercový rastr. Pro jednoduchost budeme popisovat složený strukturní element jedinou maticí. Jedničky v matici označují, že prvek patří k elementu (srovnává se s body objektů). Nuly označují, že prvek patří k elementu pozadí. Hvězdička ve strukturním elementu označuje prvek, který na srovnání nemá vliv.

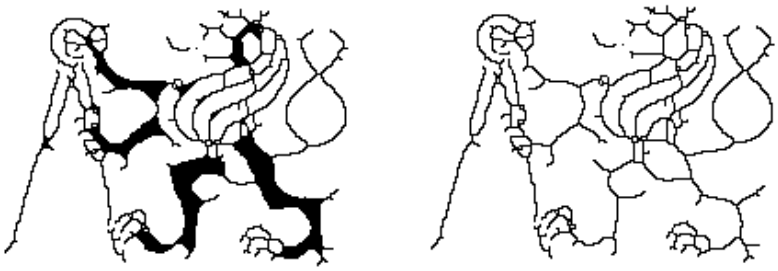
Sekvenční ztenčování a zesilování konverguje do konečného stavu. Počet iterací závisí na objektu a velikosti použitého strukturního elementu. Konečný stav se indikuje tak, že se výsledek dvou po sobě následujících iterací neliší.

Sekvenční ztenčování elementem *L* se používá jako náhrada skeletu. Objekty ve ztenčeném objektu mají tloušťku rovnu jedné. Z posloupnosti osmi strukturních elementů ukážeme jen první dvě. Následující se získají pootočením ve směru hodinových ručiček.

Strukturní elementy *L* pro čtvercový rastr jsou:

$$L_1 = \begin{bmatrix} 0 & 0 & 0 \\ * & 1 & * \\ 1 & 1 & 1 \end{bmatrix}, \; L_2 = \begin{bmatrix} * & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & * \end{bmatrix} \qquad (18.19)$$

Ukažme si sekvenční ztenčování elementem *L* na příkladě. Původní binární obraz o rozměru 256x256 bodů byl již ukázán na obr. 18.10 vlevo. Na obr. 18.10 vpravo je vidět výsledek pěti iterací ztenčování a výsledná náhrada skeletu, kde se iterovalo, dokud nebyly dva po sobě následující obrazy stejné.



obr. 18.10: Sekvenční ztenčování elementem *L* po pěti iteracích (vlevo) a úplné (vpravo).

Literatura:

- [zara98] J. Žára, B. Beneš, P. Felkel - Moderní počítačová grafika
- [grad92] M. Šonka, V. Hlaváč - Počítačové vidění